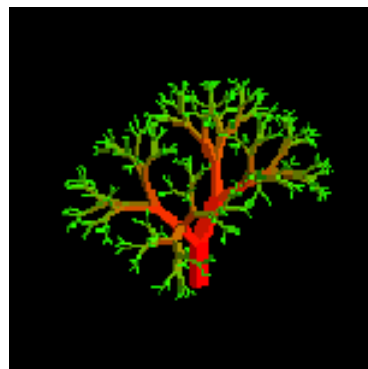
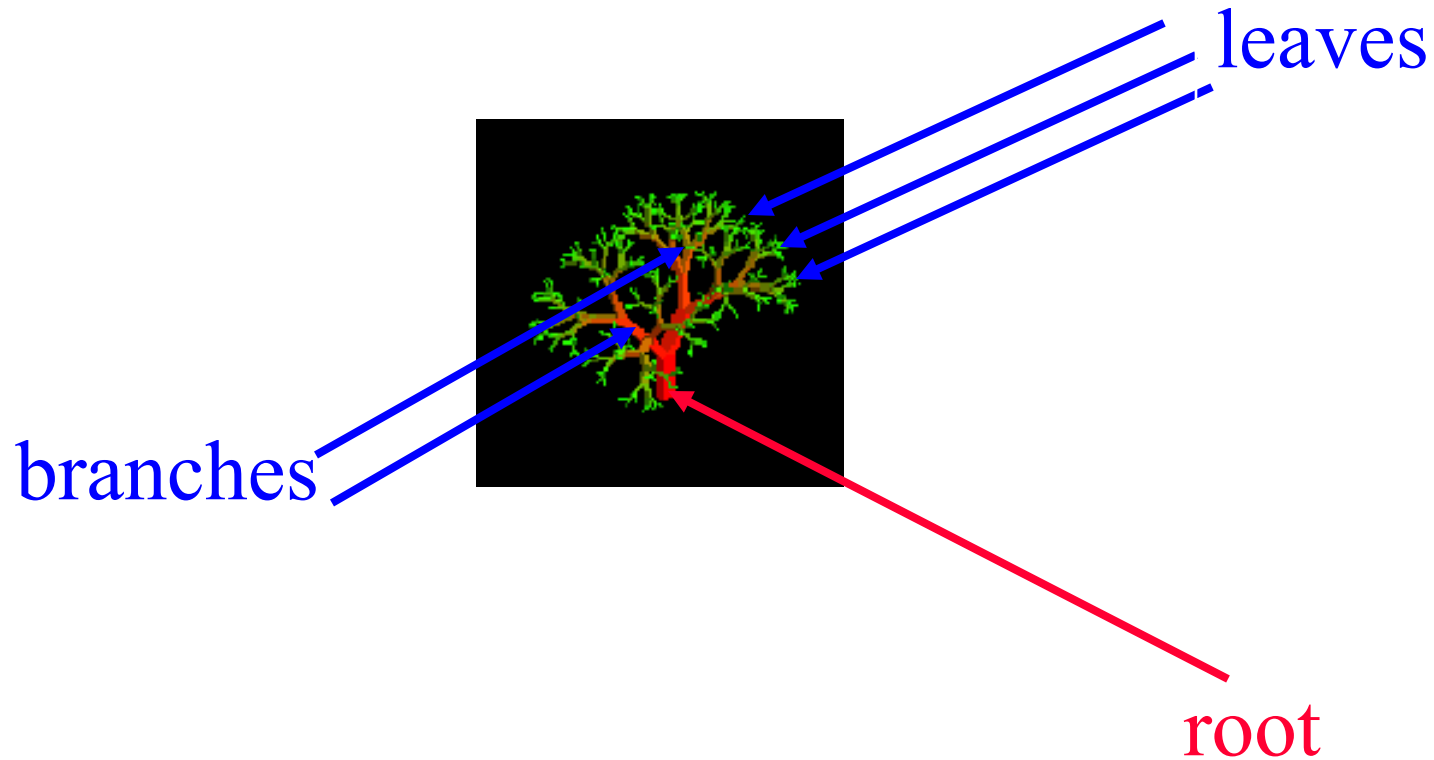




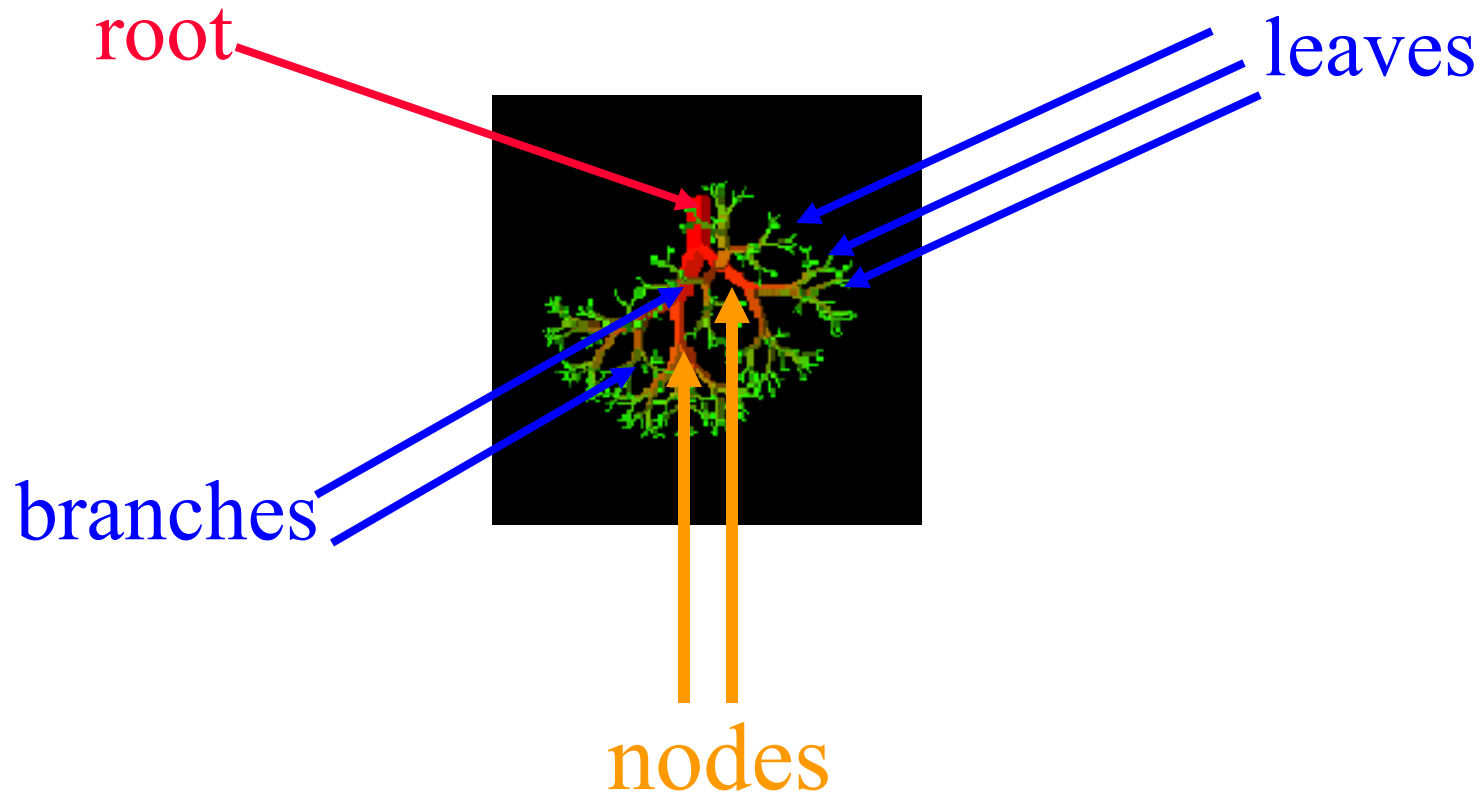
Trees



Nature Lover's View Of A Tree



Computer Scientist's View



Chapter 5

Trees

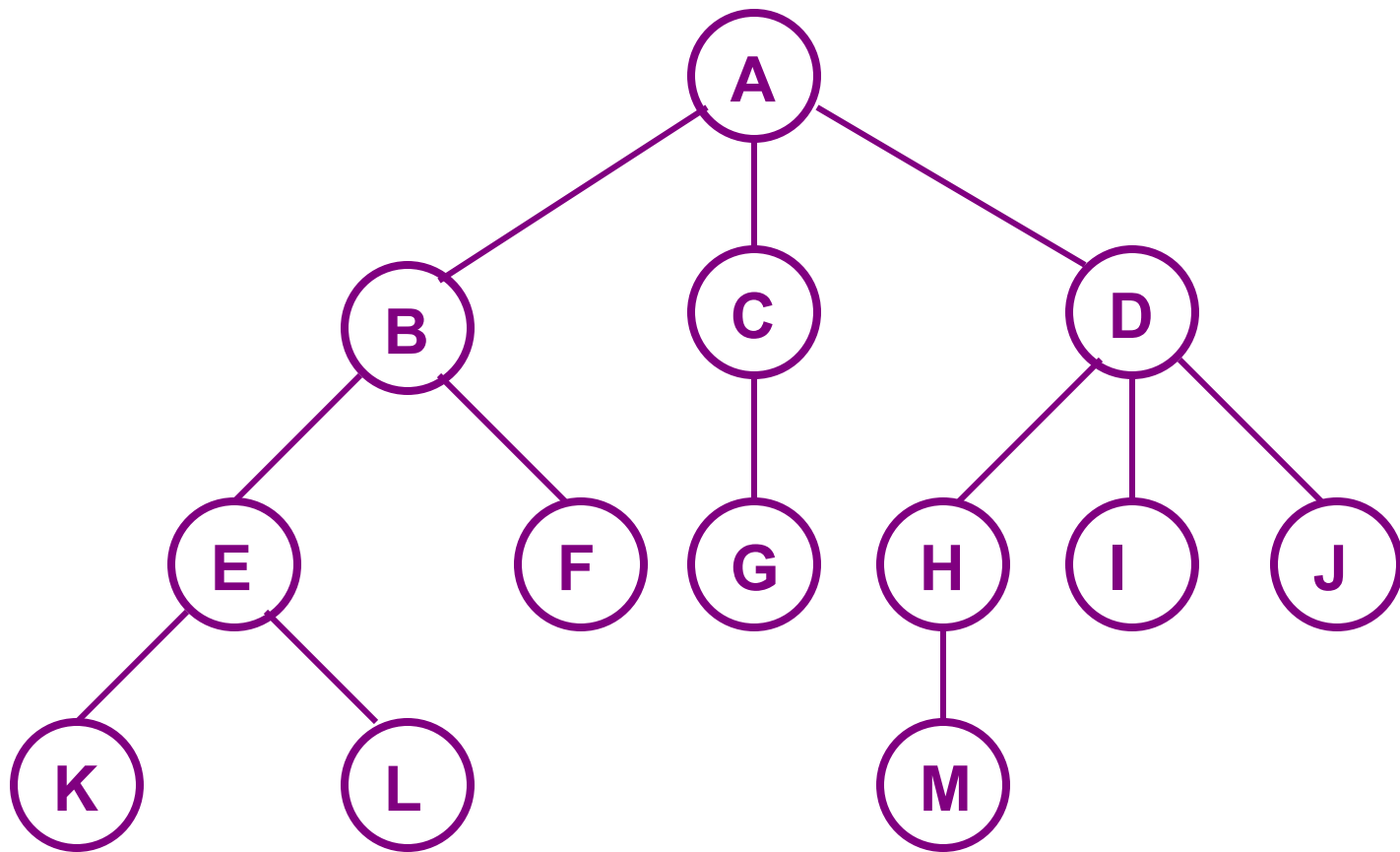
5.1 Introduction

5.1.1 Terminology

Definition: A tree is a finite set of one or more nodes such that

- (1) There is a specially designated node called root.
- (2) The remaining nodes are partitioned into $n \geq 0$ disjoint sets $T_1, \dots, T_n$, where each of these sets is a tree.

$T_1, \dots, T_n$ are called **subtrees** of the root.





Linear Lists And Trees



- Linear lists are useful for serially ordered data.
 - $(e_0, e_1, e_2, \dots, e_{n-1})$
 - Days of week.
 - Months in a year.
 - Students in this class.
- Trees are useful for hierarchically ordered data.
 - Employees of a corporation.
 - President, vice presidents, managers, and so on.
 - classes.
 - Object is at the top of the hierarchy.
 - Subclasses of Object are next, and so on.

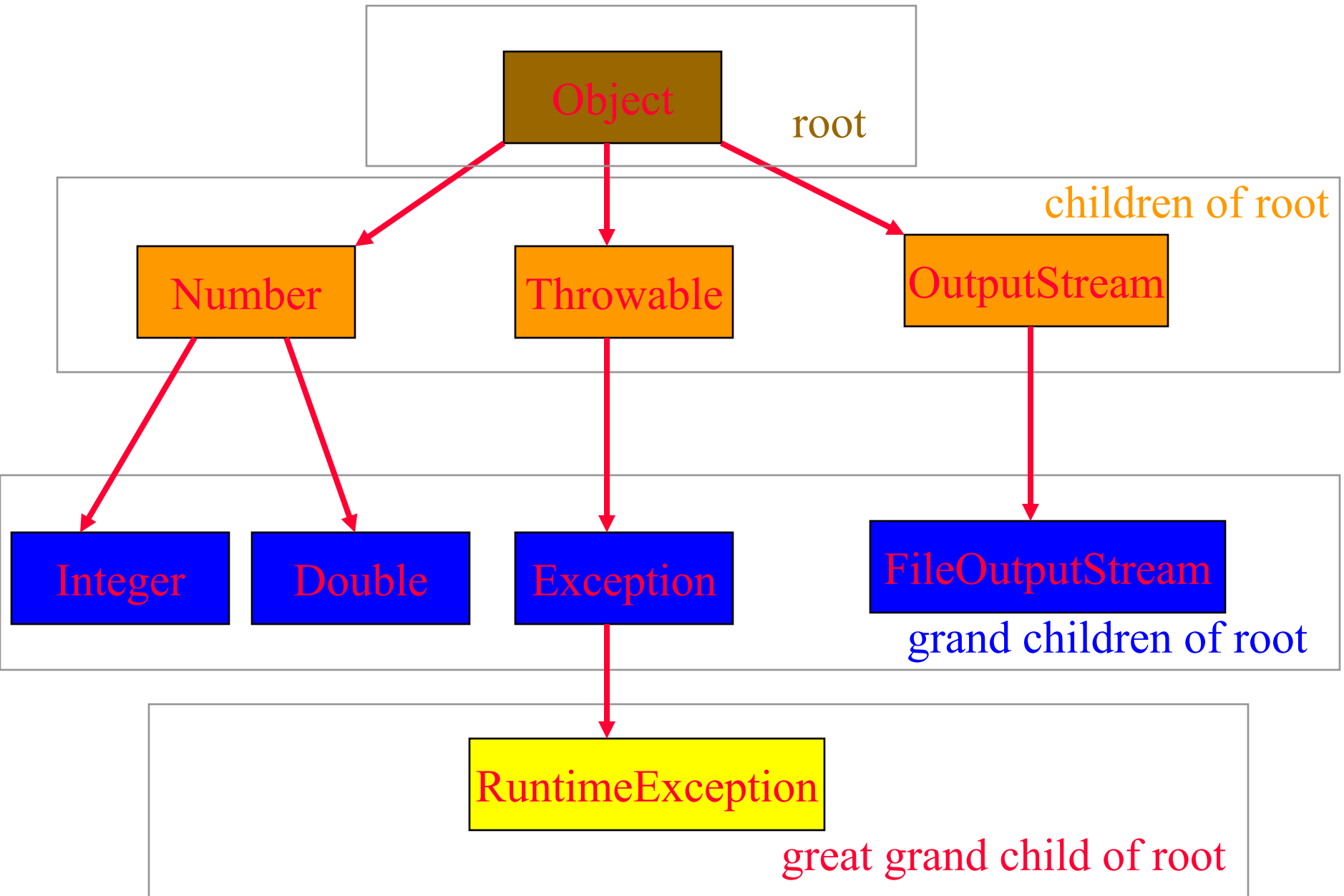


Hierarchical Data And Trees



- The element at the top of the hierarchy is the **root**.
- Elements next in the hierarchy are the **children** of the root.
- Elements next in the hierarchy are the **grandchildren** of the root, and so on.
- Elements that have no children are **leaves**.

Java's Classes (Part Of Figure 1.1)





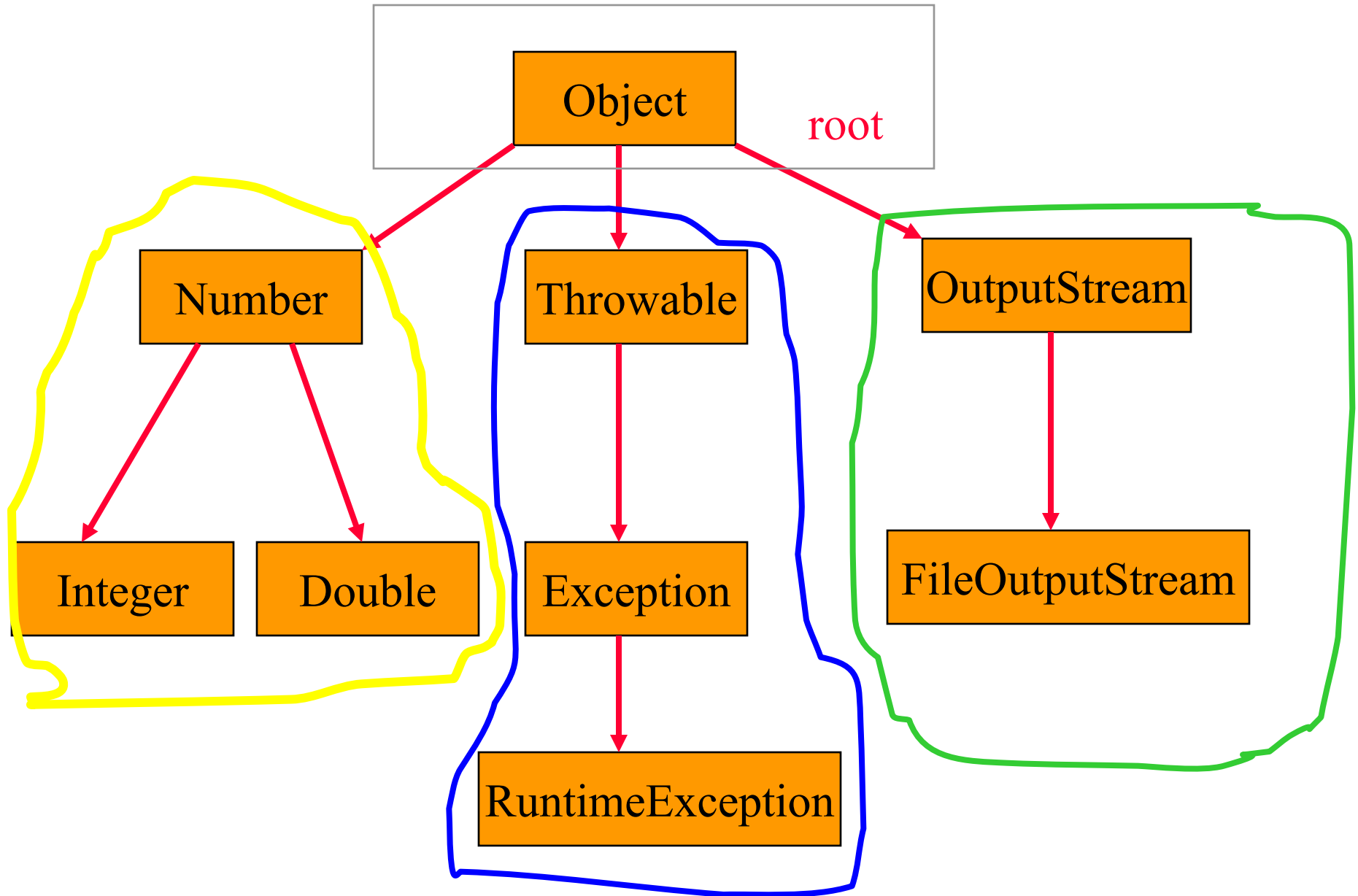
Definition



- A tree t is a finite nonempty set of elements.
- One of these elements is called the root.
- The remaining elements, if any, are partitioned into trees, which are called the subtrees of t .

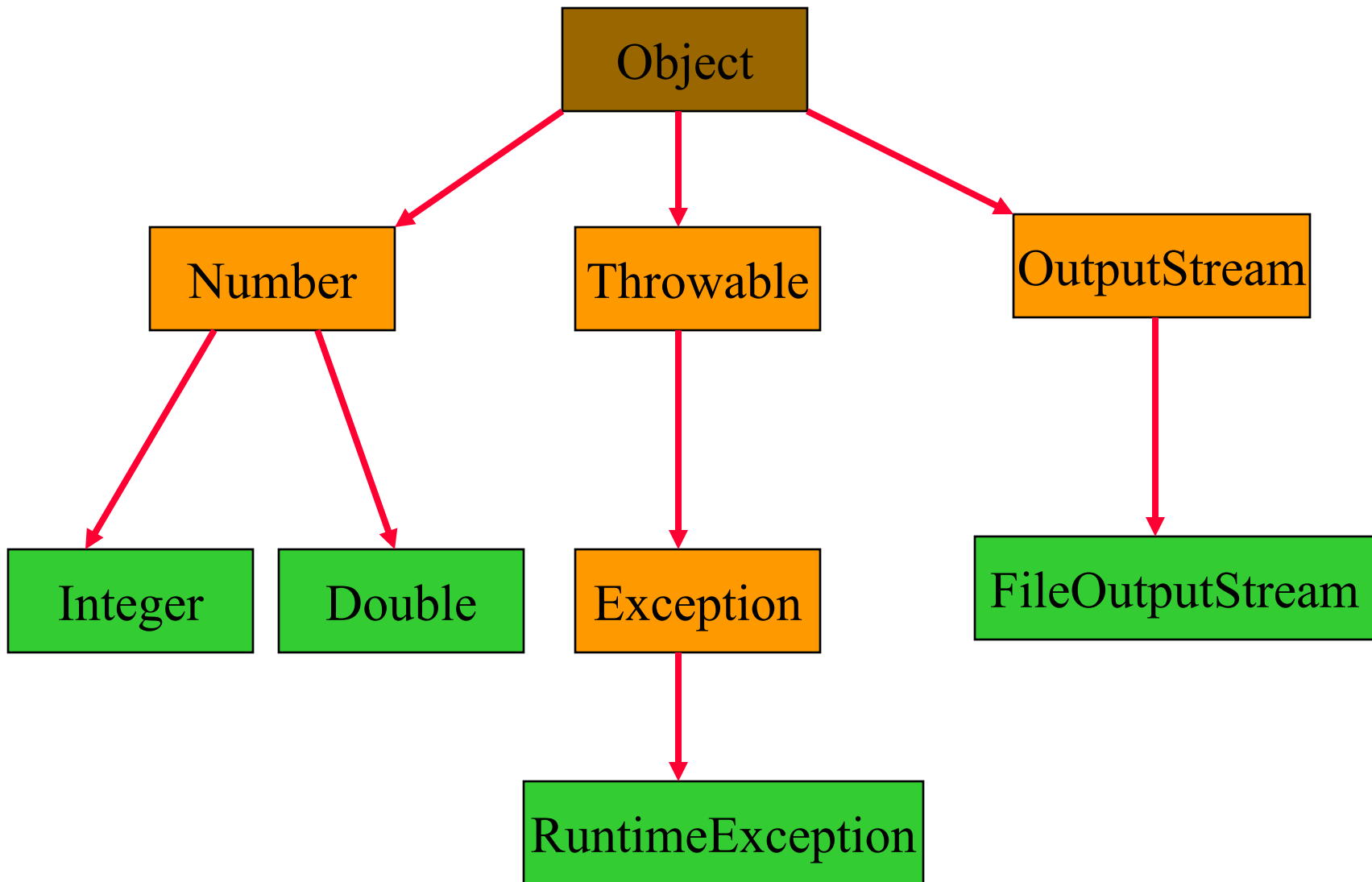


Subtrees

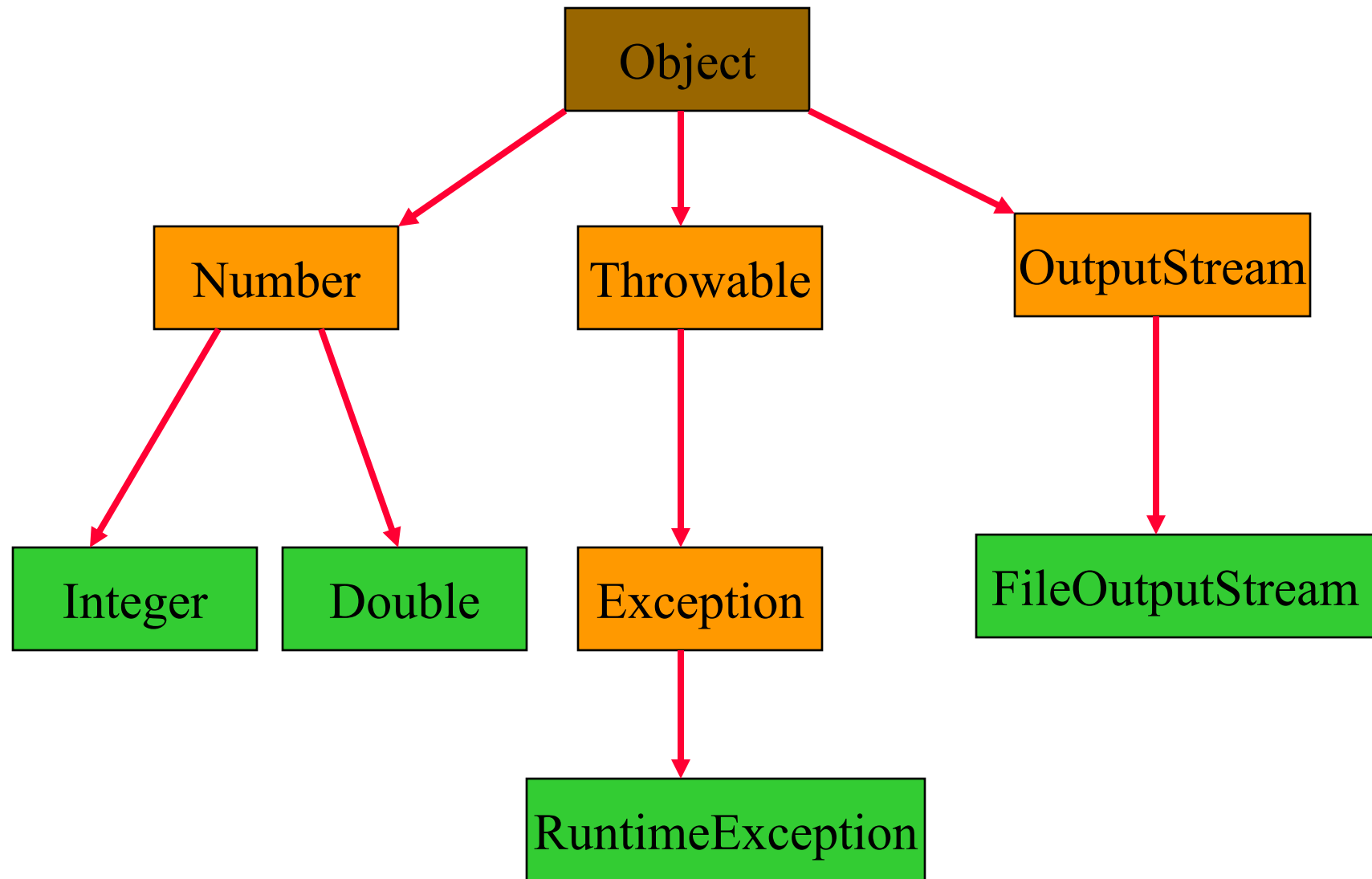




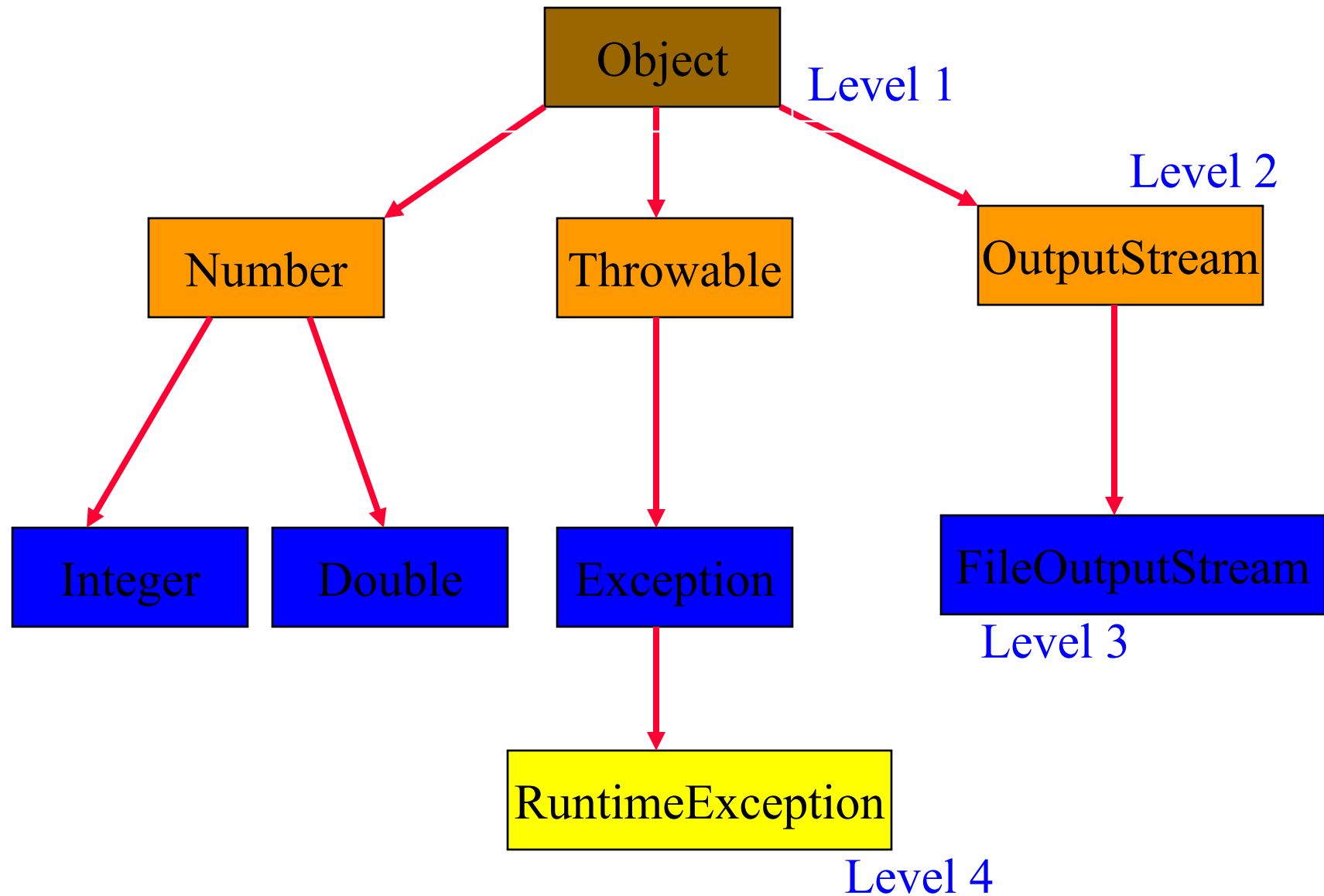
Leaves



Parent, Grandparent, Siblings, Ancestors, Descendants



Levels



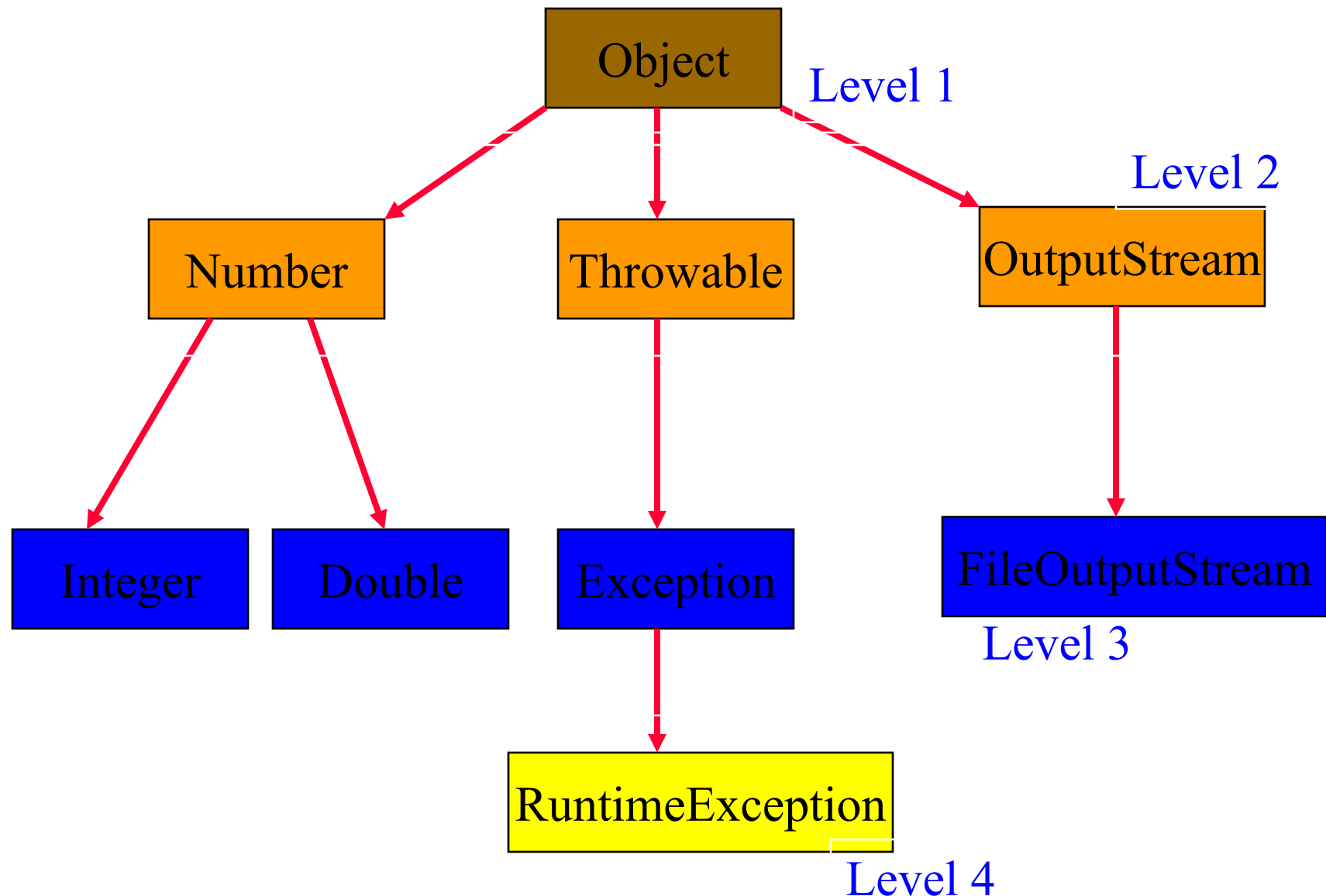


Caution

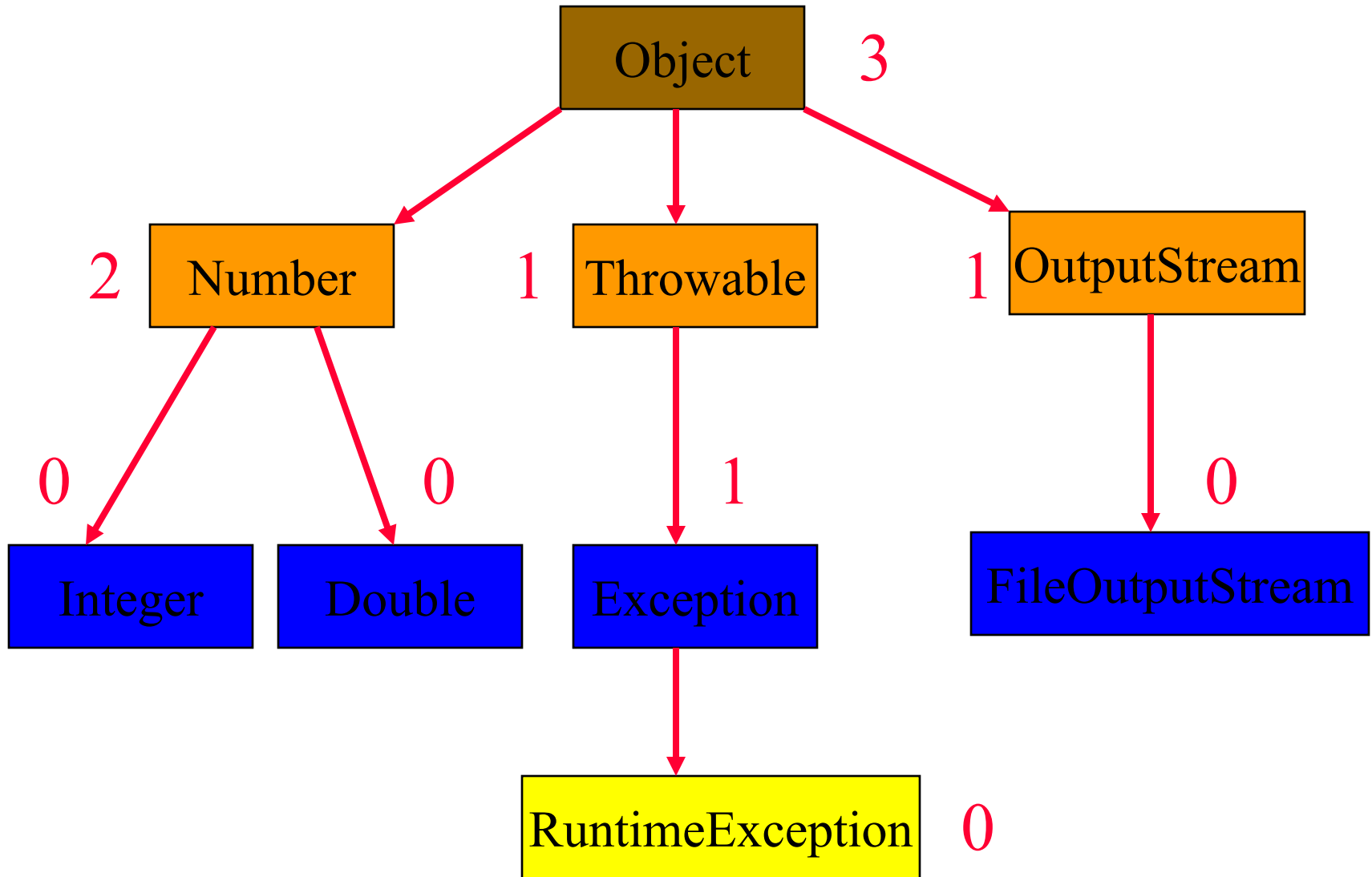


- Some texts start level numbers at 0 rather than at 1.
- Root is at level 0.
- Its children are at level 1.
- The grand children of the root are at level 2.
- And so on.
- We shall number levels with the root at level 1.

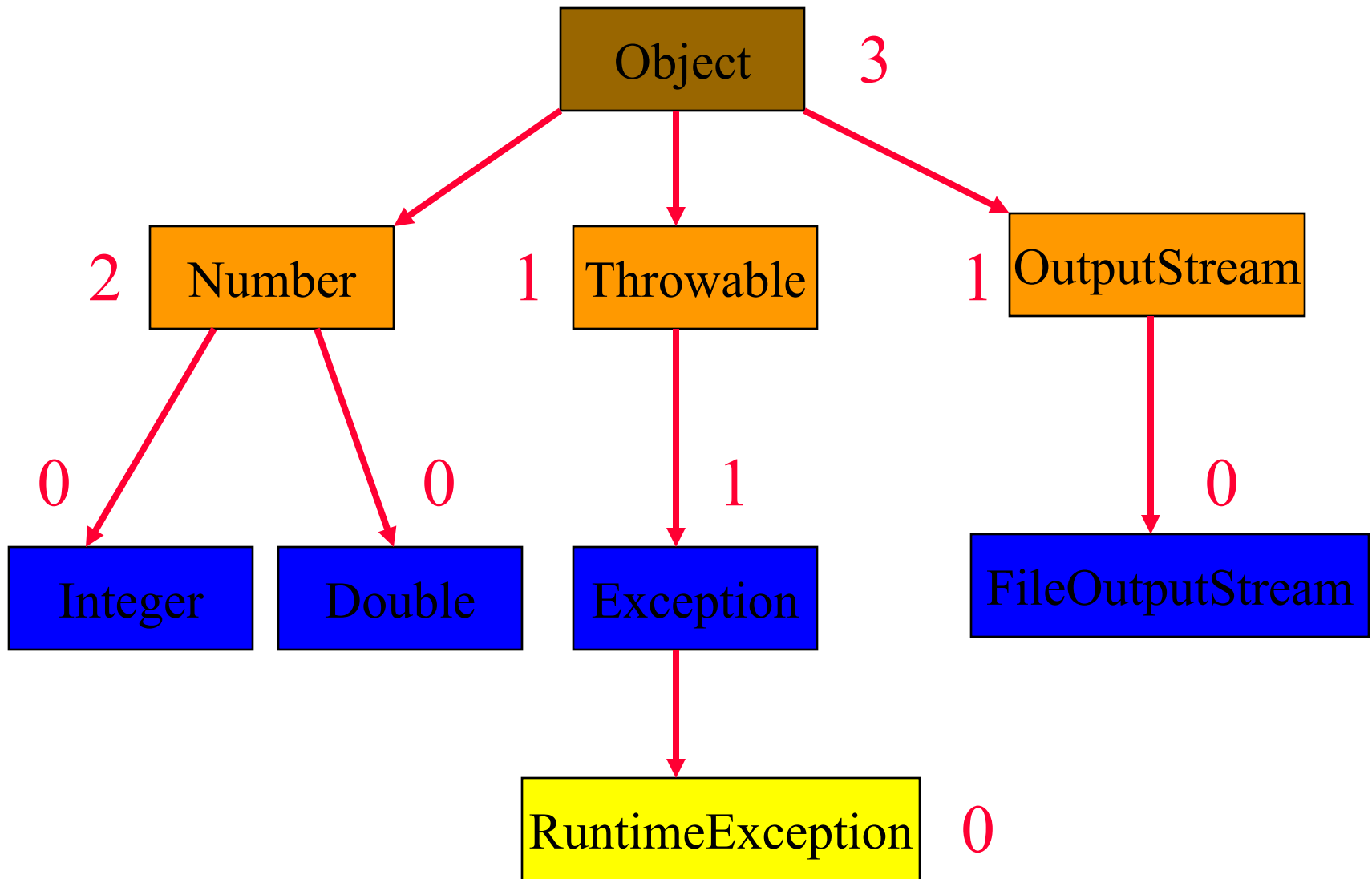
height = depth = number of levels



Node Degree = Number Of Children



Tree Degree = Max Node Degree



Degree of tree = 3.

Representation

- What to be recorded?
 - Entities
 - Nodes
 - Relationships
 - Parent-children (Edges)
- Sequential mapping: Arrays
- Linked representation

5.1.2 Representation of Trees

Node

D1	D2	D3				
-----------	-----------	-----------	--	--	--	--

Nodes ID (1,.....n)

Nodes ID (1,.....,n)

Relationships

5.1.2 Representation of Trees

What about linked lists?

To enable a Parent to locate its Children (or backwards):

For a tree of degree k , we could use a tree node that has fields for data and k pointers to the children (fixed node sizes)

Data	Child1	Child2	...
------	--------	--------	-----

Possible node structure for a tree of

Waste of space!



Lemma 5.1: If T is a k -ary tree with n nodes, each having a fixed size as in Fig. 5.4, then $n(k-1)+1$ of the $n \cdot k$ child fields are 0, $n \geq 1$.

Proof:

- each non-zero child field points to a node
- there is exactly one pointer to each node other than the root
- the number of non-zero child fields in an n node tree is:
 - $n-1$
- the number of zero fields is
 - $nk-(n-1)=n(k-1)+1$.

Binary Tree

- Finite (possibly empty) collection of elements.
- A **nonempty** binary tree has a **root** element.
- The remaining elements (if any) are partitioned into **two** binary trees.
- These are called the **left** and **right** subtrees of the binary tree.

Differences Between A Tree & A Binary Tree

- No node in a binary tree may have a degree more than 2, whereas there is no limit on the degree of a node in a tree.
- A binary tree may be empty; a tree cannot be empty.

Differences Between A Tree & A Binary Tree

- The subtrees of a binary tree are ordered; those of a tree are not ordered.



- Are different when viewed as binary trees.
- Are the same when viewed as trees.

ADT 5.1

```
template <class T>
```

```
class BinaryTree
```

```
{ // A finite set of nodes either empty or consisting of  
  // a root node, left BinaryTree and right BinaryTree.
```

```
public:
```

```
  BinaryTree ();
```

```
  // creates an empty binary tree
```

```
  bool IsEmpty ();
```

```
  // return true iff the binary tree is empty
```

```
  BinaryTree(BinaryTree<T>& bt1, T& item,
```

```
             BinaryTree<T>& bt2);
```

```
  // creates a binary tree whose left subtree is bt1,
```

```
  // right subtree is bt2, and root node contain item.
```

```
BinaryTree LeftSubtree();  
// return the left subtree of *this
```

```
T RootData();  
// return the data in the root of *this
```

```
BinaryTree RightSubtree();  
// return the right subtree of *this
```

```
};
```

Arithmetic Expressions

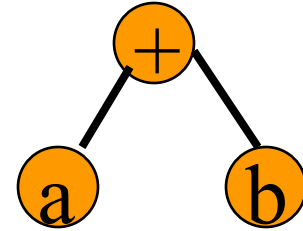
- $(a + b) * (c + d) + e - f/g * h + 3.25$
- Expressions comprise three kinds of entities.
 - Operators (+, -, /, *).
 - Operands (a, b, c, d, e, f, g, h, 3.25, (a + b), (c + d), etc.).
 - Delimiters ((,)).
- Infix form
- Suffix form

Operator Degree

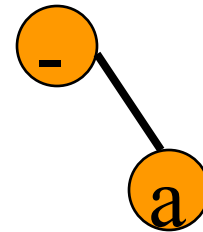
- Number of operands that the operator requires.
- Binary operator requires two operands.
 - $a + b$
 - c / d
 - $e - f$
- Unary operator requires one operand.
 - $+ g$
 - $- h$

Binary Tree Form

- $a + b$

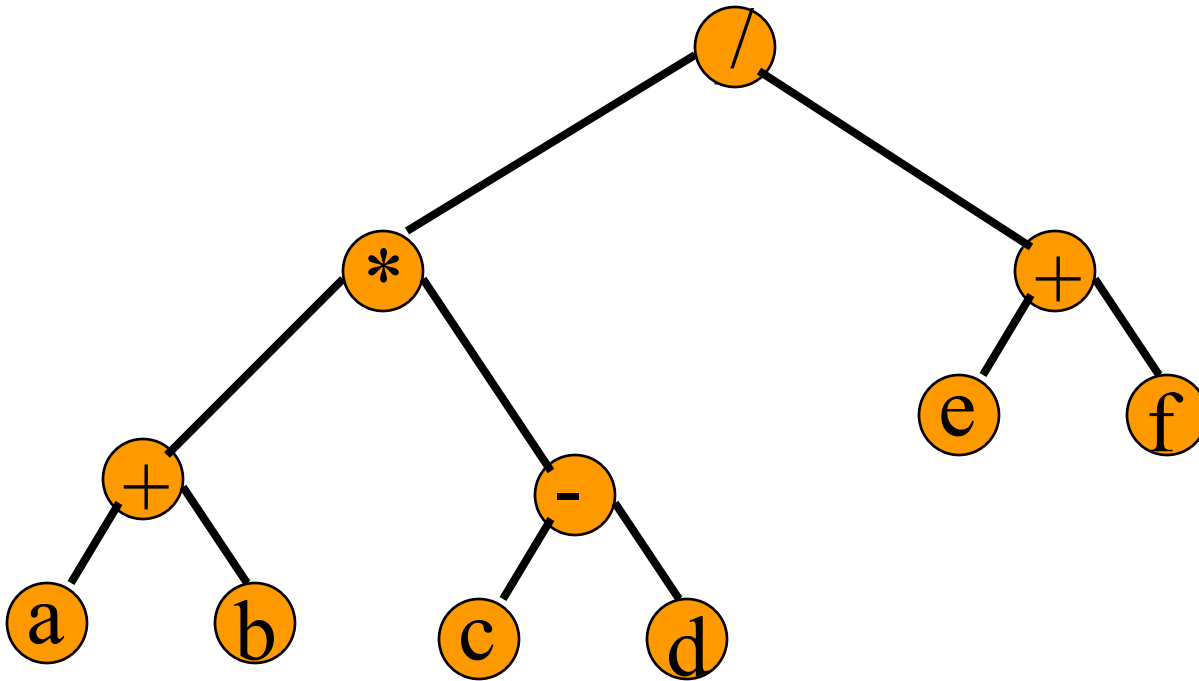


- $- a$



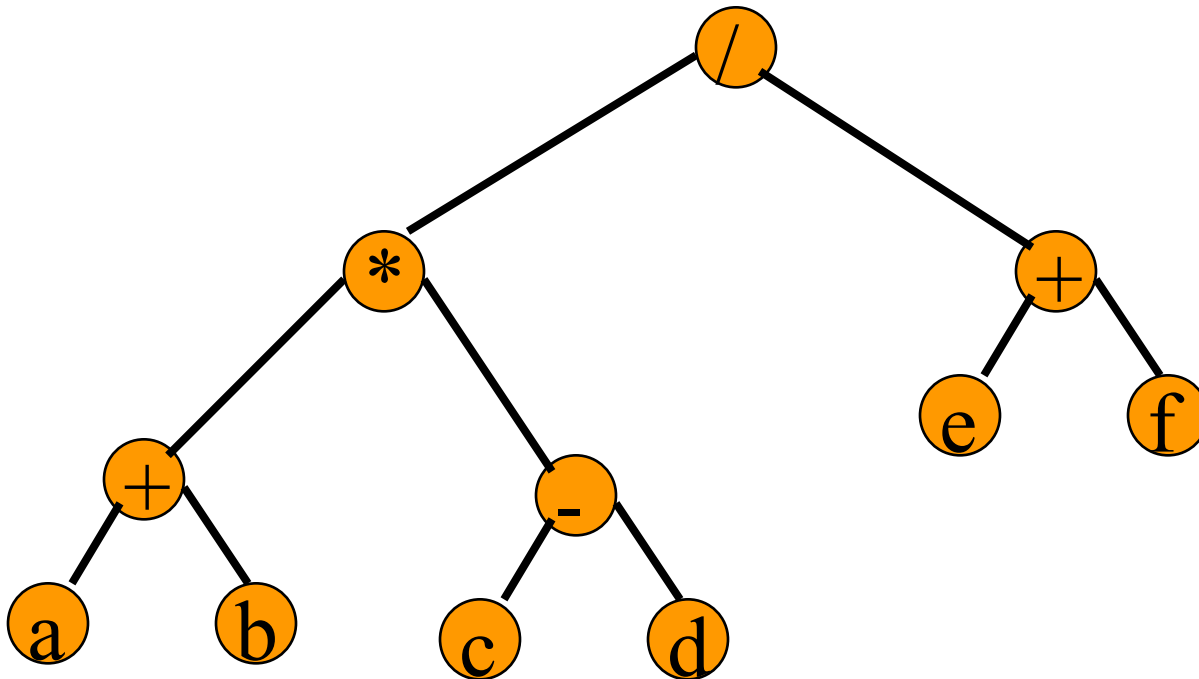
Binary Tree Form

- $(a + b) * (c - d) / (e + f)$

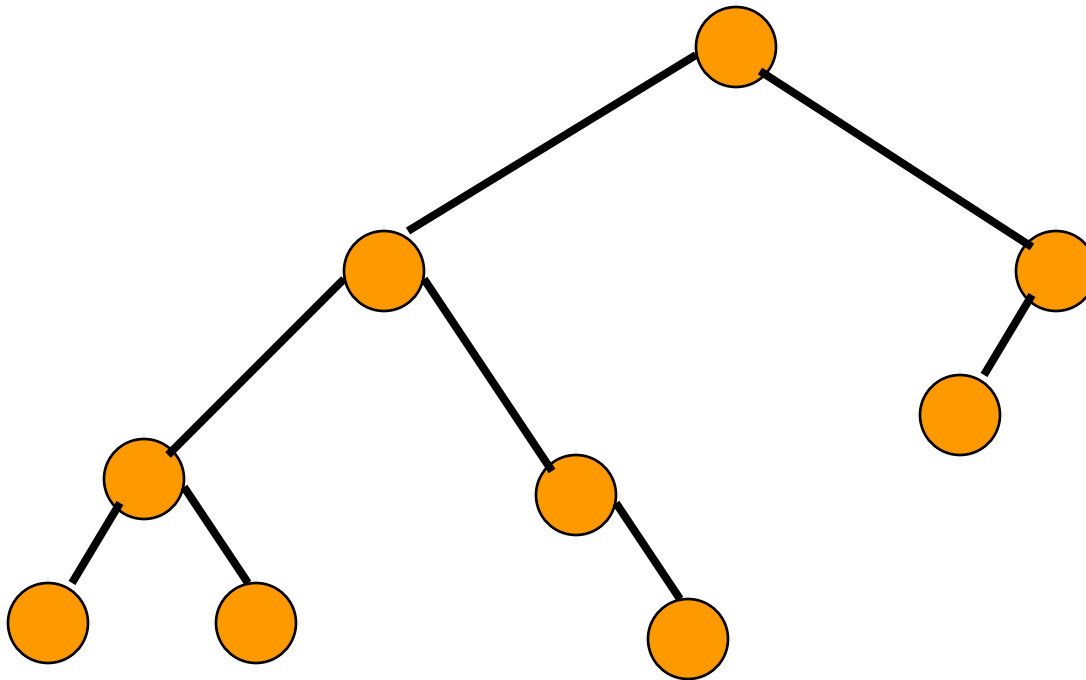
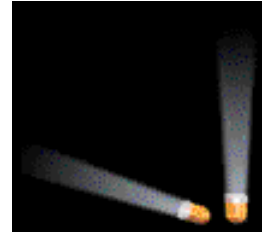
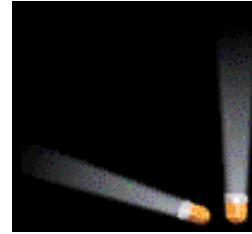
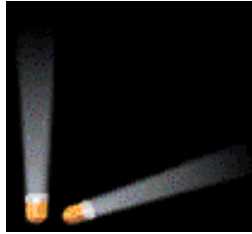
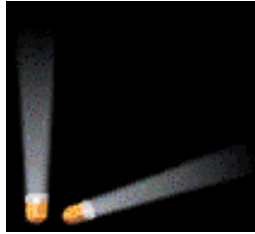


Evaluation of Binary Tree Form

- HOW?
 - Simple recursive evaluation
- Exercise: write an algorithm

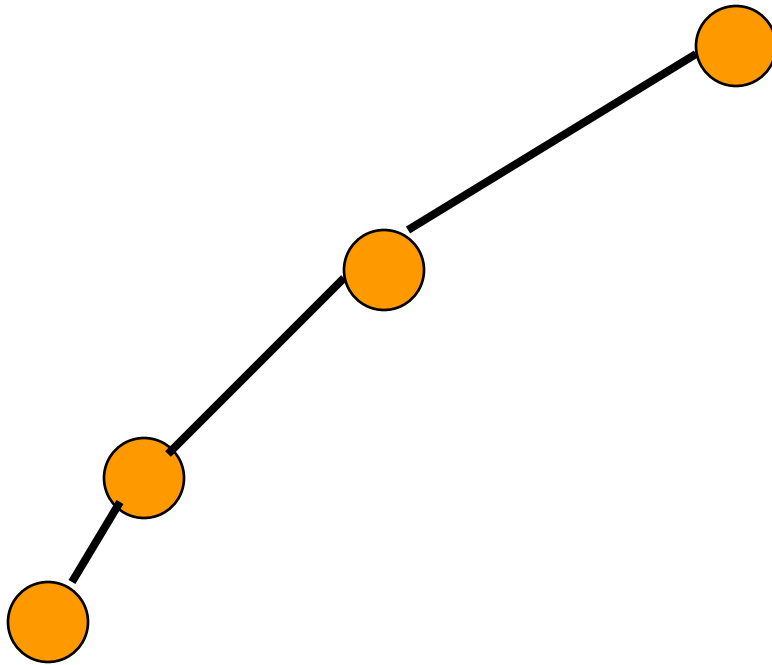


Binary Tree Properties & Representation



Minimum Number Of Nodes

- Minimum number of nodes in a binary tree whose height is h .
- At least one node at each of first h levels.

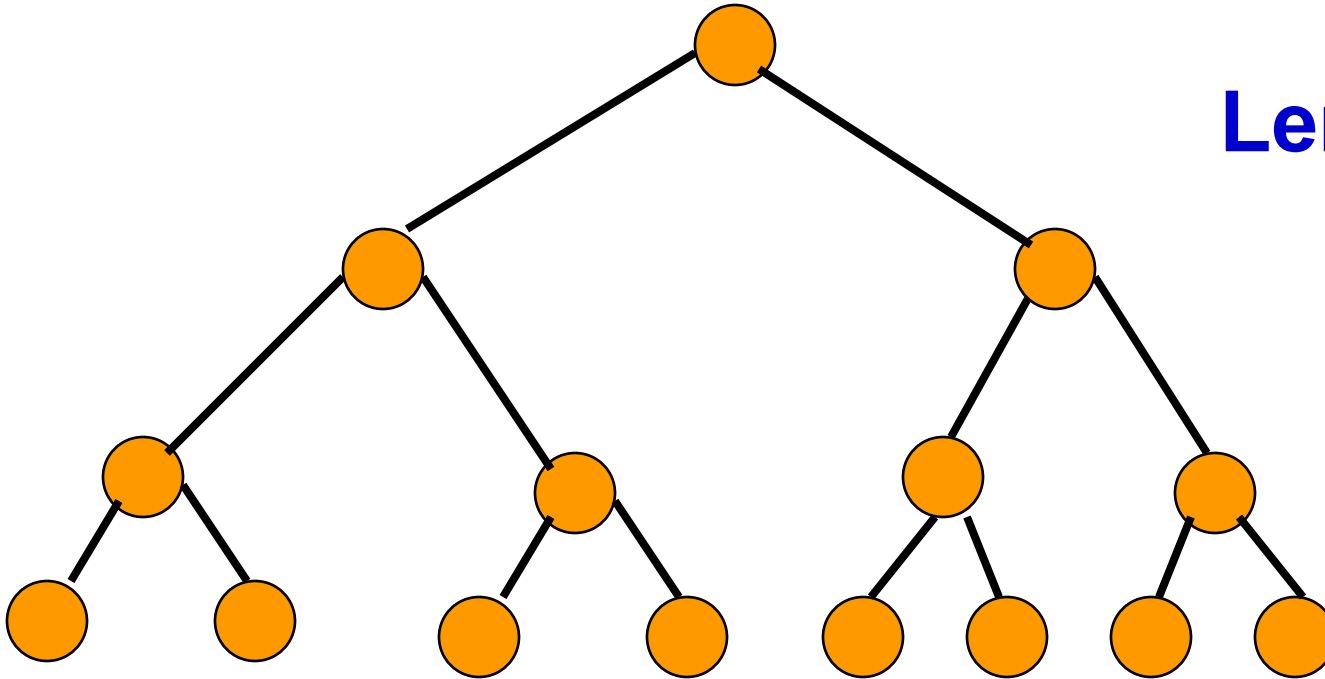


minimum number of
nodes is h

Maximum Number Of Nodes

- All possible nodes at first **h** levels are present.

Lemma 5.2



Maximum number of nodes

$$= 1 + 2 + 4 + 8 + \dots + 2^{h-1}$$

$$= 2^h - 1$$

Number Of Nodes & Height

- Let n be the number of nodes in a binary tree whose height is h .
- $h \leq n \leq 2^h - 1$
- $\log_2(n+1) \leq h \leq n$

Lemma 5.3 [Relation between number of leaf nodes and degree-2 nodes]:

For any nonempty binary tree T , if n_0 is the number of leaf nodes and n_2 is the number of nodes of degree 2, then $n_0 = n_2 + 1$.

Proof:

Let n_1 be the number of nodes of degree 1 and n the total number of nodes, we have

$$n = n_0 + n_1 + n_2 \quad (5.1)$$

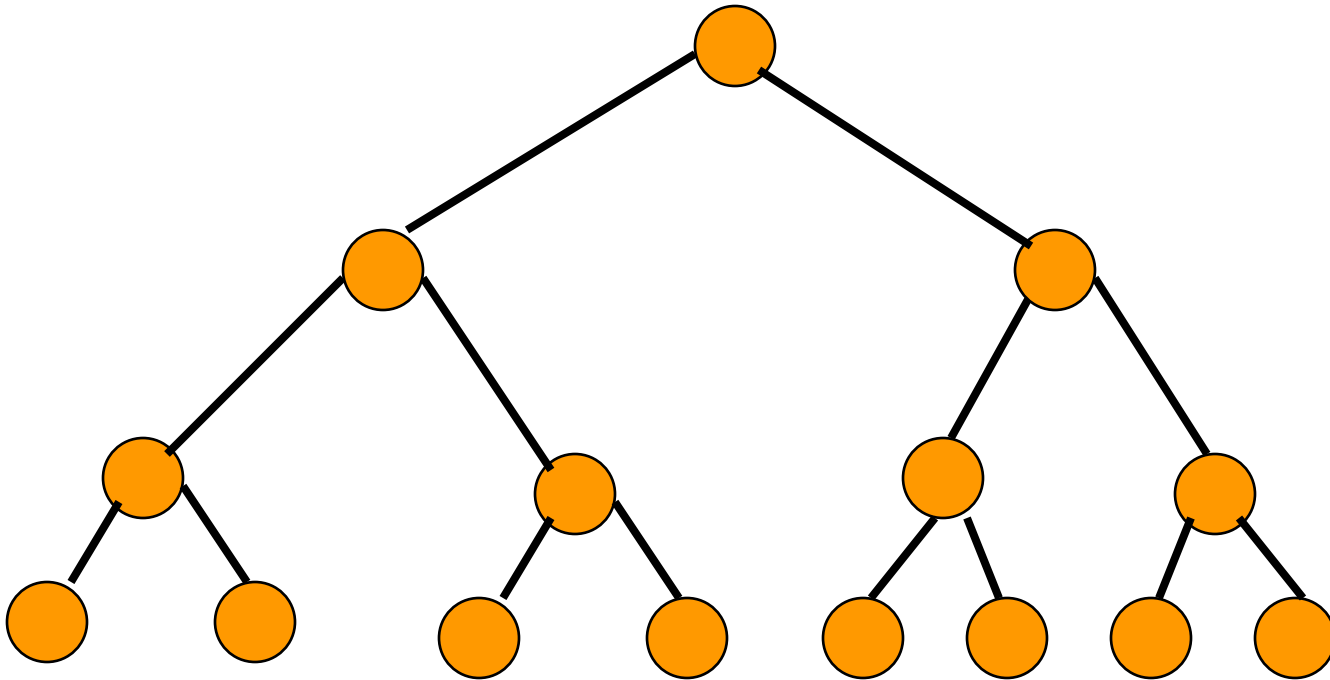
Each node except for the root has a branch leading into it. If B is the number of branches, then $n = B + 1$. And also $B = n_1 + 2n_2$, hence

$$n = n_1 + 2n_2 + 1 \quad (5.2)$$

(5.1) – (5.2): $0 = n_0 - n_2 - 1$, i.e., $n_0 = n_2 + 1$.

Full Binary Tree

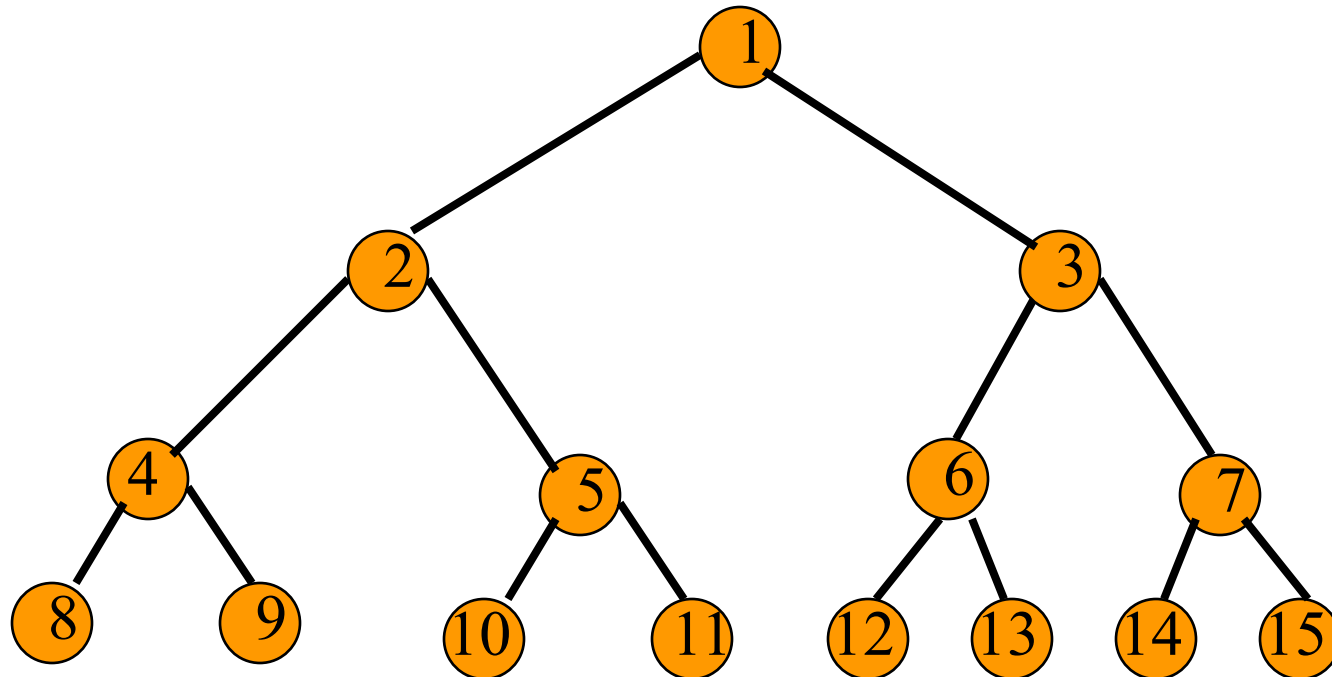
- A full binary tree of a given height h has $2^h - 1$ nodes.



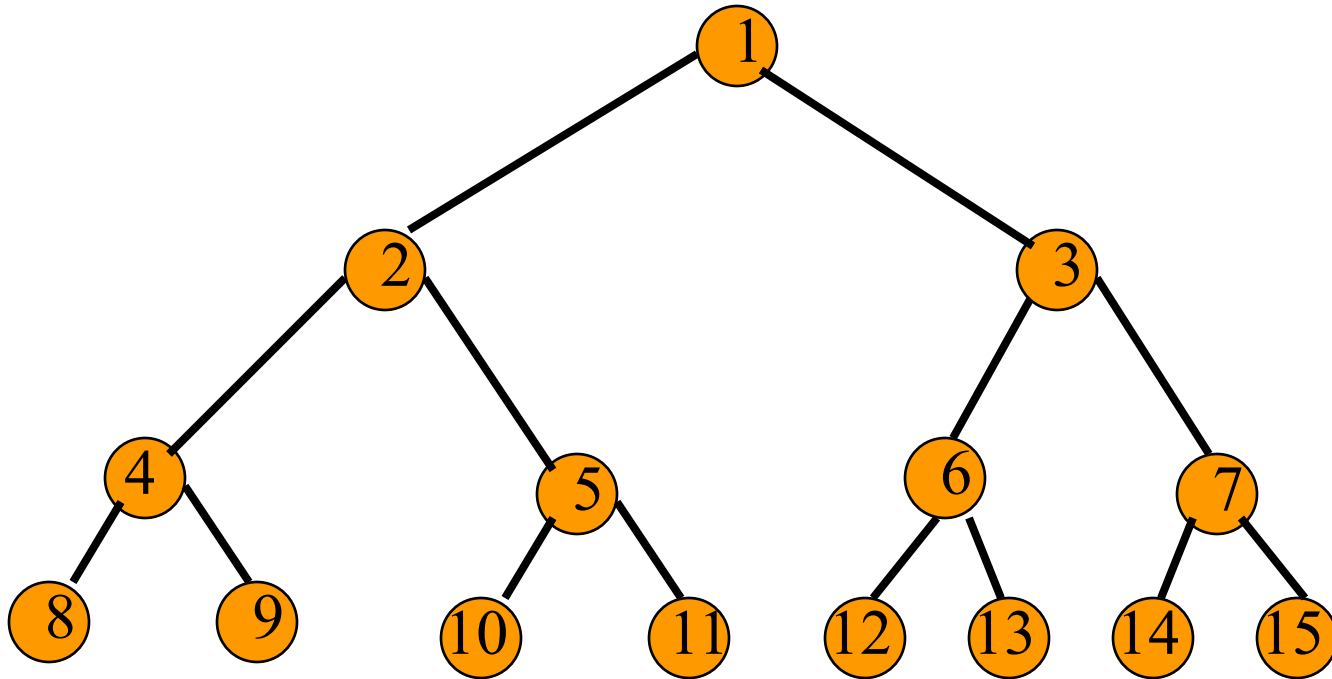
Height 4 full binary tree.

Numbering Nodes In A Full Binary Tree

- Number the nodes **1** through $2^h - 1$.
- Number by levels from top to bottom.
- Within a level number from left to right.

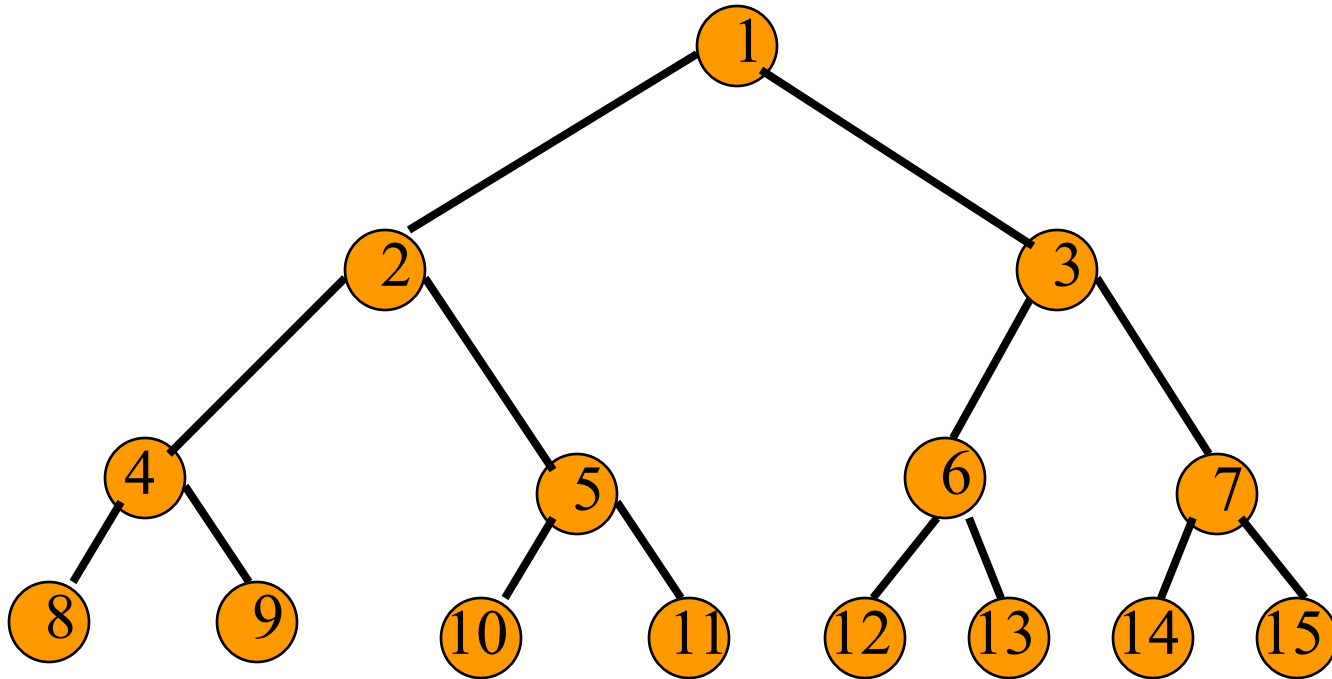


Node Number Properties



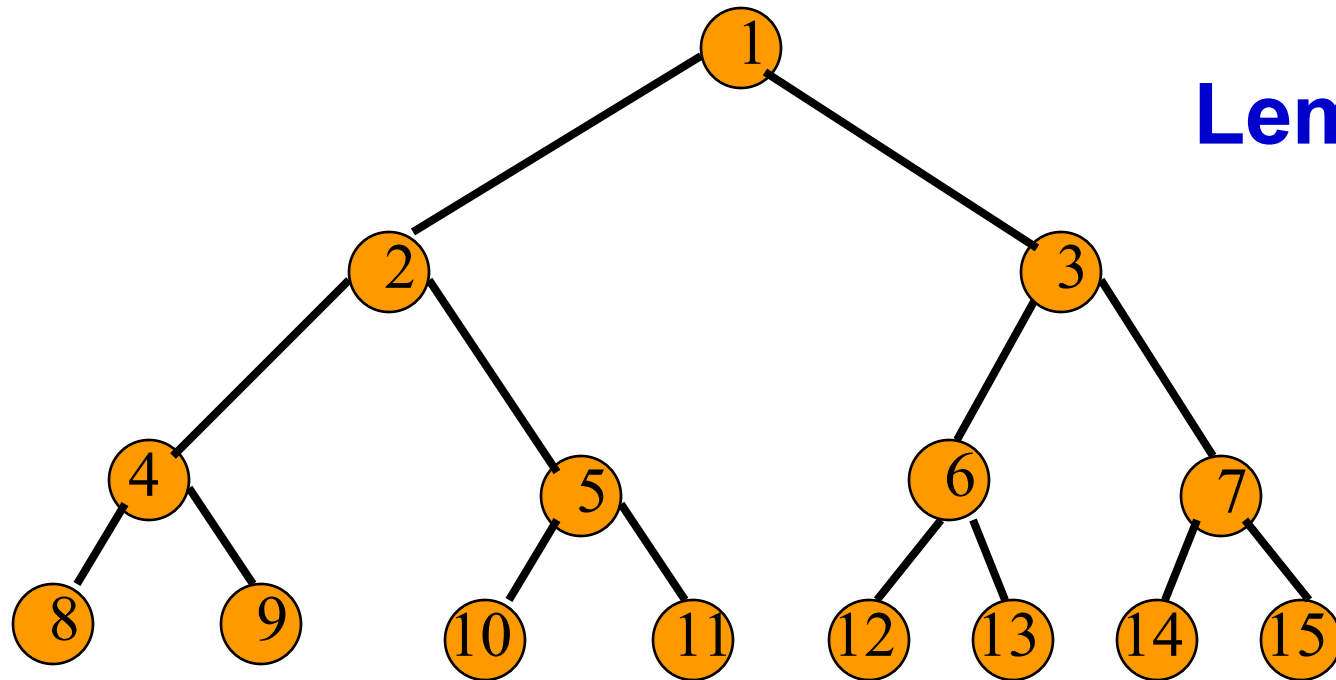
- Parent of node i is
 - node $i / 2$, unless $i = 1$.
- Node 1 is the root and has no parent.

Node Number Properties



- Left child of node i is
 - node $2i$, unless $2i > n$, where n is the number of nodes.
- If $2i > n$, node i has no left child.

Node Number Properties



Lemma 5.4

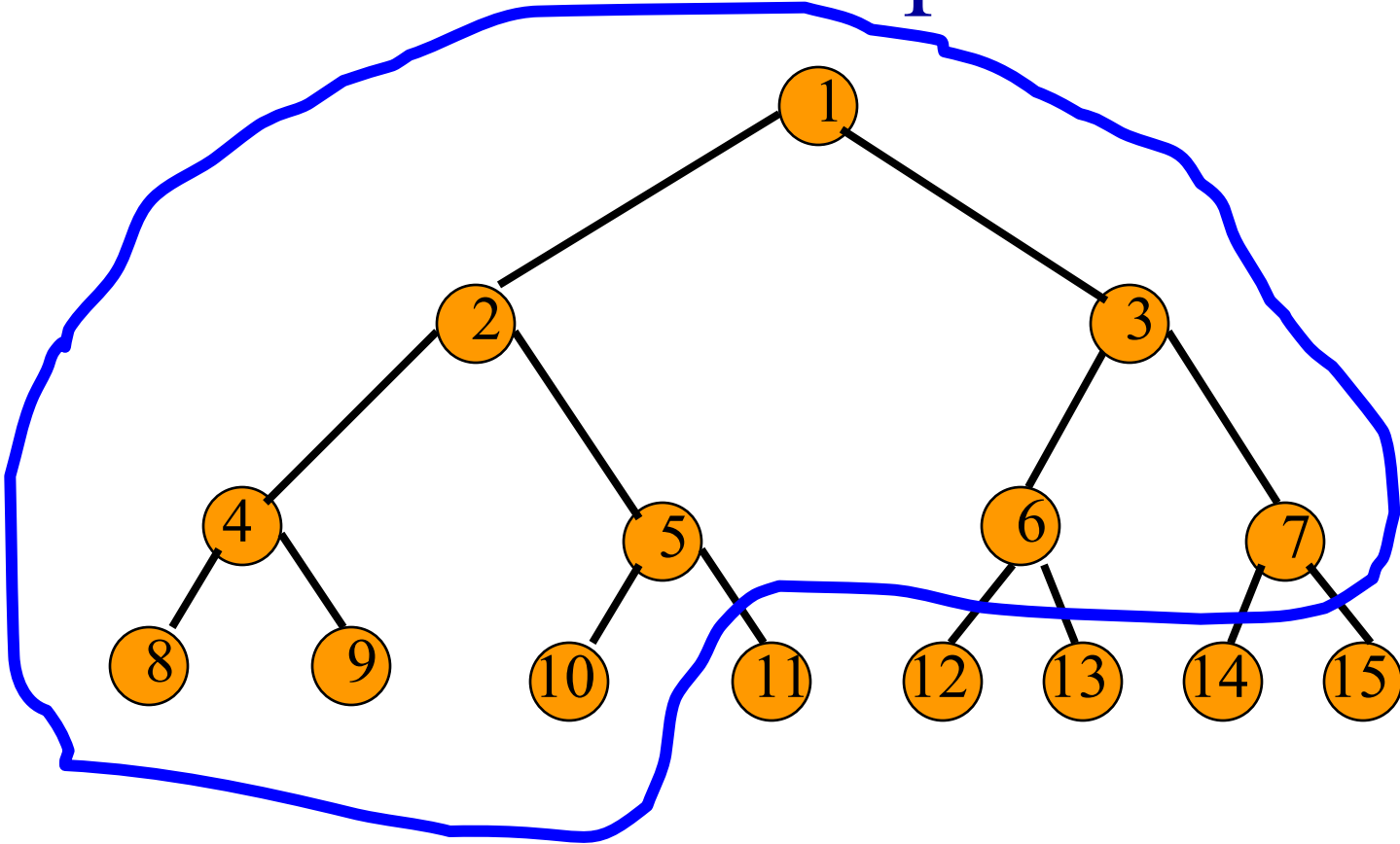
- Right child of node i is
 - node $2i+1$, unless $2i+1 > n$, where n is the number of nodes.
- If $2i+1 > n$, node i has no right child.

Complete Binary Tree With n Nodes

- Start with a full binary tree that has at least n nodes.
- Number the nodes as described earlier.
- The binary tree defined by the nodes numbered 1 through n is the unique n node complete binary tree.

Definition: a binary tree with n nodes and depth k is **complete** iff its nodes corresponding to the nodes numbered from 1 to n in the full binary tree of depth k .

Example



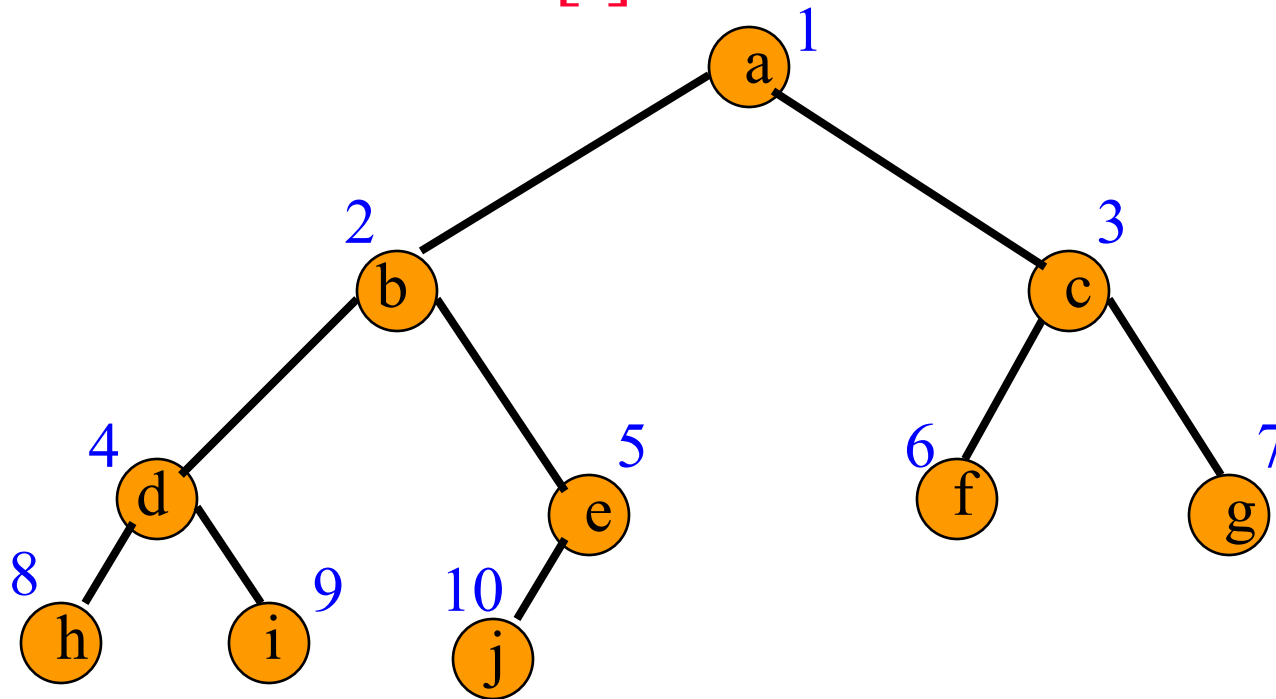
- Complete binary tree with 10 nodes.

Binary Tree Representation

- Array representation.
- Linked representation.

Array Representation

- Number the nodes using the numbering scheme for a full binary tree. The node that is numbered i is stored in $tree[i]$.

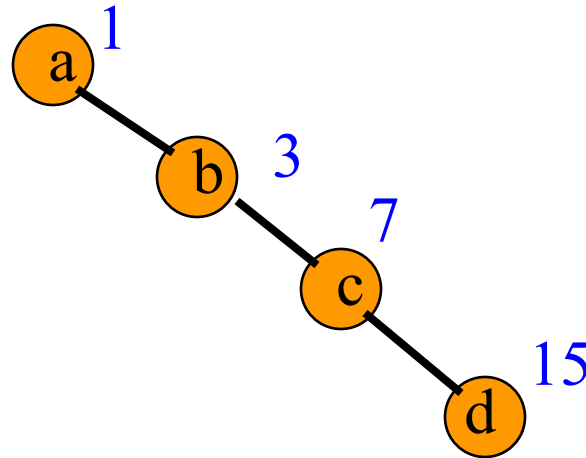


$tree[]$

	a	b	c	d	e	f	g	h	i	j
0					5					10

Drawback?

Right-Skewed Binary Tree



tree[]

	a	-	b	-	-	-	c	-	-	-	-	-	-	-	d
0				5				10							15

- An **n** node binary tree needs an array whose length is between **n+1** and **2ⁿ**.

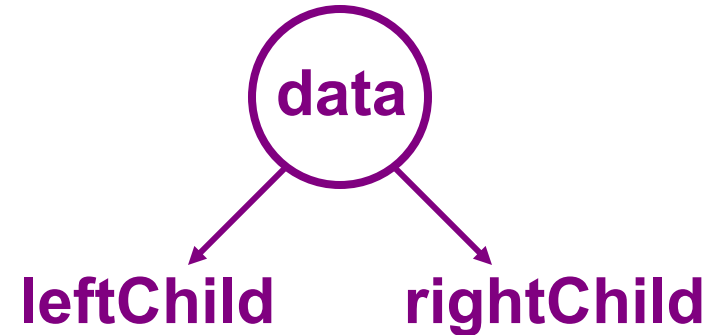
Linked Representation

- Each binary tree node is represented as an object whose data type is **TreeNode**.
- The space required by an **n** node binary tree is **$n * (\text{space required by one node})$** .

Exponential v.s. Linear

The Class BinaryTreeNode

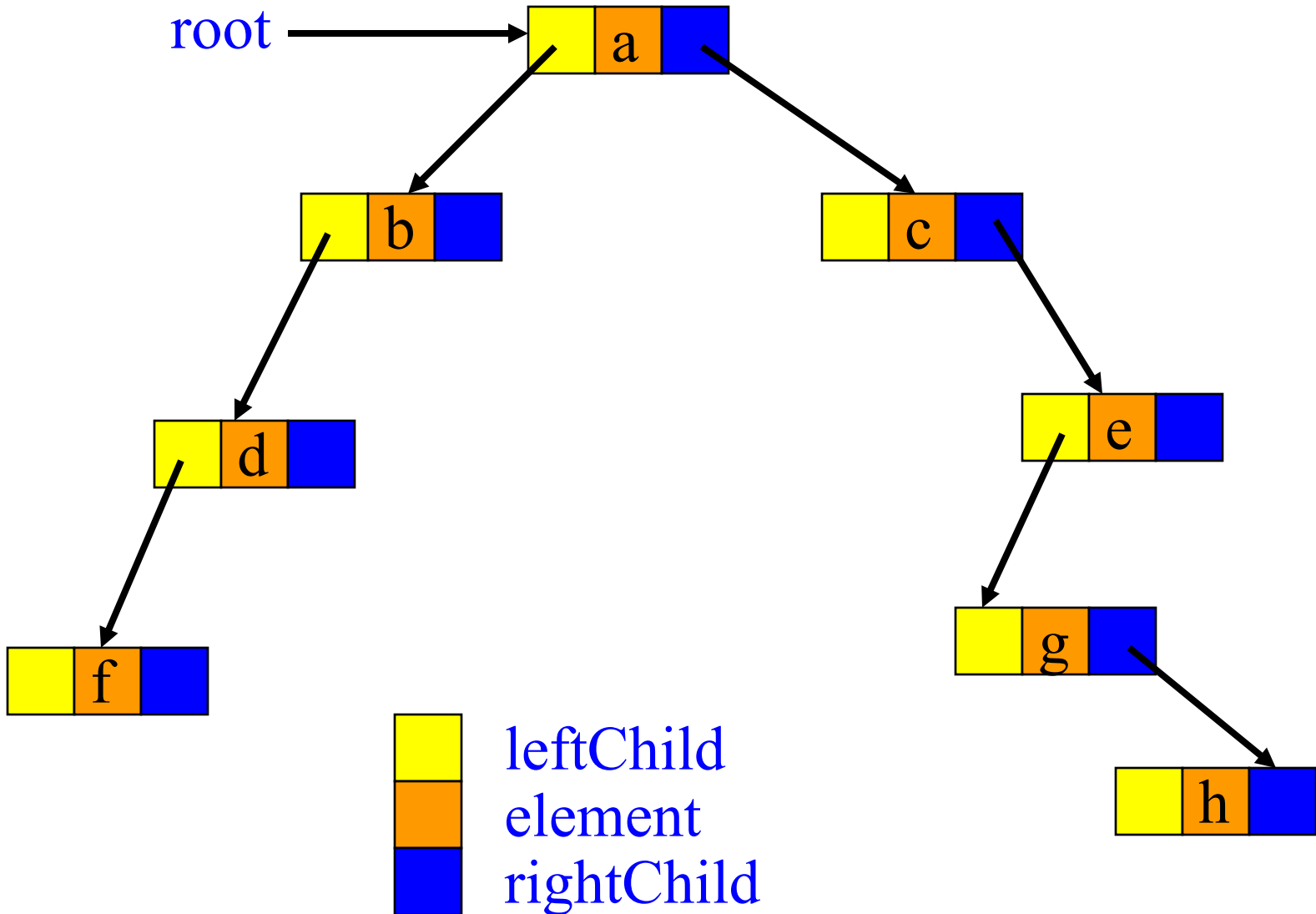
- **template <class T> class Tree;**
- **class TreeNode {**
- **friend class Tree<T>;**
- **public:**
- TreeNode (T& e, TreeNode<T>* left, TreeNode<T>* right)
- {data=e; leftChild=left; rightChild=right;}
- **private:**
- T data;
- TreeNode<Y>* leftChild;
- TreeNode<Y>* rightChild;
- **};**



```
template <class T>
class Tree {
public:
    // Tree operations
    ...
private:
    TreeNode<T>* root;
};
```

If necessary, a 4th field, **parent**, may be included in the node.

Linked Representation Example



Some Binary Tree Operations

- Determine the height.
- Determine the number of nodes.
- Make a clone.
- Determine if two binary trees are clones.
- Display the binary tree.
- Evaluate the arithmetic expression represented by a binary tree.
- Obtain the infix form of an expression.
- Obtain the prefix form of an expression.
- Obtain the postfix form of an expression.

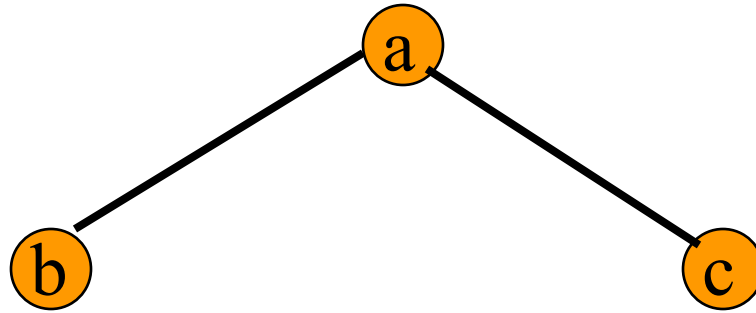
Binary Tree Traversal

- Many binary tree operations are done by performing a **traversal** of the binary tree.
- In a traversal, each element of the binary tree is **visited** exactly once.
- During the **visit** of an element, all action (make a clone, display, evaluate the operator, etc.) with respect to this element is taken.

Binary Tree Traversal Methods

- Preorder
- Inorder
- Postorder
- Level order

Preorder Example (visit = print)

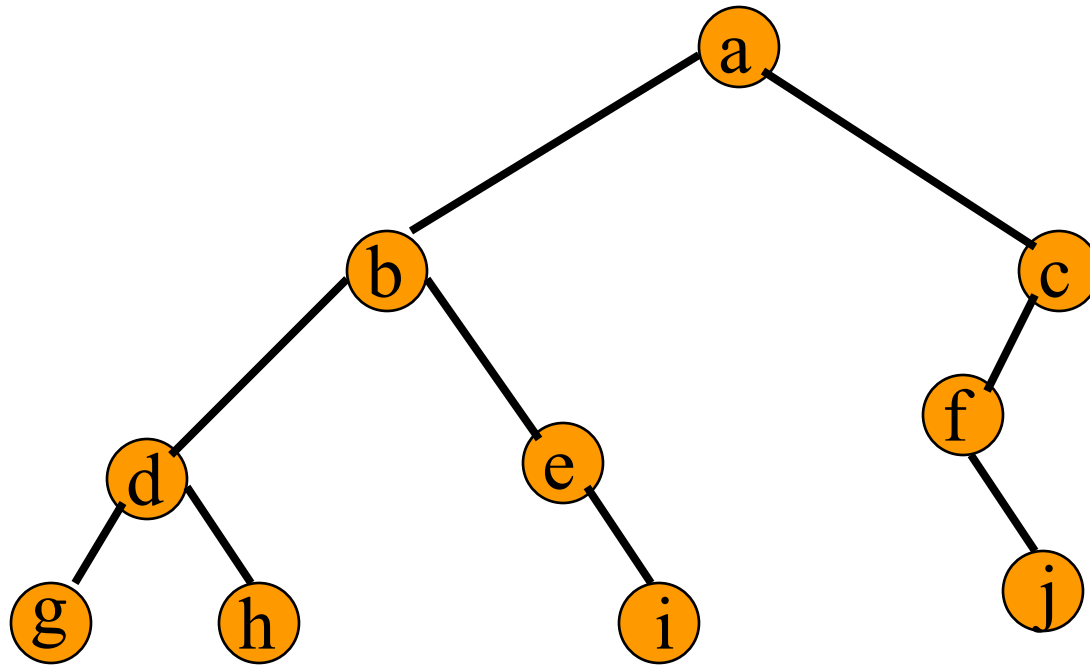


a b c

Preorder Traversal

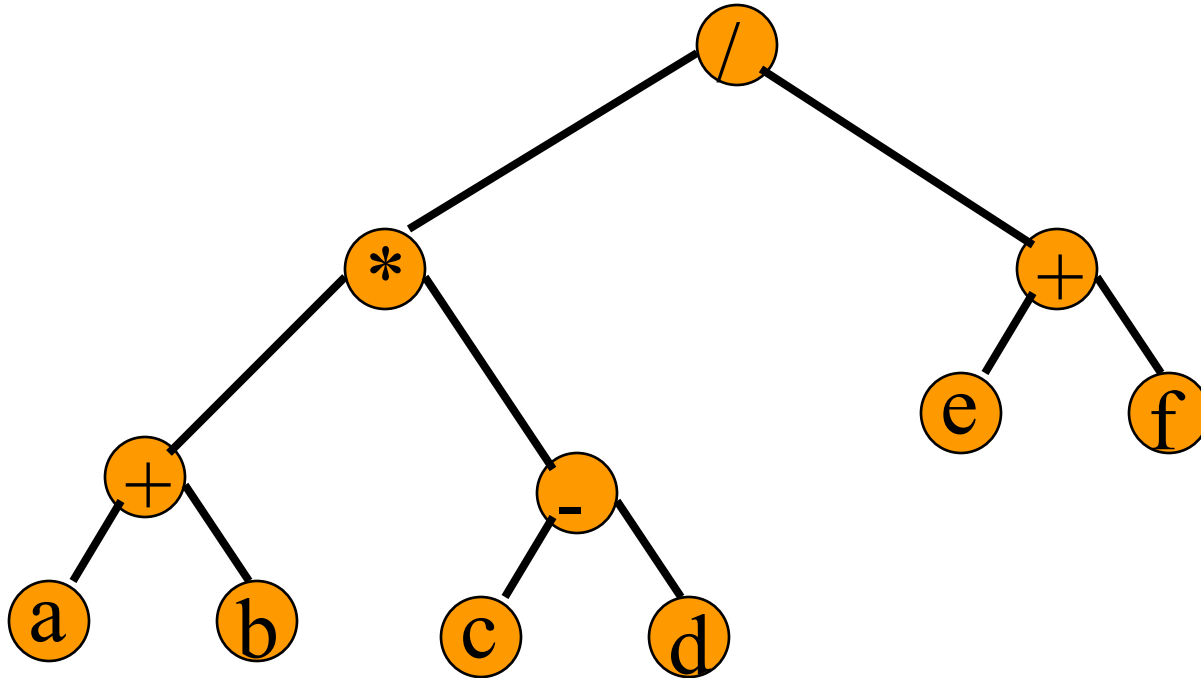
- **template** <class T>
- **void** Tree<T>::Preorder()
- { // Driver.
- Preorder(root);
- }
- **template** <class T>
- **void** Tree<T>::Preorder(TreeNode<T>* currentNode)
- { // workhorse.
- **if** (currentNode) {
- Visit(currentNode);
- Preorder(currentNode→leftChild);
- Preorder(currentNode→rightChild);
- }
- }

Preorder Example (visit = print)



a b d g h e i c f j

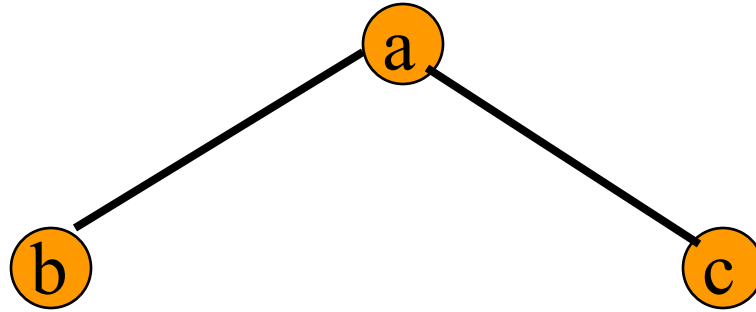
Preorder Of Expression Tree



$/ * + a b - c d + e f$

Gives prefix form of expression!

Inorder Example (visit = print)

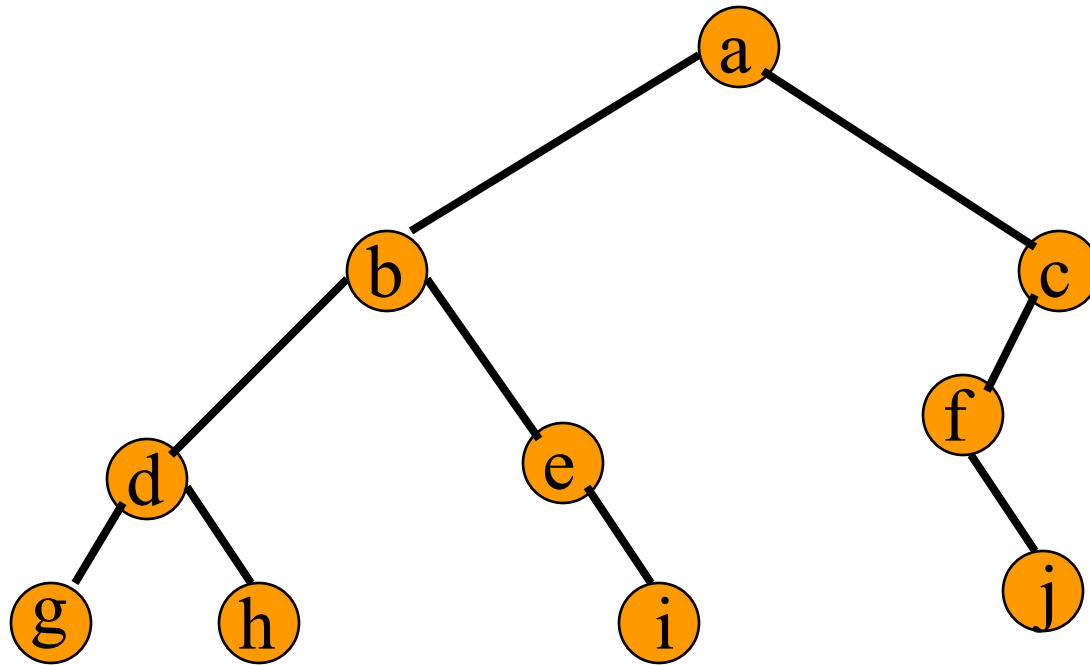


b a c

Inorder Traversal

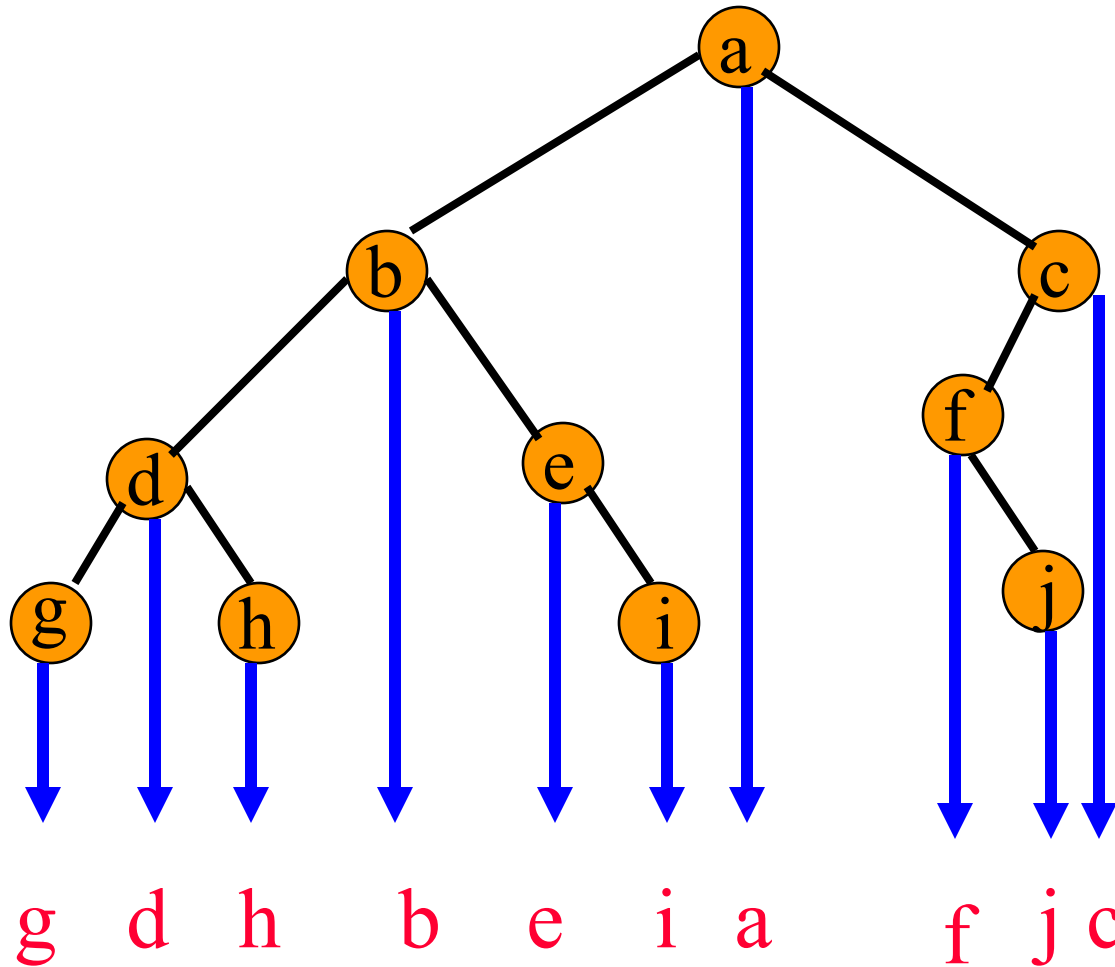
- **template** <class T>
- **void** Tree<T>::Inorder()
- { // driver as a public member
- Inorder(root);
- }
- **template** <class T>
- **void** Tree<T>::Inorder (TreeNode<T>* currentNode)
- { // workhorse as a private member of Tree
- **if** (CurrentNode) {
- Inorder(currentNode→leftChild);
- Visit(currentNode)
- Inorder(currentNode→rightChild);
- }
- }

Inorder Example (visit = print)

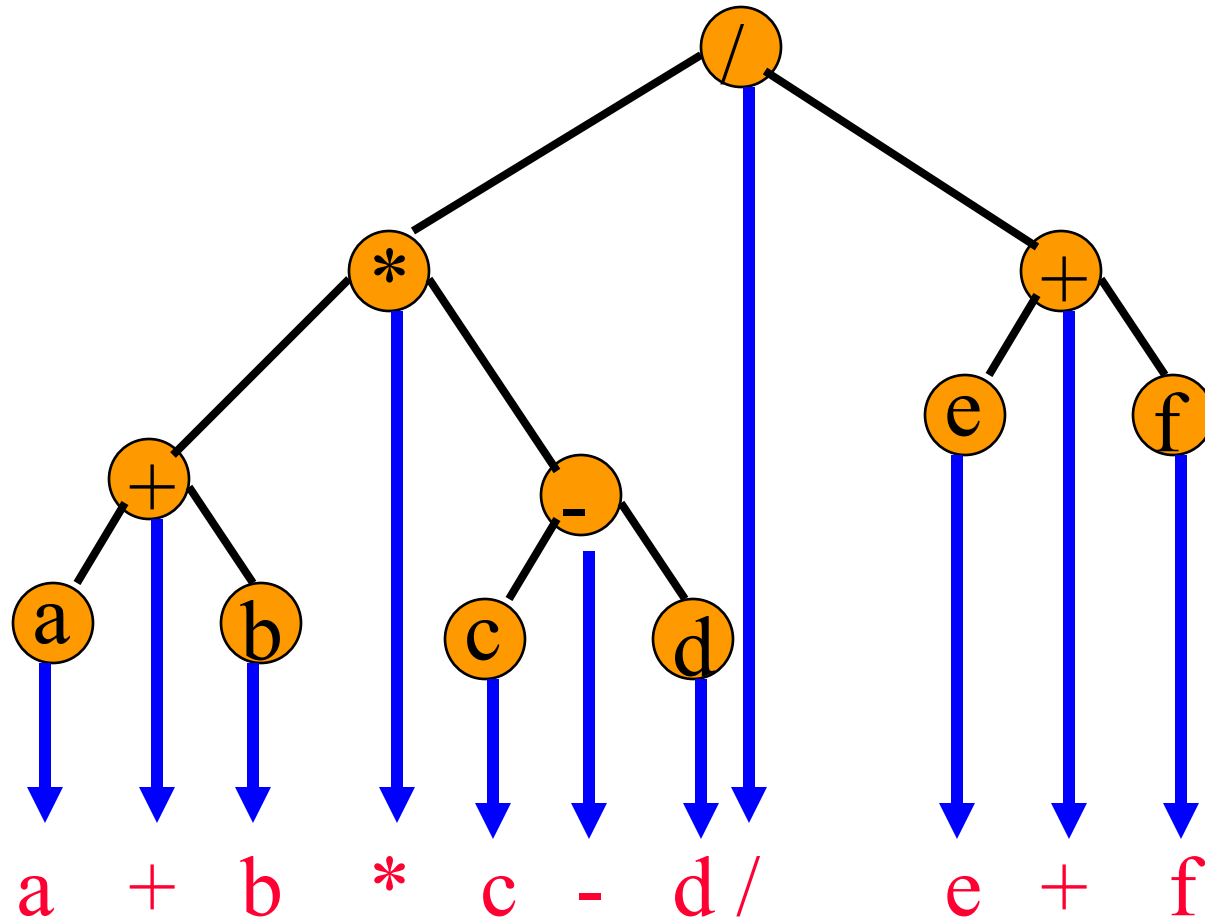


g d h b e i a f j c

Inorder By Projection

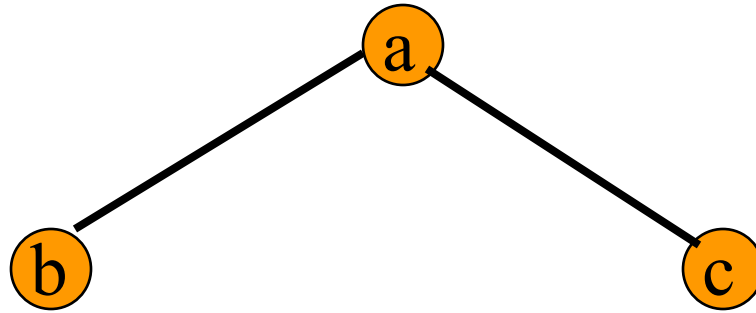


Inorder Of Expression Tree



Gives infix form of expression (sans parentheses)!

Postorder Example (visit = print)

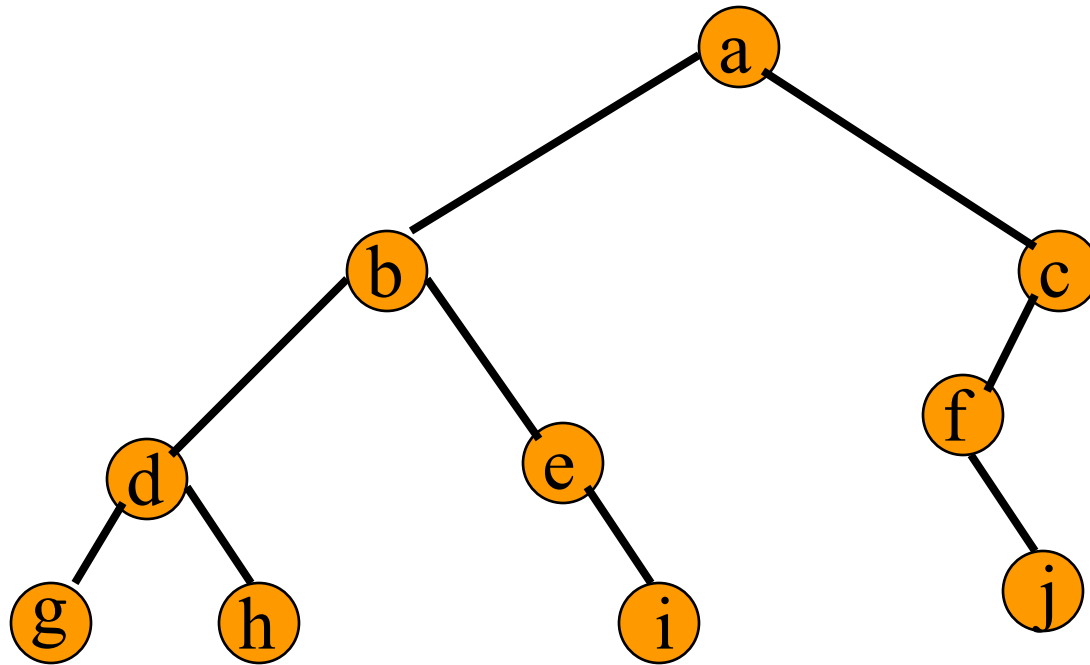


b c a

Postorder Traversal

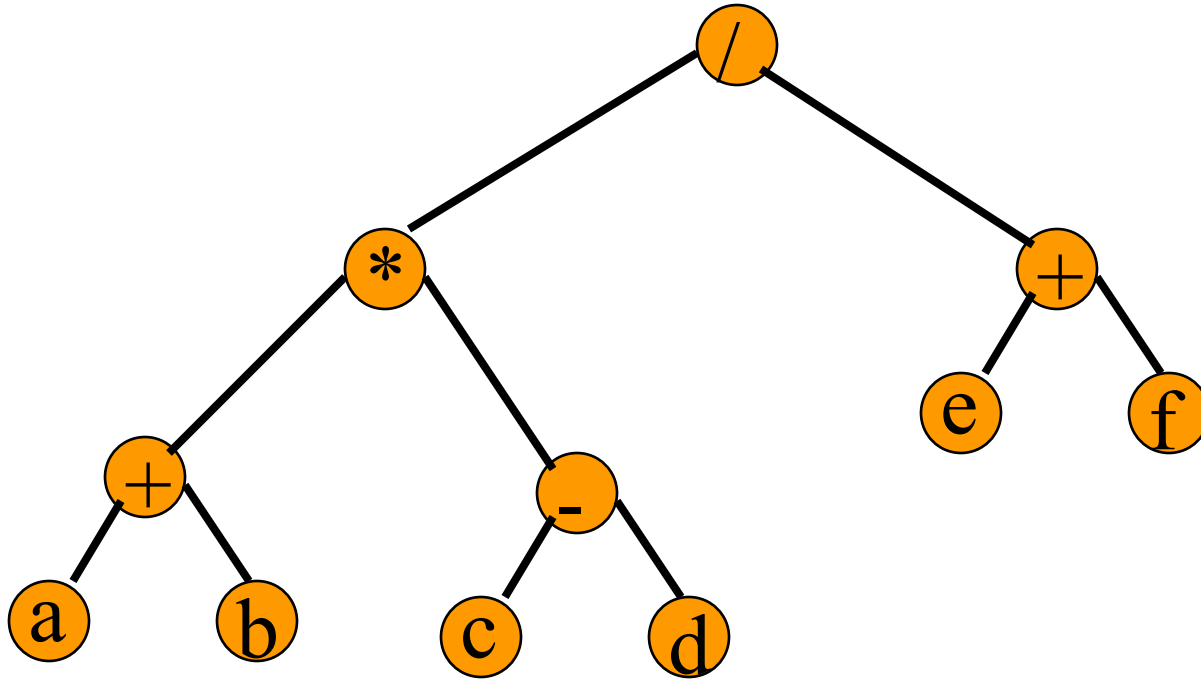
- **template** <class T>
- **void** Tree<T>::Postorder()
- { // Driver.
- Postorder(root);
- }
- **template** <class T>
- **void** Tree<T>::Postorder (TreeNode<T>* currentNode)
- { // Workhorse.
- **if** (currentNode) {
- Postorder(currentNode→leftChild);
- Postorder(currentNode→rightChild);
- Visit(currentNode);
- }
- }

Postorder Example (visit = print)



g h d i e b j f c a

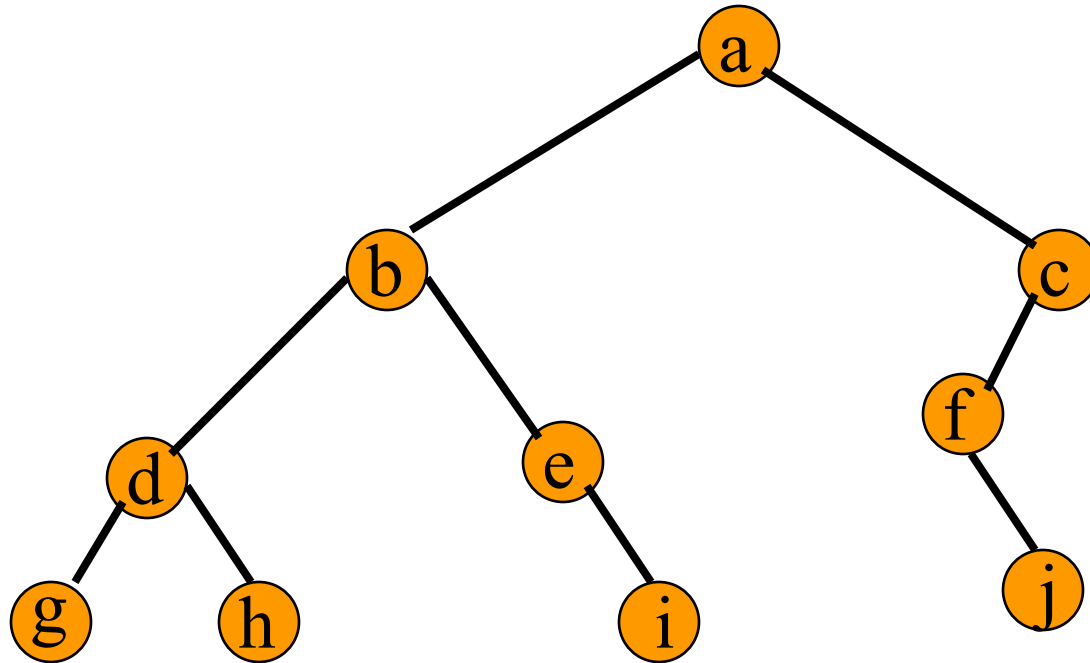
Postorder Of Expression Tree



a b + c d - * e f + /

Gives postfix form of expression!

Traversal Applications

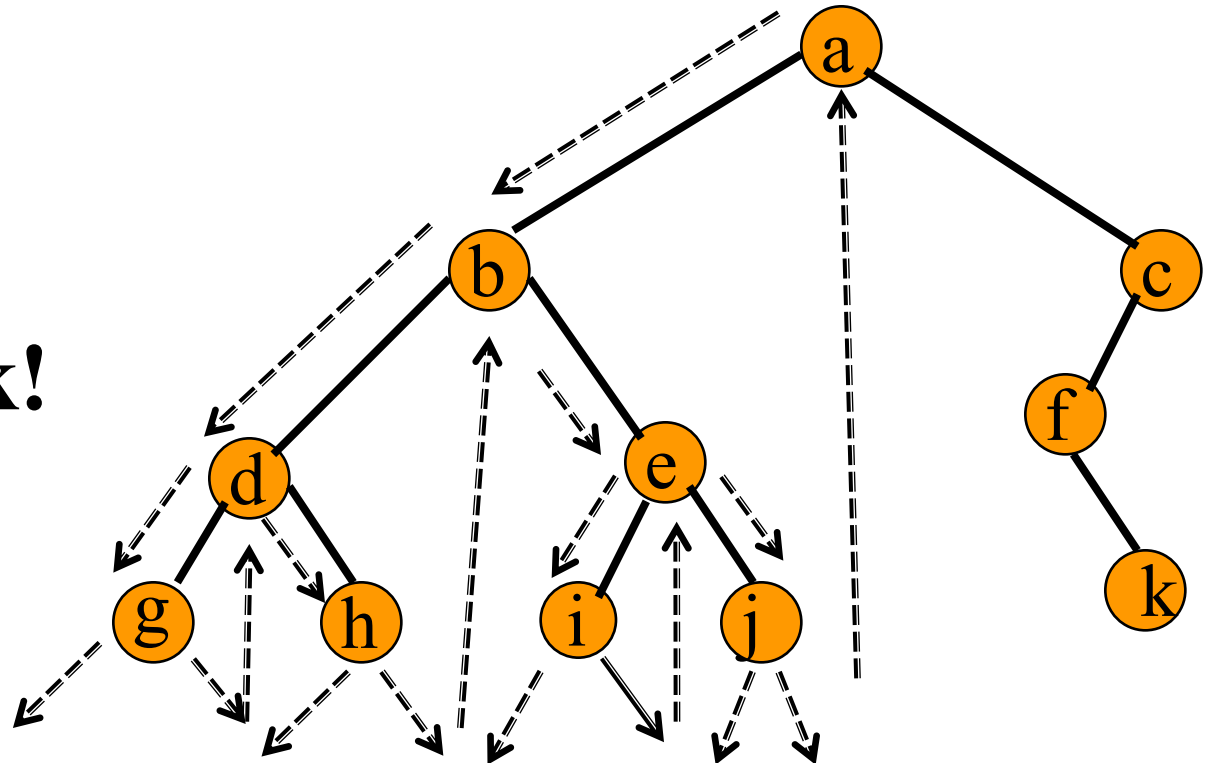


- Make a clone.
- Determine height.
- Determine number of nodes.

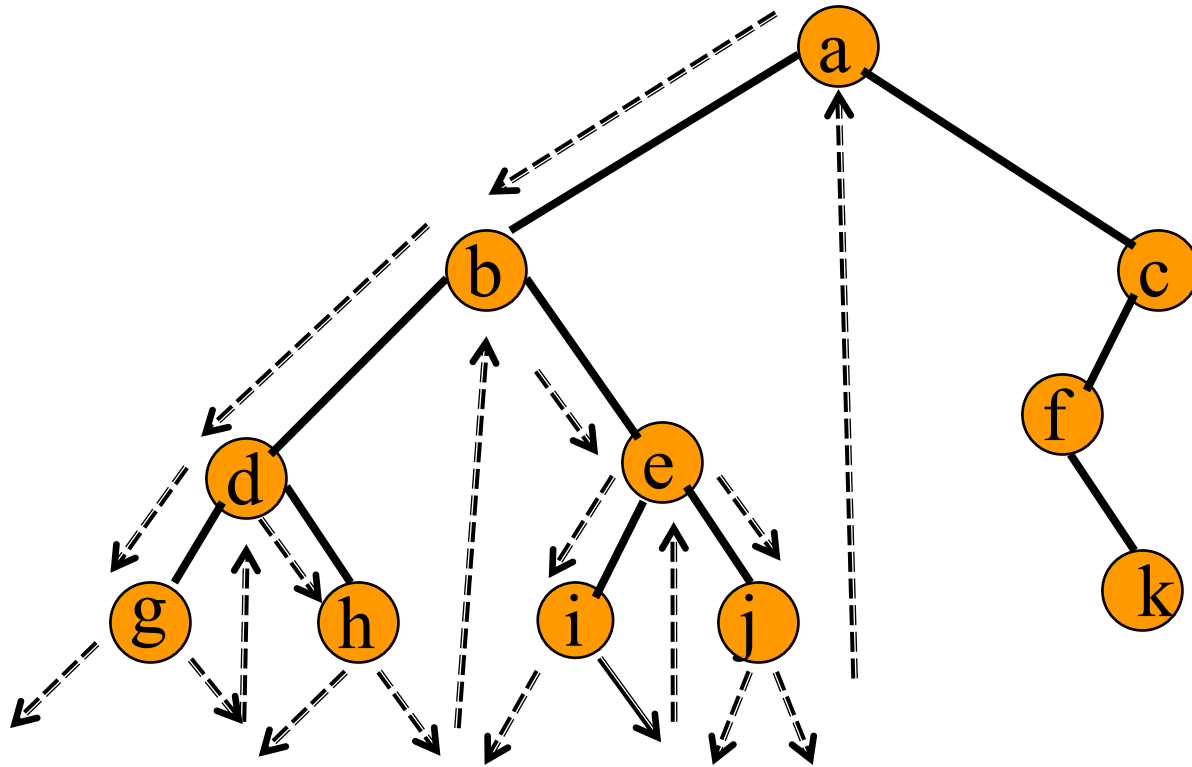
- Code here

Iterative Inorder Traversal

- **Tree iterator**
 - **Access nodes one by one**
- **Non-recursive tree traversal algorithm**
 - **Inorder**
- **Data structure:**
 - **Stack!**



Iterative Inorder Traversal




```
1 template <class T>
2 void Tree<T>::NonrecInorder()
3 { // Nonrecursive inorder traversal using a stack
4   Stack<TreeNode<T>*> s; // declare and initialize a stack
5   TreeNode<T>* currentNode=root;
6   while (1) {
7     while (currentNode) { // move down leftChild
8       s.Push(currentNode); // add to stack
9       currentNode=currentNode→leftChild;
10    }
11    If (s.IsEmpty()) return;
12    currentNode=s.Top();
13    s.Pop(); // delete from stack
14    Visit(currentNode);
15    currentNode=currentNode→rightChild;
16  }
17}
```

The NonrecInorder USES-A template stack.

Definition: A data object of Type A USES-A data object of Type B if a Type A object uses a Type B object to perform a task. Typically, a Type B object is employed in a member function of Type A.

USES-A is similar to IS-IMPLEMENTED-IN-TERMS-OF, but the degree of using the Type B object is less.

Analysis of NonrecInorder:

- n ---the number of nodes in the tree.
- every node is placed on the stack once, line 8, 9 and 11 to 15 are executed n times.
- `currentNode` will equal 0 once for every 0 link, which is $2n_0 + n_1 = n_0 + n_1 + n_2 + 1 = n + 1$.

The computing time: $O(n)$.

The space required for the stack is equal to the depth of the tree.

Now we use the function `NonrecInorder` to obtain an inorder iterator for a tree.

The key observation is that each iteration of the while loop of line 6-16 yields the next element in the inorder traversal of the tree.

```
class InorderIterator { // a public nested member class of  
Tree
```

```
public:
```

```
    InorderIterator() {currentNode=root;};
```

```
    T* Next( );
```

```
private:
```

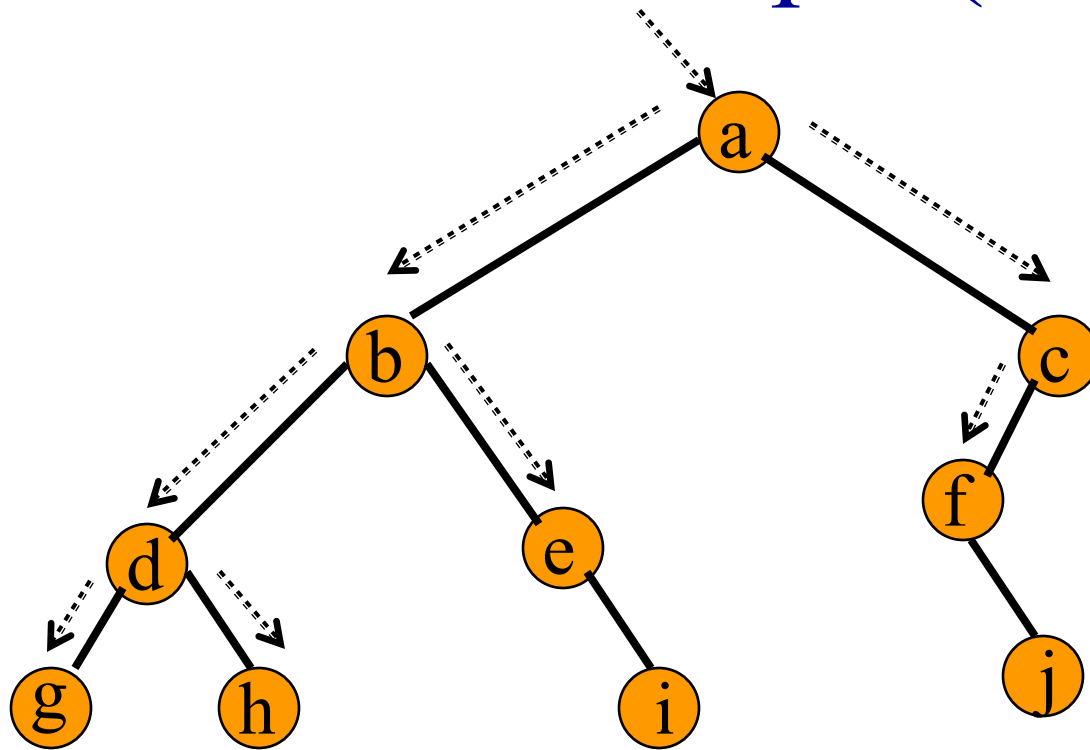
```
    Stack<TreeNode<T>*> s;
```

```
    TreeNode<T>* currentNode;
```

```
};
```

```
T* InorderIterator::Next()
{
    while (currentNode) {
        s.Push(currentNode);
        currentNode=currentNode→LeftChild;
    }
    if (s.IsEmpty()) return 0;
    currentNode=s.Top();
    T& temp=currentNode→data;
    currentNode=currentNode→rightChild;
    return &temp;
}
```

Level-Order Example (visit = print)



Storage?

a b c d e f g h i j

Level Order

Let **t** be the tree root.

```
while (t != null)
```

```
{
```

```
    visit t and put its children on a FIFO queue;
```

```
    remove a node from the FIFO queue and  
    call it t;
```

```
    // remove returns null when queue is empty
```

```
}
```

Additional Binary Tree Operations

Copying Binary Trees

```
template <class T>
Tree<T>::Tree(const Tree<T>& s) // driver
{ // Copy constructor
    root = Copy( s.root );
}
```


Testing Equality

```
template <class T>
bool Tree<T>::operator==(const Tree& t) const
{
    return Equal(root, t.root);
}
```

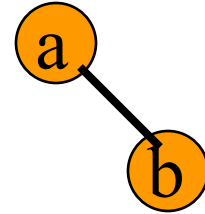
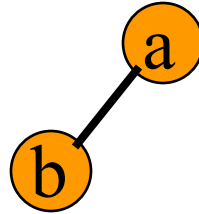
```
template <class T>
bool Tree<T>::Equal(TreeNode<T>* a, TreeNode<T>* b)
{ // Workhorse-
    if ((!a) && (!b)) return true; // both a and b are 0
    return (a && b                // both a and b are non-0
            && (a->data == b->data) //data is the same
            && Equal(a->leftChild, b->leftChild) //left equal
            && Equal(a->rightChild, b->rightChild)); //right equal
}
```

Binary Tree Construction

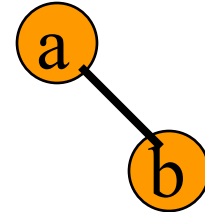
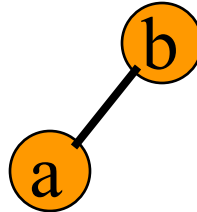
- Suppose that the elements in a binary tree are distinct.
- Can you construct the binary tree from which a given traversal sequence came?
- When a traversal sequence has more than one element, the binary tree is not uniquely defined.
- Therefore, the tree from which the sequence was obtained cannot be reconstructed uniquely.

Some Examples

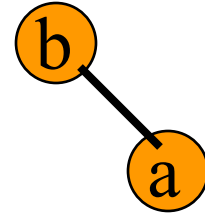
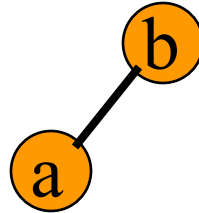
preorder
= ab



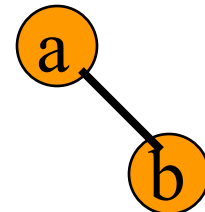
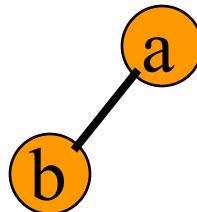
inorder
= ab



postorder
= ab



level order
= ab



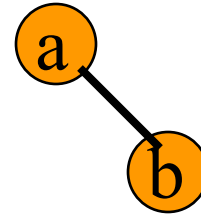
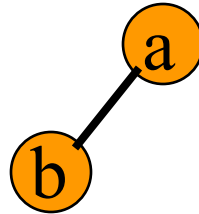
Binary Tree Construction

- Can you construct the binary tree, given two traversal sequences?
- Depends on which two sequences are given.

Preorder And Postorder

preorder = **ab**

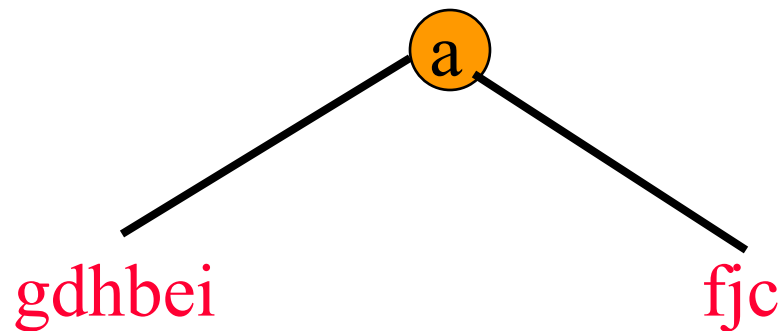
postorder = **ba**



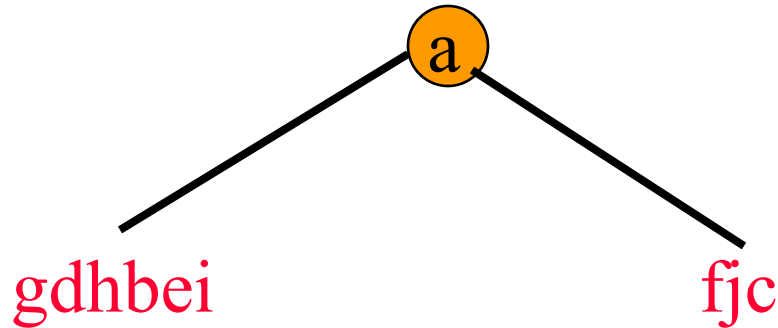
- Preorder and postorder do not uniquely define a binary tree.
- Nor do preorder and level order (same example).
- Nor do postorder and level order (same example).

Inorder And Preorder

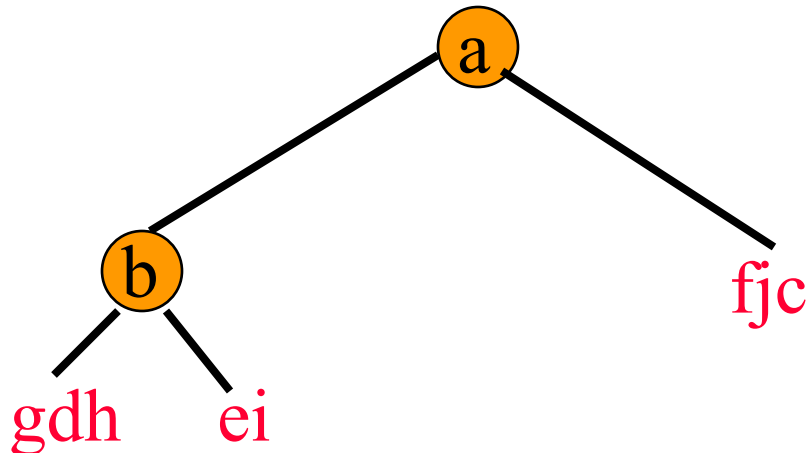
- inorder = g d h b e i a f j c
- preorder = a b d g h e i c f j
- Scan the preorder left to right using the inorder to separate left and right subtrees.
- a is the root of the tree; gdhbei are in the left subtree; fjc are in the right subtree.



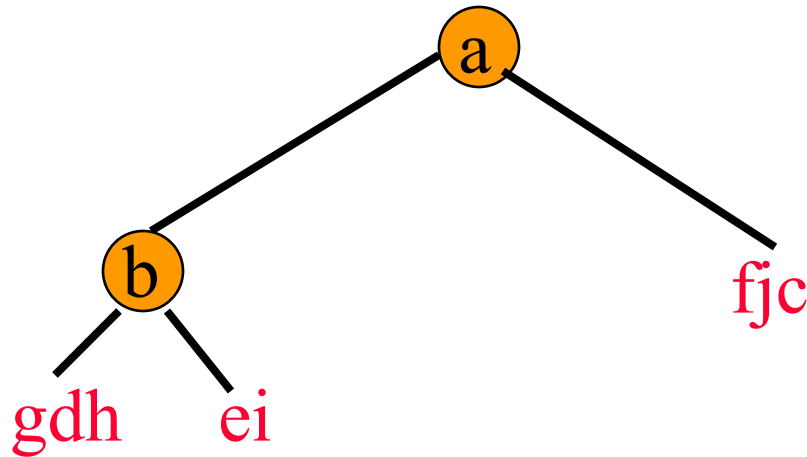
Inorder And Preorder



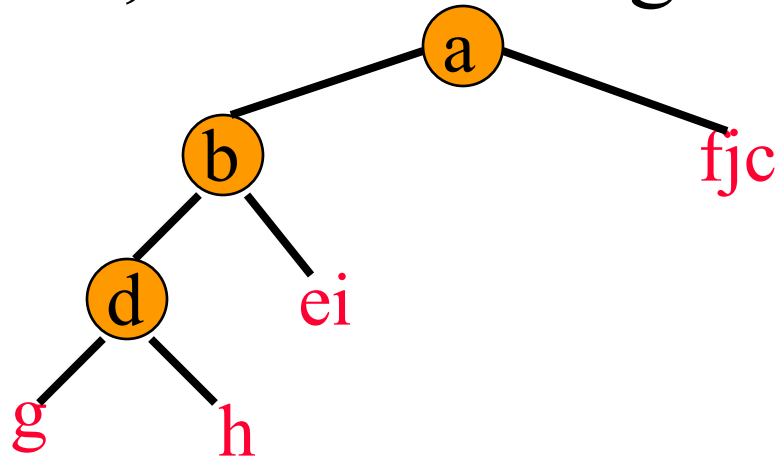
- preorder = a b d g h e i c f j
- b is the next root; gdh are in the left subtree; ei are in the right subtree.



Inorder And Preorder



- preorder = a b d g h e i c f j
- d is the next root; g is in the left subtree; h is in the right subtree.



Inorder And Postorder

- Scan postorder from right to left using inorder to separate left and right subtrees.
- inorder = g d h b e i a f j c
- postorder = g h d i e b j f c a
- Tree root is a; gdhbei are in left subtree; fjc are in right subtree.

Inorder And Level Order

- Scan level order from left to right using inorder to separate left and right subtrees.
- inorder = g d h b e i a f j c
- level order = a b c d e f g h i j
- Tree root is a; gdhbei are in left subtree; fjc are in right subtree.

Exercises: P267-4, P267-6, ~~P272-1~~, P273-4

Experiment: P267-10

Exercises:

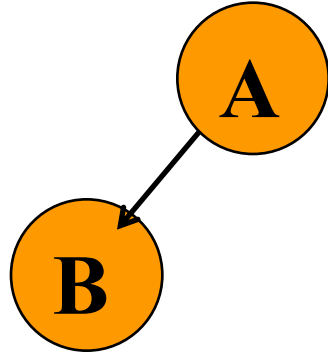
1. A binary tree, inorder = g d h b e i a f j c,
postorder = g h d i e b j f c a, construct the tree.
2. A Tree with degree 3, Preorder = ABCDEFGHI,
Postorder=BCGHFEIDA, construct the tree.

Preorder = ABCDEFGHI,

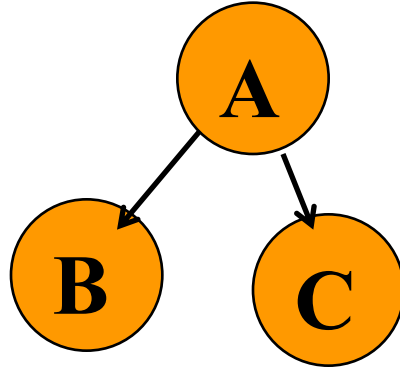
Postorder=BCGFHFEIDA



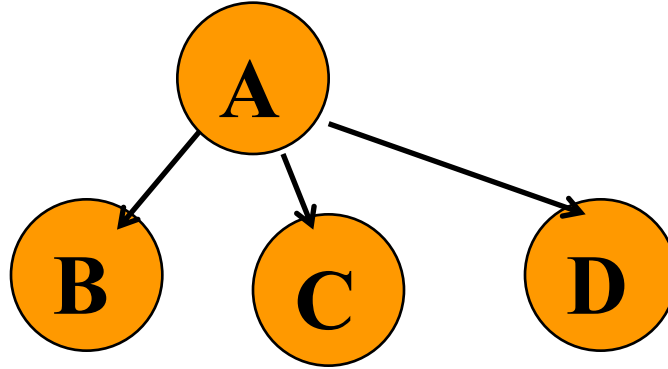
Preorder = ABCDEFGHI,
Postorder=BCGFHFEIDA



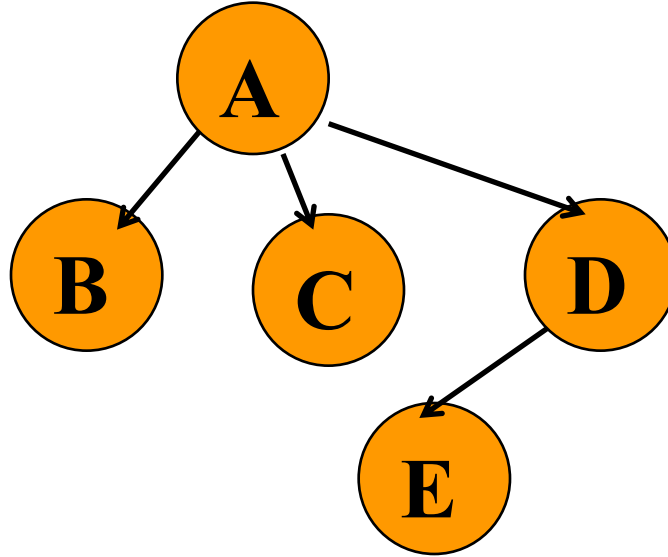
Preorder = ABCDEFGHI,
Postorder=BCGFHEIDA



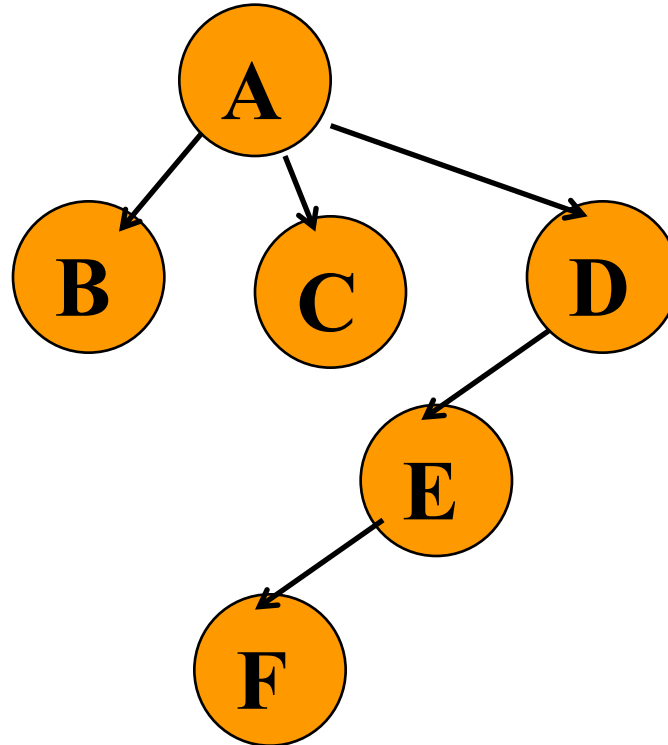
Preorder = ABCDEFGHI,
Postorder=BCGFHEIDA



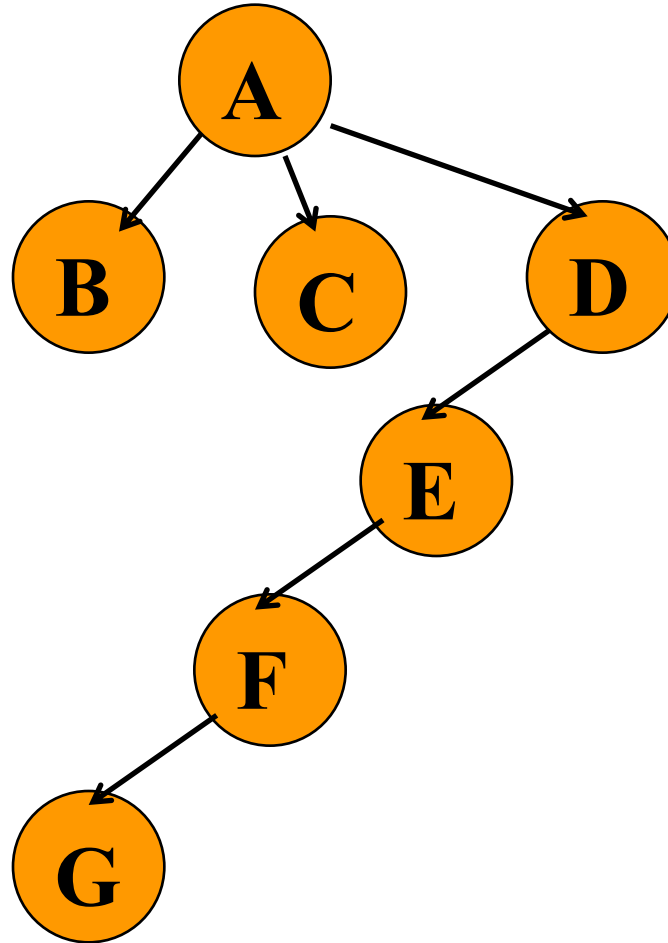
Preorder = ABCDEFGHI,
Postorder=BCGFHEIDA



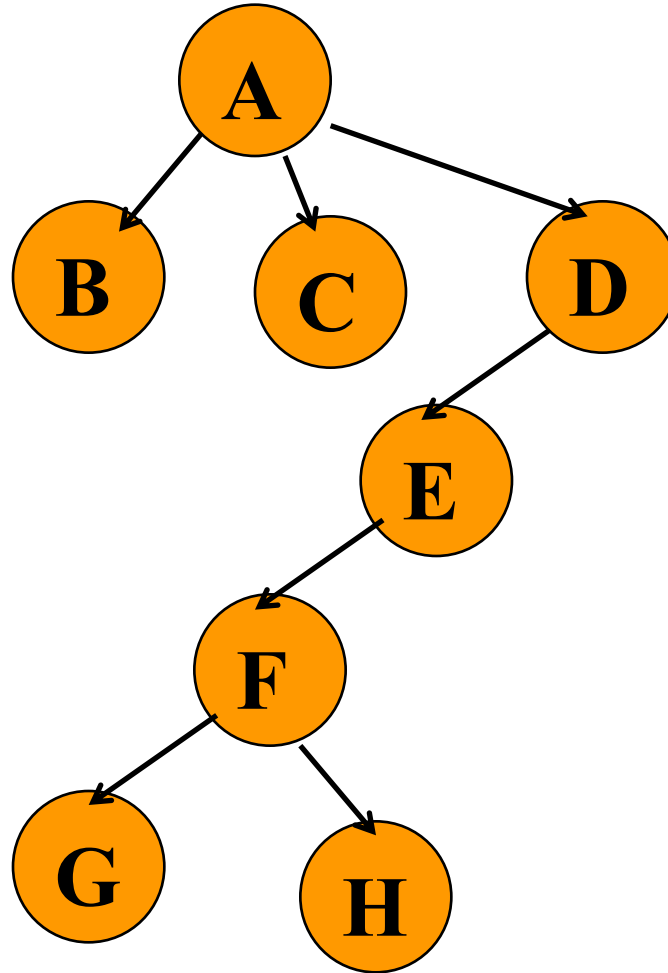
Preorder = ABCDEFGHI,
Postorder=BCGFHEIDA



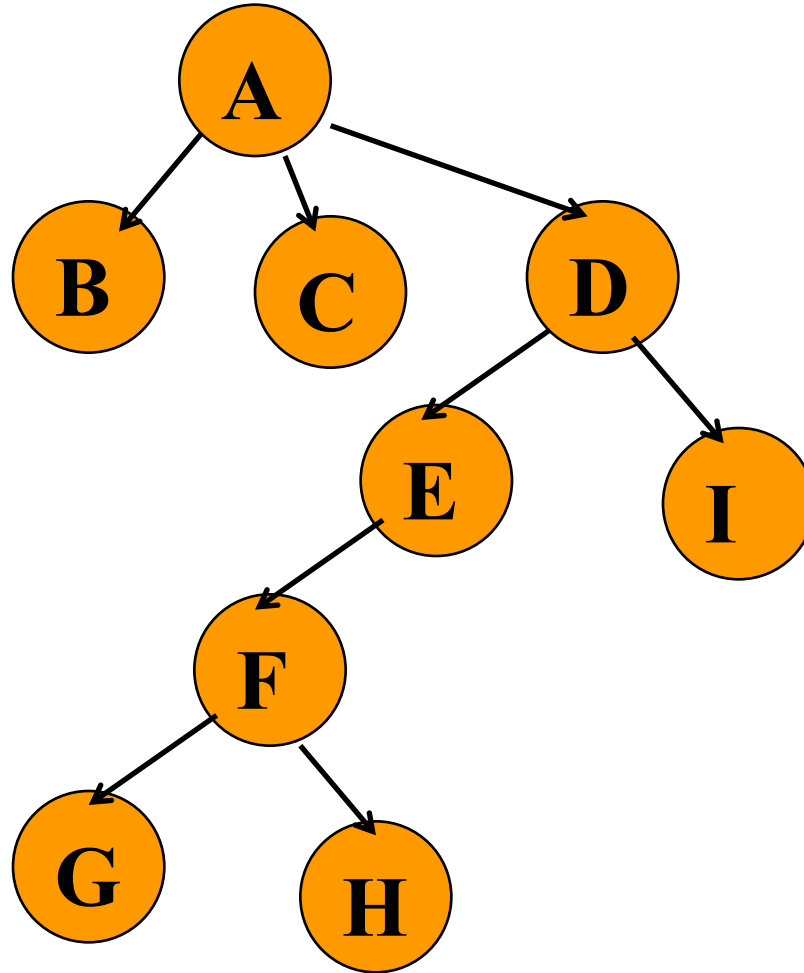
Preorder = ABCDEFGHI,
Postorder=BCGFHFEIDA



Preorder = ABCDEFGHI,
Postorder=BCGFHFEIDA



Preorder = ABCDEFGHI,
Postorder=BCGFHFEIDA



Binary Search Trees



- Dictionary Operations:
 - `get(key)`
 - `put(key, value)`
 - `remove(key)`
- Additional operations:
 - `ascend()`
 - `get(index)` (indexed binary search tree)
 - `remove(index)` (indexed binary search tree)
- **no two pairs have the same key**

ADT 5.3

```
template <class K, class E>
```

```
class Dictionary {
```

```
public:
```

```
    virtual bool IsEmpty () const = 0;
```

```
    // return true iff the dictionary is empty
```

```
    virtual pair<K,E>* Get(const K&) const = 0;
```

```
    // return pointer to the pair with specified key;
```

```
    // return 0 if no such pair
```

```
    virtual void Insert(const pair<K,E>&) = 0;
```

```
    // insert the given pair; if key is a duplicate
```

```
    // update associated element
```

```
    virtual void Delete(const K&) = 0;
```

```
    // delete pair with specified key
```

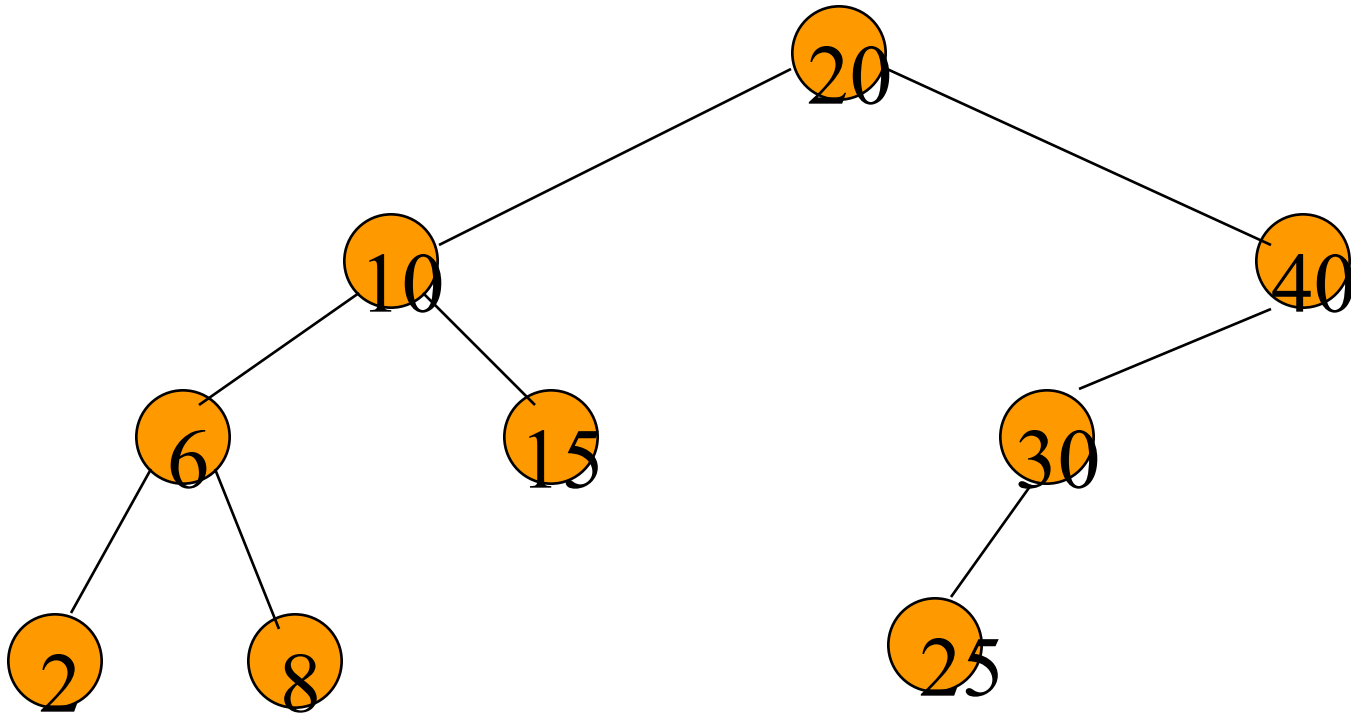
```
};
```


- **template** <**class** K, **class** E>
- **struct** pair
- {
- K first; // **Key**
- E second; // **Value**
- };

Definition Of Binary Search Tree

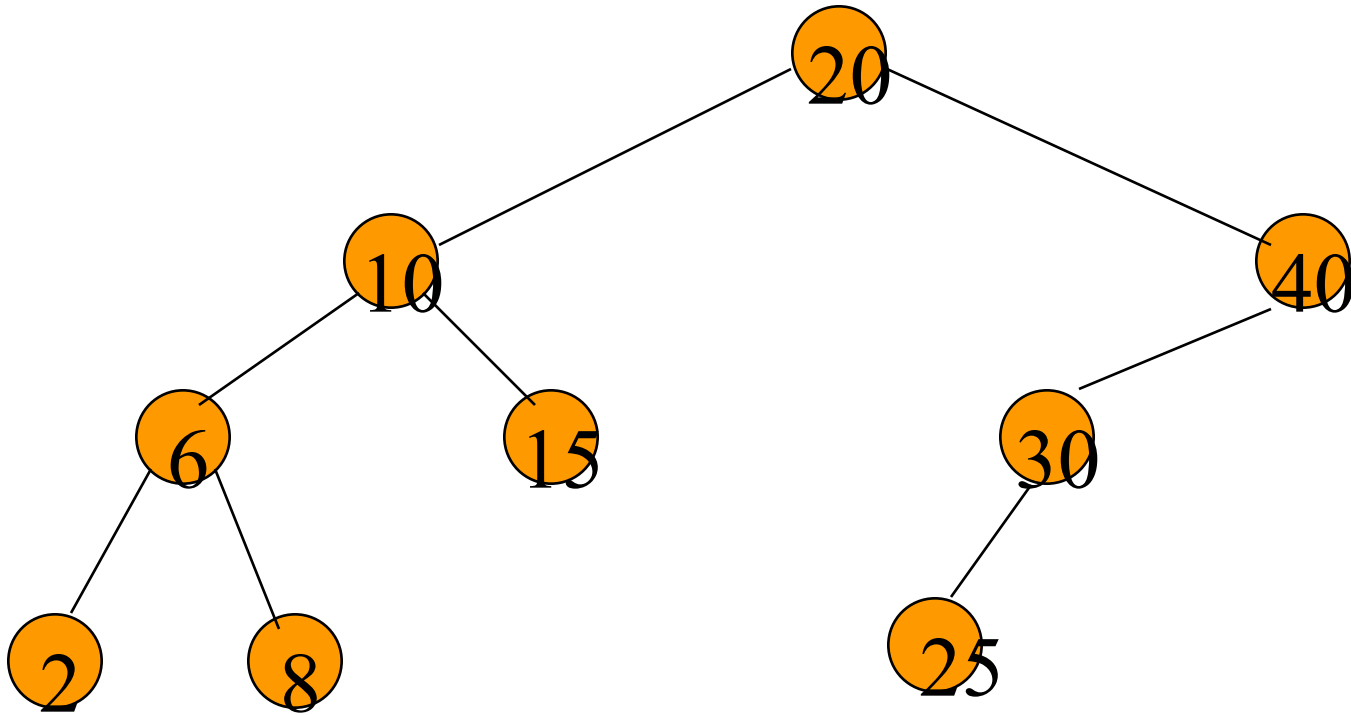
- A binary tree.
- Each node has a (key, value) pair.
- For every node **x**, all keys in the left subtree of **x** are smaller than that in **x**.
- For every node **x**, all keys in the right subtree of **x** are greater than that in **x**.

Example Binary Search Tree



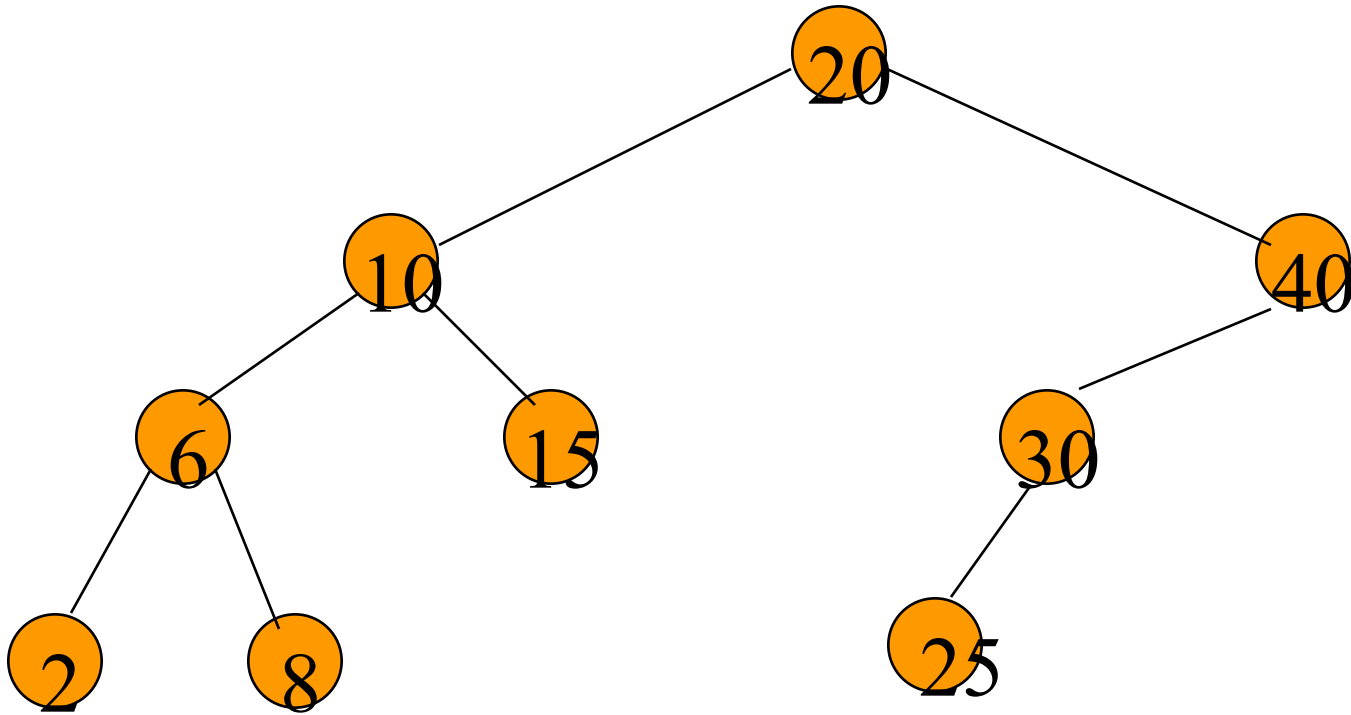
Only keys are shown.

The Operation ascend()



Do an inorder traversal. $O(n)$ time.

The Operation get()



Complexity is $O(\text{height}) = O(n)$, where n is number of nodes/elements.

The Operation `get()`

- Search for an element with key k :
 - If $k = \text{the key in root}$, success
 - If $x < \text{the key in root}$, search the left subtree
 - If $x > \text{the key in root}$, search the right subtree

The Operation get()

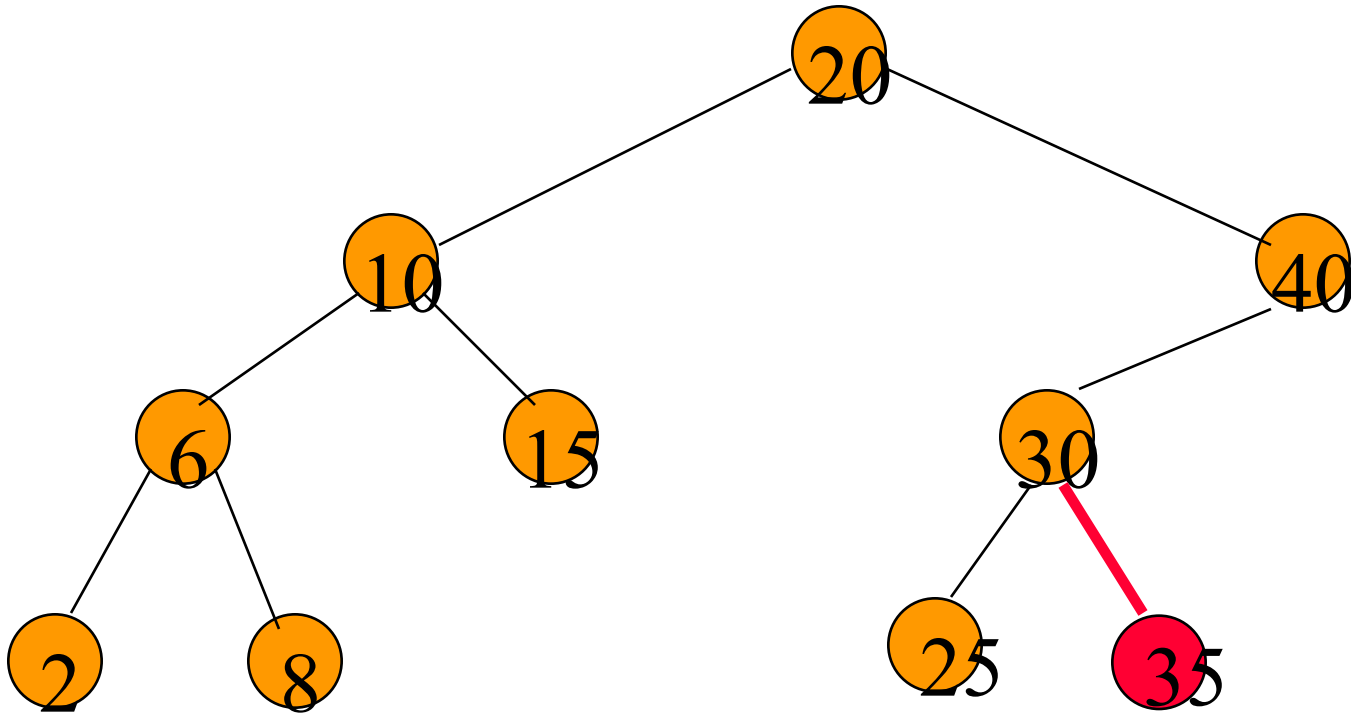
```
template <class K, class E> // Driver
pair<K,E>* BST<K,E>::Get(const K& k)
{ // Search *this for a pair with key k.
    return Get(root, k);
}
```

```
template <class K, class E> // Workhorse
pair<K,E>* BST<K,E>::Get(treeNode<pair<K,E>>* p,
                        const K& k)
{
    if (!p) return 0;
    if (k < p→data.first) return Get(p→leftChild, k);
    if (k > p→data.first) return Get(p→rightChild, k);
    return &p→data;
}
```

```
template <class K, class E> // Iterative version
pair<K,E>* BST<K,E>::Get(const K& k)
{
    TreeNode<pair<K,E>>* currentNode = root;
    while (currentNode)
        if (k < currentNode→data.first)
            currentNode = currentNode→leftChild;
        else if (k > currentNode→data.first)
            currentNode = currentNode→rightChild;
        else return &currentNode→data;

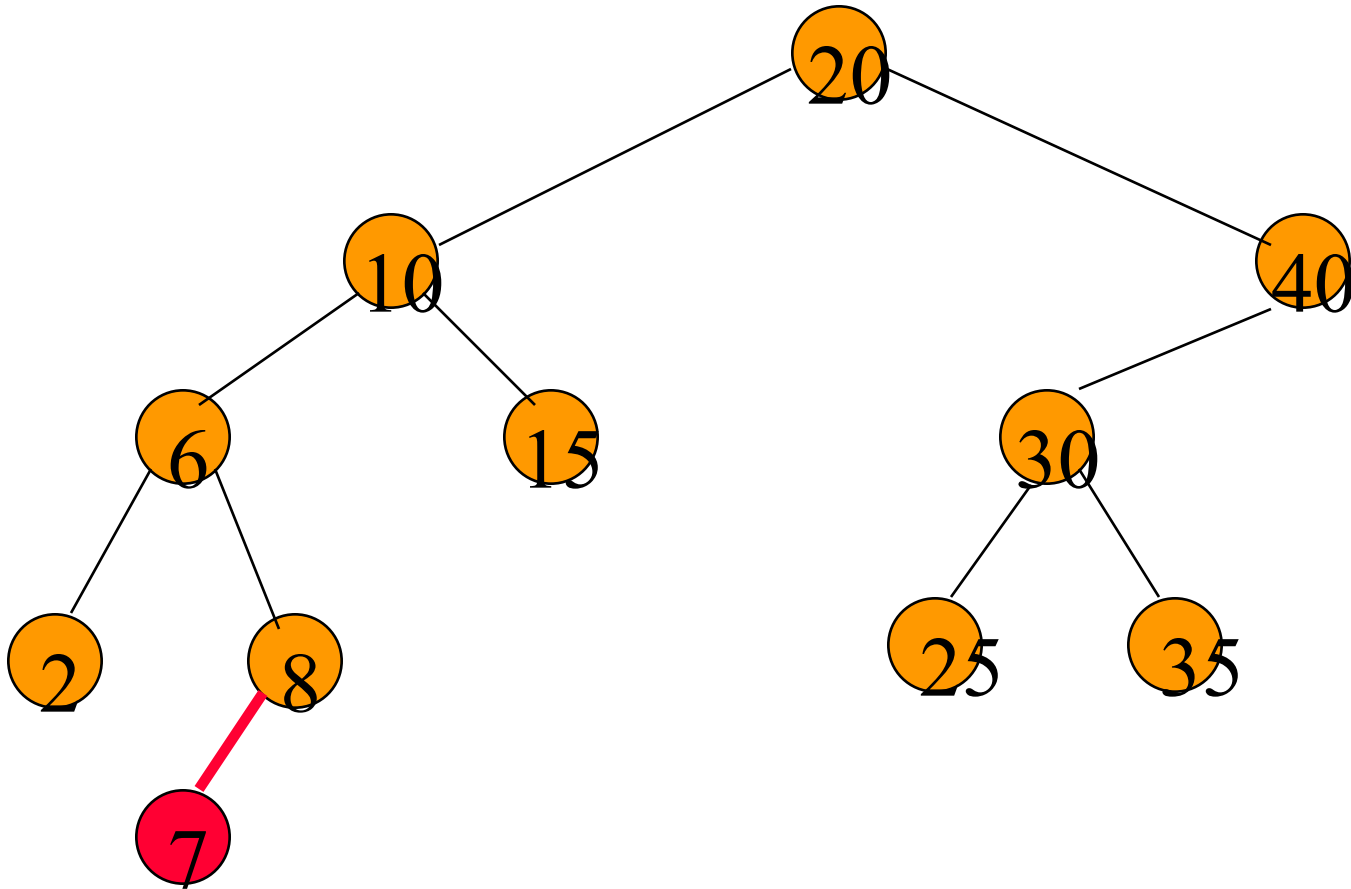
    // no matching pairs
    return 0;
}
```


The Operation put()



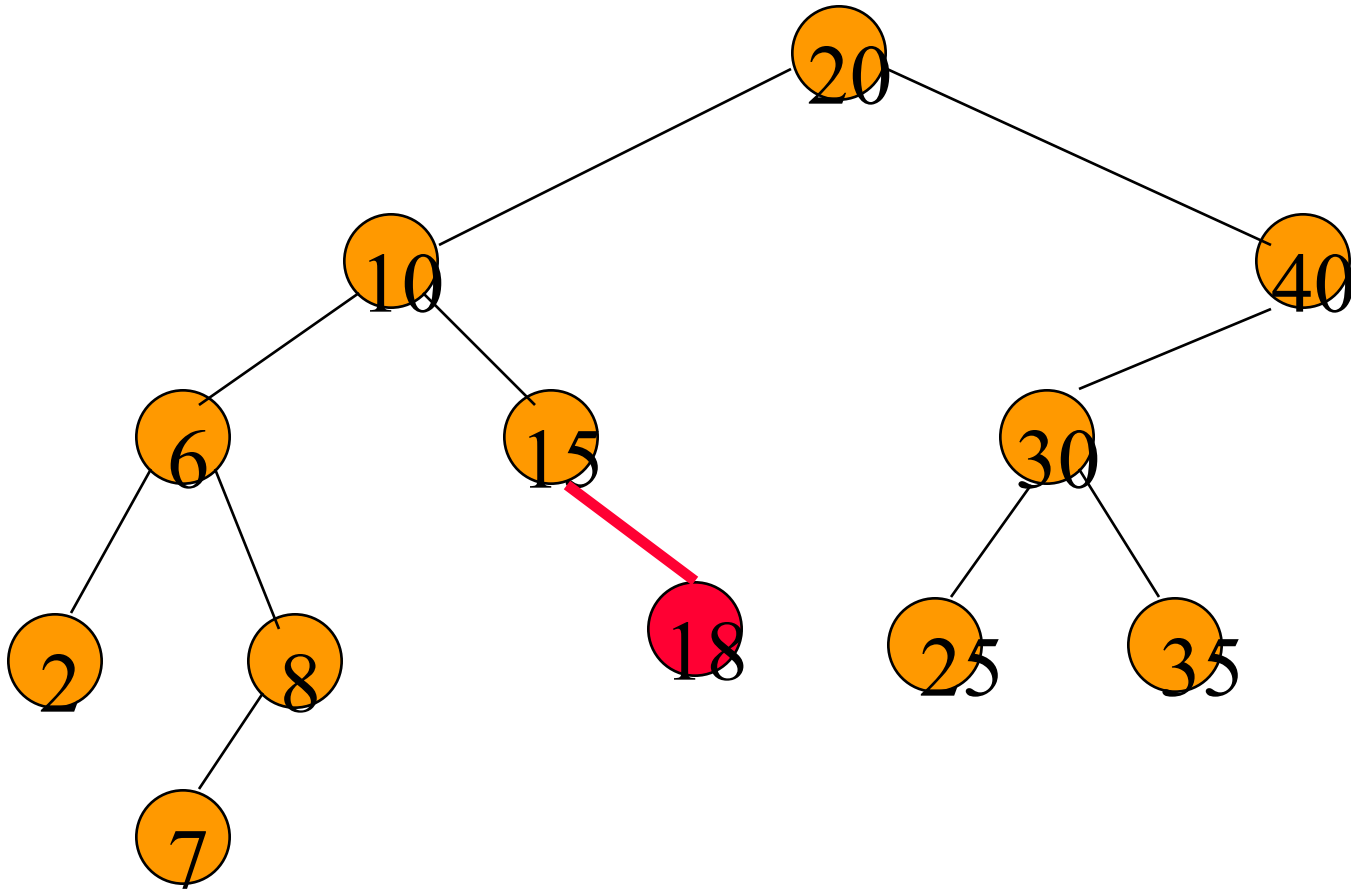
Put a pair whose key is 35.

The Operation put()



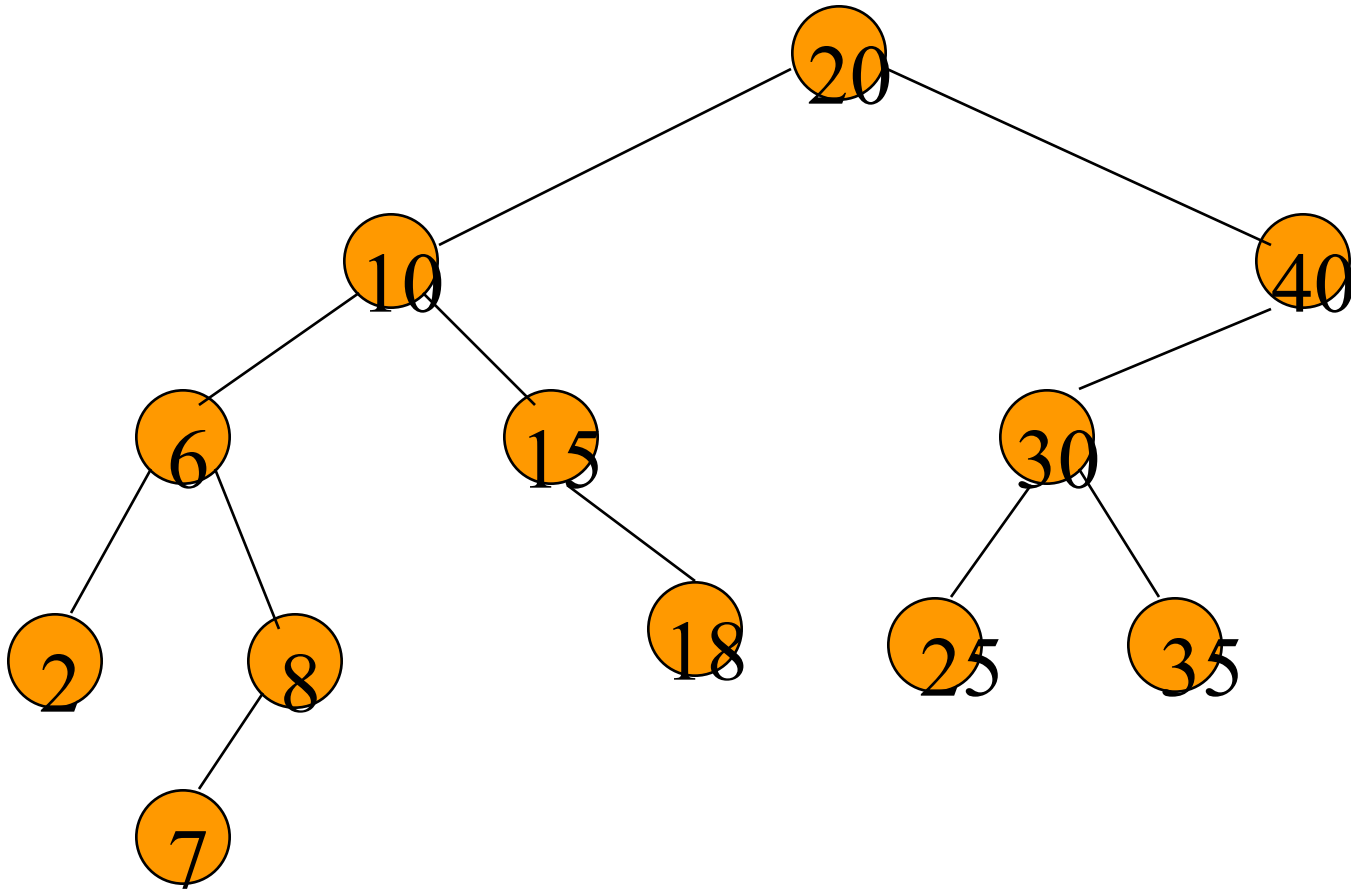
Put a pair whose key is 7.

The Operation put()



Put a pair whose key is 18.

The Operation put()



Complexity of `put()` is $O(\text{height})$.

When the dictionary already contains
a pair with key k

Simply update the element associated
with this key to e

```
template <class K, class E>
```

```
void BST<K,E>::Insert(const pair<K,E>& thePair) {
```

```
    TreeNode<pair<K,E>> *p=root, *pp=0;
```

```
    while (p) {
```

```
        pp=p;
```

```
        if (thePair.first < p→data.first) p=p→leftChild;
```

```
        else if (thePair.first > p→data.first) p=p→rightChild;
```

```
        else // duplicate, update associated element
```

```
            {p→data.second=thePair.second;return;}  
    }
```

```
    p=new TreeNode<pair<K,E>>(thePair,0,0);
```

```
    if (root) // tree not empty
```

```
        if (thePair.first < pp→data.first) pp→leftChild=p;
```

```
        else pp→rightChild=p;
```

```
    else root=p;
```

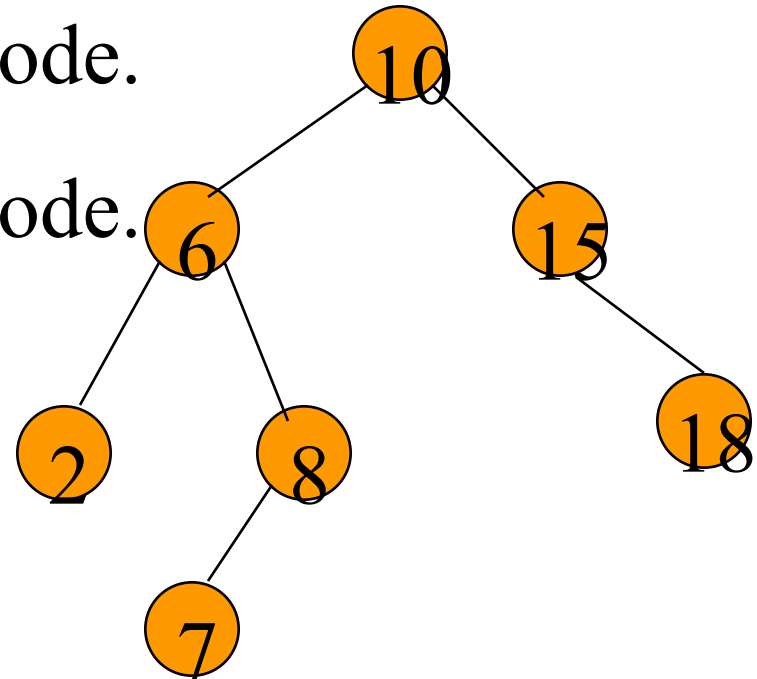
```
}
```

$O(h)$

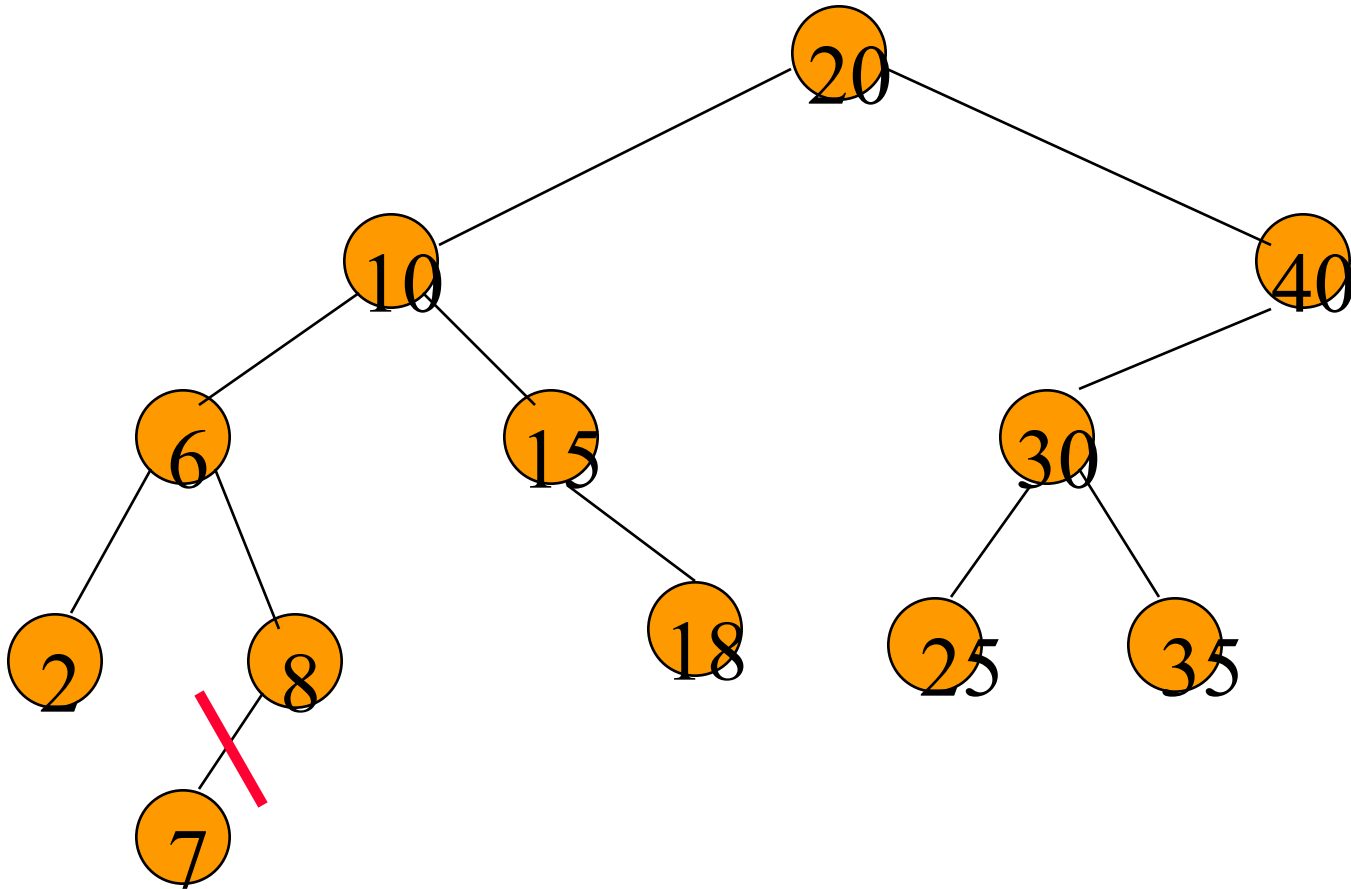
The Operation remove()

Three cases:

- Element is in a leaf.
- Element is in a degree 1 node.
- Element is in a degree 2 node.

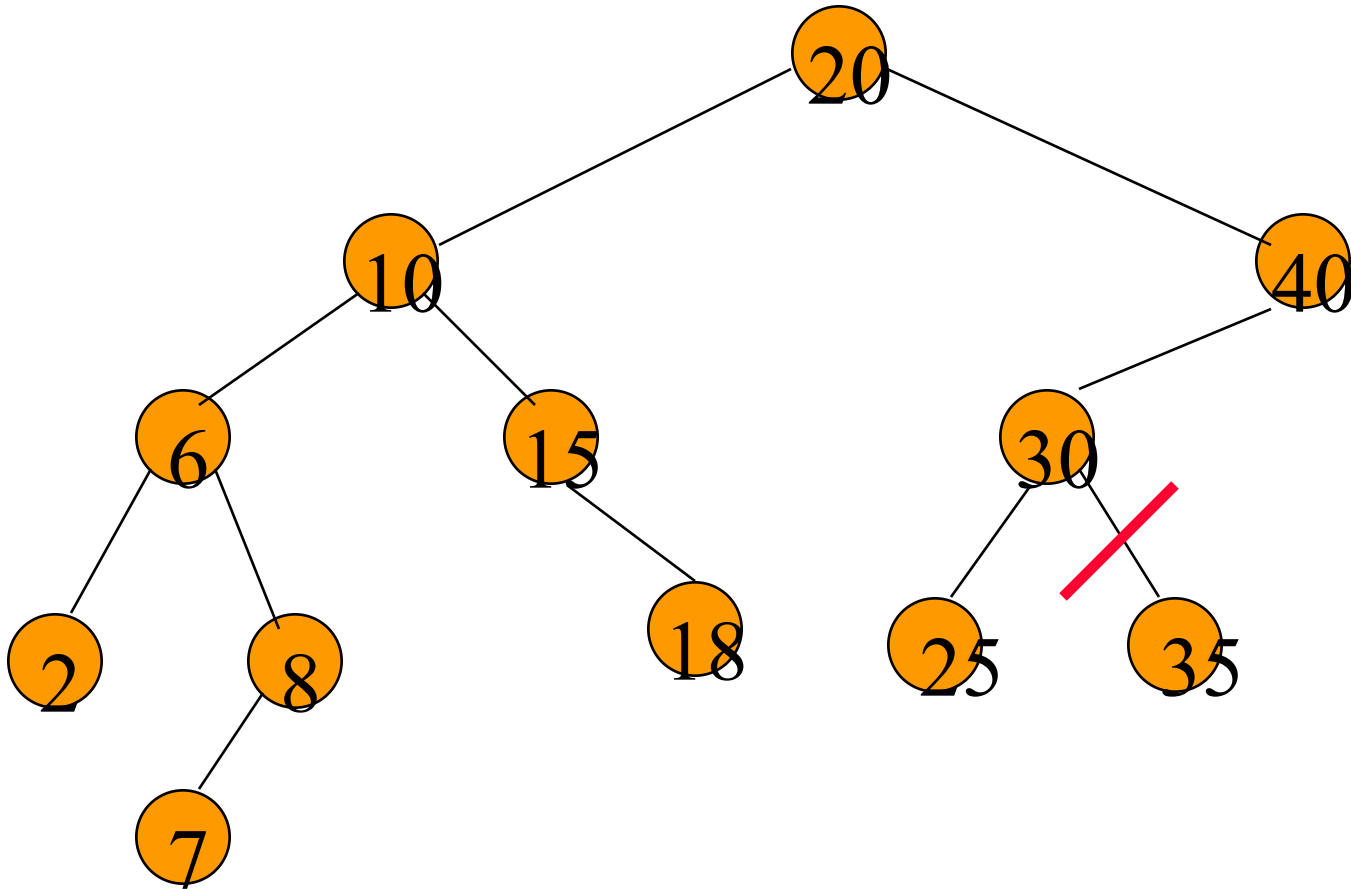


Remove From A Leaf



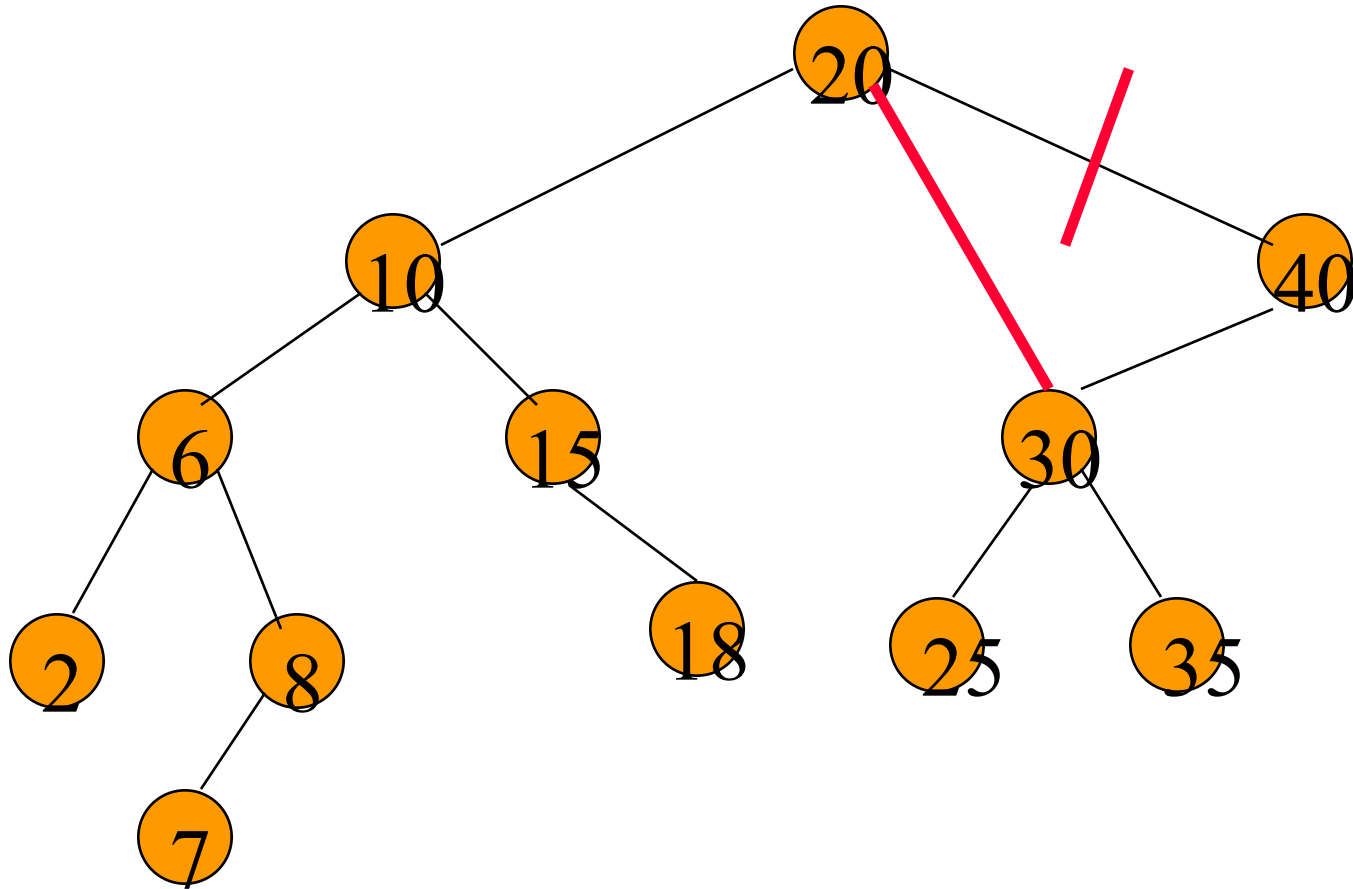
Remove a leaf element. key = 7

Remove From A Leaf (contd.)



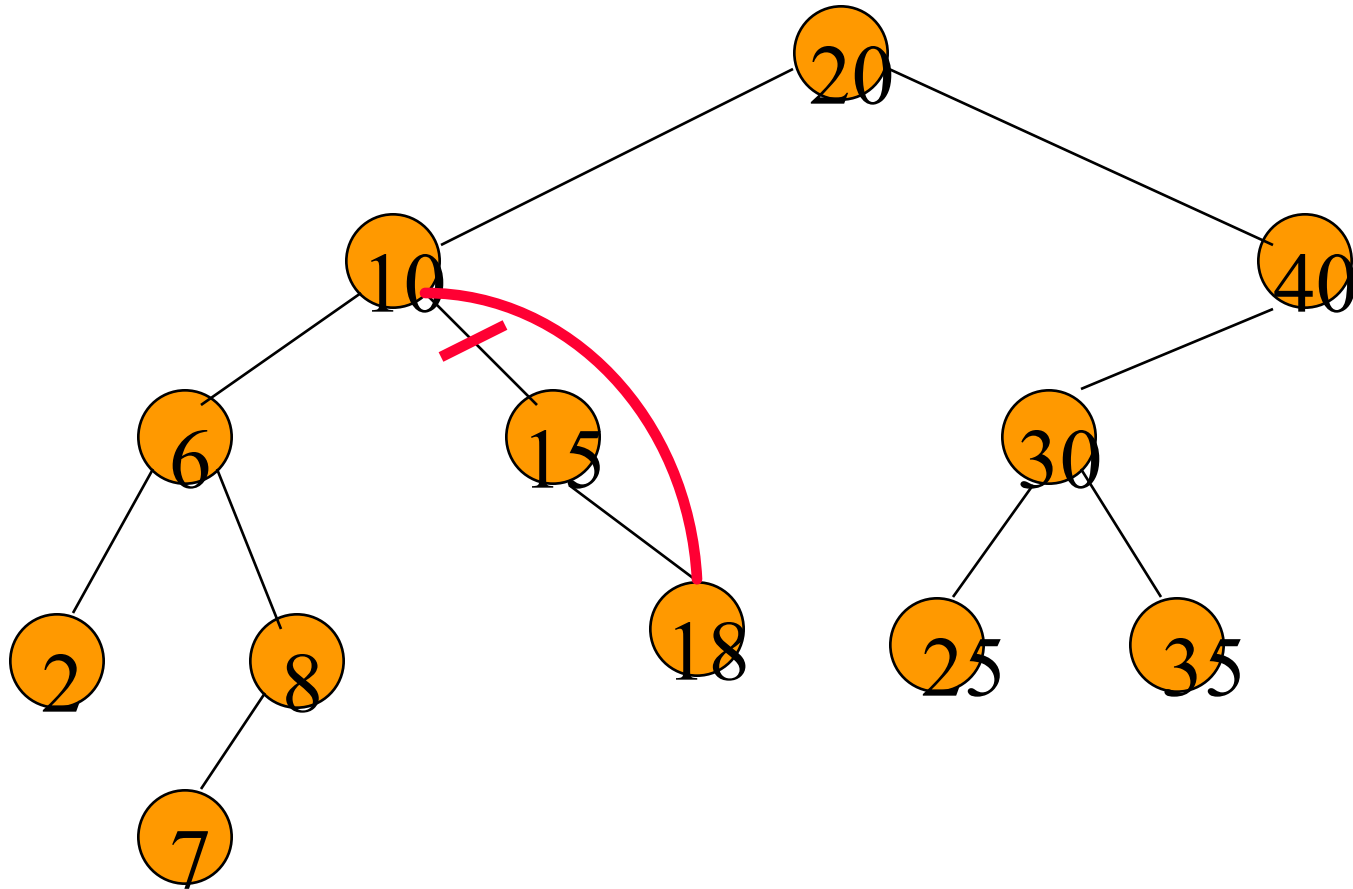
Remove a leaf element. key = 35

Remove From A Degree 1 Node



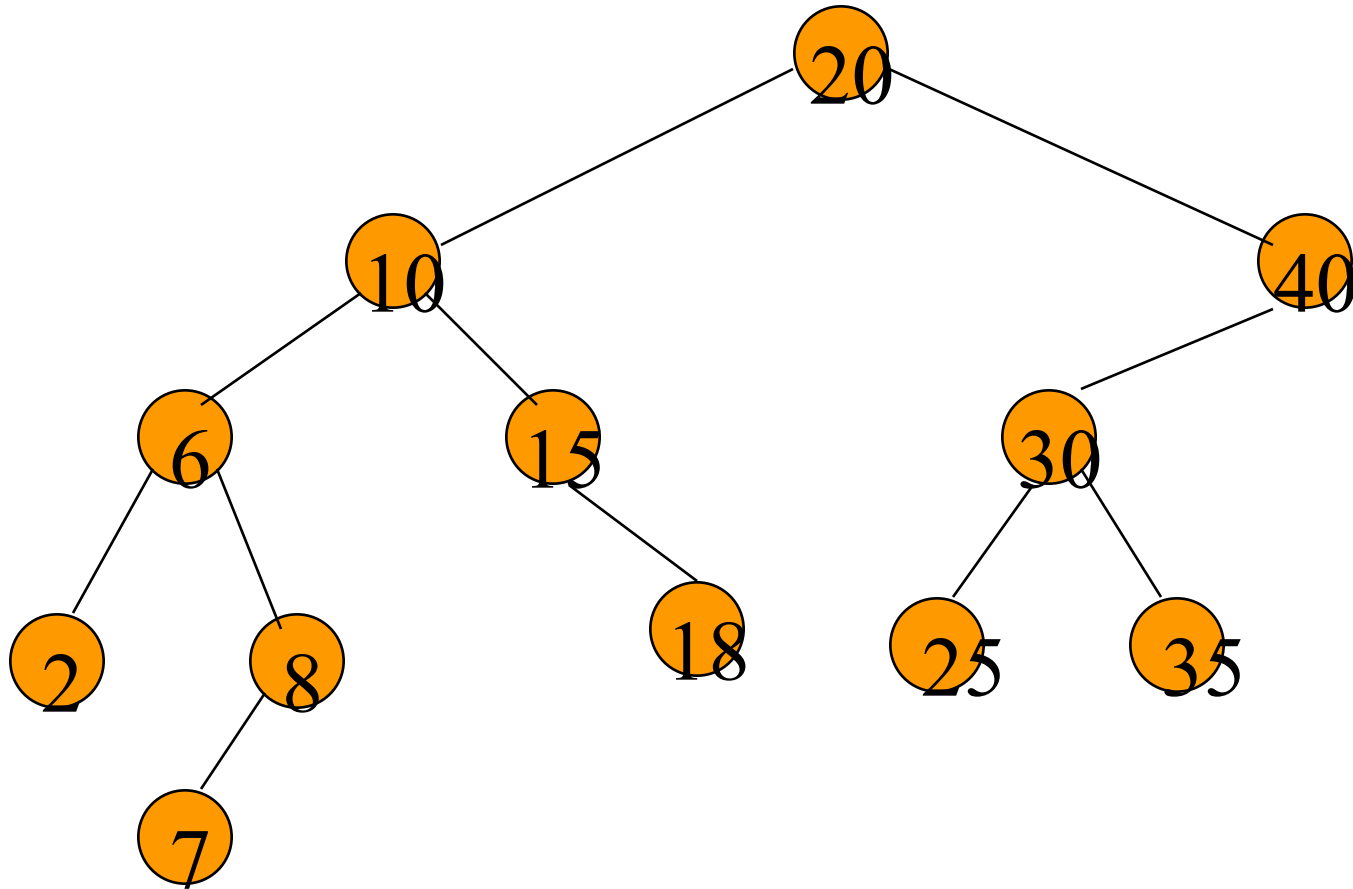
Remove from a degree 1 node. key = 40

Remove From A Degree 1 Node (contd.)



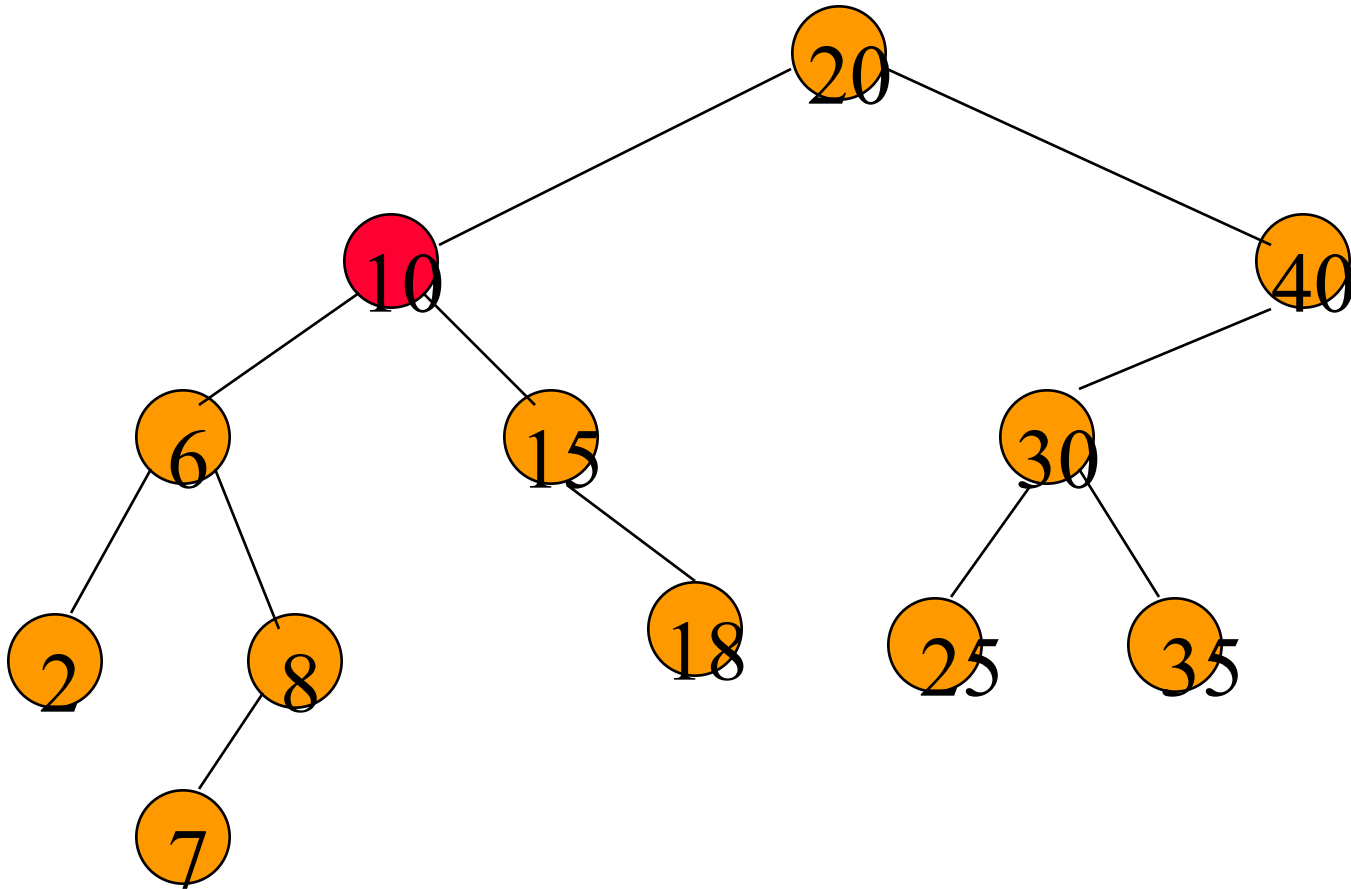
Remove from a degree 1 node. key = 15

Remove From A Degree 2 Node



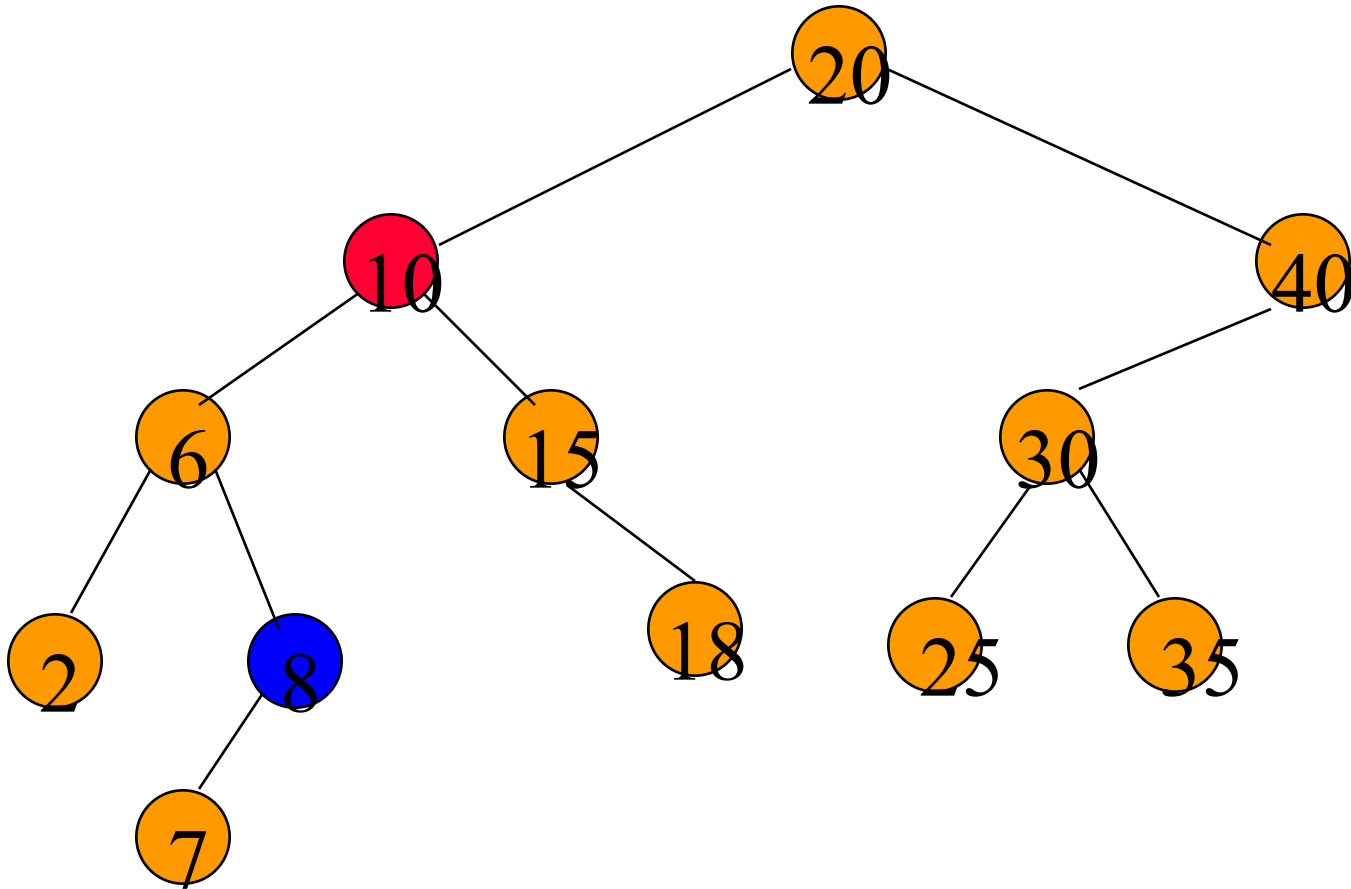
Remove from a degree 2 node. key = 10

Remove From A Degree 2 Node



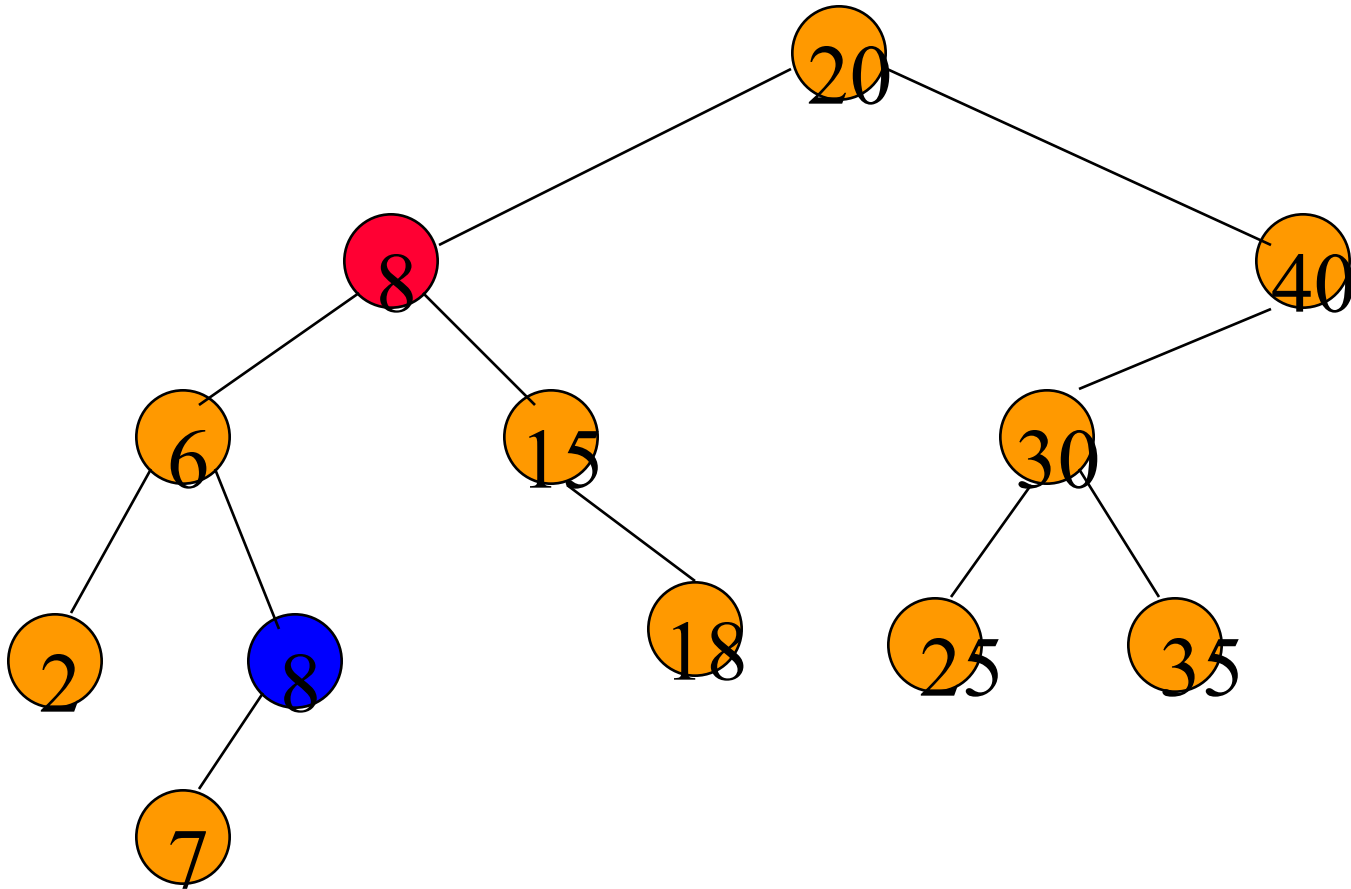
Replace with largest key in left subtree (or smallest in right subtree).

Remove From A Degree 2 Node



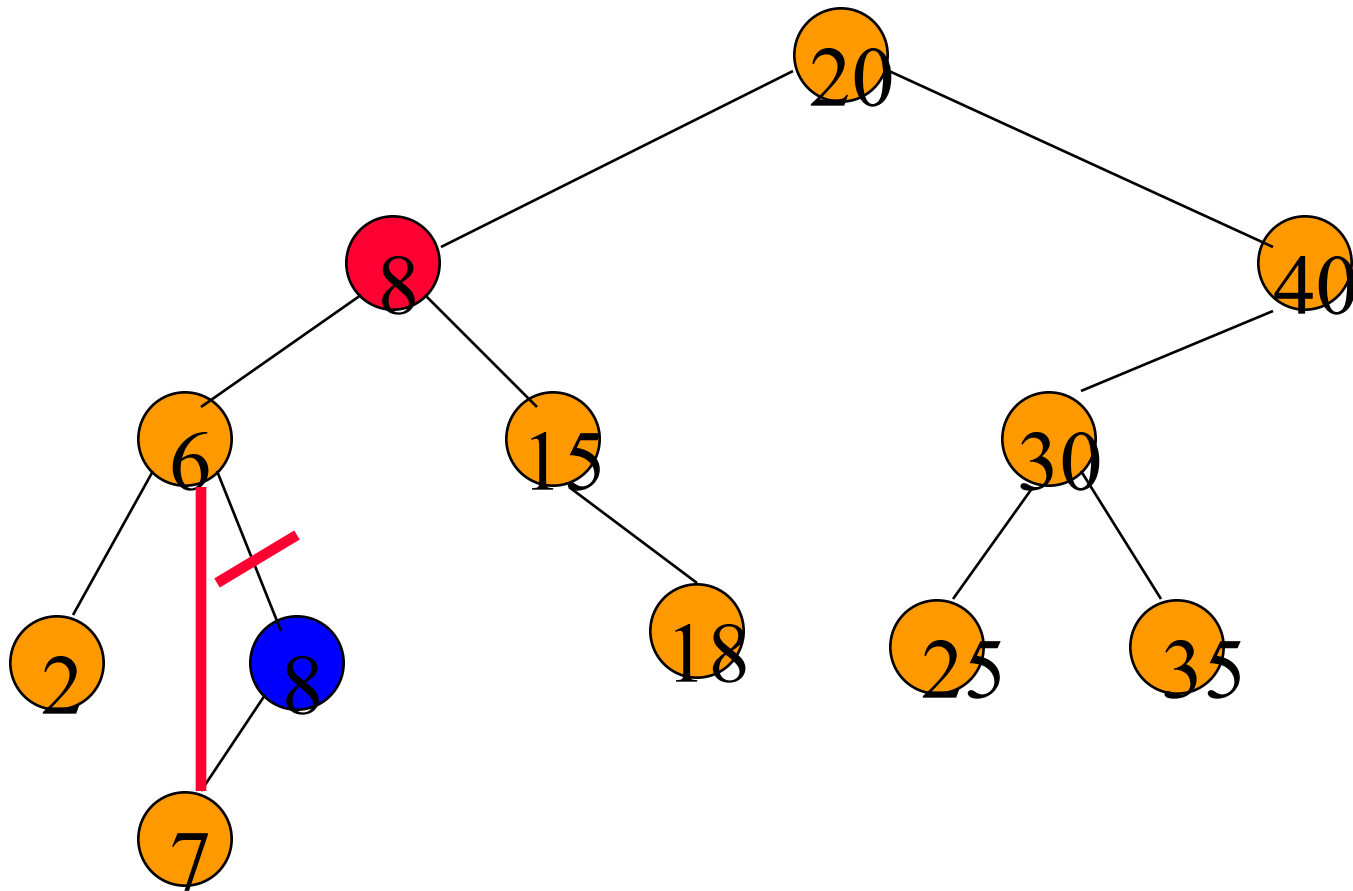
Replace with largest key in left subtree (or smallest in right subtree).

Remove From A Degree 2 Node



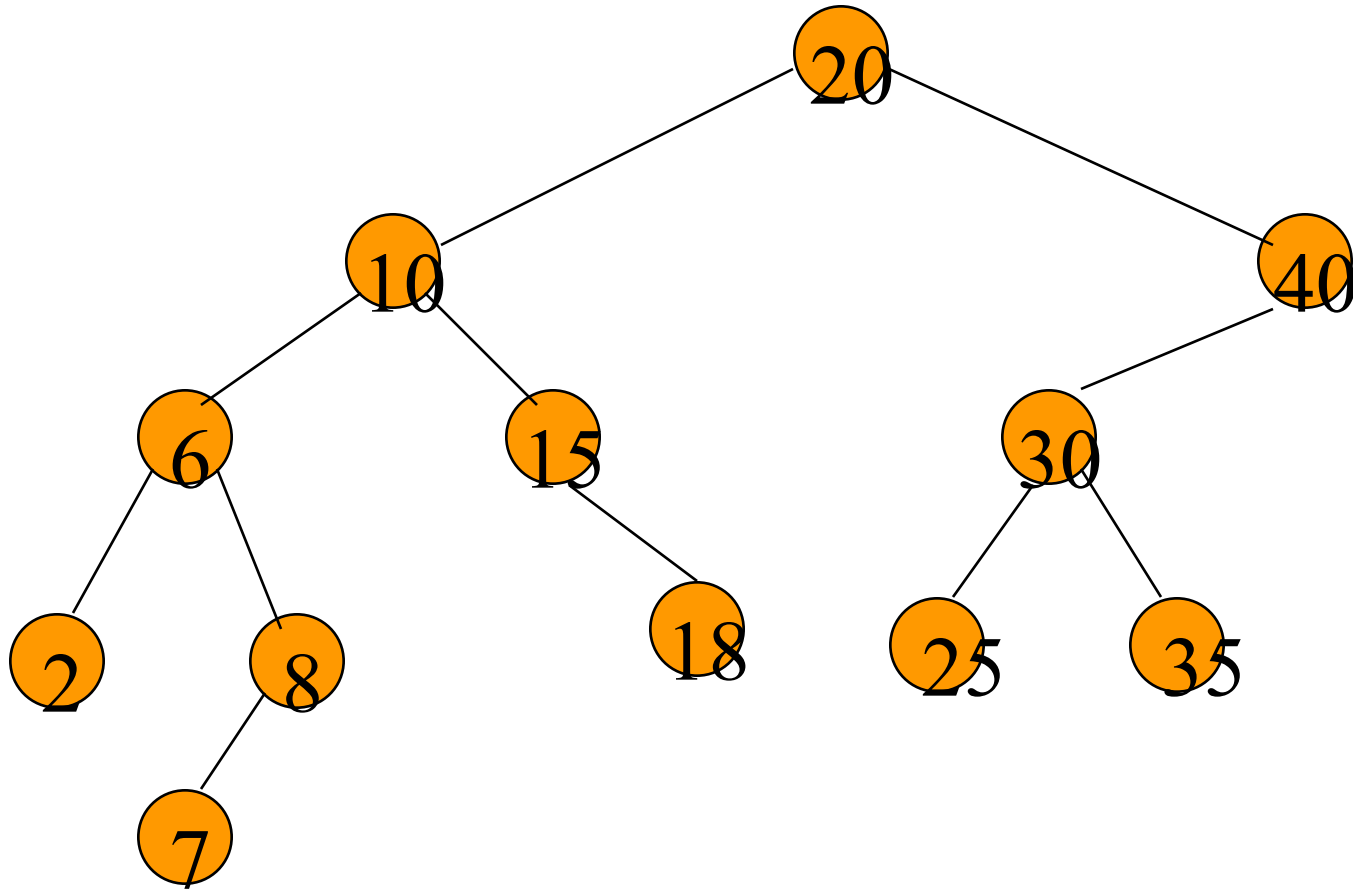
Replace with largest key in left subtree (or smallest in right subtree).

Remove From A Degree 2 Node



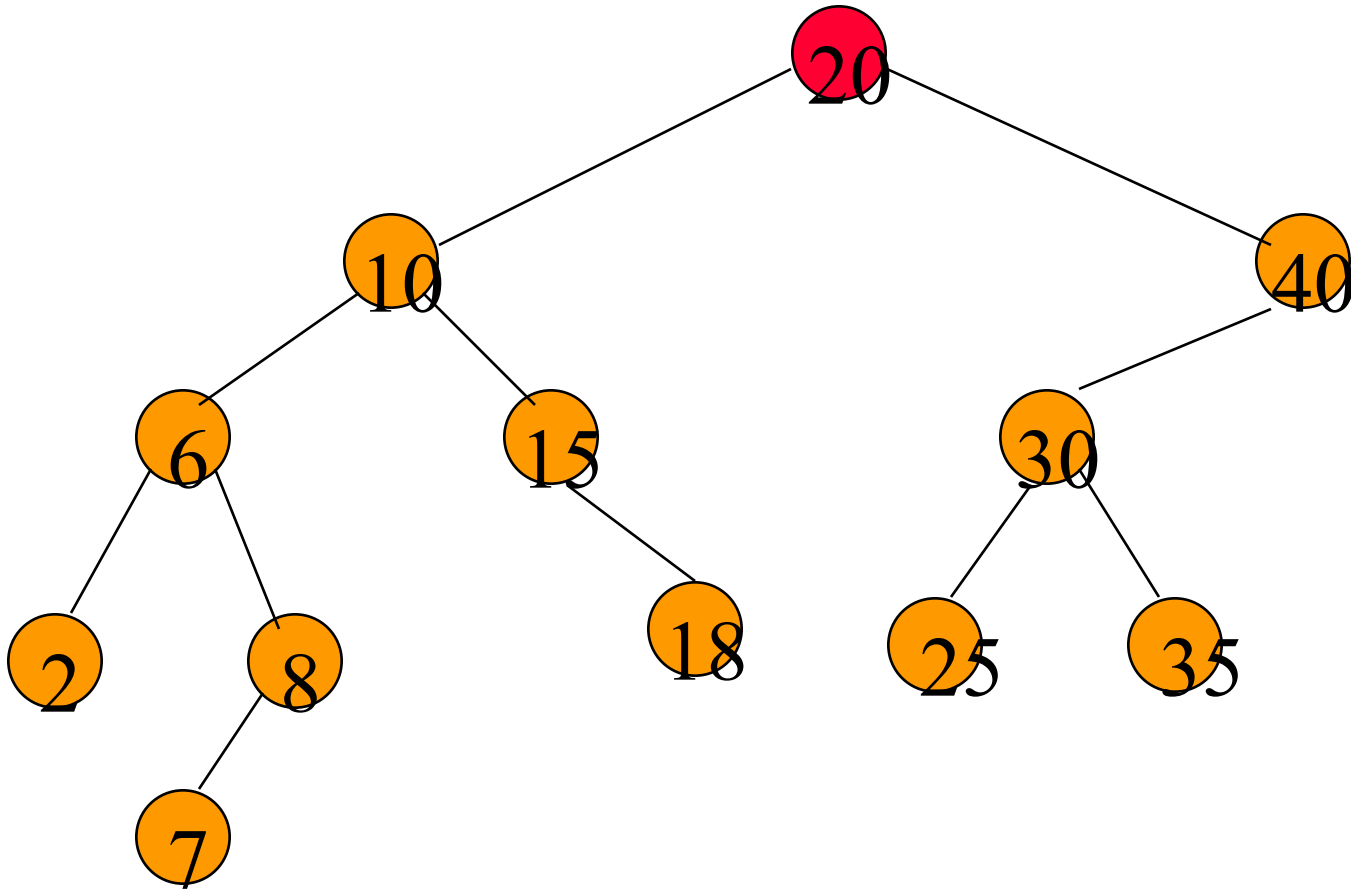
Largest key must be in a leaf or degree 1 node.

Another Remove From A Degree 2 Node



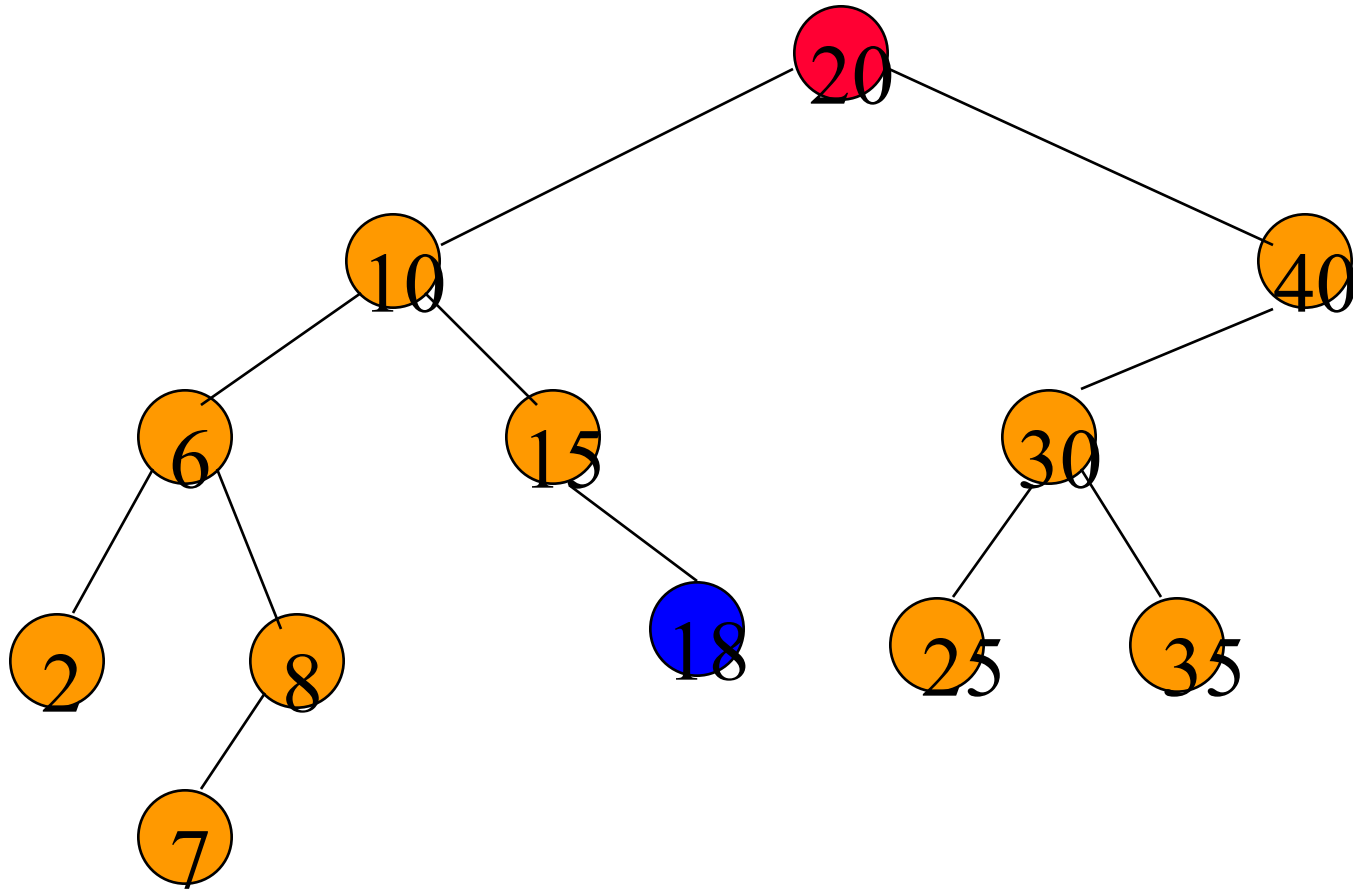
Remove from a degree 2 node. key = 20

Remove From A Degree 2 Node



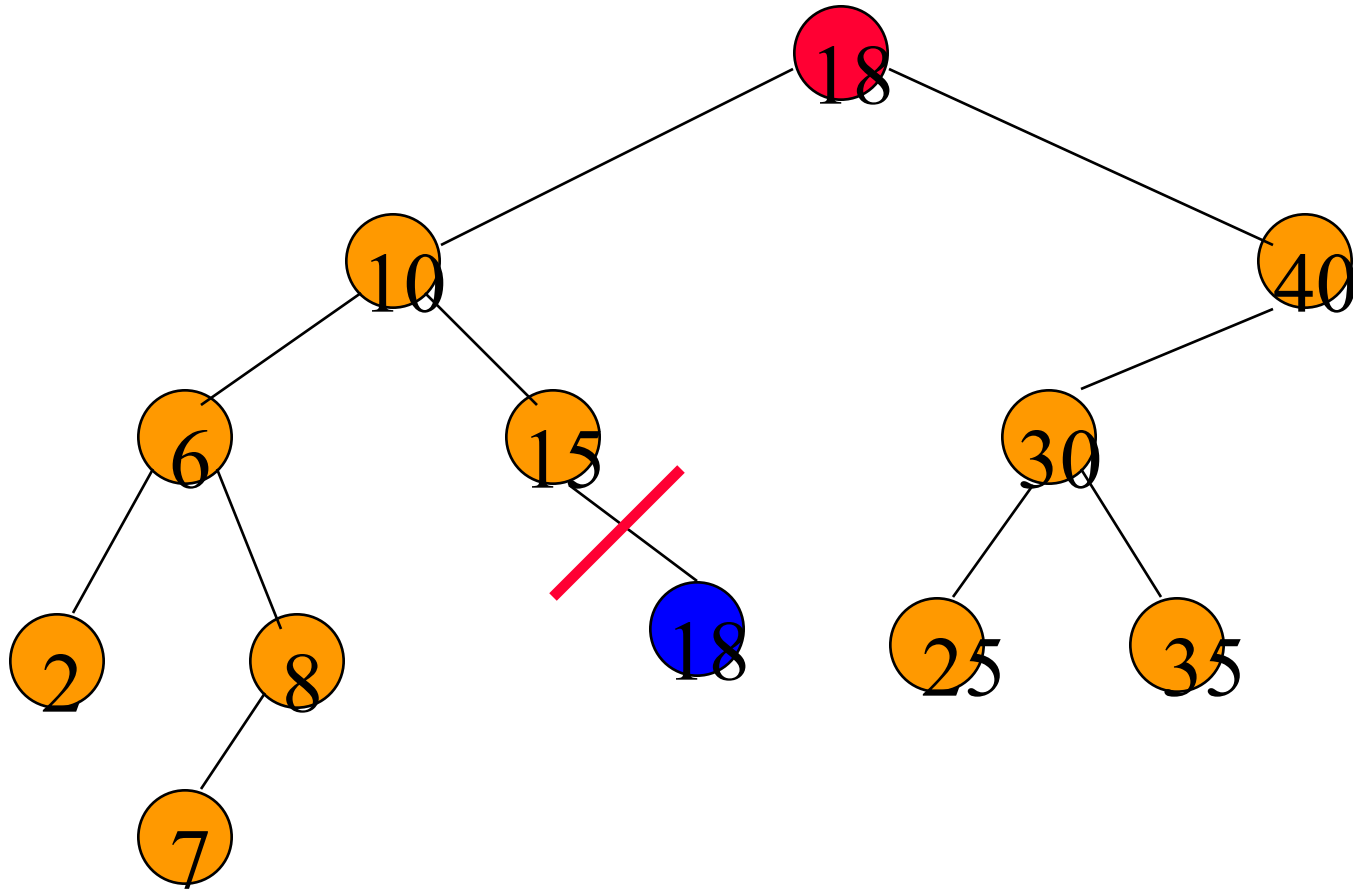
Replace with largest in left subtree.

Remove From A Degree 2 Node



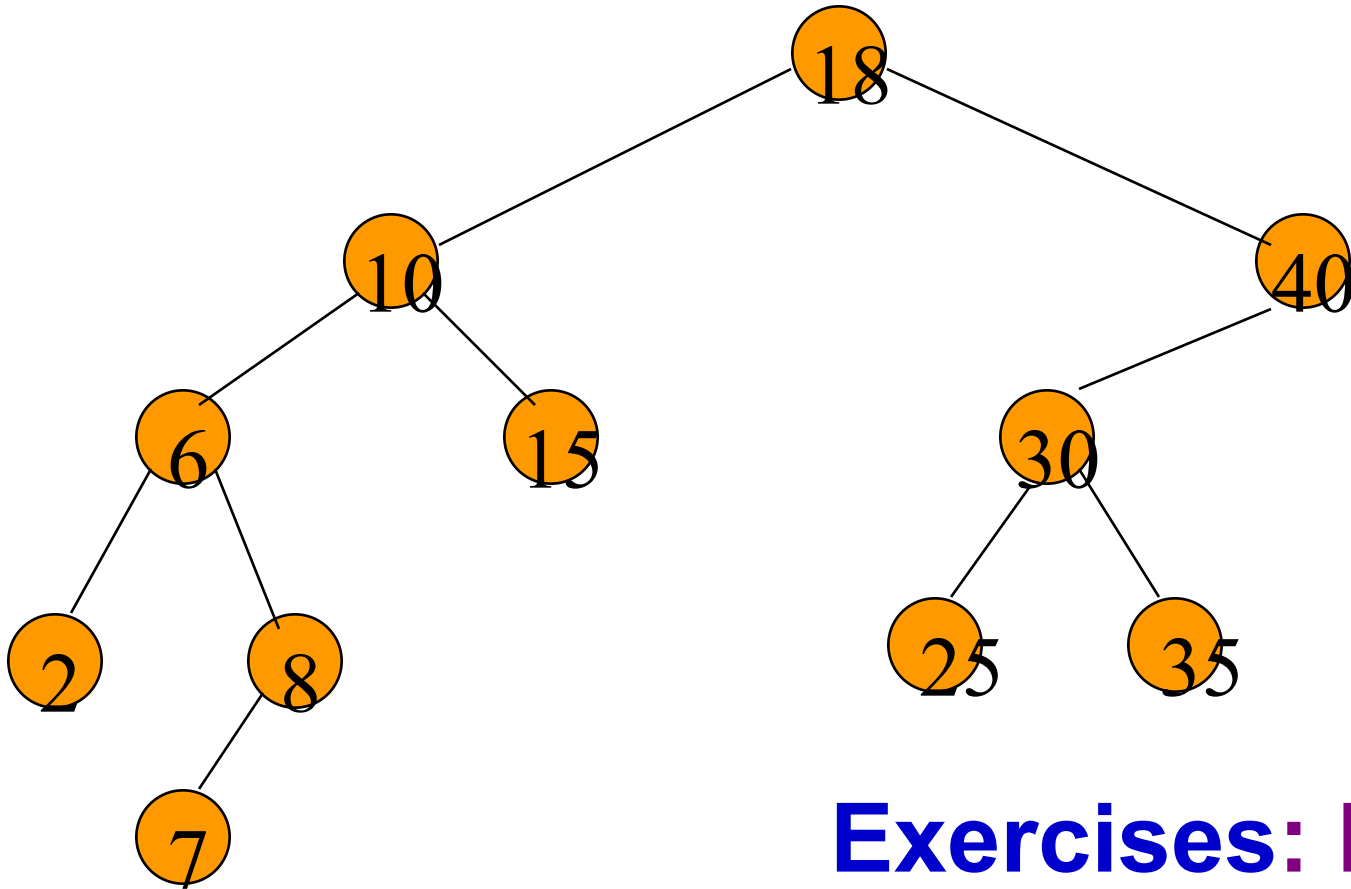
Replace with largest in left subtree.

Remove From A Degree 2 Node



Replace with largest in left subtree.

Remove From A Degree 2 Node



Exercises: P296-1,2

Complexity is $O(\text{height})$.

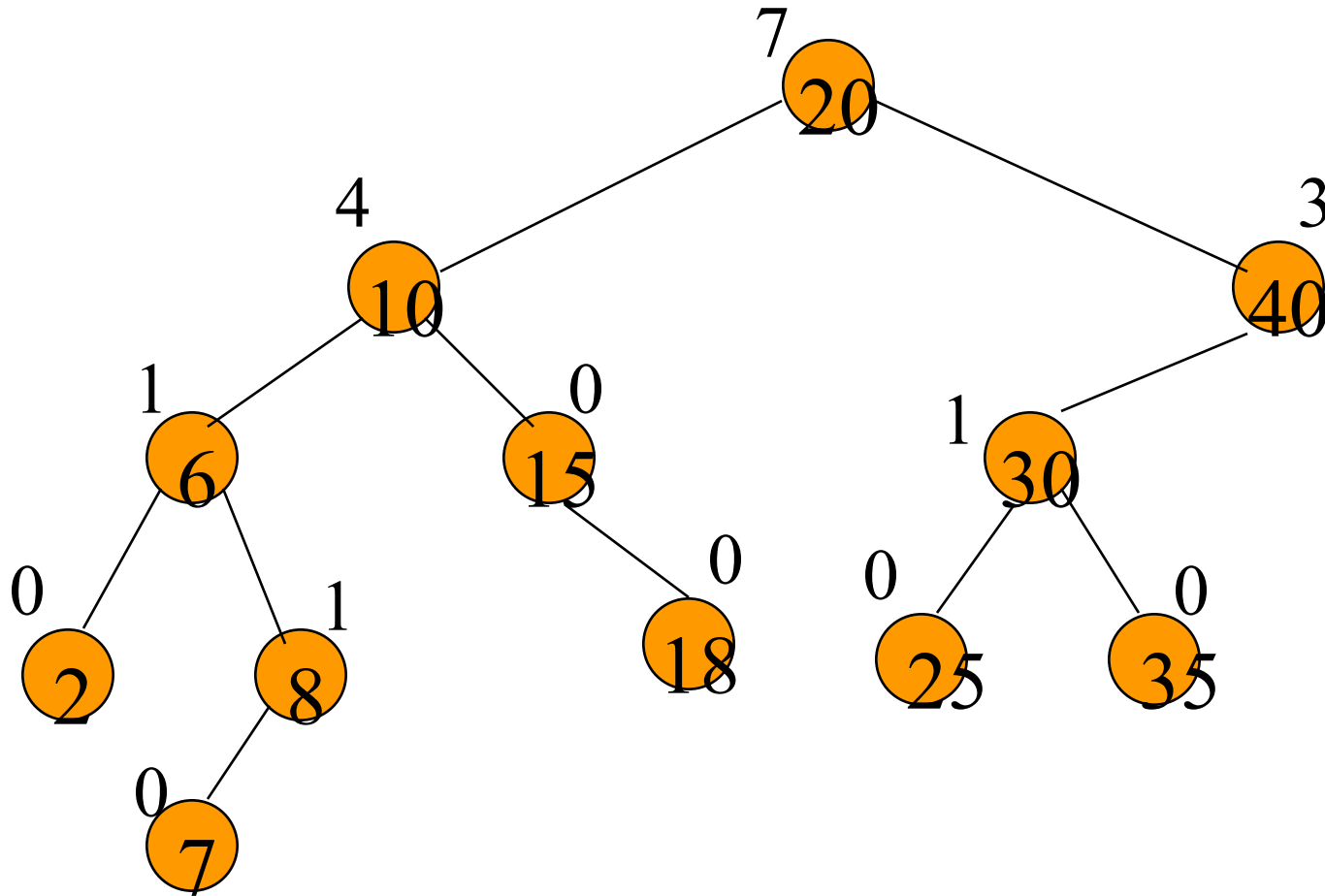
Exercise (chain)

- **2. Given a sorted single linked list $L = \langle a_1, \dots, a_n \rangle$, where $a_i.data \leq a_j.data$ ($i < j$).**
 - **Try to write an algorithm of inserting a new data element X to L , and analyze its complexities.**
- **3. Given a linear list $L = \langle a_1, \dots, a_n \rangle$, implemented by a single linked list.**
 - **Delete data a_i with Time Complexity $O(1)$. We have a pointer to $node(a_i)$.**

Indexed Binary Search Tree

- Binary search tree.
- Each node has an additional field.
 - **leftSize** = number of nodes in its left subtree

Example Indexed Binary Search Tree



leftSize values are out of the circle

leftSize And Rank

Rank of an element is its position in inorder
(inorder = ascending key order).

[2,6,7,8,10,15,18,20,25,30,35,40]

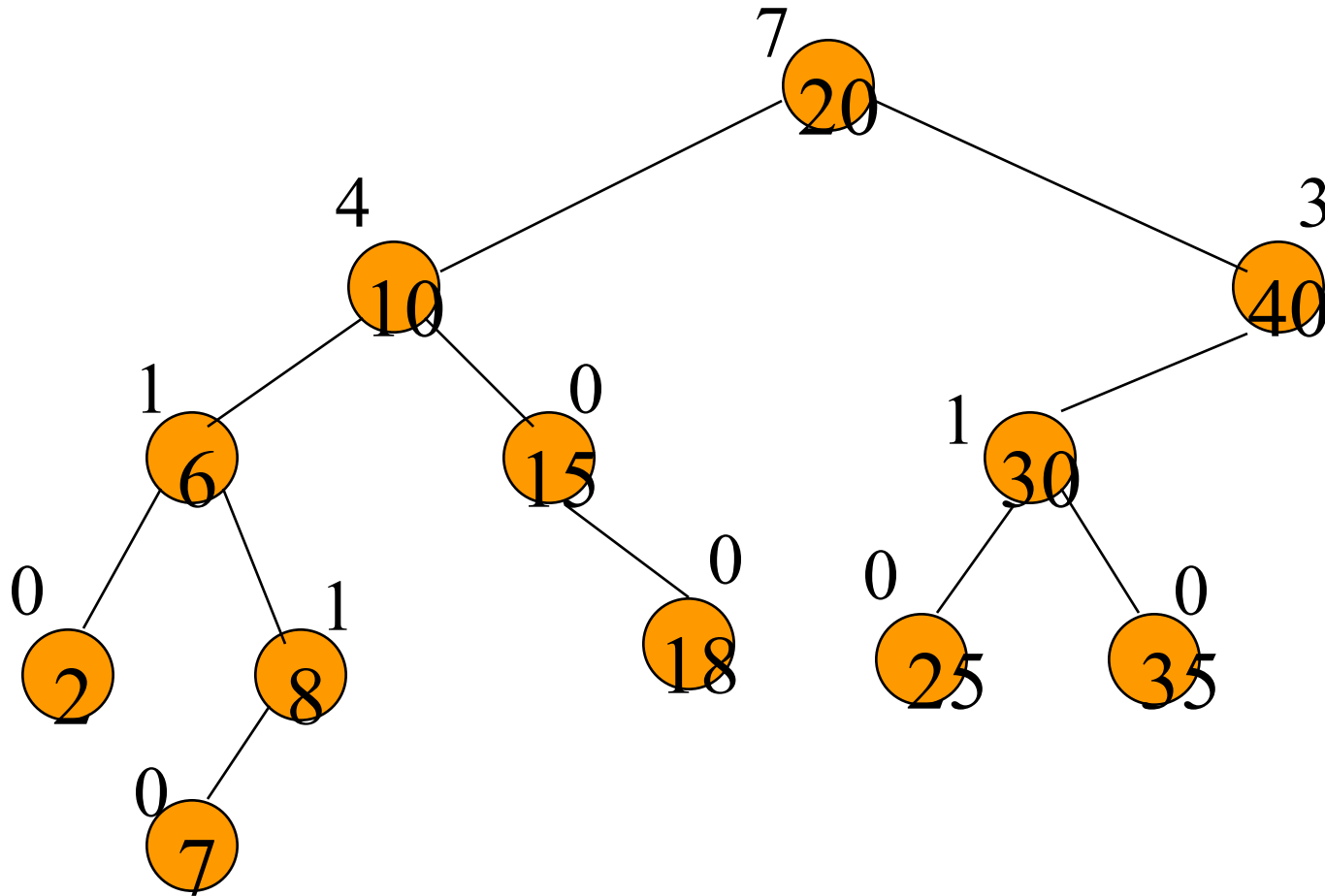
$$\text{rank}(2) = 0$$

$$\text{rank}(15) = 5$$

$$\text{rank}(20) = 7$$

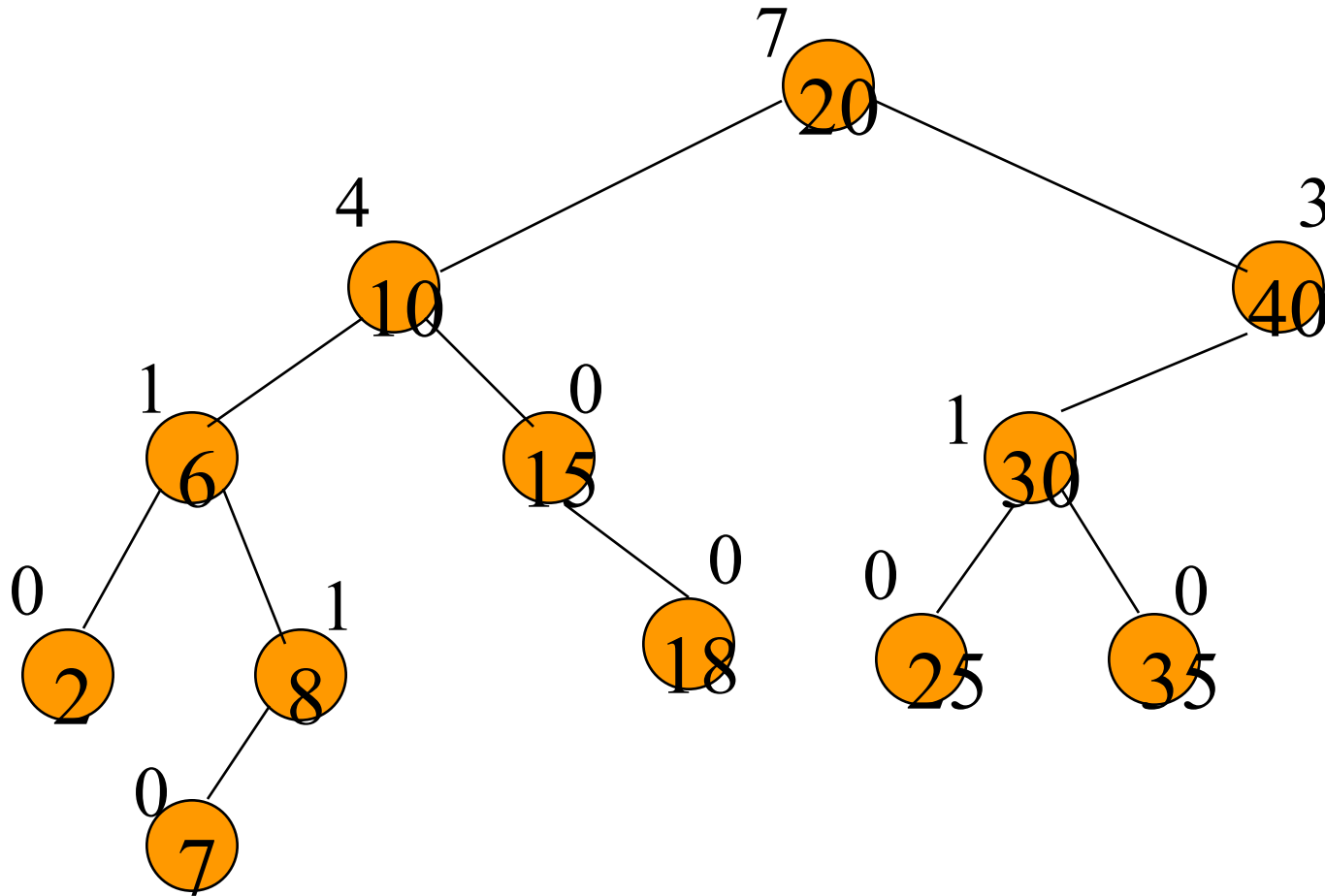
$\text{leftSize}(x) = \text{rank}(x)$ with respect to elements in
subtree rooted at x

leftSize And Rank



sorted list = [2,6,7,8,10,15,18,20,25,30,35,40]

get(index) And remove(index)



sorted list = [2,6,7,8,10,15,18,20,25,30,35,40]

get(index) And remove(index)

- if $\text{index} = \text{x.leftSize}$ desired element is x.element
- if $\text{index} < \text{x.leftSize}$ desired element is index 'th element in left subtree of x
- if $\text{index} > \text{x.leftSize}$ desired element is $(\text{index} - \text{x.leftSize} - 1)$ 'th element in right subtree of x

Applications

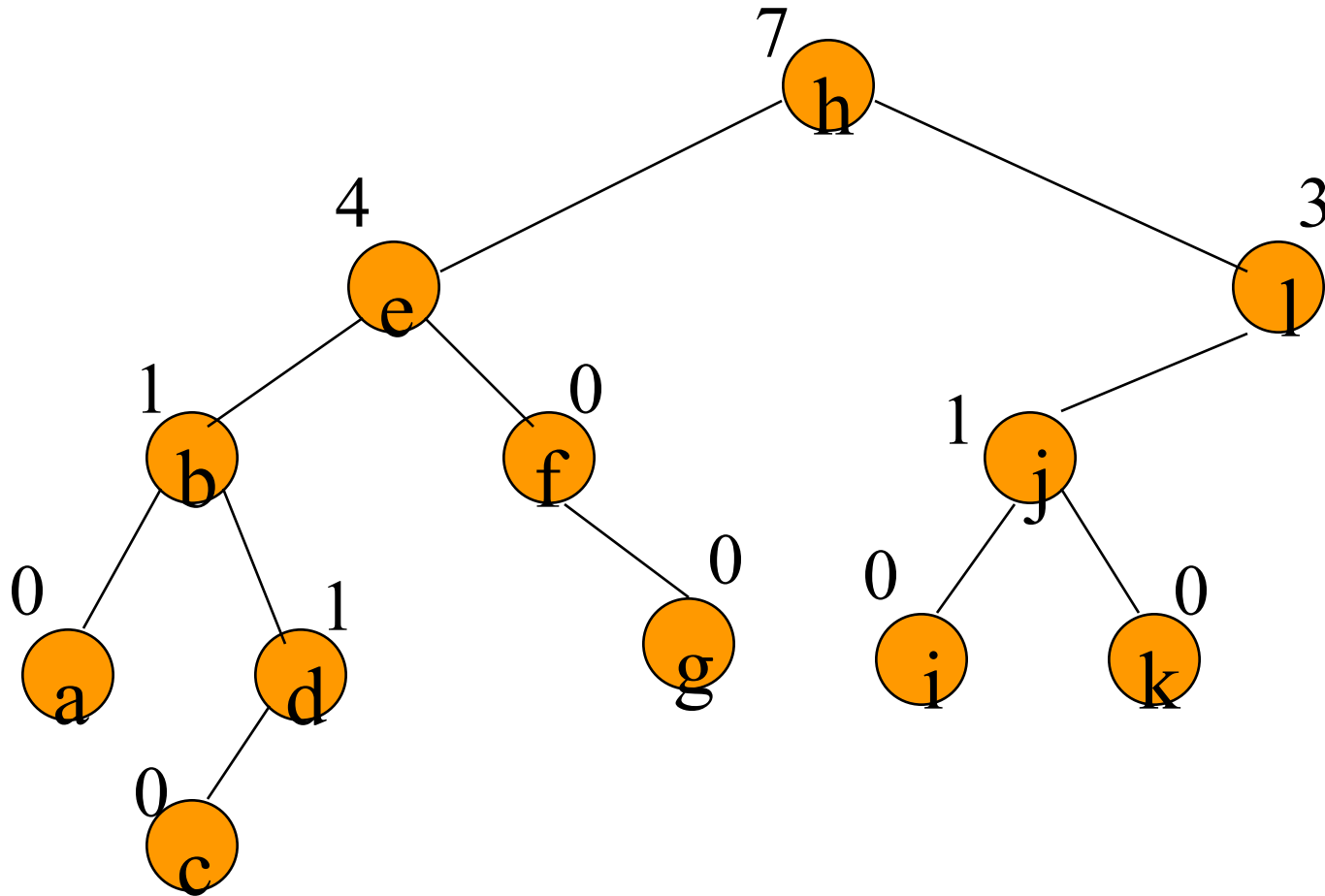
(Complexities Are For Balanced Trees)

Best-fit bin packing in $O(n \log n)$ time.

Representing a linear list so that `get(index)`, `add(index, element)`, and `remove(index)` run in $O(\log(\text{list size}))$ time (uses an indexed binary tree, not indexed binary search tree).

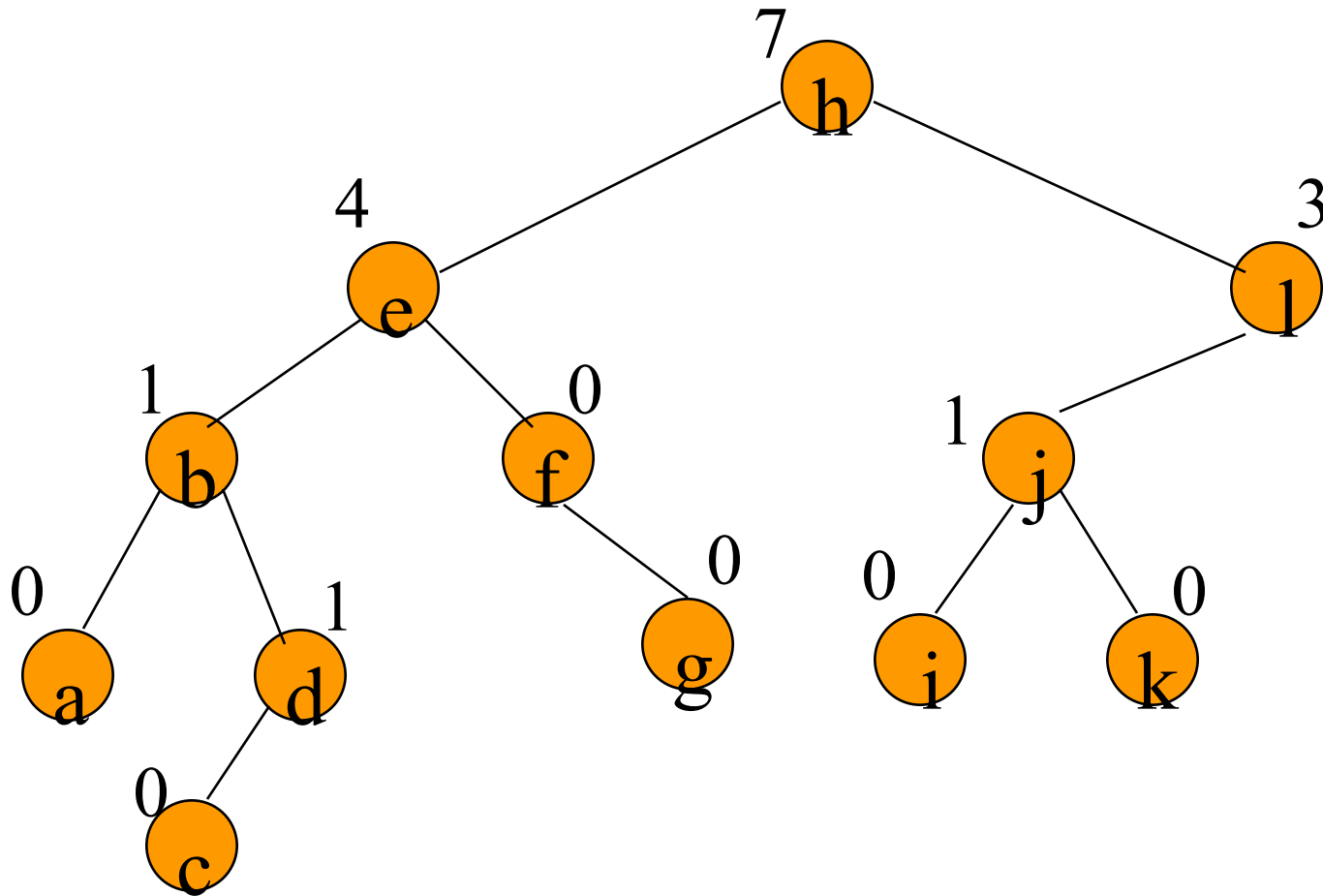
Can't use hash tables for either of these applications.

Linear List As Indexed Binary Tree



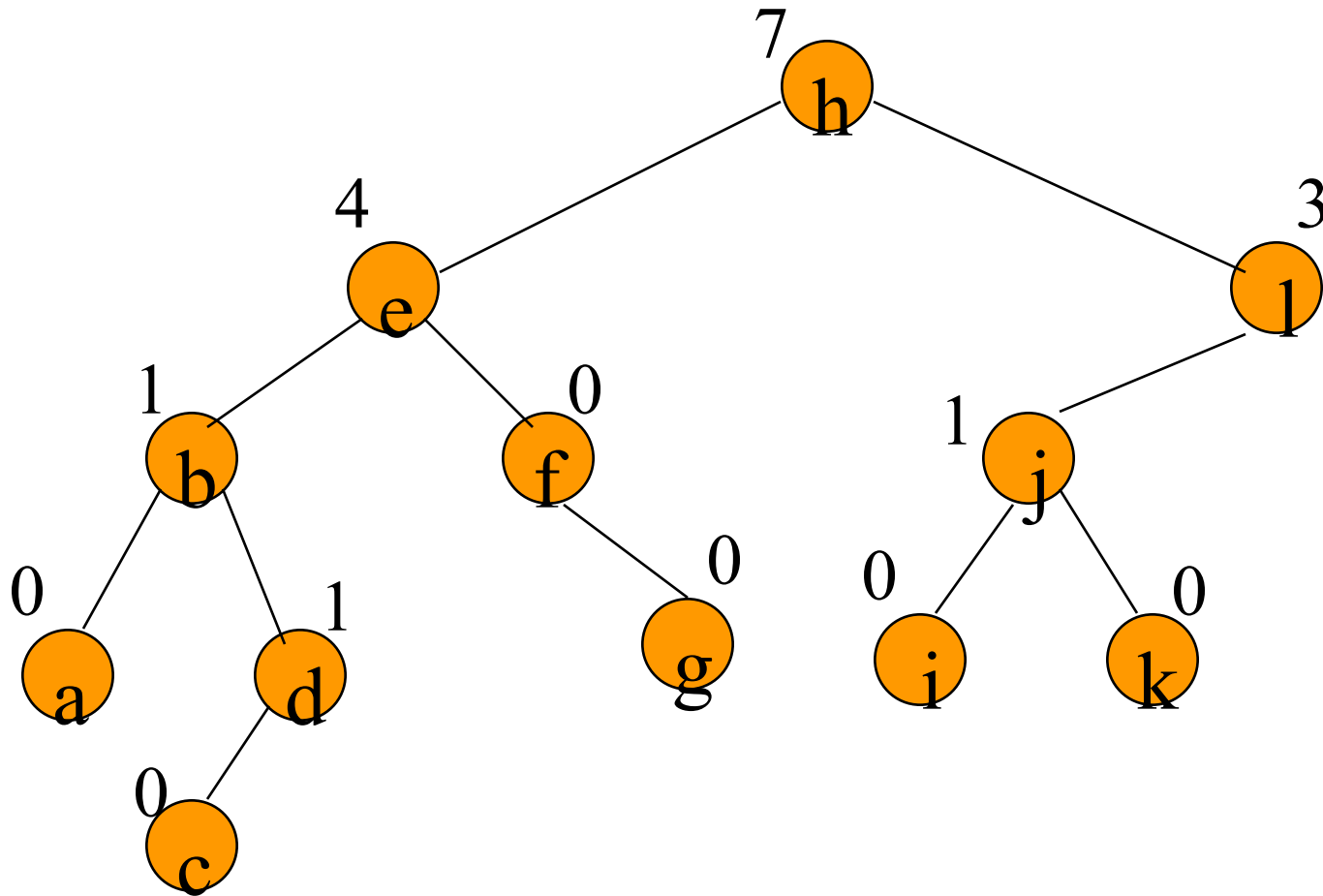
list = [a,b,c,d,e,f,g,h,i,j,k,l]

add(5,'m')



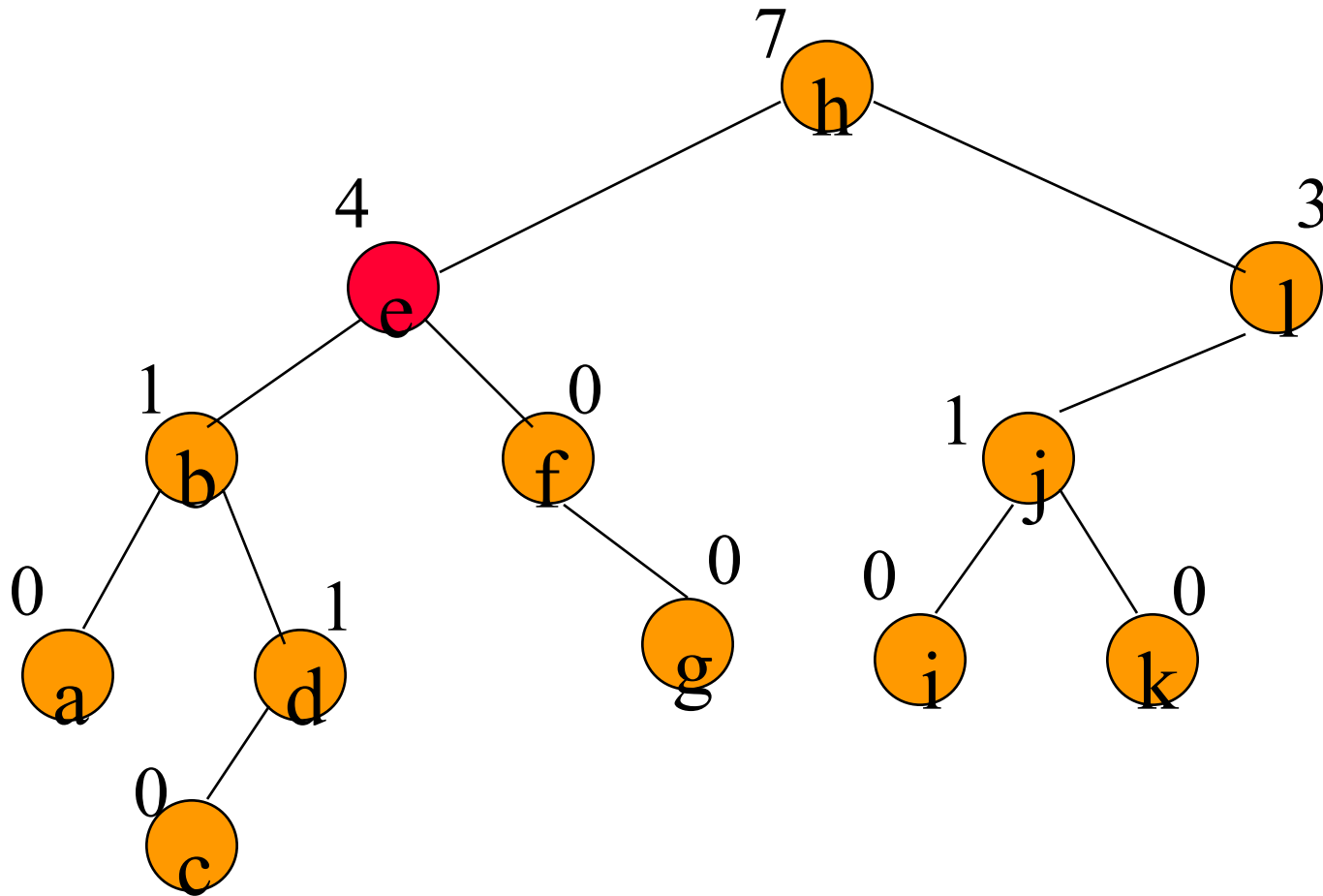
list = [a,b,c,d,e,f,g,h,i,j,k,l]

add(5,'m')



list = [a,b,c,d,e,**m**,f,g,h,i,j,k,l]
find node with element 4 (e)

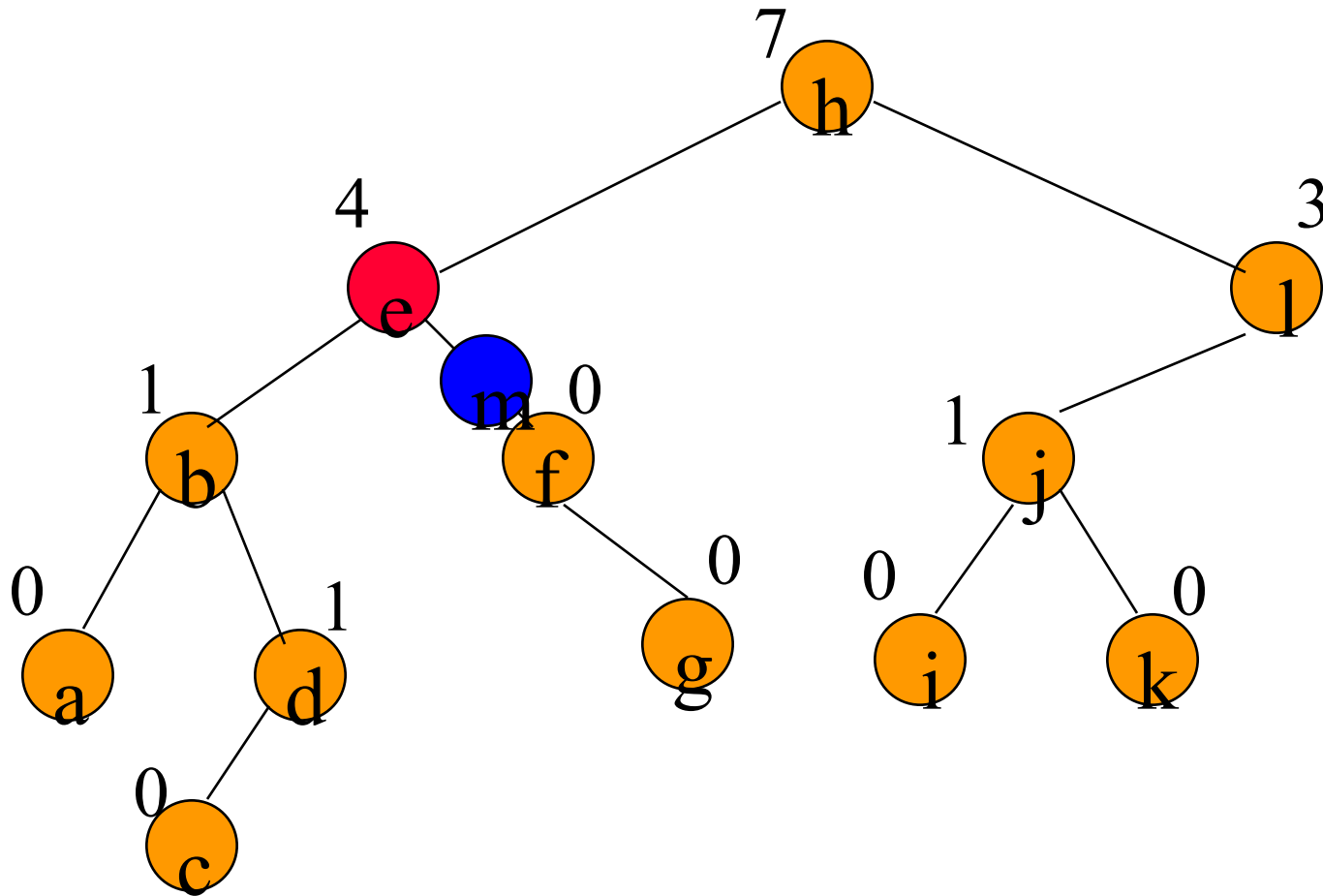
add(5, 'm')



list = [a,b,c,d,e, **m**,f,g,h,i,j,k,l]

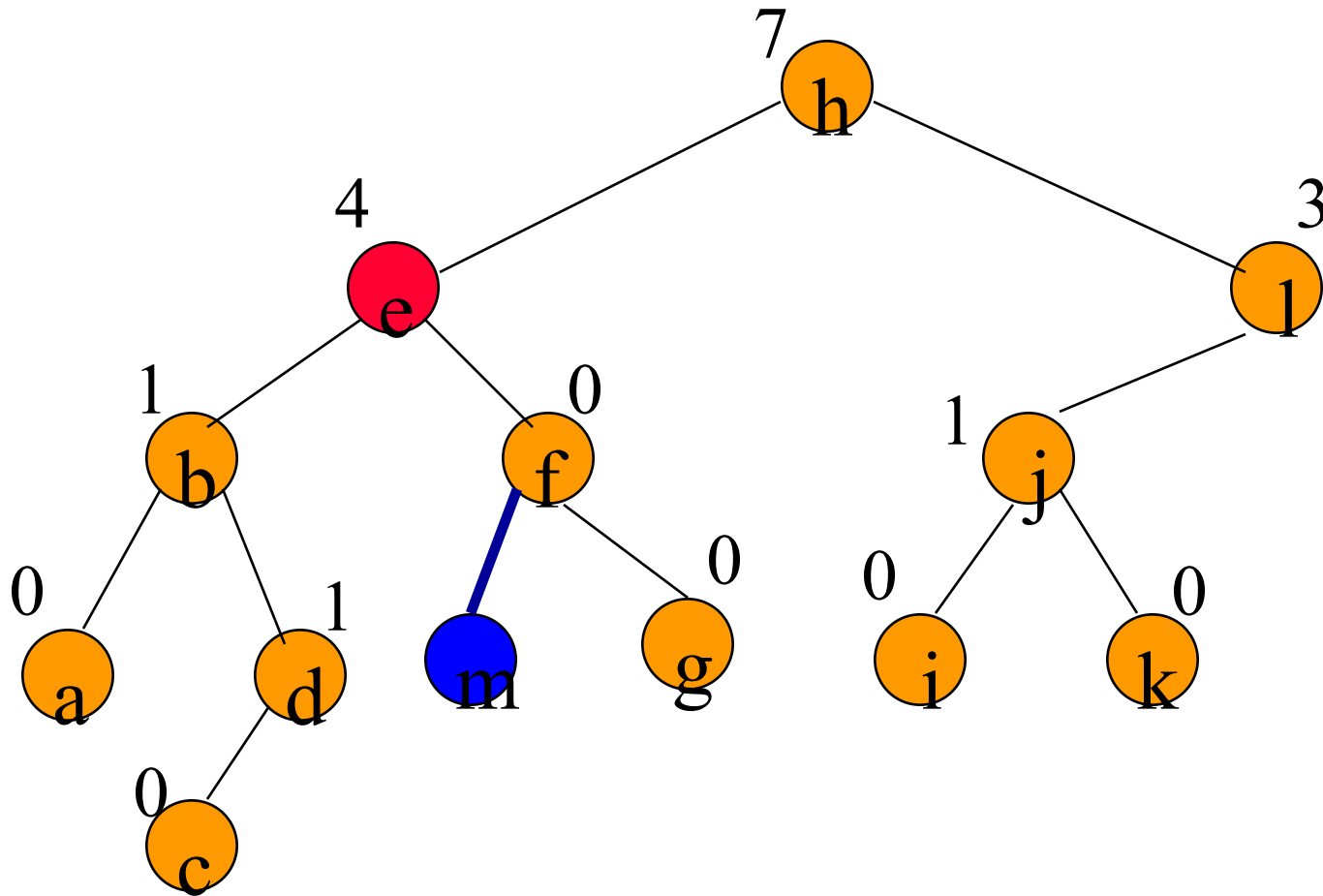
find node with element 4 (e)

add(5, 'm')



add m as right child of e; former right subtree of e becomes right subtree of m

add(5, 'm')



add m as leftmost node in right subtree
of e

add(5, 'm')

- Other possibilities exist.
- Must update some **leftSize** values on path from root to new node.
- Complexity is **$O(\text{height})$** .

- Exercise:
 - `Data IndexedBinaryTree::Search(int idx);`
 - `Bool IndexedBinaryTree::Insert(int idx, int data);`

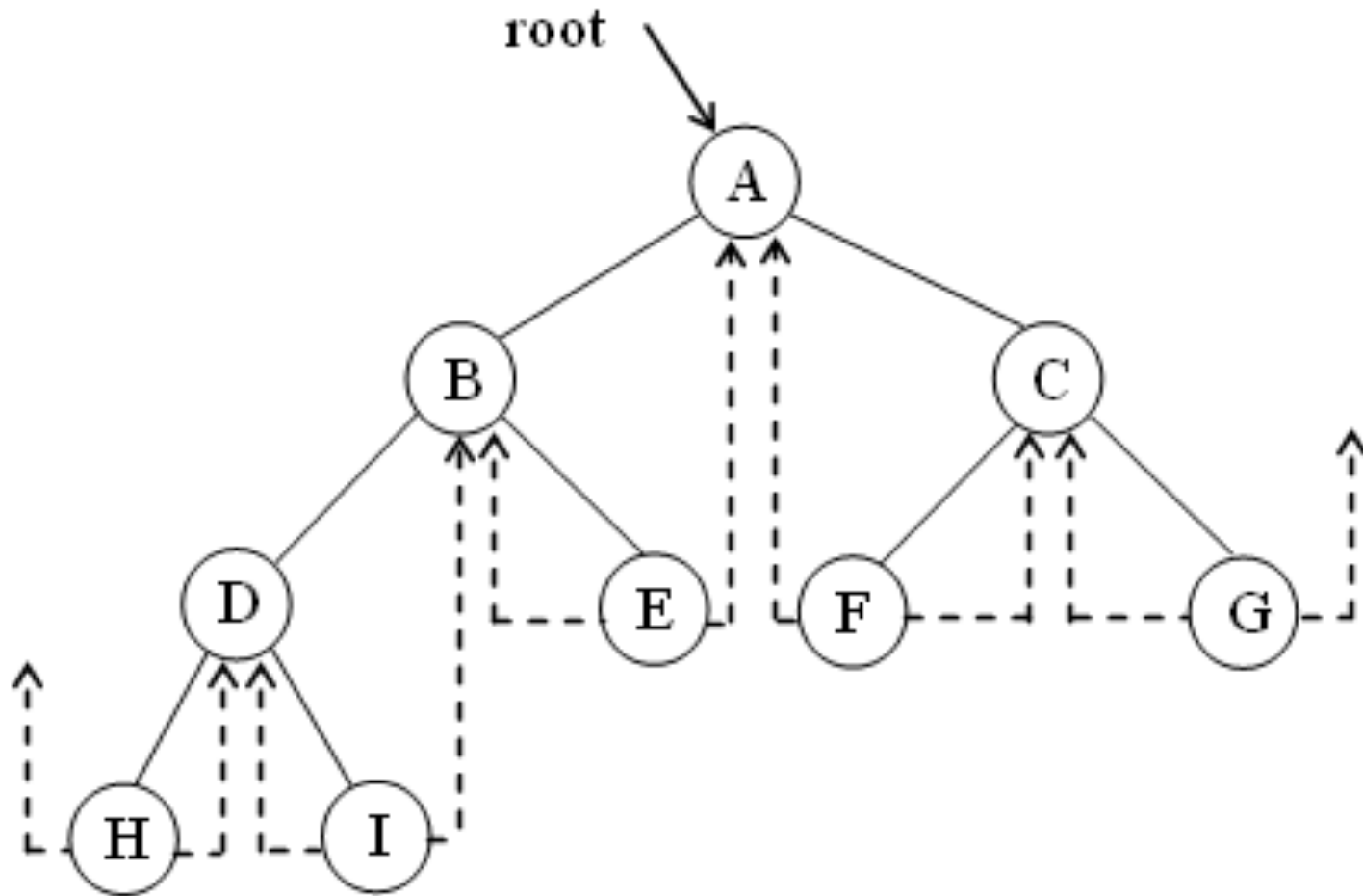
Threaded Trees

- Binary trees have a lot of wasted space:
 - the leaf nodes each have 2 null pointers
- We can use these pointers to help us in **inorder** traversals
- We have the pointers reference the **next/previous** node in an inorder traversal; called *threads*
- We need to know if a pointer is an actual link or a thread, so we keep a boolean for each pointer

The threads are constructed using the following rules:

- (1) A 0 **rightChild** field at node p is replaced by a pointer to the inorder successor of p.
- (2) A 0 **leftChild** field at node p is replaced by a pointer to the inorder predecessor of p.

The following is a threaded tree, in which node E has a predecessor thread pointing to B and a successor thread to A.



To distinguish between threads and normal pointers, add two bool fields:

- leftThread
- rightThread

If $t \rightarrow \text{leftThread} == \text{true}$, then $t \rightarrow \text{leftChild}$ contains a thread
otherwise a pointer to left child

Similar for $t \rightarrow \text{rightThread}$.

```
template <class T>
class ThreadedNode {
friend class ThreadedTree;
private:
    bool leftThread;
    ThreadedNode * leftChild;
    T data;
    ThreadedNode * rightChild;
    bool rightThread;
};
```

```
template <class T>
class ThreadedTree {
public:
    // Tree operations
    ...
private:
    ThreadedNode *root;
};
```

Let ThreadedInorderIterator be a nested class of ThreadedTree:

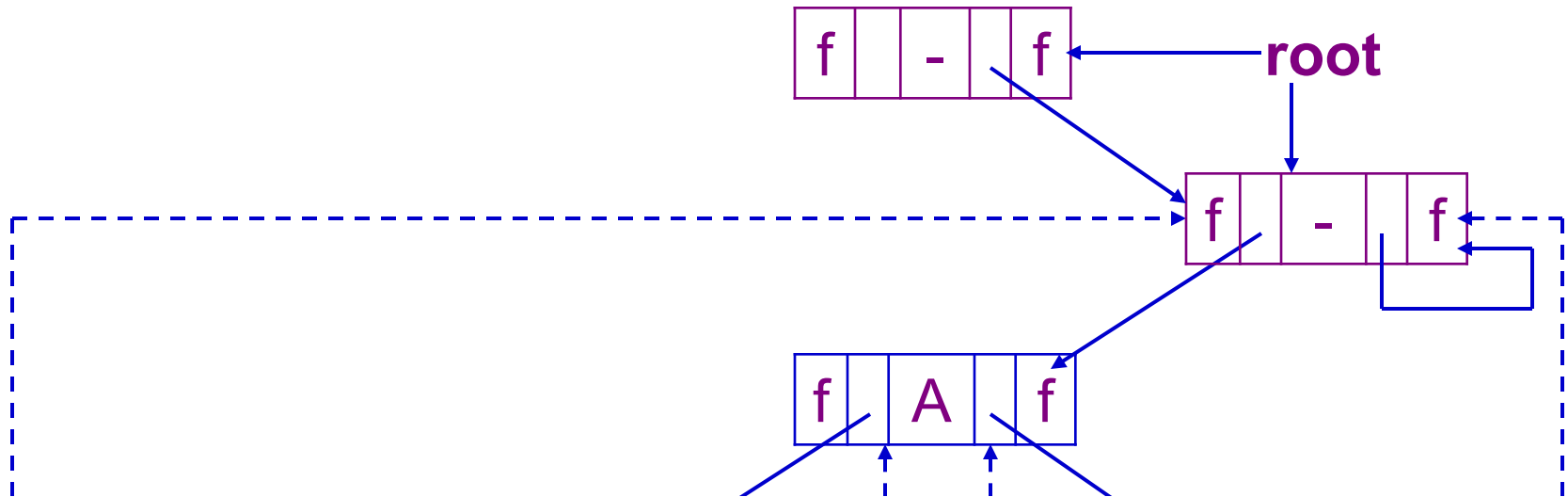
```
class ThreadedInorderIterator {  
public:  
    T* Next();  
    ThreadedInorderIterator()  
        { currentNode = root; }  
private:  
    ThreadedNode<T>* currentNode;  
};
```

To make the left thread of the first node in inorder and the right thread of the last node in inorder un-dangle, we assume a **head** node for all threaded binary tree, let the two threads point to the head.

The original tree is the **left subtree** of the head, and the rightChild of head points to the head itself.

leftThread	leftChild	data	rightChild	rightThread
true				false

An empty threaded binary tree



we can see:

- (1) The inorder successor of the head node is the first node in inorder;
- (2) The inorder successor of the last node in inorder is the head node.



memory representation of threaded tree

Inorder Traversal of a Threaded Binary Tree

Observe:

(1) If $x \rightarrow \text{rightThread} == \text{true}$

the inorder successor of x is

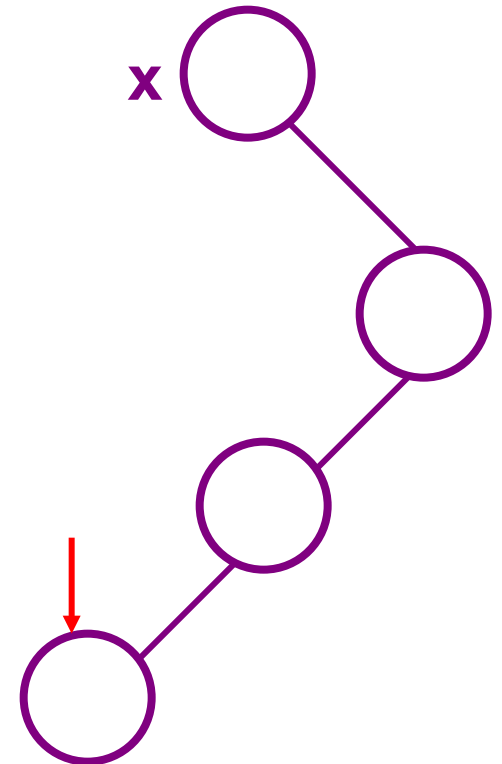
$x \rightarrow \text{rightChild};$

(2) If $x \rightarrow \text{rightThread} == \text{false}$

the inorder successor of x is obtained by

following a path of `leftChild` from the right
child of x until

a node with `leftThread == true` is reached.



Thus we have:

```
T* ThreadedInorderIterator::Next()
{ // Return the inorder successor of currentNode in a
  threaded
    // binary tree
    ThreadedNode<T>* temp=currentNode→rightChild;
    if (! currentNode→rightThread)
        while (!temp→leftThread)
            temp=temp→leftChild;
    currentNode=temp;
    if (currentNode==root)
        return 0; //no next
    else
        return &currentNode→data;
}
```

Note that when `currentNode == root`, `Next()` return the 1st node of inorder, thus we can use the following function to do an inorder traversal of a threaded binary tree:

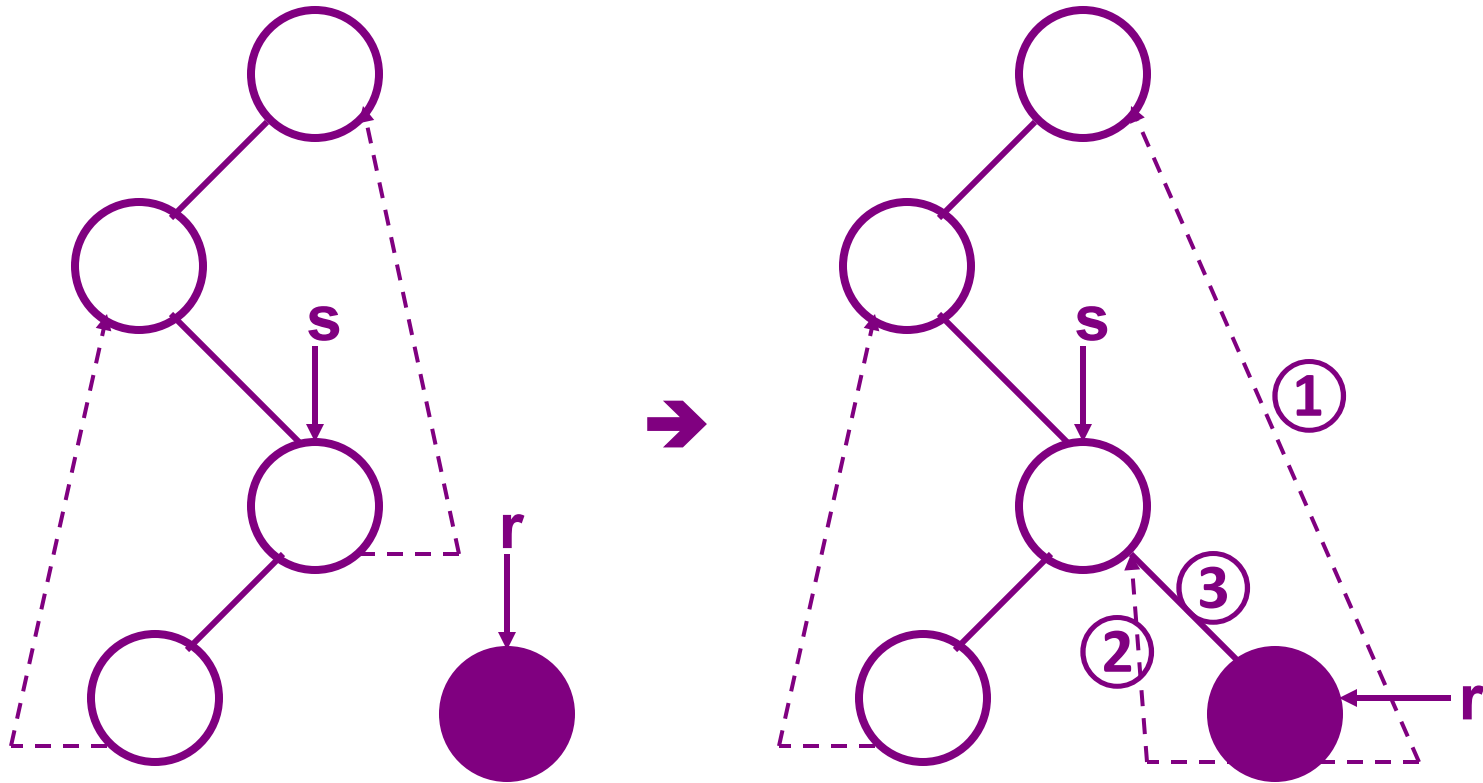
```
template <class T>  
void ThreadedTree::Inorder()  
{  
    ThreadedInorderiterator ti;  
    for (T* p = ti.Next(); p; p = ti.Next())  
        Visit(*p);  
}
```

Inserting a Node into a Threaded Binary Tree

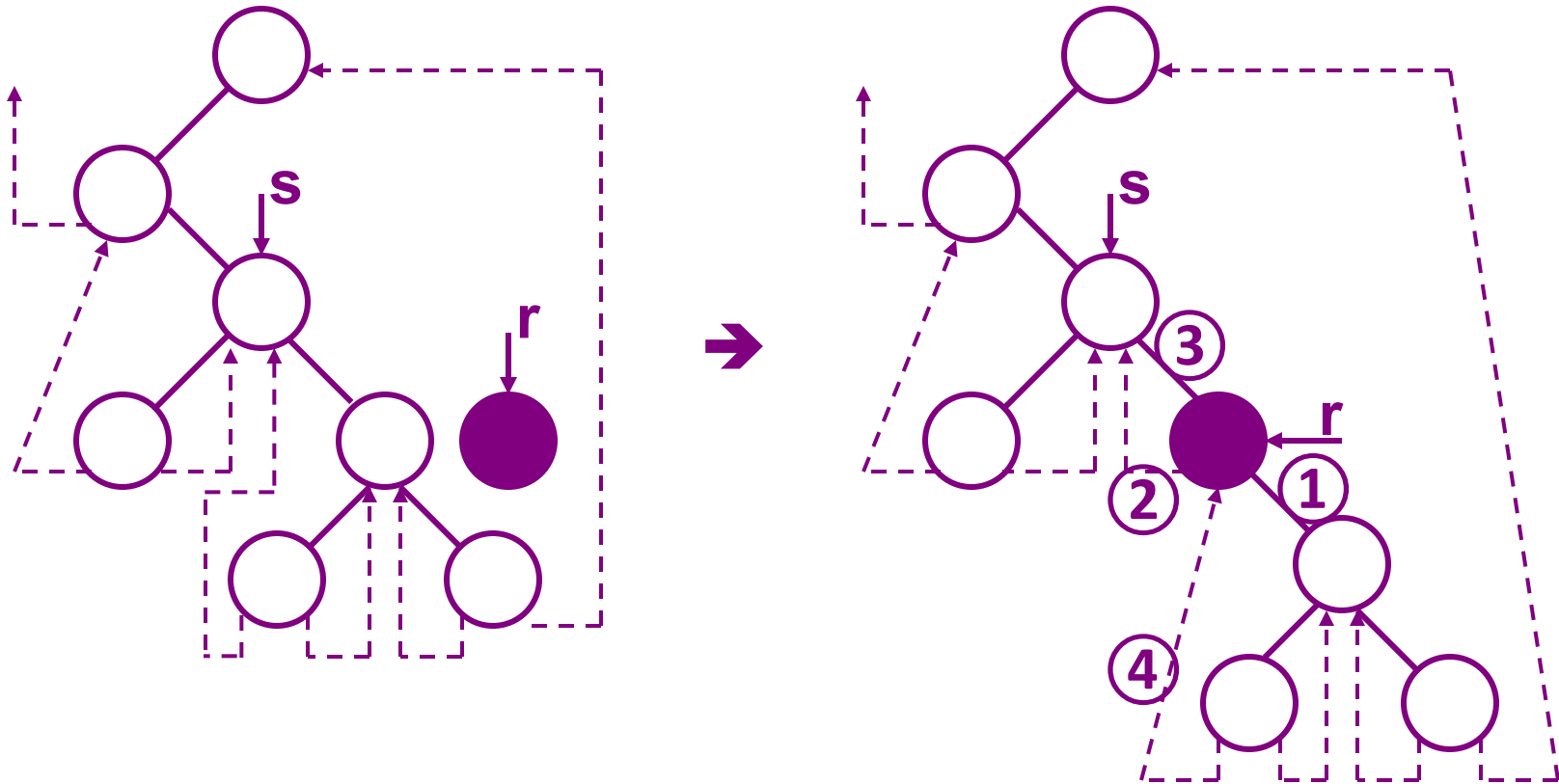
Insertion into a threaded tree provides the function for growing threaded tree.

We shall study only the case of inserting r as the **right child** of s . The left child case is similar.

(1) If $s \rightarrow \text{rightThread} == \text{true}$, as:



(2) If $s \rightarrow \text{rightThread} == \text{false}$, as:



In both (1) and (2), actions ①,②, ③ are the same,④ is special for (2).

```
template <class T>
```

```
void ThreadedTree<T>::InsertRight(ThreadedNode<T>* s,  
                                   ThreadedNode<T>* r)
```

```
{ // insert r as the right child of s
```

```
    r→rightChild=s→rightChild;           // ①
```

```
    r→rightThread=s→rightThread;         // ① note s!=t.root,
```

```
    r→leftChild=s;                        // ②
```

```
    r→leftThread=true;                    // ②
```

```
    s→rightChild=r;                       // ③
```

```
    s→rightThread=false;                  // ③
```

```
    if (! r→rightThread) { // case (2)
```

```
        ThreadedNode<T>* temp=InorderSucc(r); // ④
```

```
        temp→leftChild=r;                 // ④
```

```
    }
```

```
}
```

Exercises: P277-1, P278-4

Given a binary tree, make it an inorder threaded binary tree.

Priority Queues



Two kinds of priority queues:

- Min priority queue.
- Max priority queue.

Min Priority Queue

- Collection of elements.
- Each element has a priority or key.
- Supports following operations:
 - isEmpty
 - size
 - add/put an element into the priority queue
 - get element with min priority
 - remove element with min priority

Max Priority Queue

- Collection of elements.
- Each element has a priority or key.
- Supports following operations:
 - isEmpty
 - size
 - add/put an element into the priority queue
 - get element with **max** priority
 - remove element with **max** priority

ADT MaxHeap

- **template <class T> class MaxPQ {**
- **public:**
- **virtual ~MaxPQ { }** // virtual destructor
- **virtual bool IsEmpty() const = 0;**
- // return **true** iff the priority queue is empty
- **virtual const T& Top() const = 0;**
- // return reference to the max element
- **virtual void Push(const T&) = 0;**
- // add an element to the priority queue
- **virtual void Pop() = 0;**
- // delete the max element
- **};**

Complexity Of Operations

First idea:

Linear List

Unordered Linear List

Ordered Linear List

Complexity

isEmpty

Push

Pop

Complexity Of Operations

Two good implementations are heaps and leftist trees.

isEmpty, size, and get $\Rightarrow O(1)$ time

put and remove $\Rightarrow O(\log n)$ time

where n is the size of the priority queue

Applications

Sorting

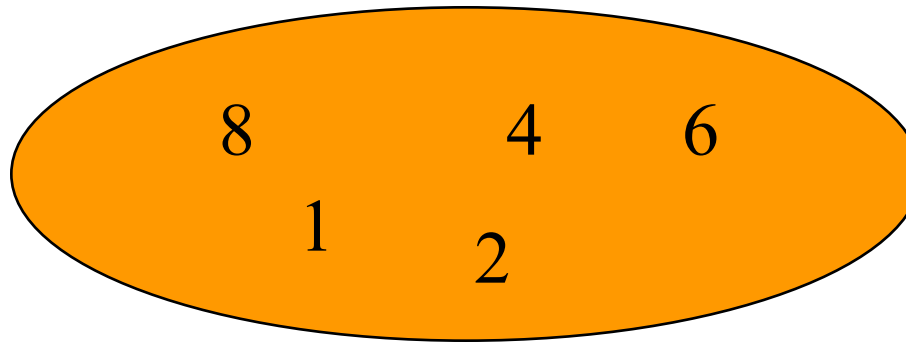
- use element key as priority
- put elements to be sorted into a priority queue
- extract elements in priority order
 - if a min priority queue is used, elements are extracted in ascending order of priority (or key)
 - if a max priority queue is used, elements are extracted in descending order of priority (or key)

Sorting Example

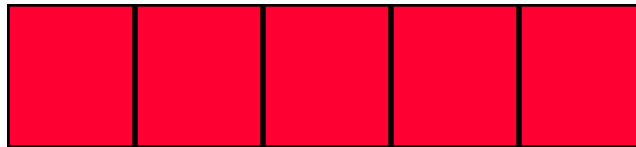
Sort five elements whose keys are 6, 8, 2, 4, 1 using a max priority queue.

- Put the five elements into a max priority queue.
- Do five remove max operations placing removed elements into the sorted array from right to left.

After Putting Into Max Priority Queue

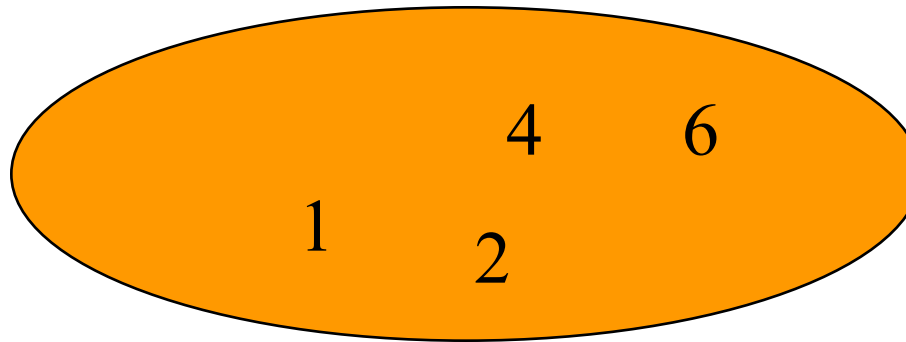


Max Priority
Queue



Sorted Array

After First Remove Max Operation

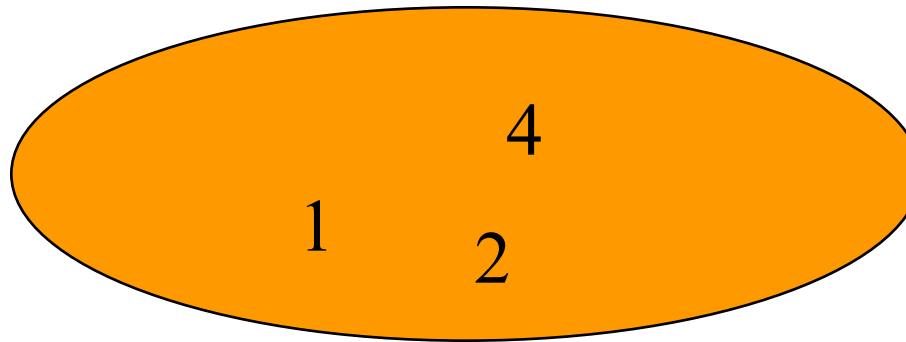


Max Priority
Queue



Sorted Array

After Second Remove Max Operation

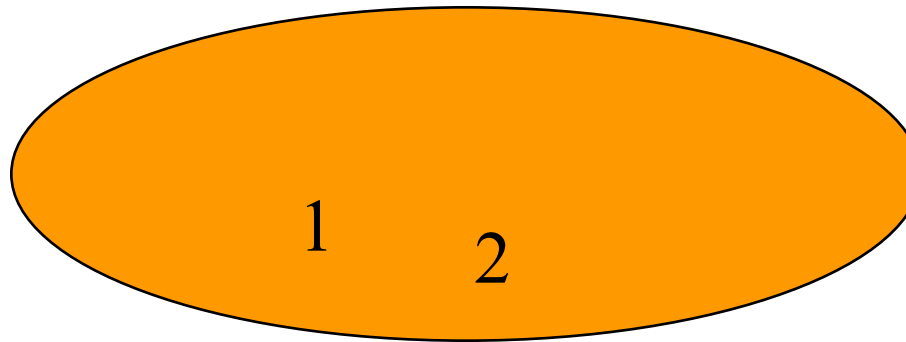


Max Priority
Queue



Sorted Array

After Third Remove Max Operation

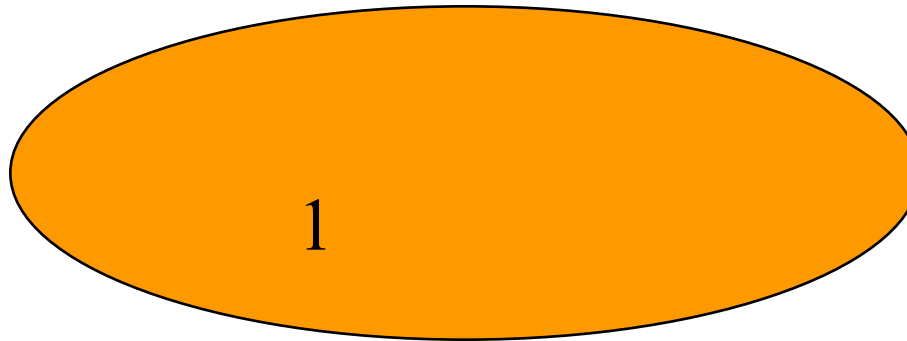


Max Priority
Queue



Sorted Array

After Fourth Remove Max Operation

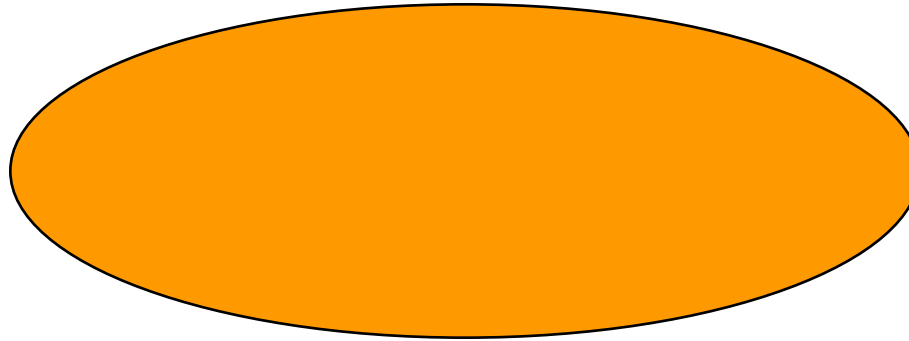


Max Priority
Queue



Sorted Array

After Fifth Remove Max Operation



Max Priority
Queue

1	2	4	6	8
---	---	---	---	---

Sorted Array

Complexity Of Sorting

Sort n elements.

- n put operations $\Rightarrow O(n \log n)$ time.
- n remove max operations $\Rightarrow O(n \log n)$ time.
- total time is $O(n \log n)$.
- compare with sort methods $O(n^2)^*$

Machine Scheduling

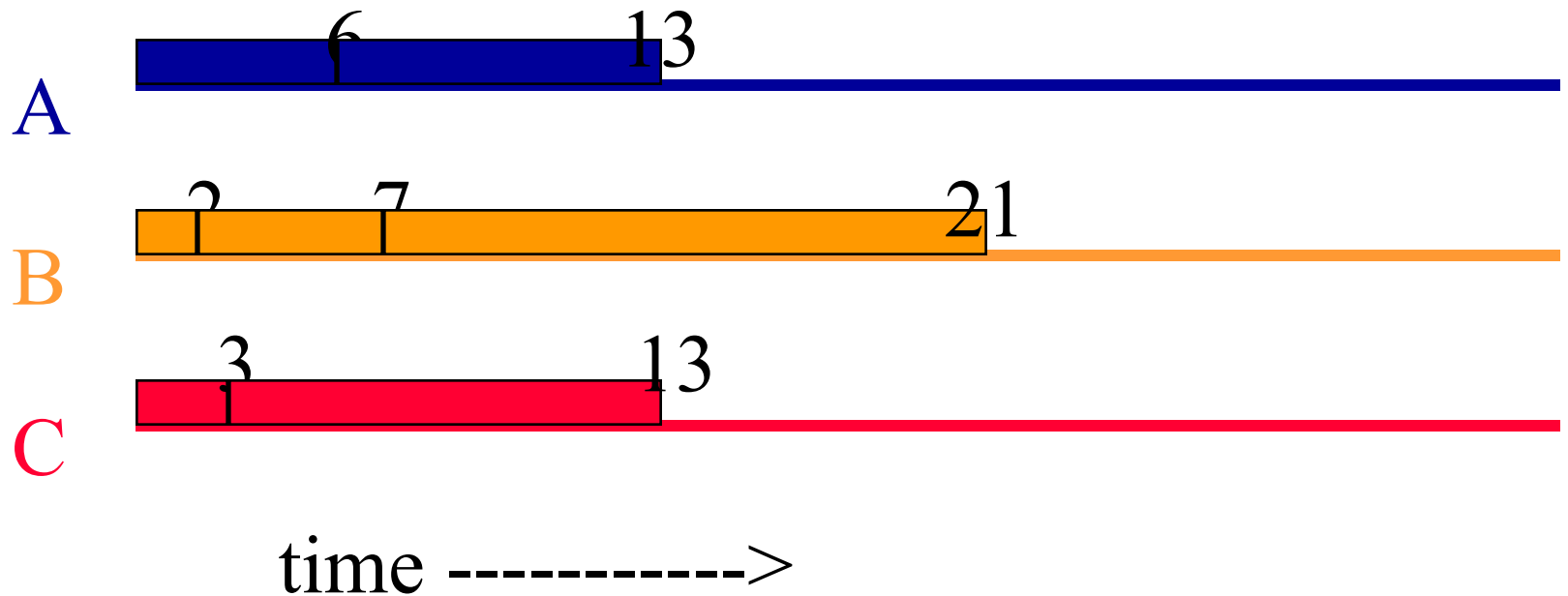
- m identical machines
- n jobs/tasks to be performed
- assign jobs to machines so that the time at which the last job completes is minimum

Machine Scheduling Example

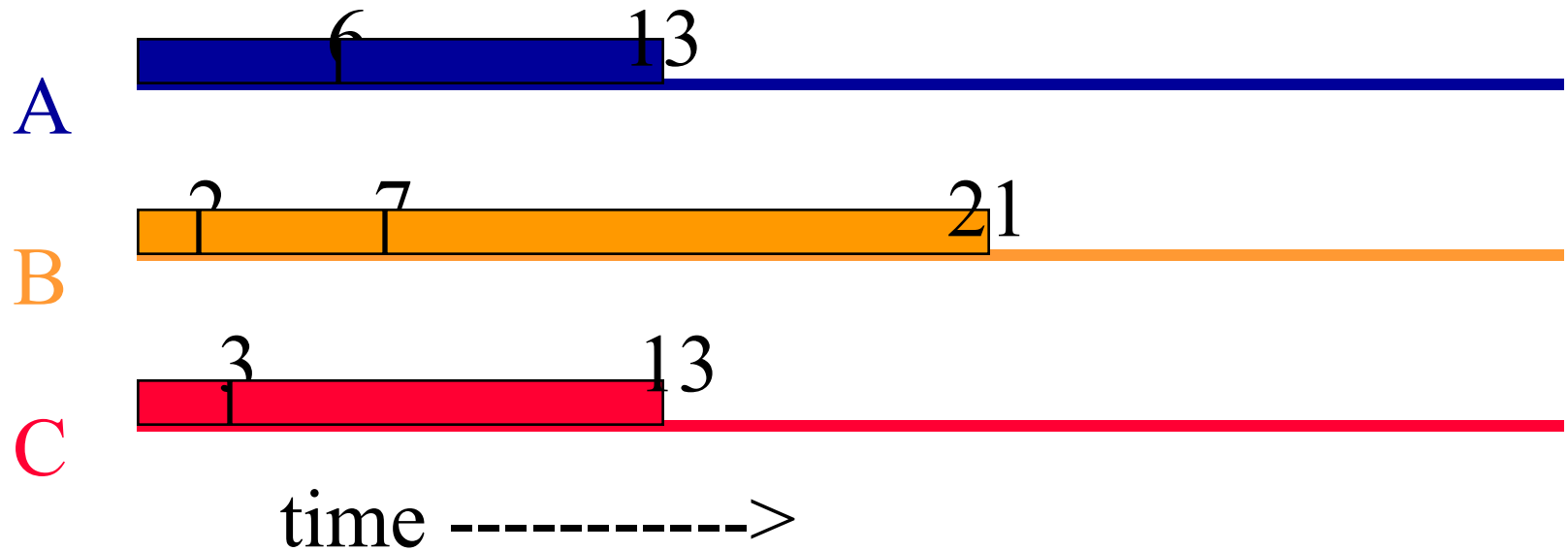
3 machines and 7 jobs

job times are [6, 2, 3, 5, 10, 7, 14]

possible schedule



Machine Scheduling Example



Finish time = 21

Objective: Find schedules with minimum finish time.

LPT Schedules

Longest Processing Time first.

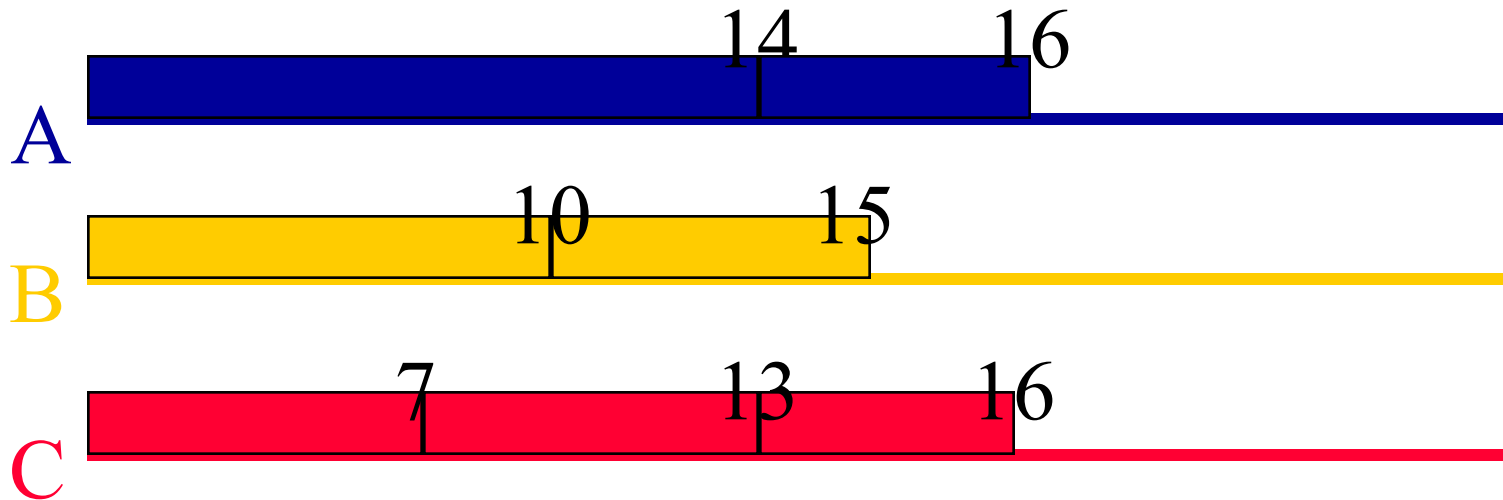
Jobs are scheduled in the order

14, 10, 7, 6, 5, 3, 2

Each job is scheduled on the machine on which it finishes earliest.

LPT Schedule

[14, 10, 7, 6, 5, 3, 2]



Finish time is 16!

LPT Schedule

- LPT rule does not guarantee minimum finish time schedules.
- Usually LPT finish time is much closer to minimum finish time.
- Minimum finish time scheduling is NP-hard.

NP-hard Problems

- Infamous class of problems for which no one has developed a polynomial time algorithm.
- That is, no algorithm whose complexity is $O(n^k)$ for any constant k is known for any NP-hard problem.
- The class includes thousands of real-world problems.
- Highly unlikely that any NP-hard problem can be solved by a polynomial time algorithm.

NP-hard Problems

- Since even polynomial time algorithms with degree $k > 3$ (say) are not practical for large n , we must change our expectations of the algorithm that is used.
- Usually develop fast heuristics for NP-hard problems.
 - Algorithm that gives a solution close to best.
 - Runs in acceptable amount of time.
- LPT rule is good heuristic for minimum finish time scheduling.

Complexity Of LPT Scheduling

- Sort jobs into decreasing order of task time.
 - $O(n \log n)$ time (n is number of jobs)
- Schedule jobs in this order.
 - assign job to machine that becomes available first
 - must find minimum of m (m is number of machines) finish times
 - takes $O(m)$ time using simple strategy
 - so need $O(mn)$ time to schedule all n jobs.

Using A Min Priority Queue

- Min priority queue has the finish times of the **m** machines.
- Initial finish times are all **0**.
- To schedule a job, remove machine with minimum finish time from the priority queue.
- Update the finish time of the selected machine and put the machine back into the priority queue.

Using A Min Priority Queue

- m put operations to initialize priority queue
- 1 remove min and 1 put to schedule each job
- each put and remove min operation takes $O(\log m)$ time
- time to schedule is $O(n \log m)$
- overall time is

$$O(n \log n + n \log m) = O(n \log (mn))$$

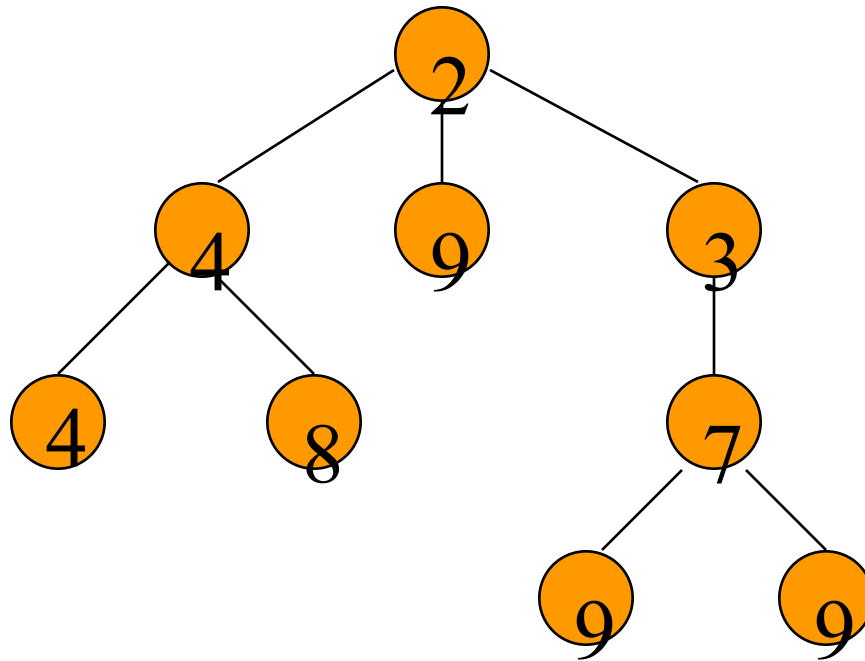
Min Tree Definition

Each tree node has a value.

Value in any node is the minimum value in the subtree for which that node is the root.

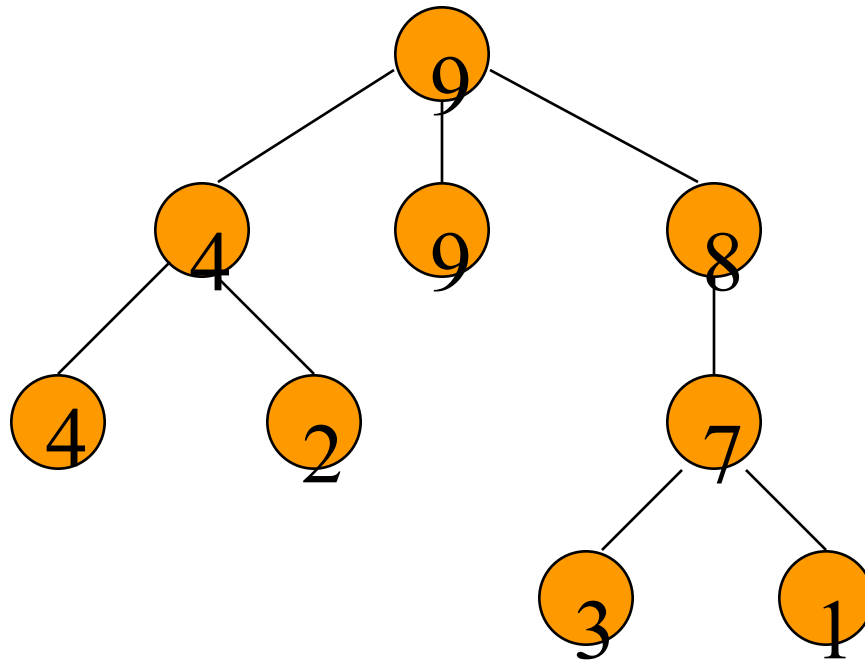
Equivalently, no descendent has a smaller value.

Min Tree Example



Root has minimum element.

Max Tree Example

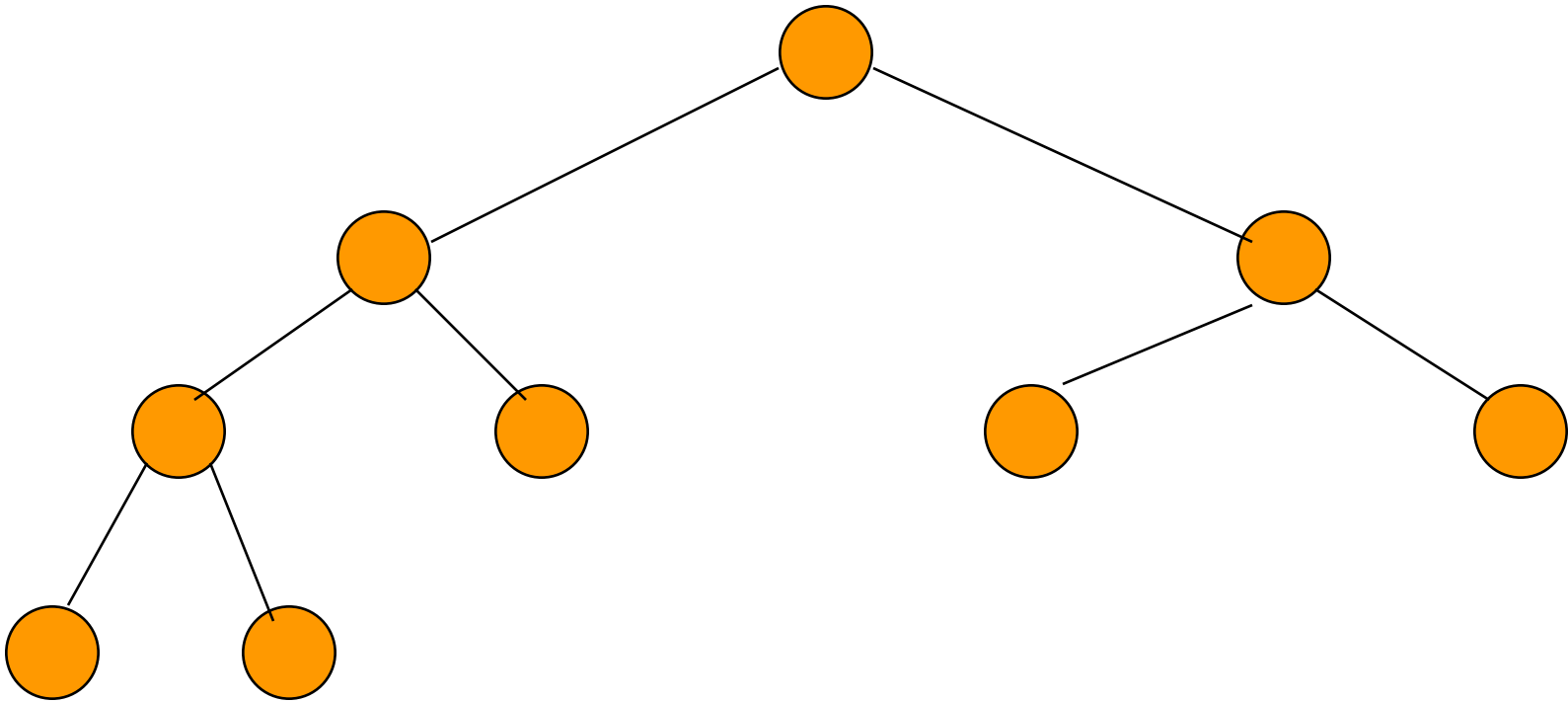


Root has maximum element.

Min Heap Definition

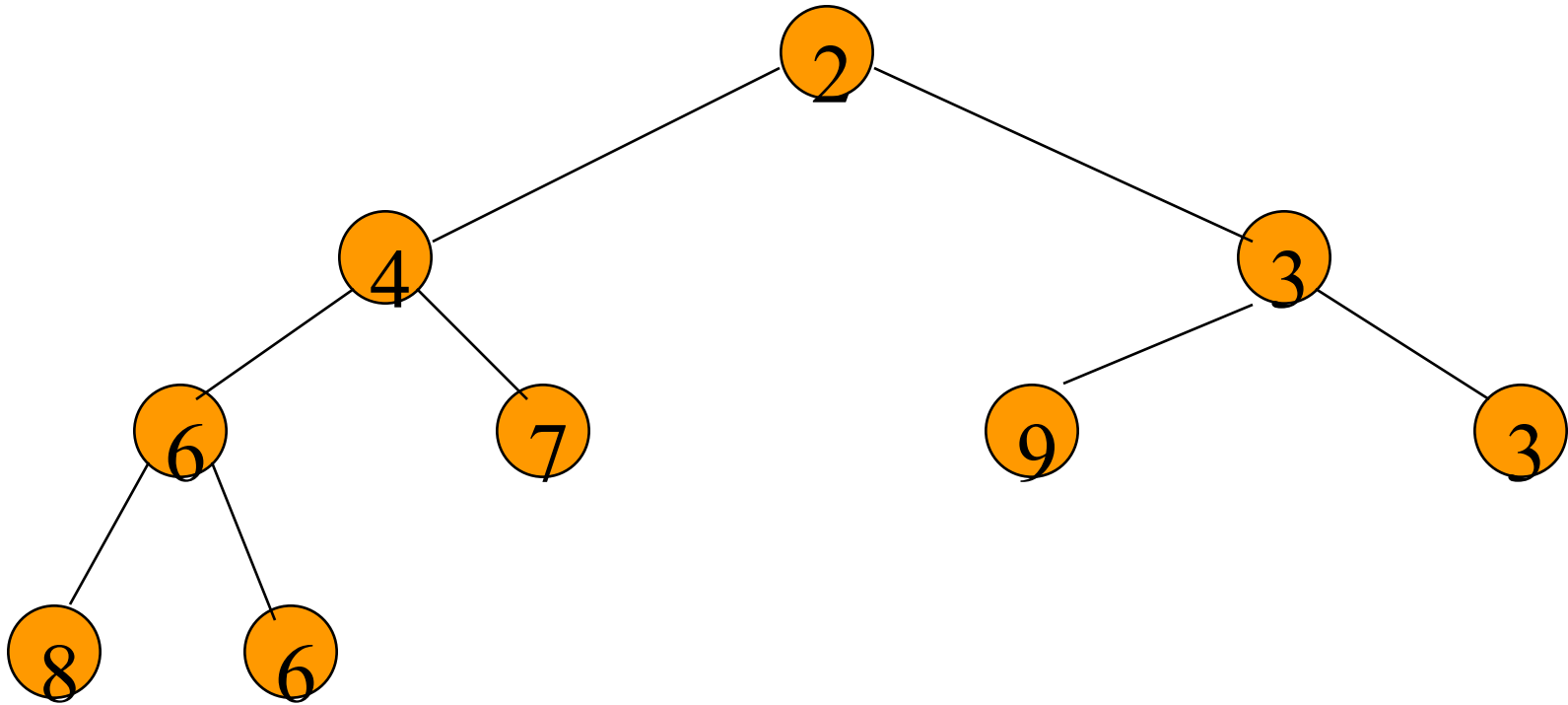
- complete binary tree
- min tree

Min Heap With 9 Nodes



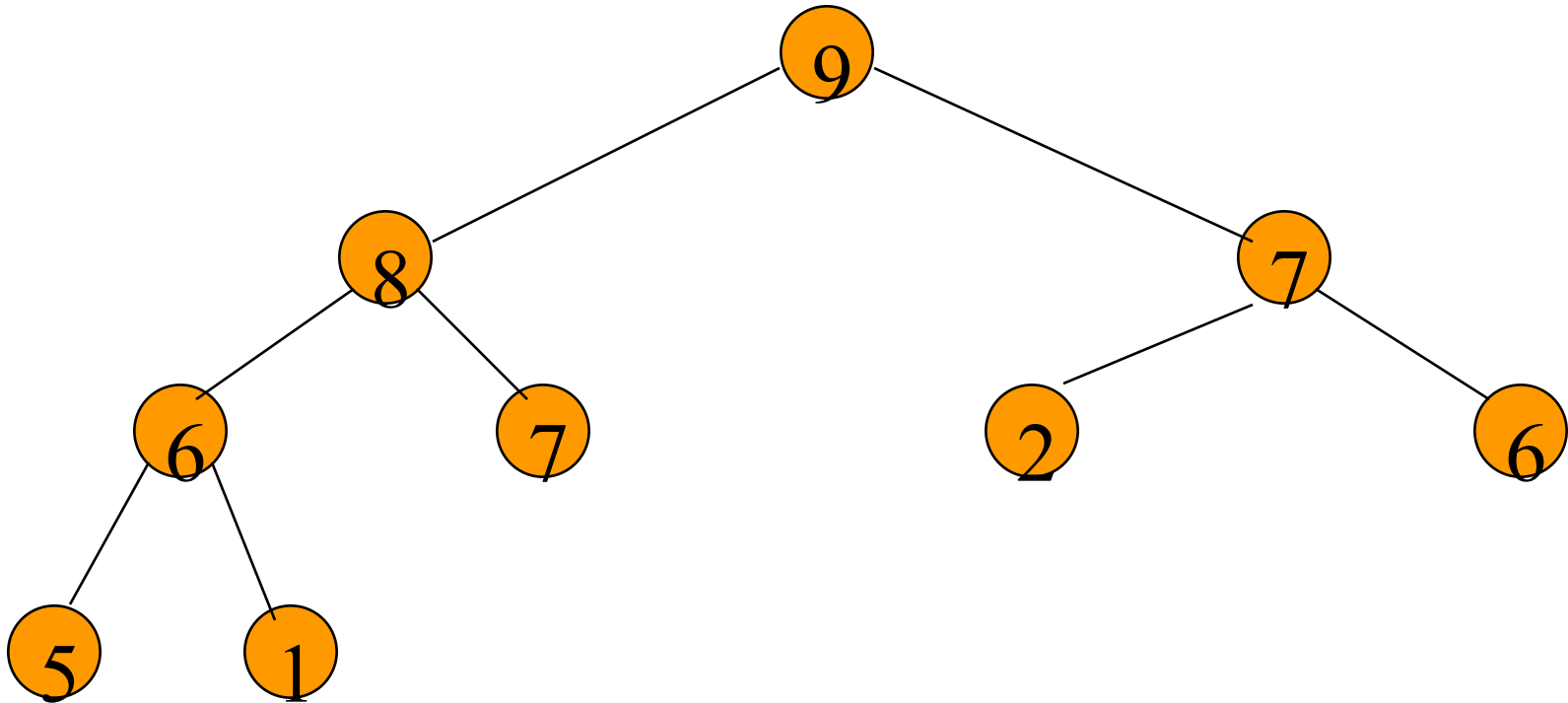
Complete binary tree with 9 nodes.

Min Heap With 9 Nodes



Complete binary tree with 9 nodes
that is also a min tree.

Max Heap With 9 Nodes



Complete binary tree with 9 nodes
that is also a max tree.

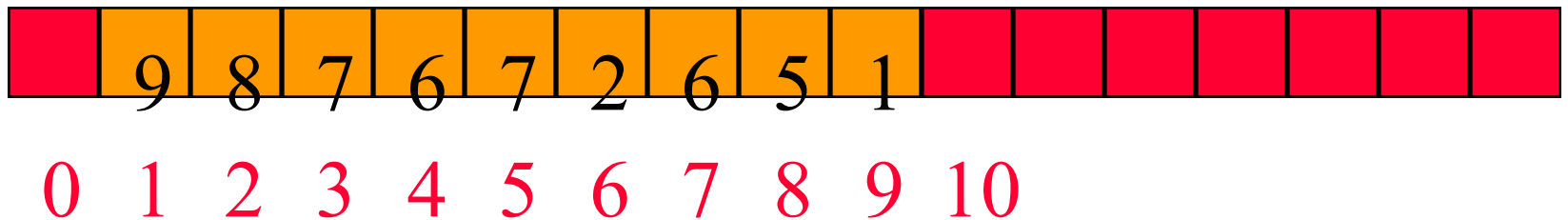
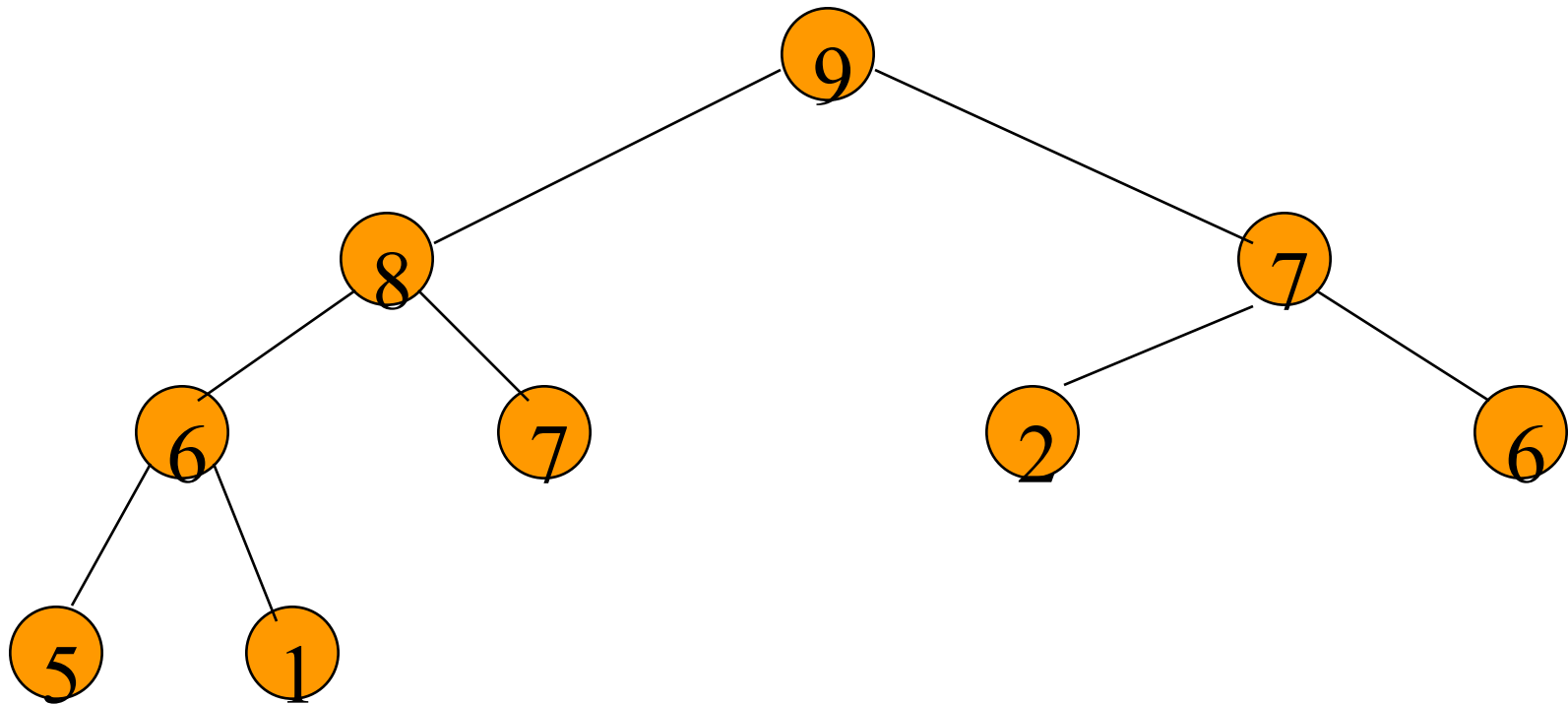
Heap Height

Since a heap is a complete binary tree, the height of an **n** node heap is $\log_2 (n+1)$.

How to represent a Heap?

```
template <class T>  
class MaxHeap: public MaxPQ <T>  
{  
public:  
    MaxHeap (int theCapacity=10);  
    bool IsEmpty () { return heapSize==0;}  
    const T& Top() const;  
    void Push(const T&);  
    void Pop();  
private:  
    T* heap;           // element array  
    int heapSize;      // number of elements in heap  
    int capacity;      // size of the array heap  
};
```

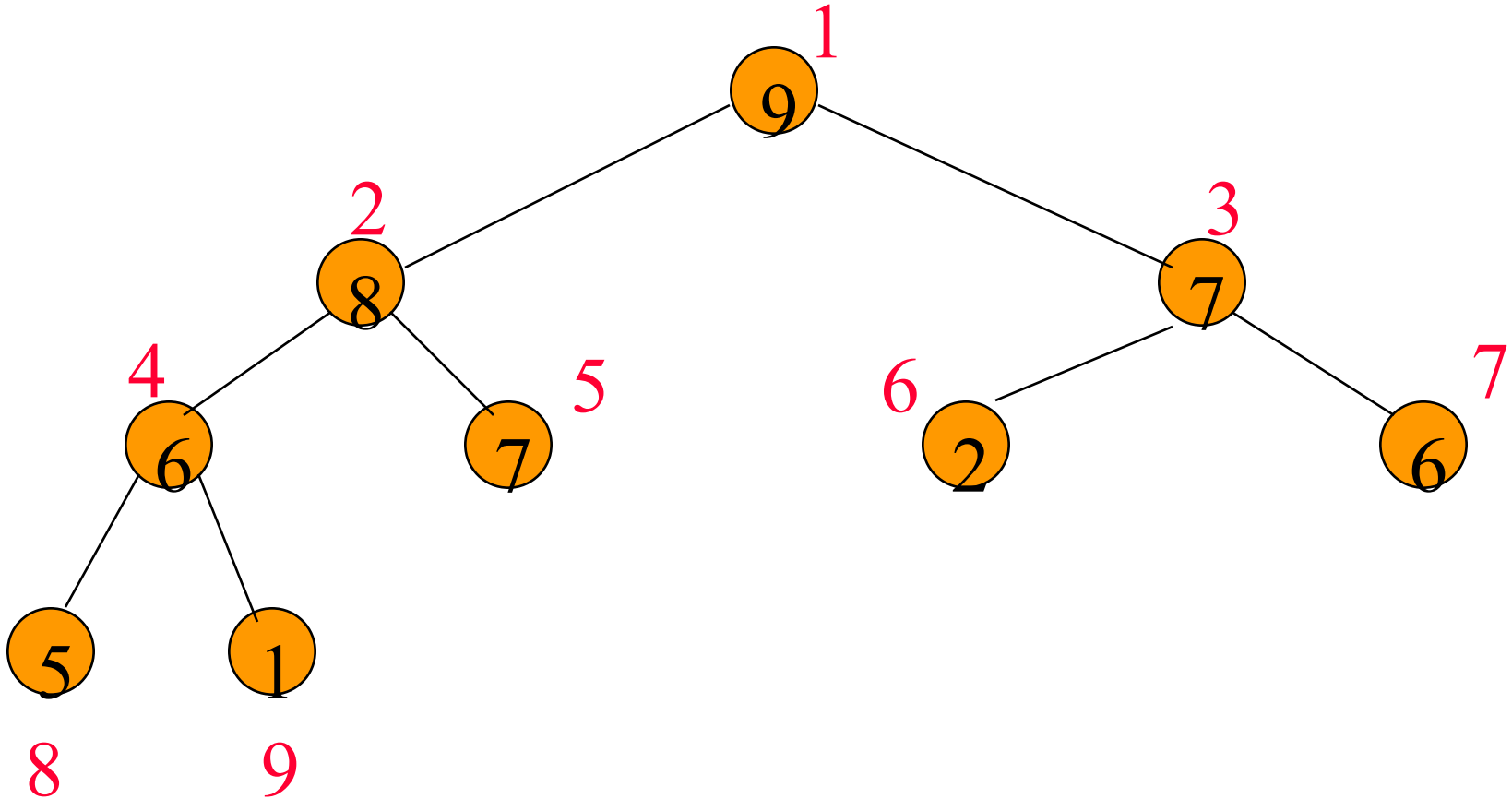
A Heap Is Efficiently Represented As An Array



```
template <class T>
MaxHeap<T>::MaxHeap (int theCapacity=10)
{ //constructor
    if (theCapacity < 1) throw “Capacity must be >= 1”;
    capacity = theCapacity;
    heapSize = 0;
    heap = new T[capacity+1]; //heap[0] not used
}
```

```
template <class T>
Inline T& MaxHeap<T>::Top()
{
    if (IsEmpty()) throw “The heap is empty”;
    return heap[1];
}
```

Moving Up And Down A Heap

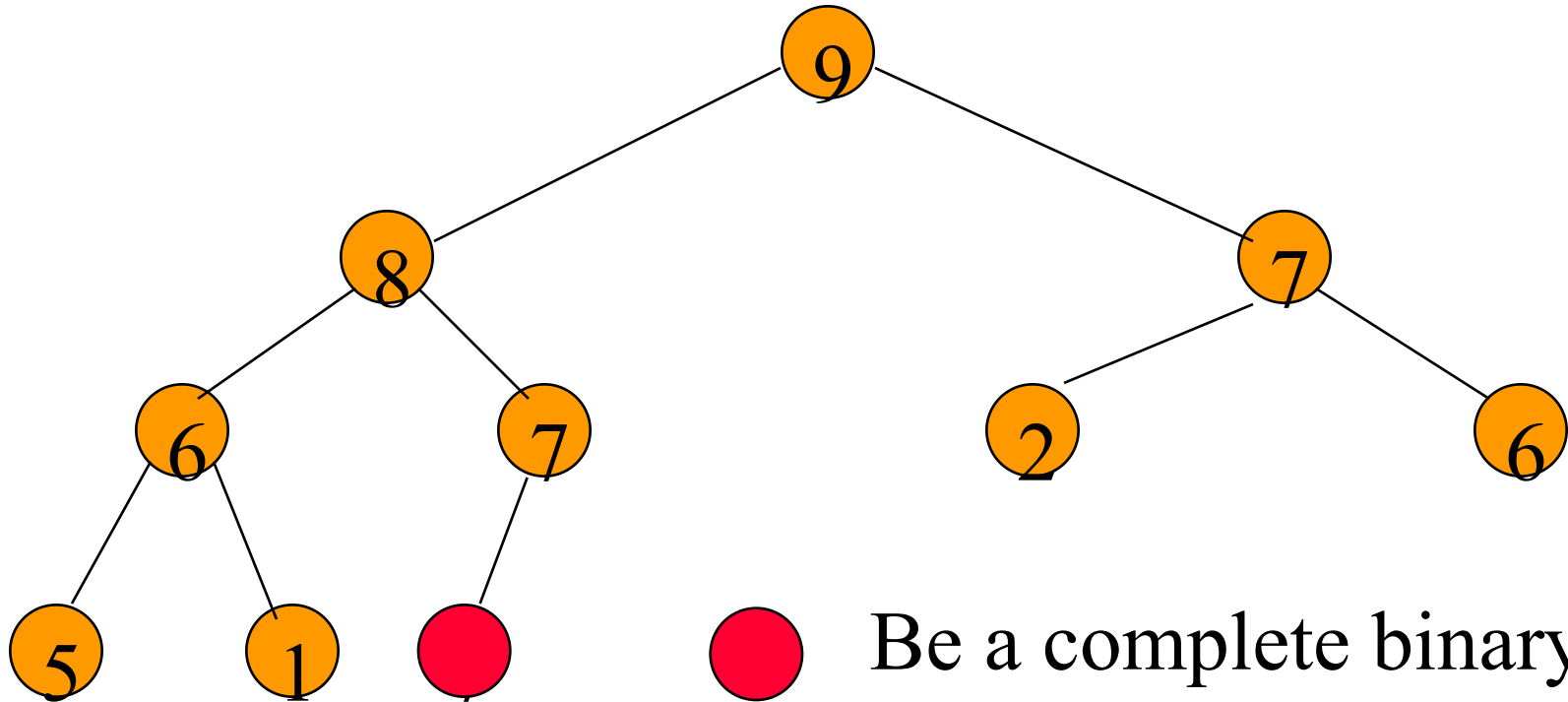


Putting An Element Into A Max Heap

Complete Binary Tree

Max Tree

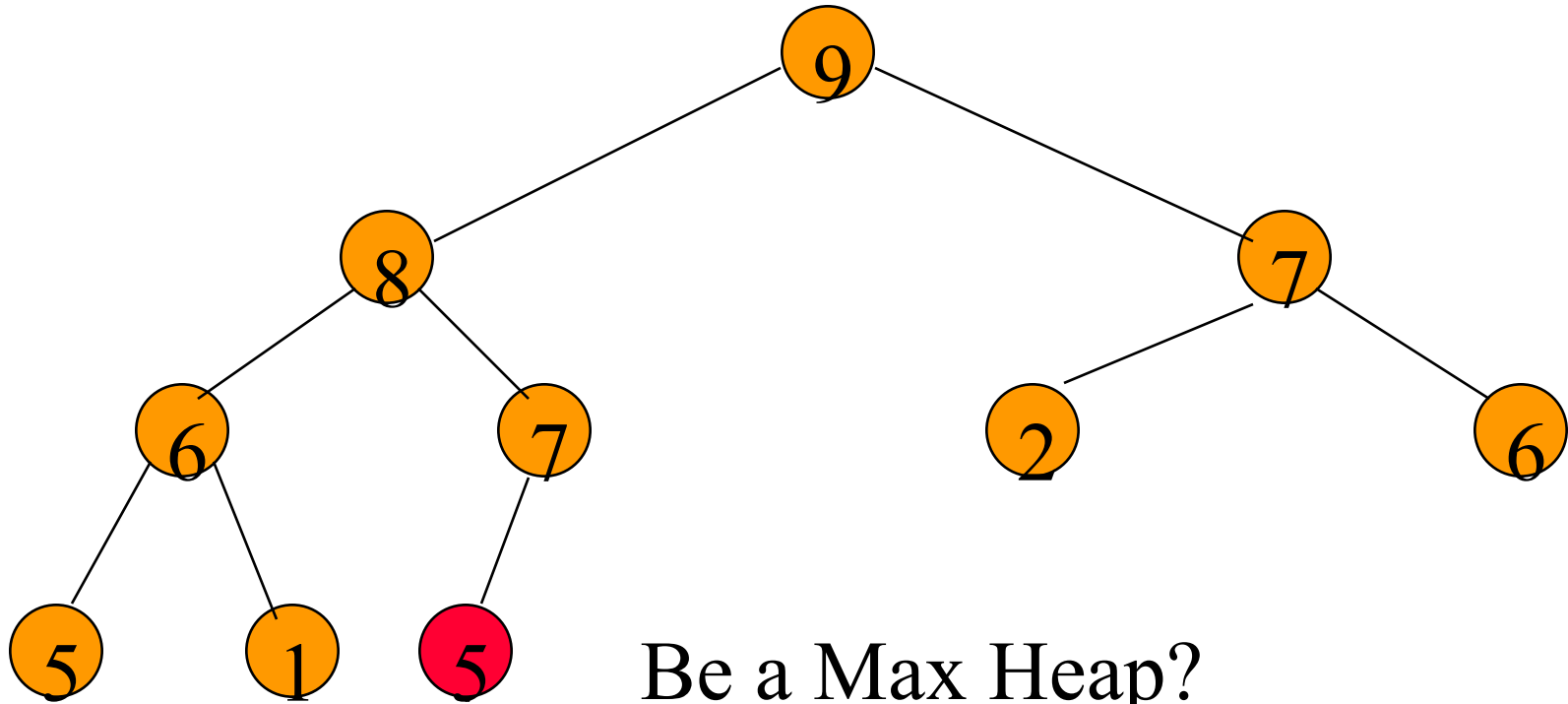
Putting An Element Into A Max Heap



Be a complete binary tree?

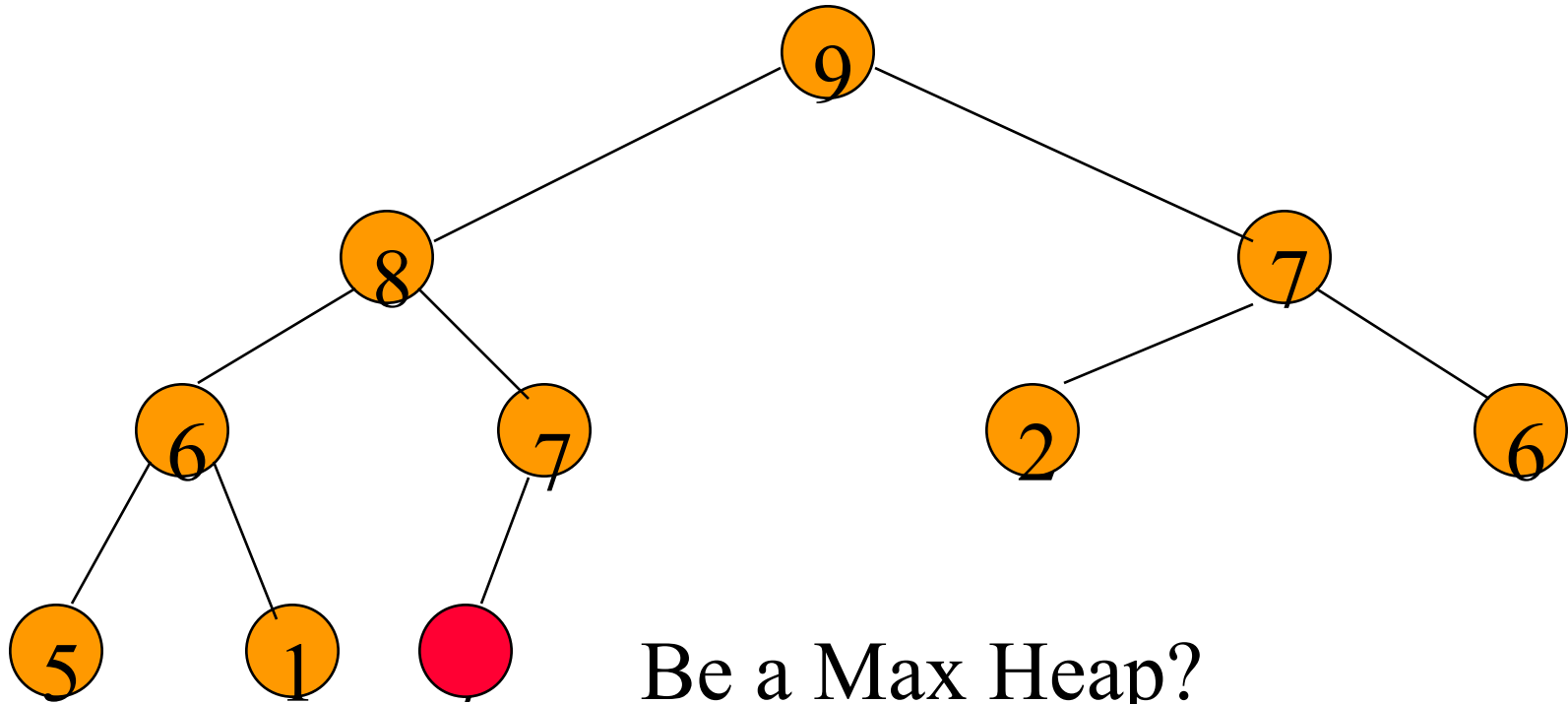
Complete binary tree with 10 nodes.

Putting An Element Into A Max Heap



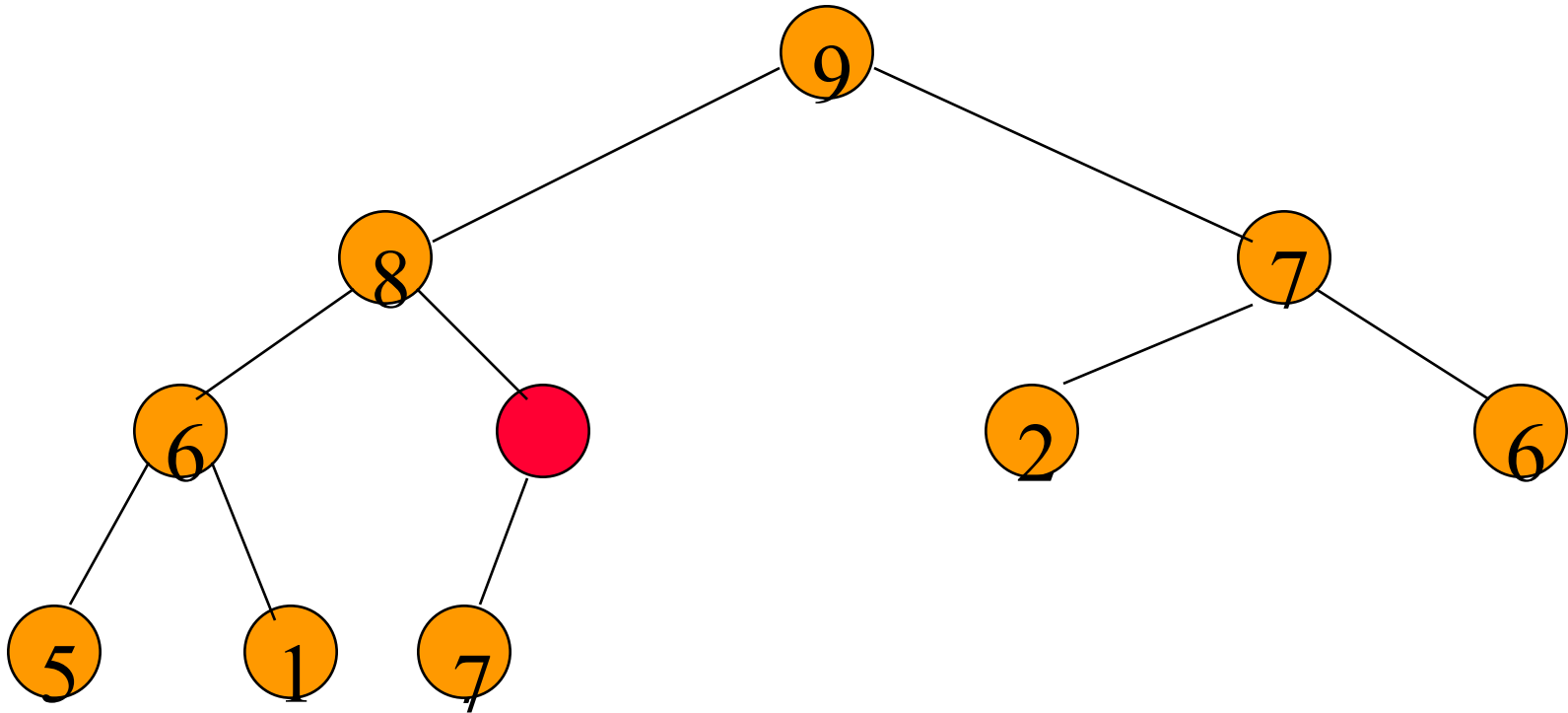
New element is 5.

Putting An Element Into A Max Heap



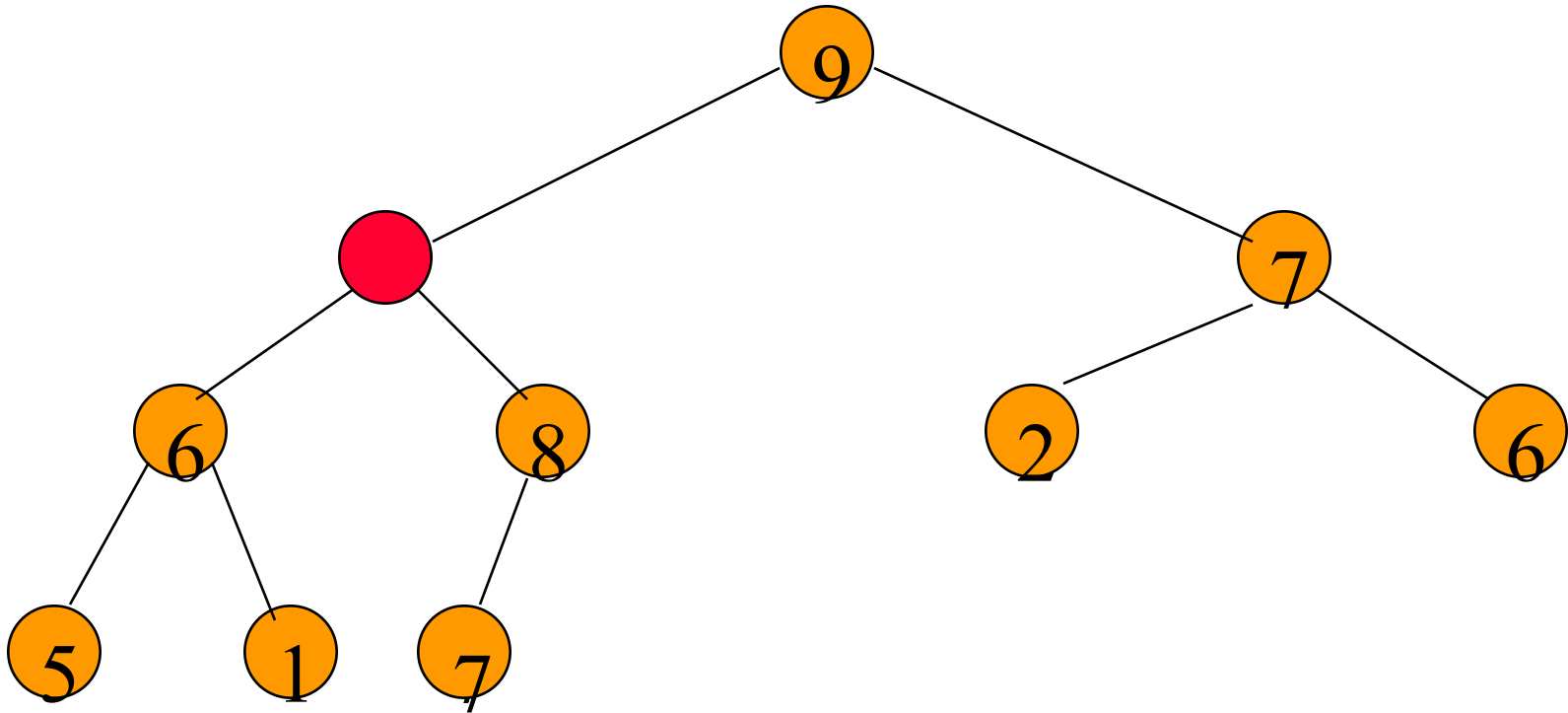
New element is 20.

Putting An Element Into A Max Heap



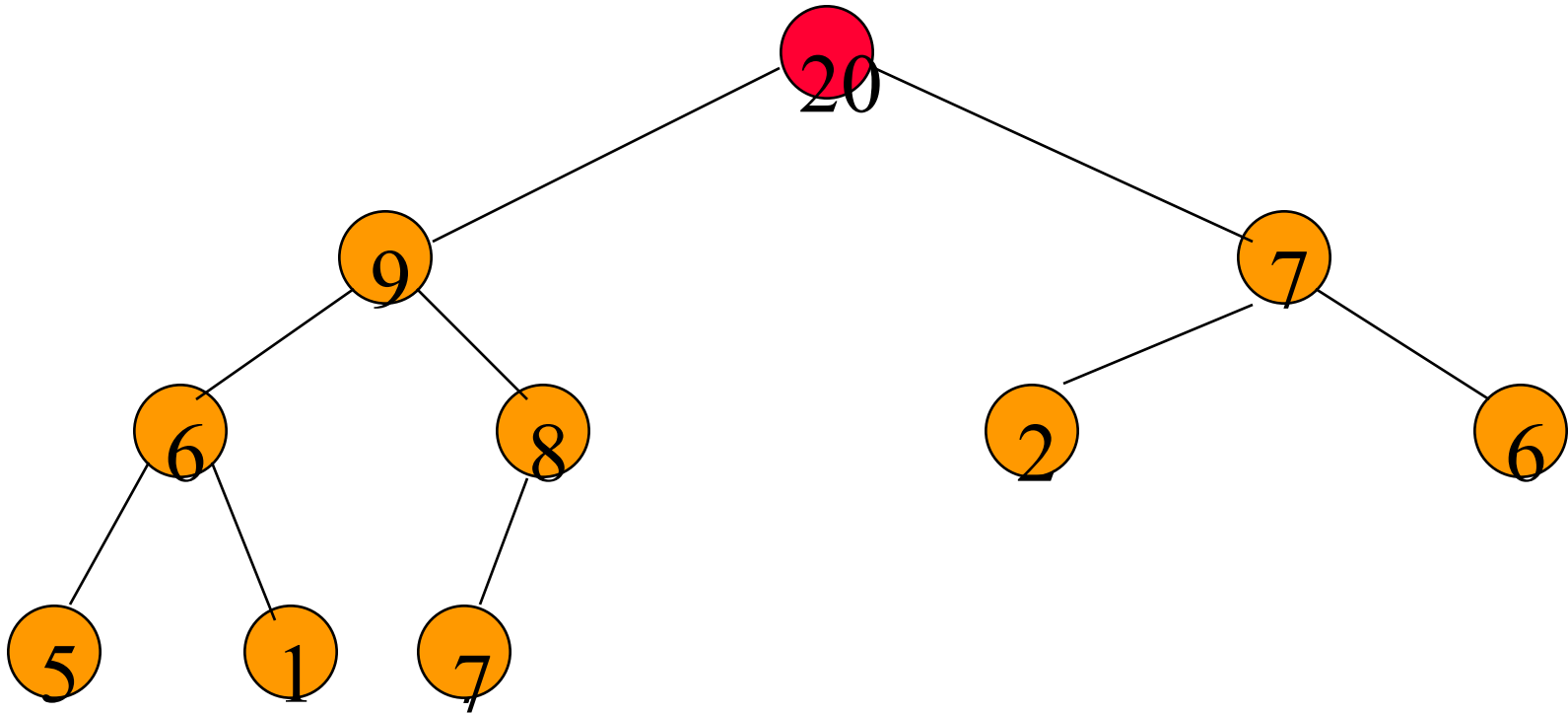
New element is 20.

Putting An Element Into A Max Heap



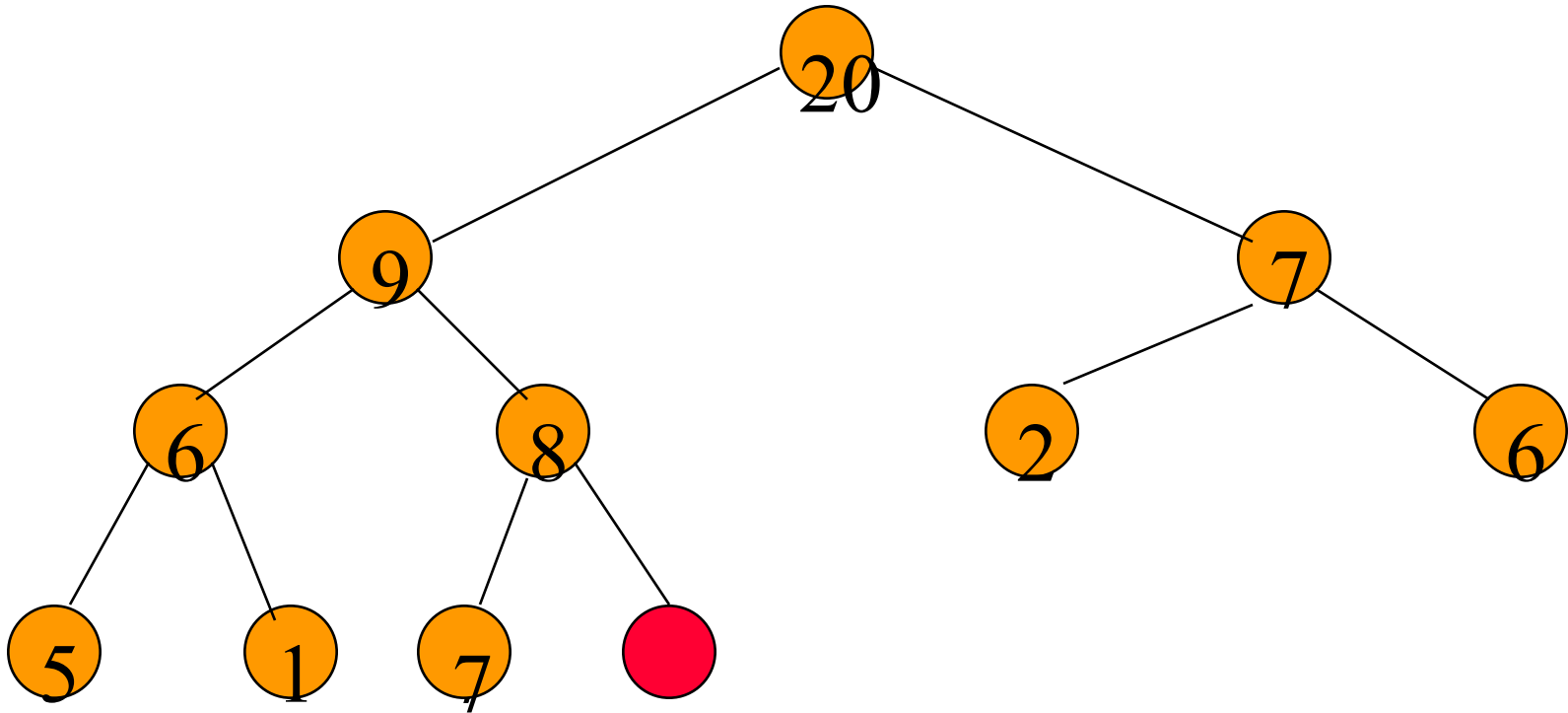
New element is 20.

Putting An Element Into A Max Heap



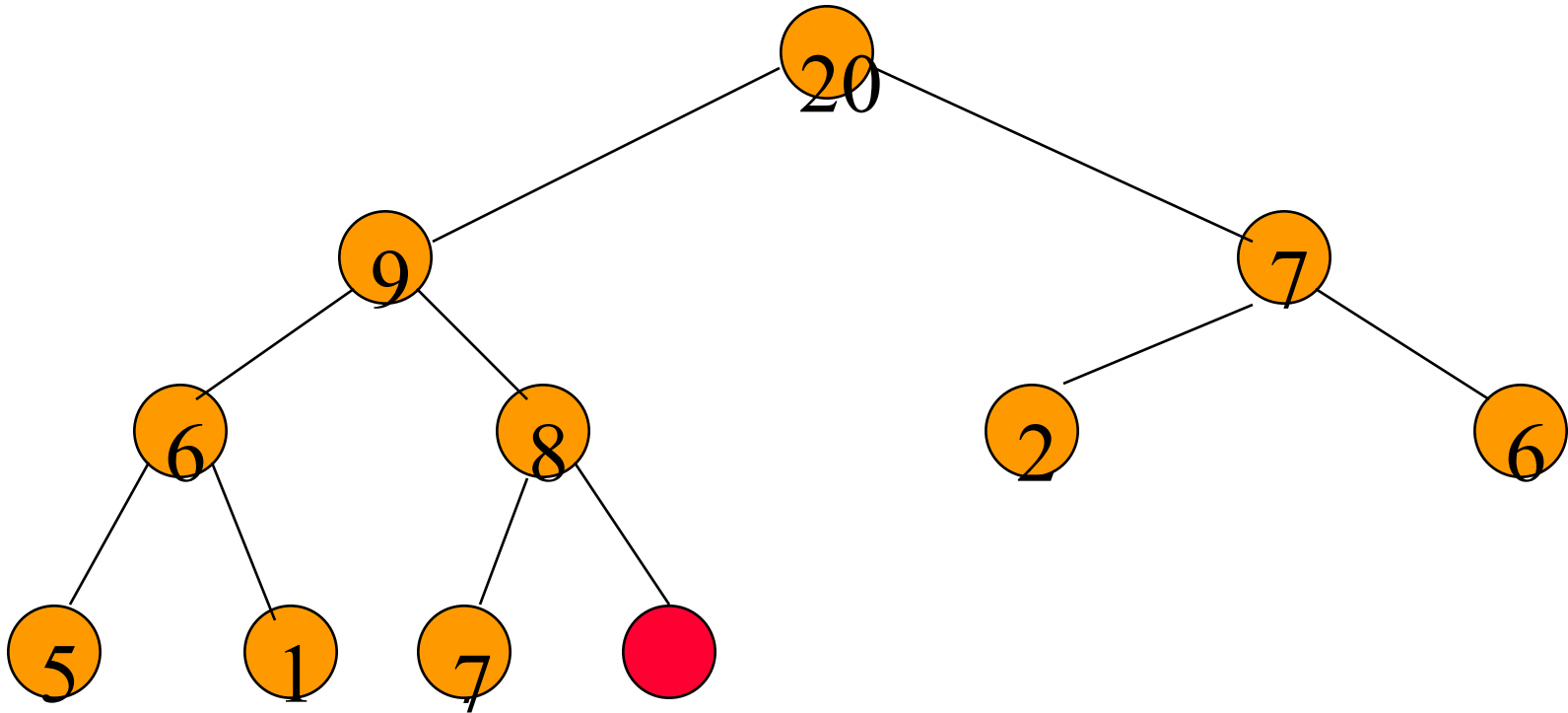
New element is 20.

Putting An Element Into A Max Heap



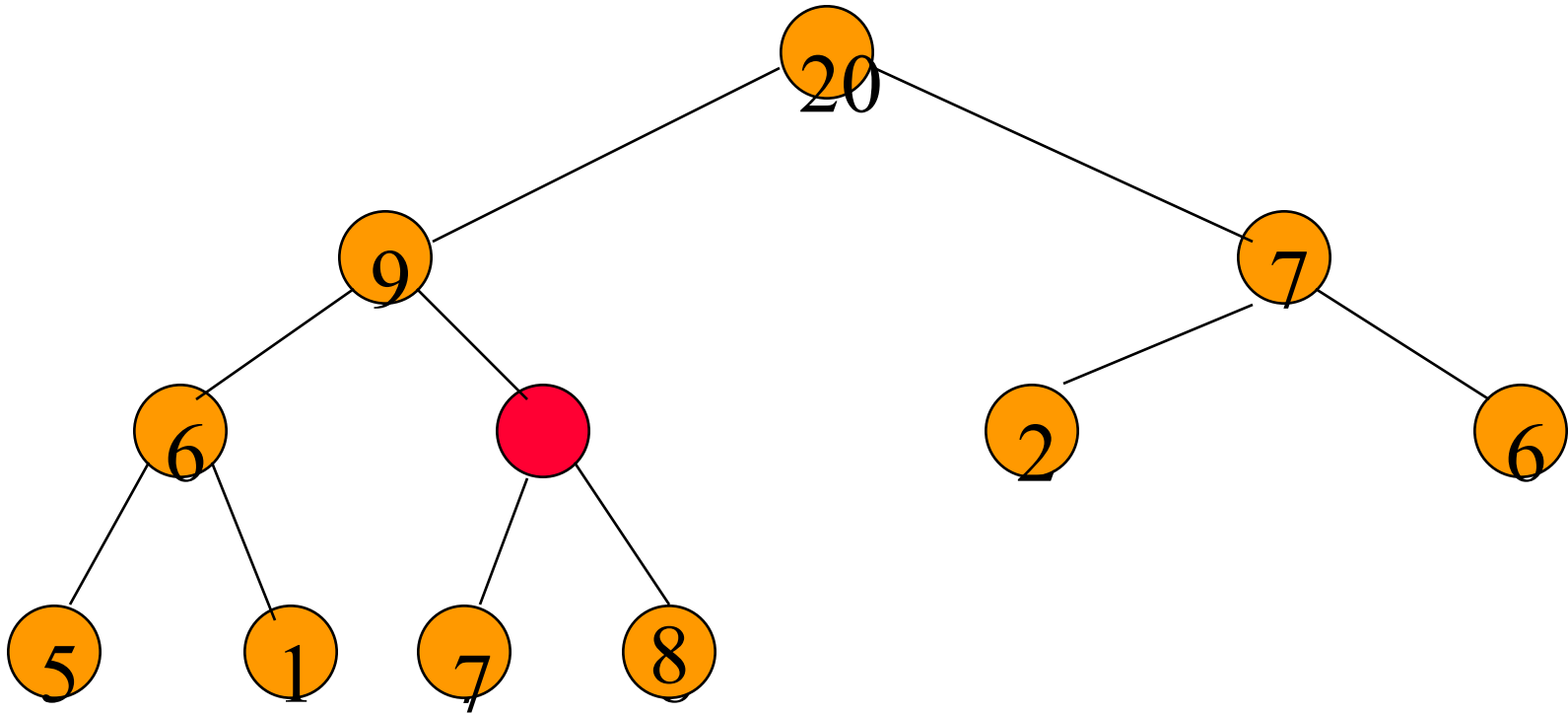
Complete binary tree with 11 nodes.

Putting An Element Into A Max Heap



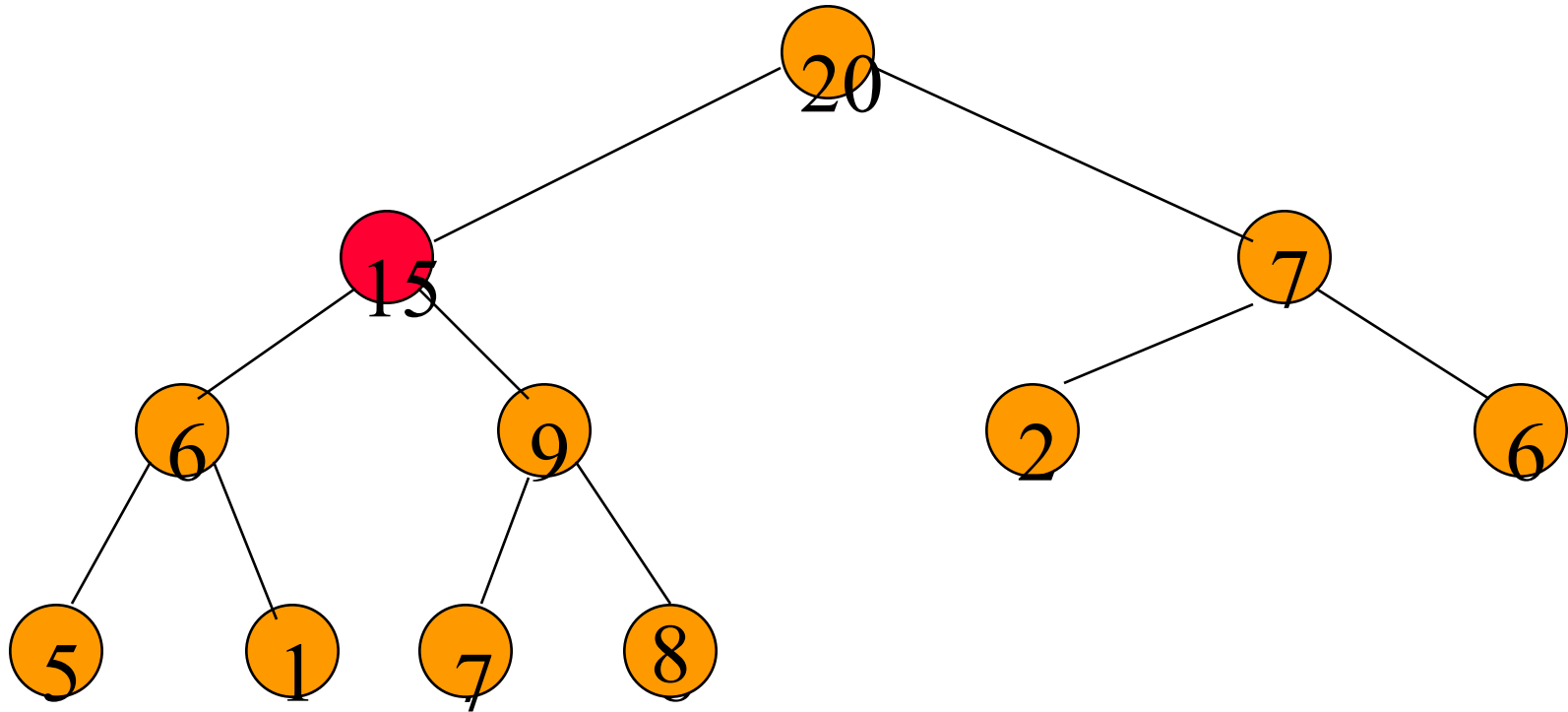
New element is 15.

Putting An Element Into A Max Heap



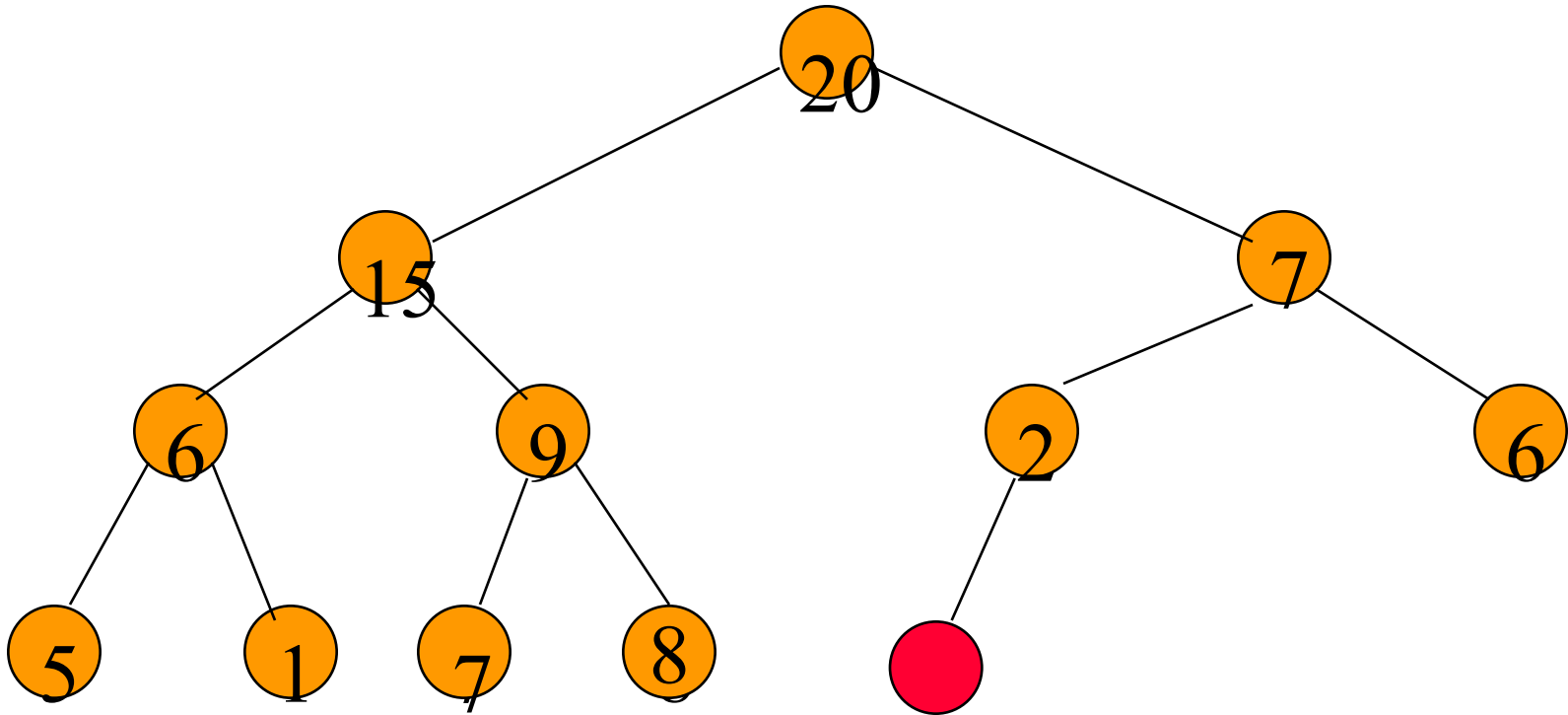
New element is 15.

Putting An Element Into A Max Heap



New element is 15.

Complexity Of Put



Complexity is $O(\log n)$, where n is heap size.

```

template <class T>
void MaxHeap<Type>::Push(const T& e)
{ // insert e into the max heap
    if (heapSize == capacity) { // double the capacity
        ChangeSize1D(heap, capacity, 2*capacity);
        capacity *= 2;
    }
    int currentNode = ++heapSize;
    while (currentNode != 1 && heap[currentNode/2] < e)
    { // bubble up
        heap[currentNode] = heap[currentNode/2];
        currentNode /= 2;
    }
    heap[currentNode] = e;
}

```

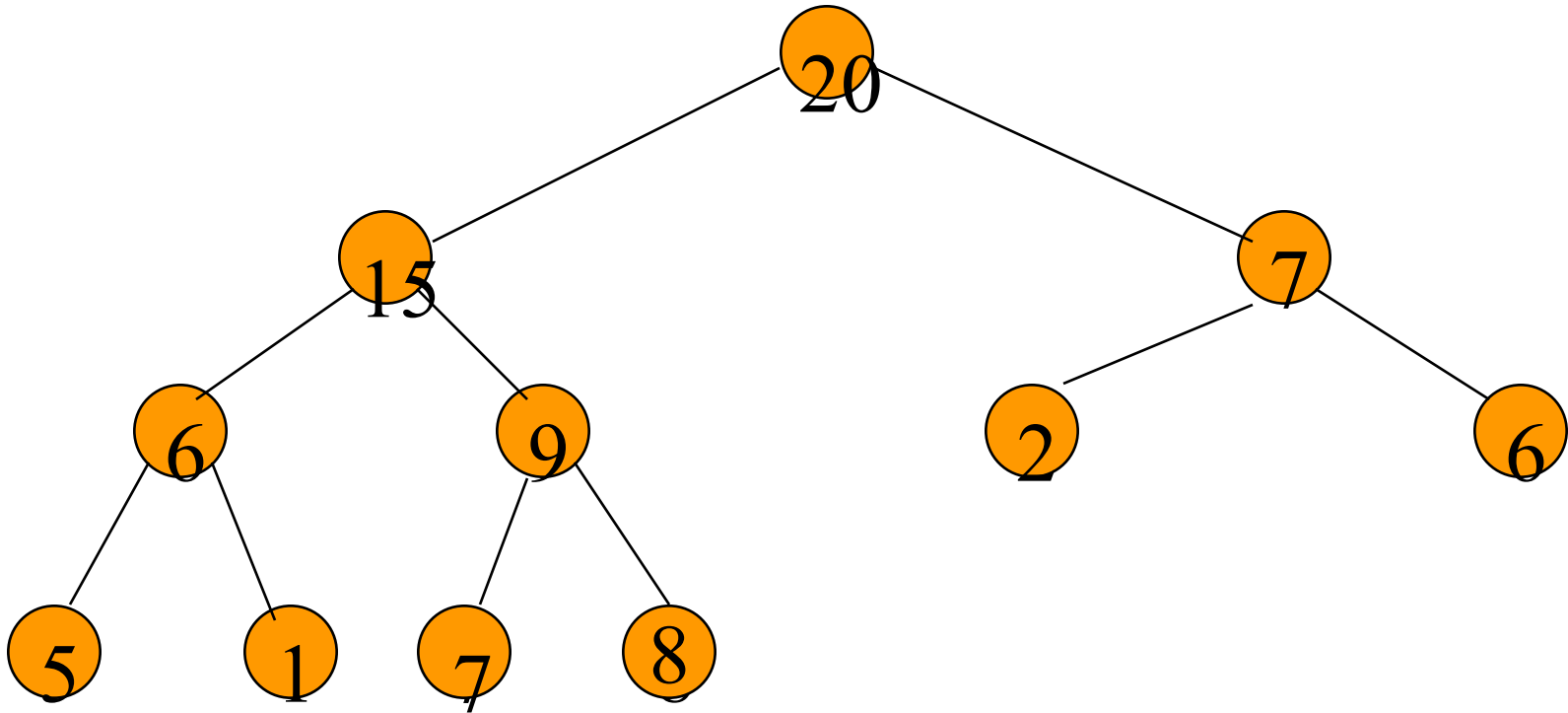
$O(\log n)$

Removing The Max Element

Complete Binary Tree

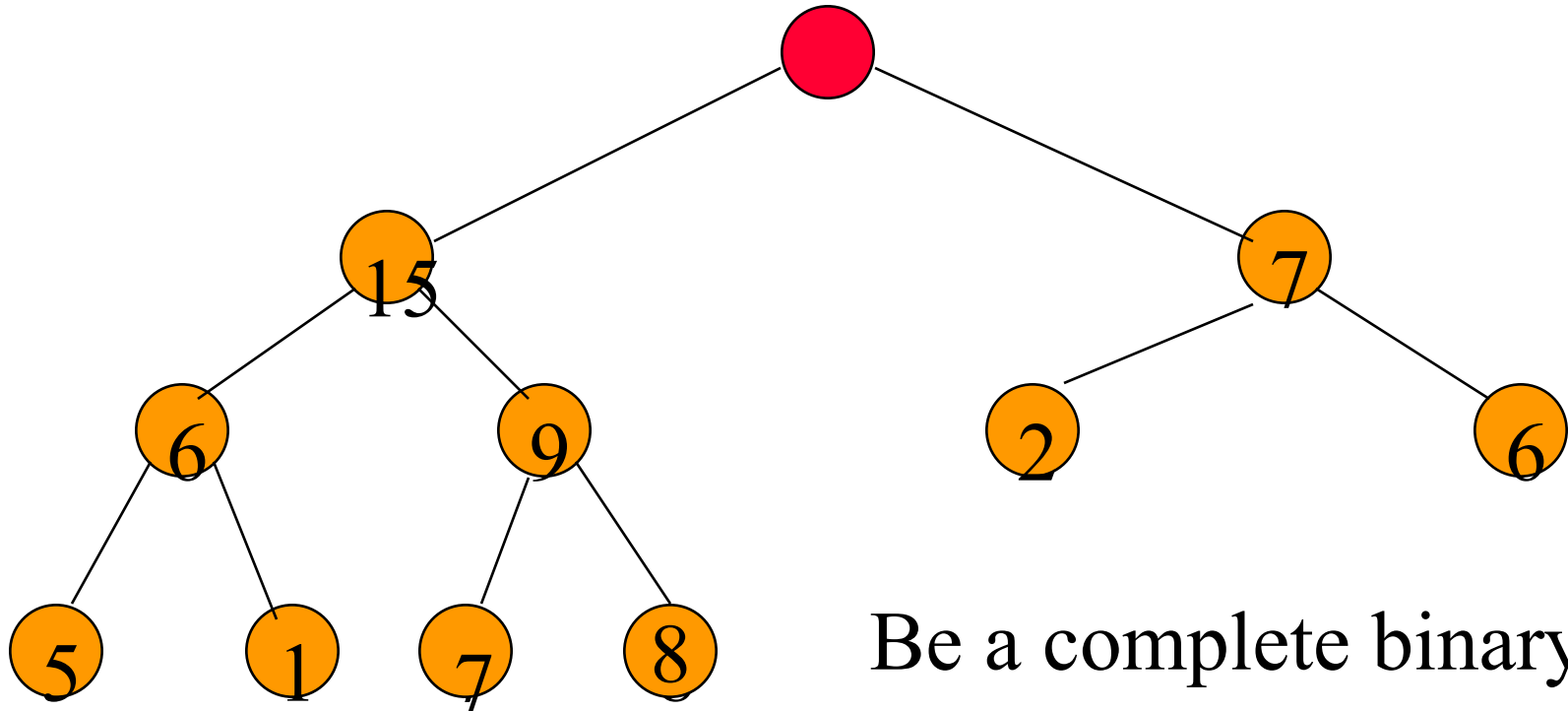
Max Tree

Removing The Max Element



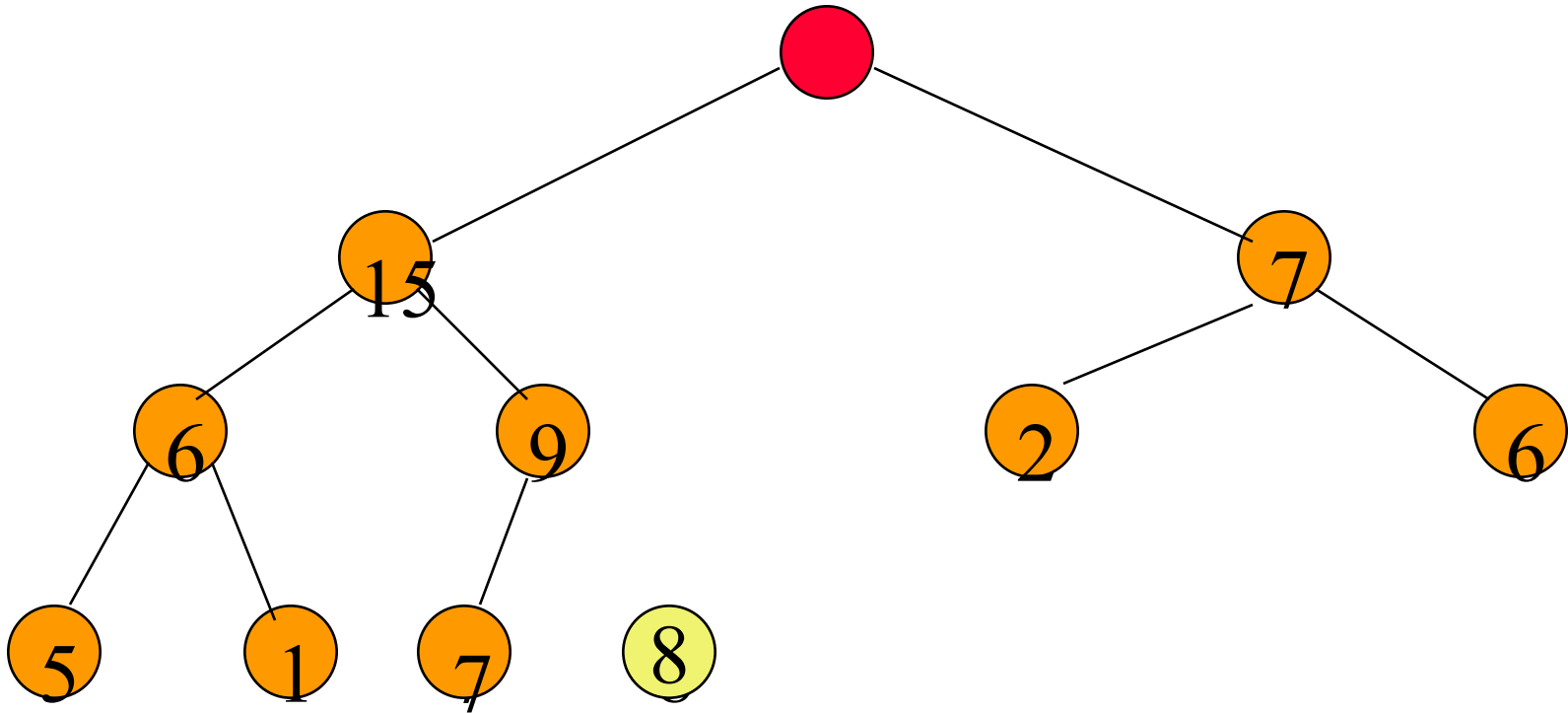
Max element is in the root.

Removing The Max Element



After max element is removed.

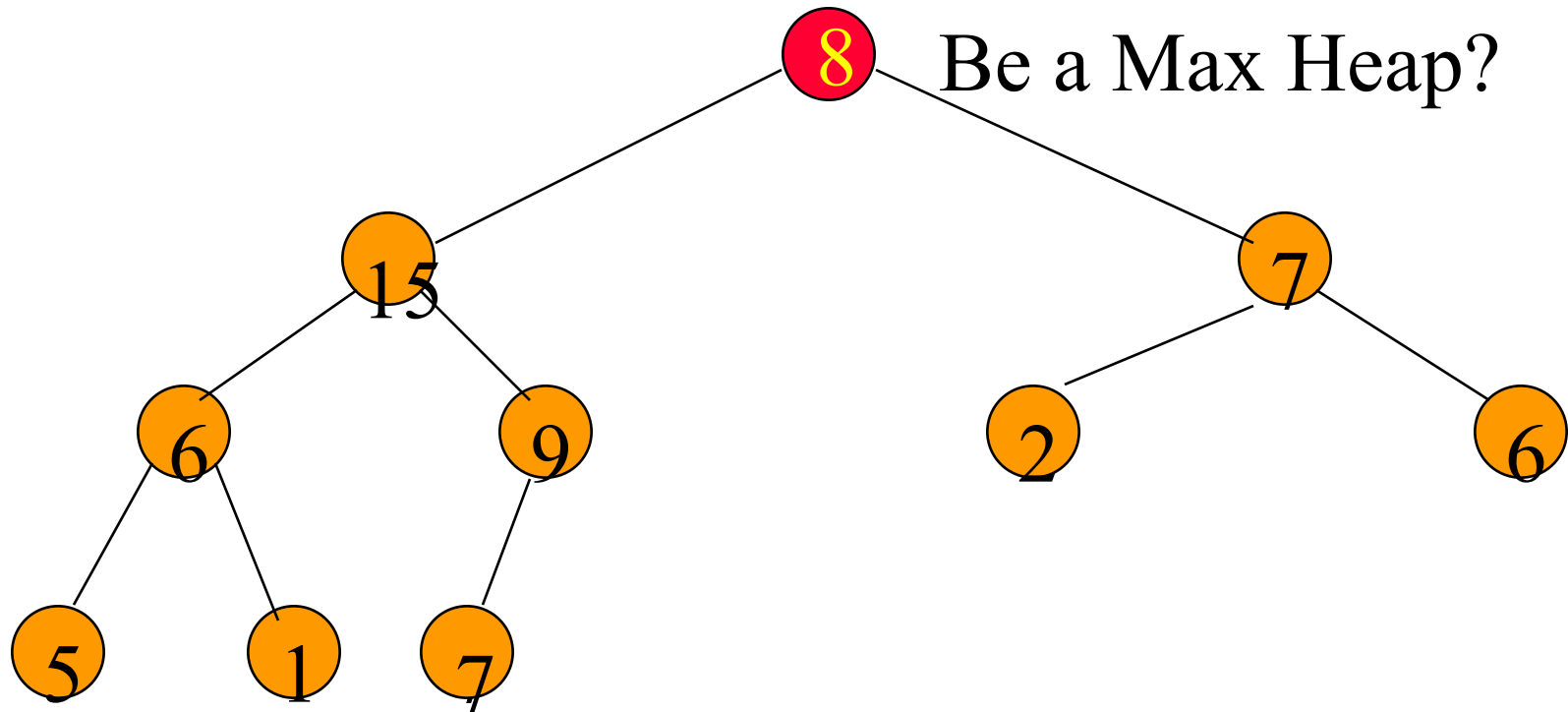
Removing The Max Element



Heap with 10 nodes.

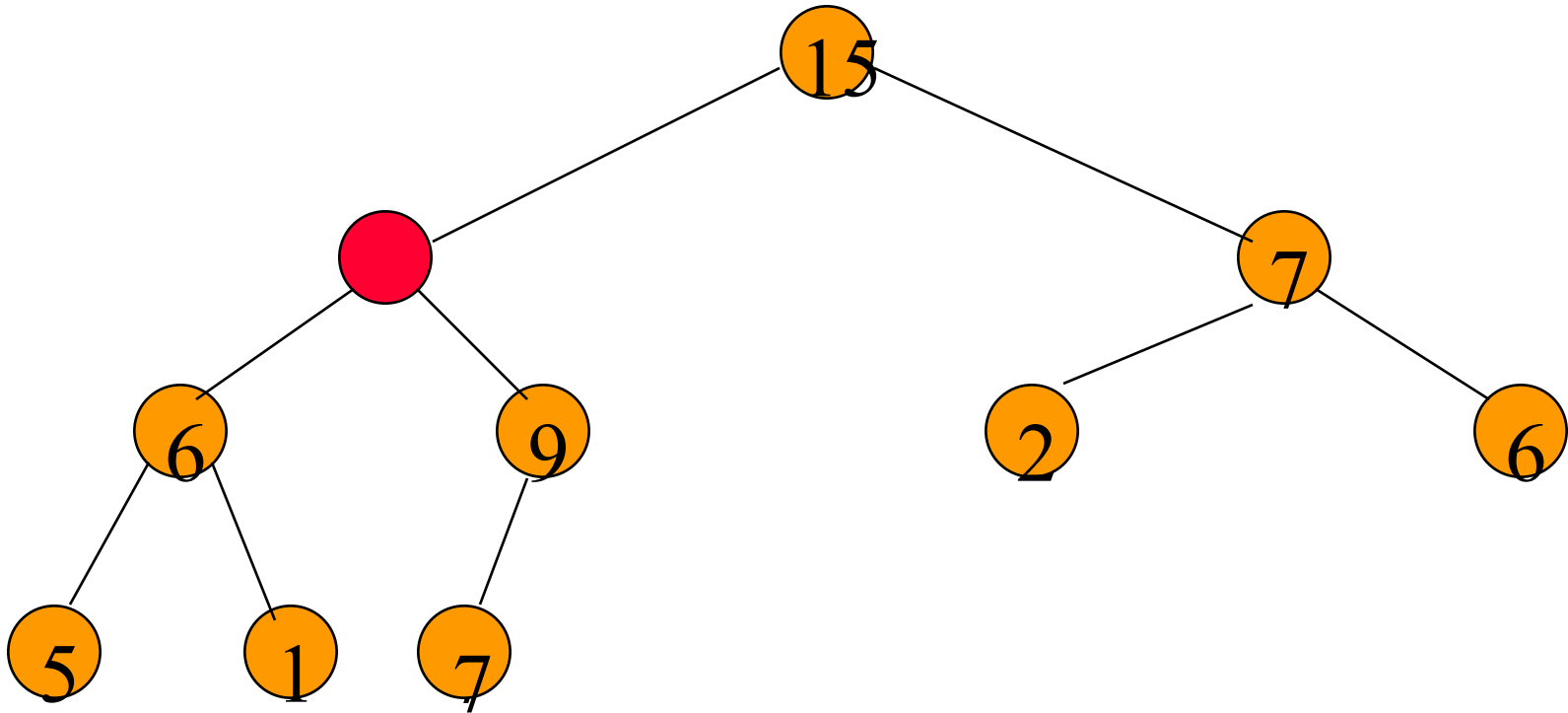
Reinsert 8 into the heap.

Removing The Max Element



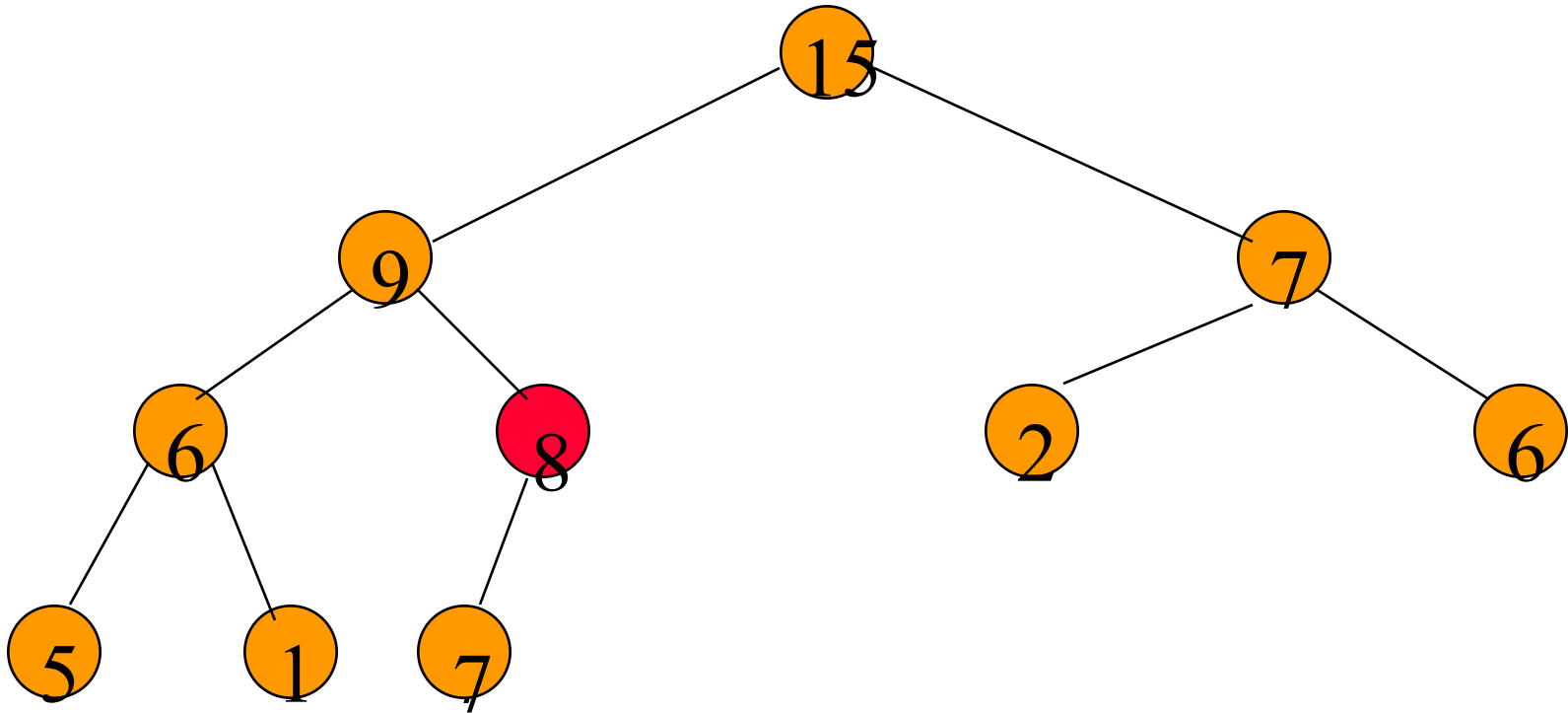
Reinsert **8** into the heap.

Removing The Max Element



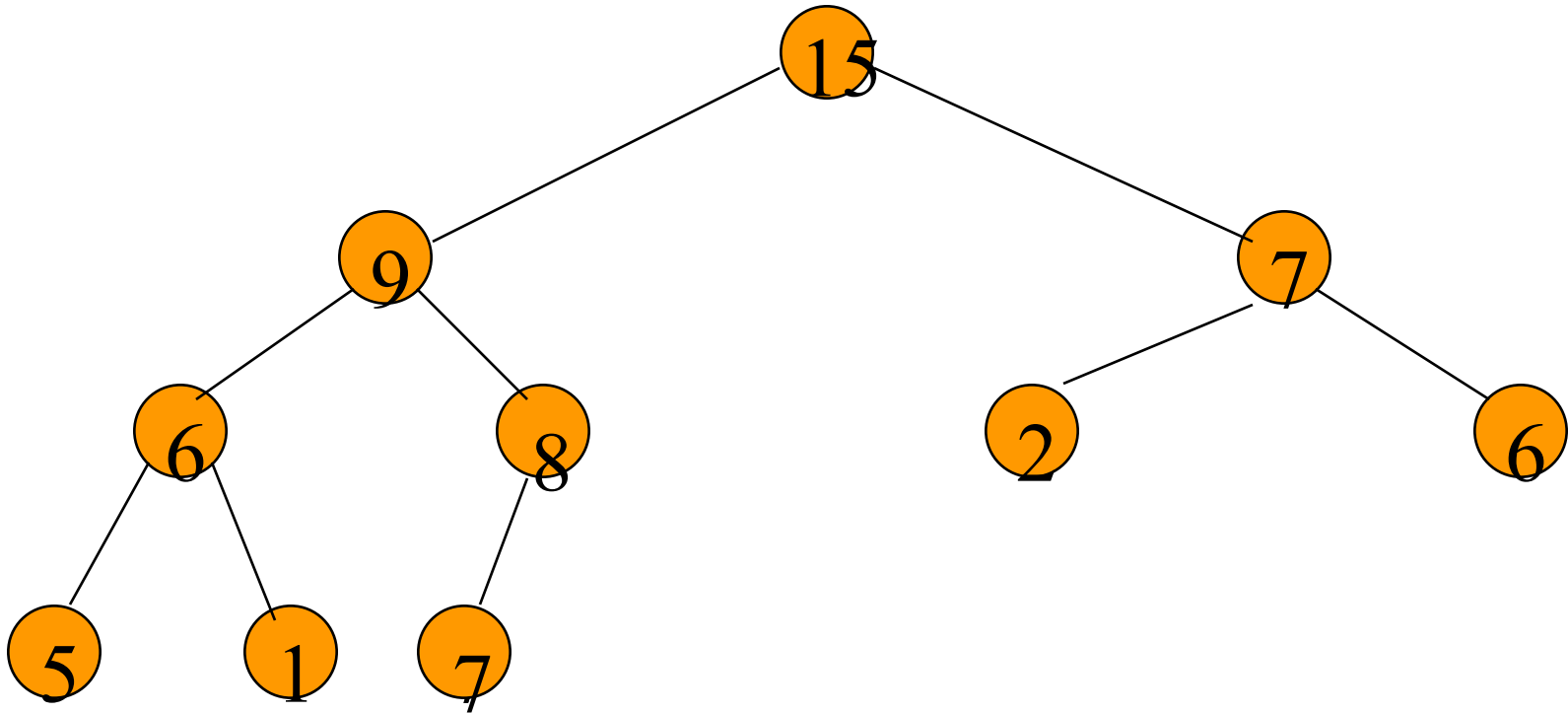
Reinsert 8 into the heap.

Removing The Max Element



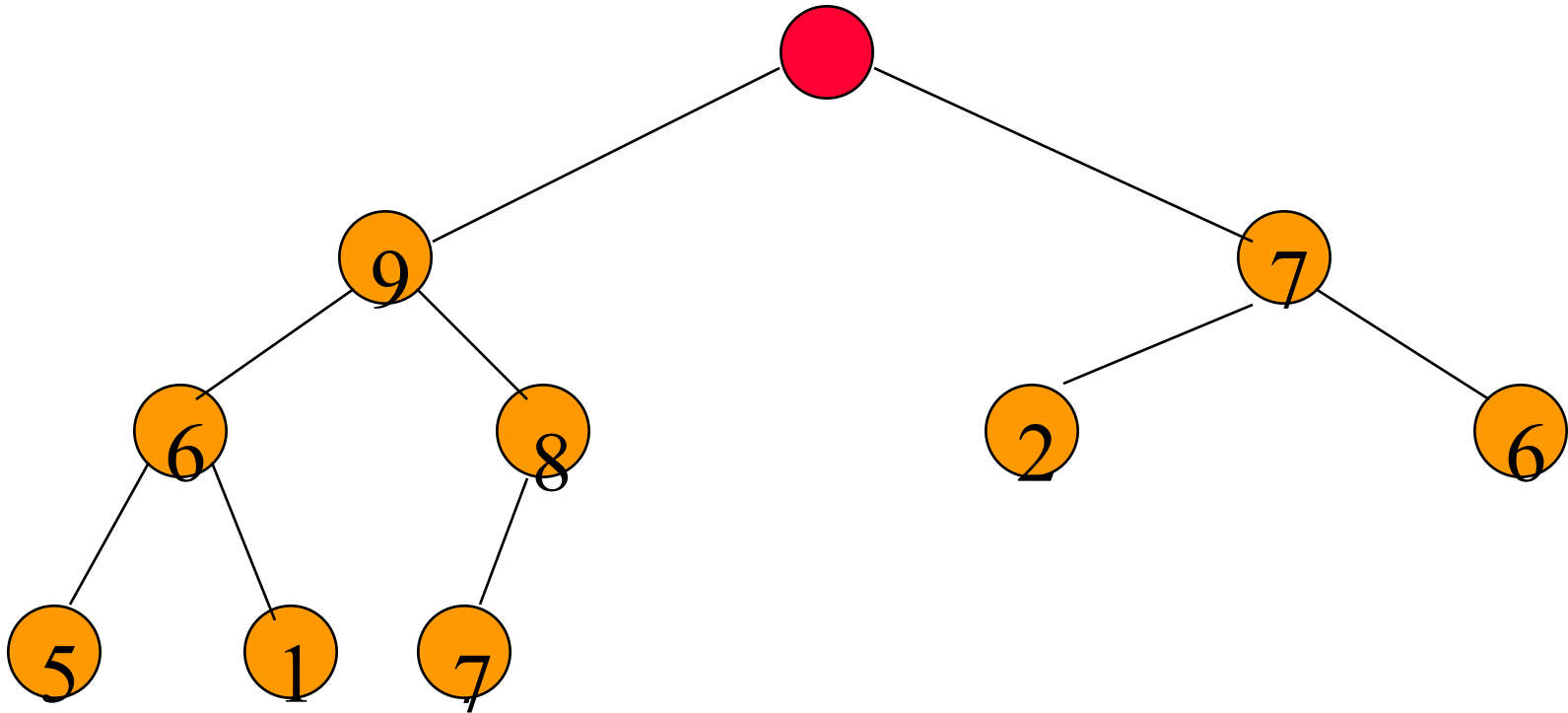
Reinsert **8** into the heap.

Removing The Max Element



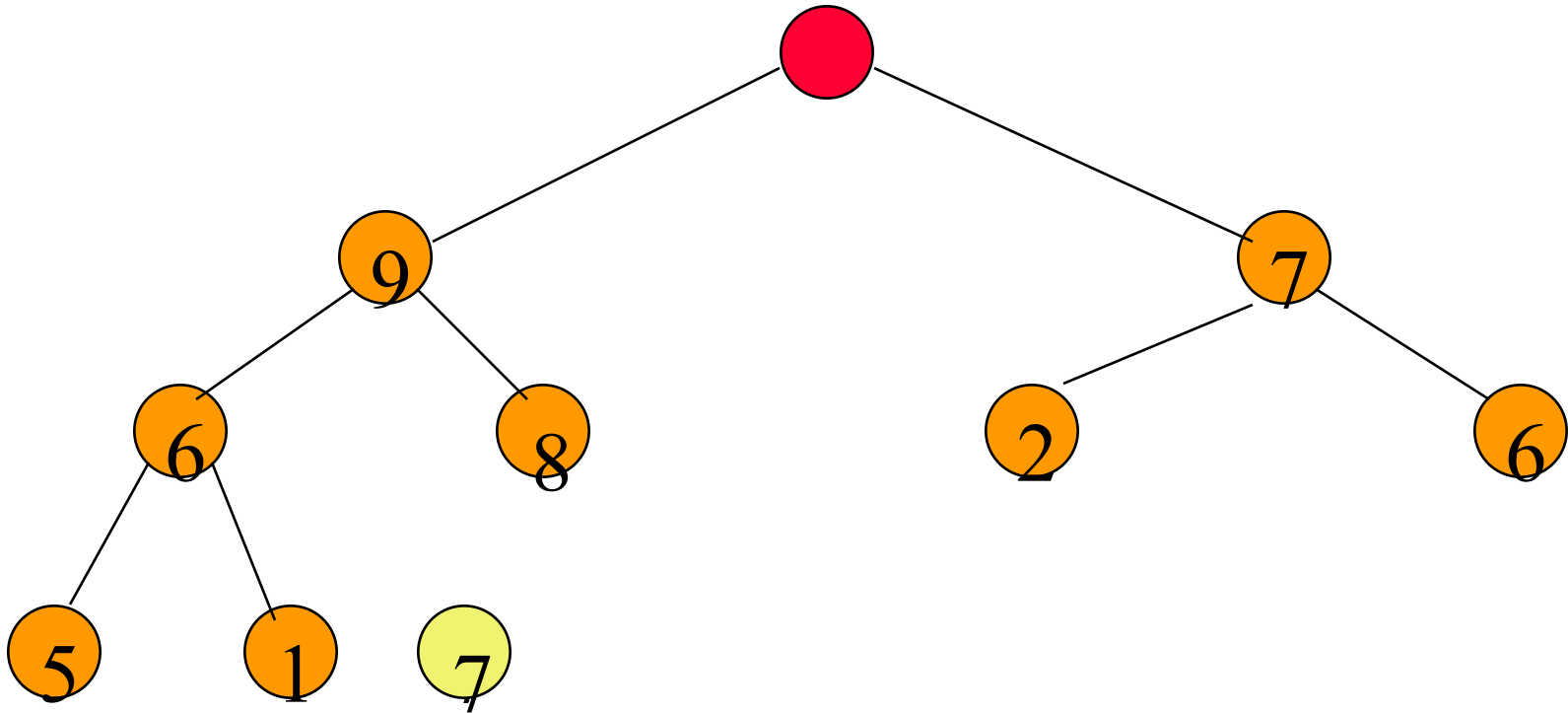
Max element is 15.

Removing The Max Element



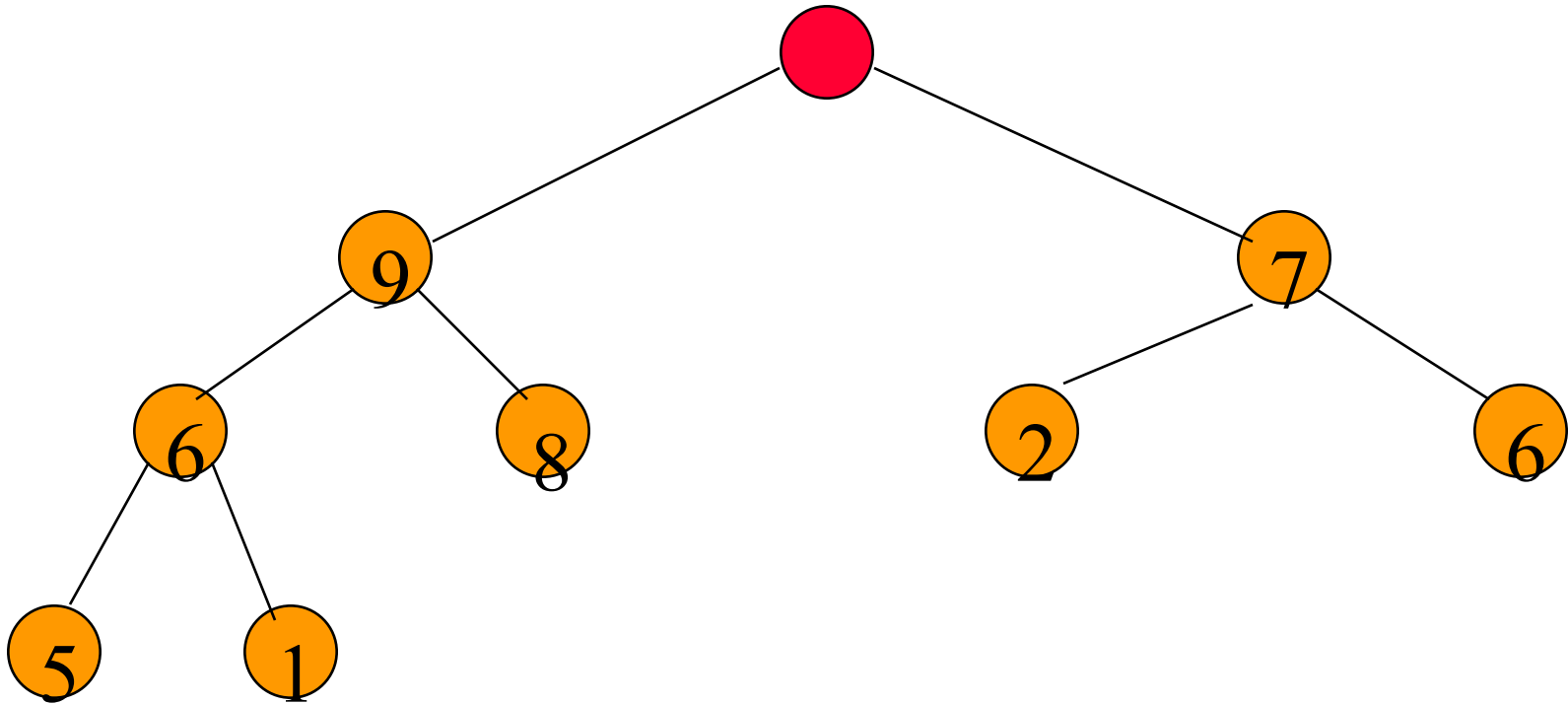
After max element is removed.

Removing The Max Element



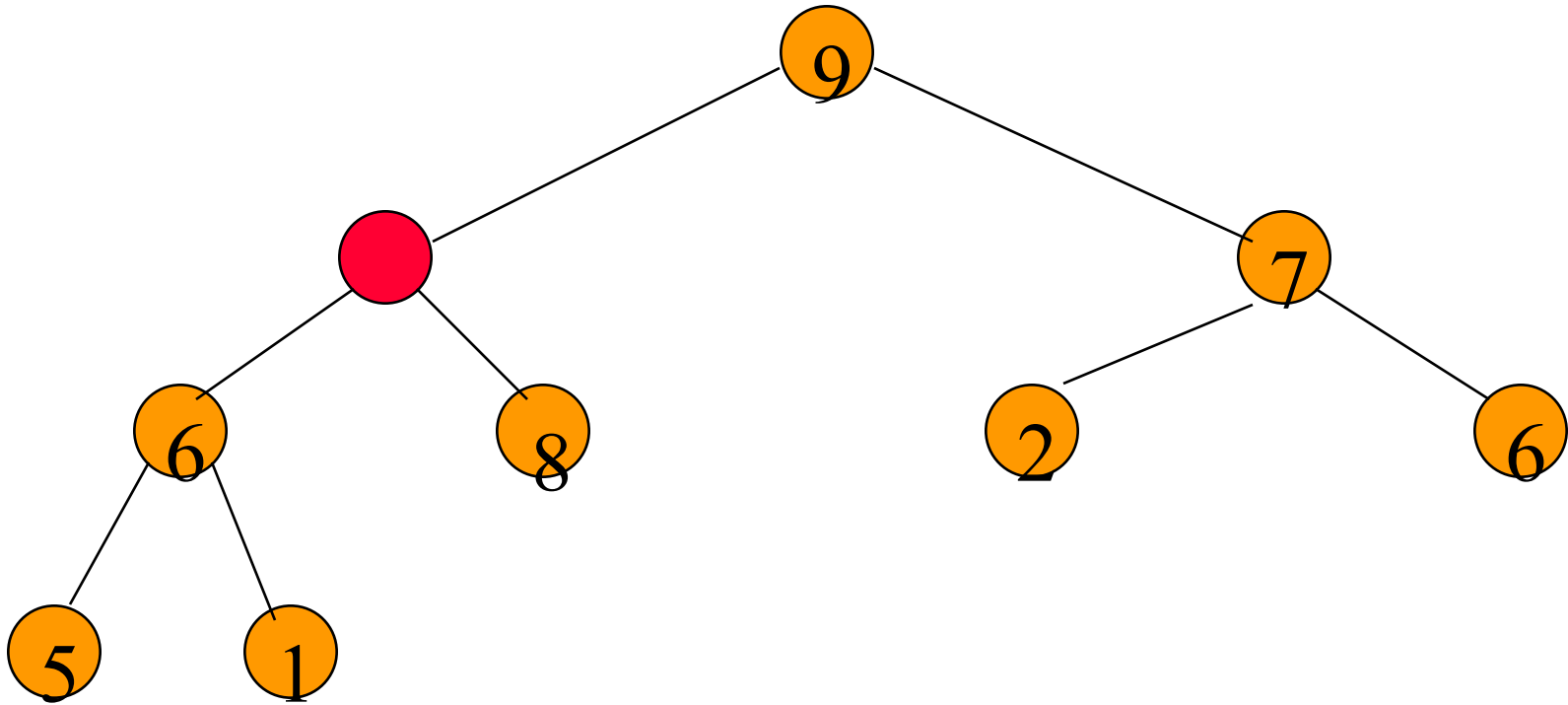
Heap with 9 nodes.

Removing The Max Element



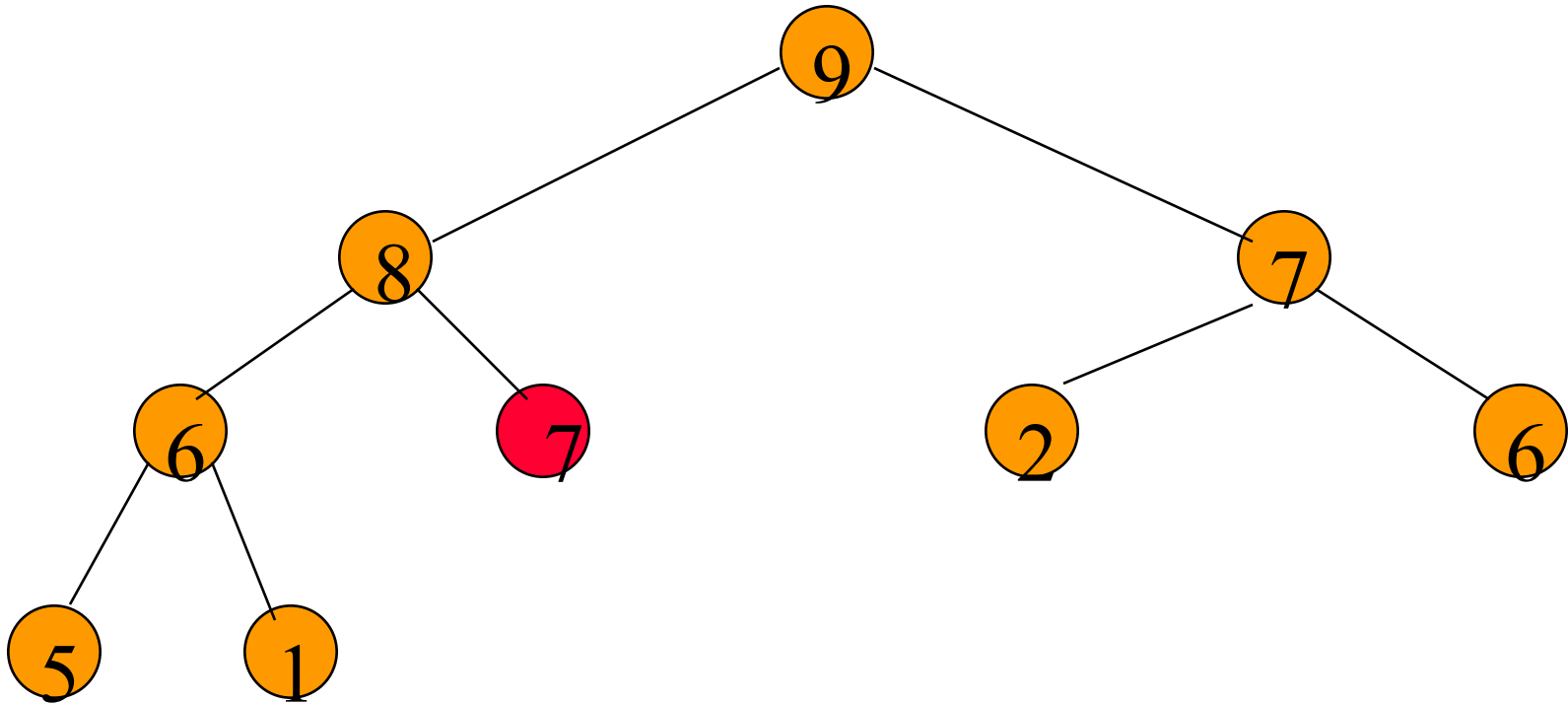
Reinsert **7**.

Removing The Max Element



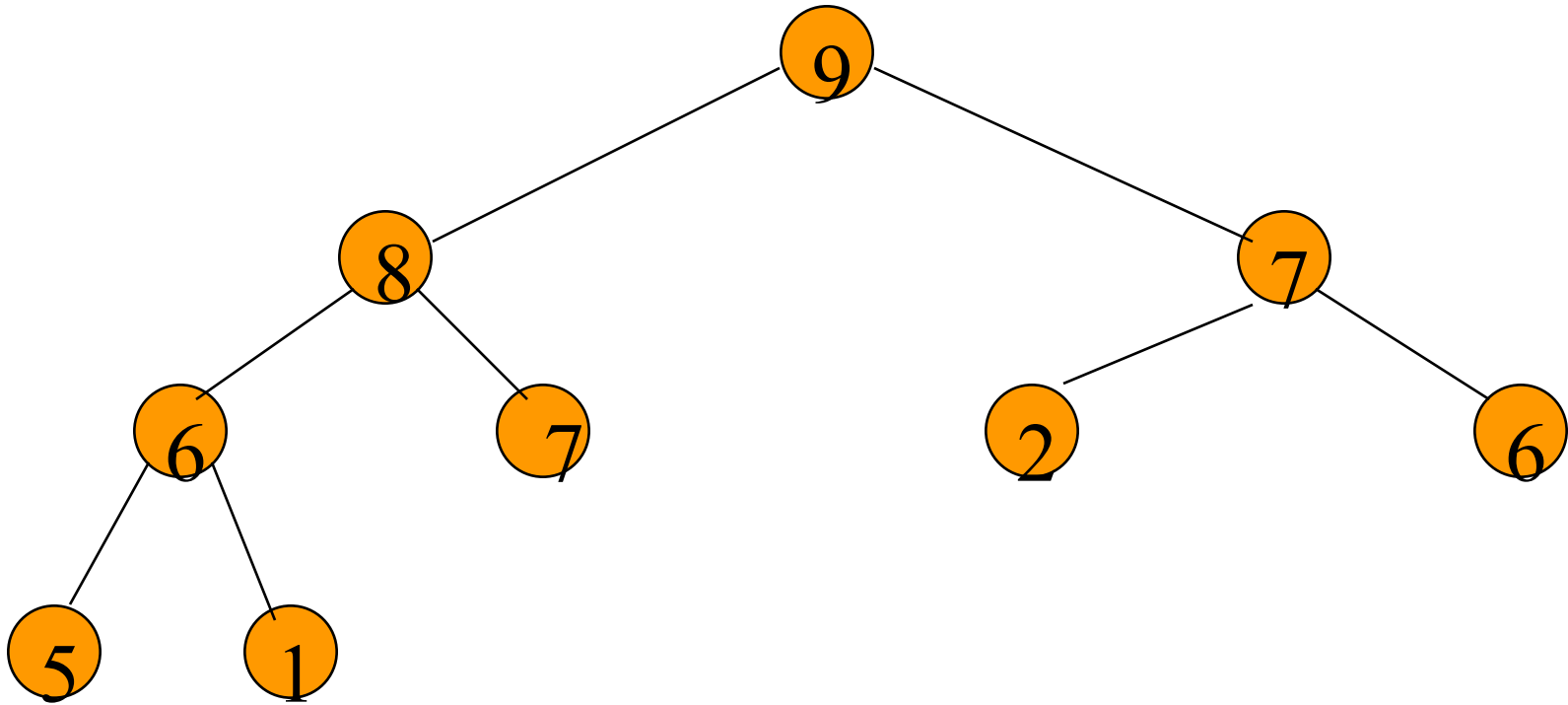
Reinsert **7**.

Removing The Max Element



Reinsert **7**.

Complexity Of Remove Max Element



Complexity is $O(\log n)$.


```
template <class T>
void MaxHeap<T>::Pop()
{ // delete the max element.
    if (IsEmpty()) throw “Heap is empty. Cannot delete.”;
    heap[1].~T(); // delete the max

    // remove the last element from heap
    T lastE = heap[heapSize--];

    // trickle down
    int currentNode = 1; // root
    int child = 2;      // left child of currentNode
```

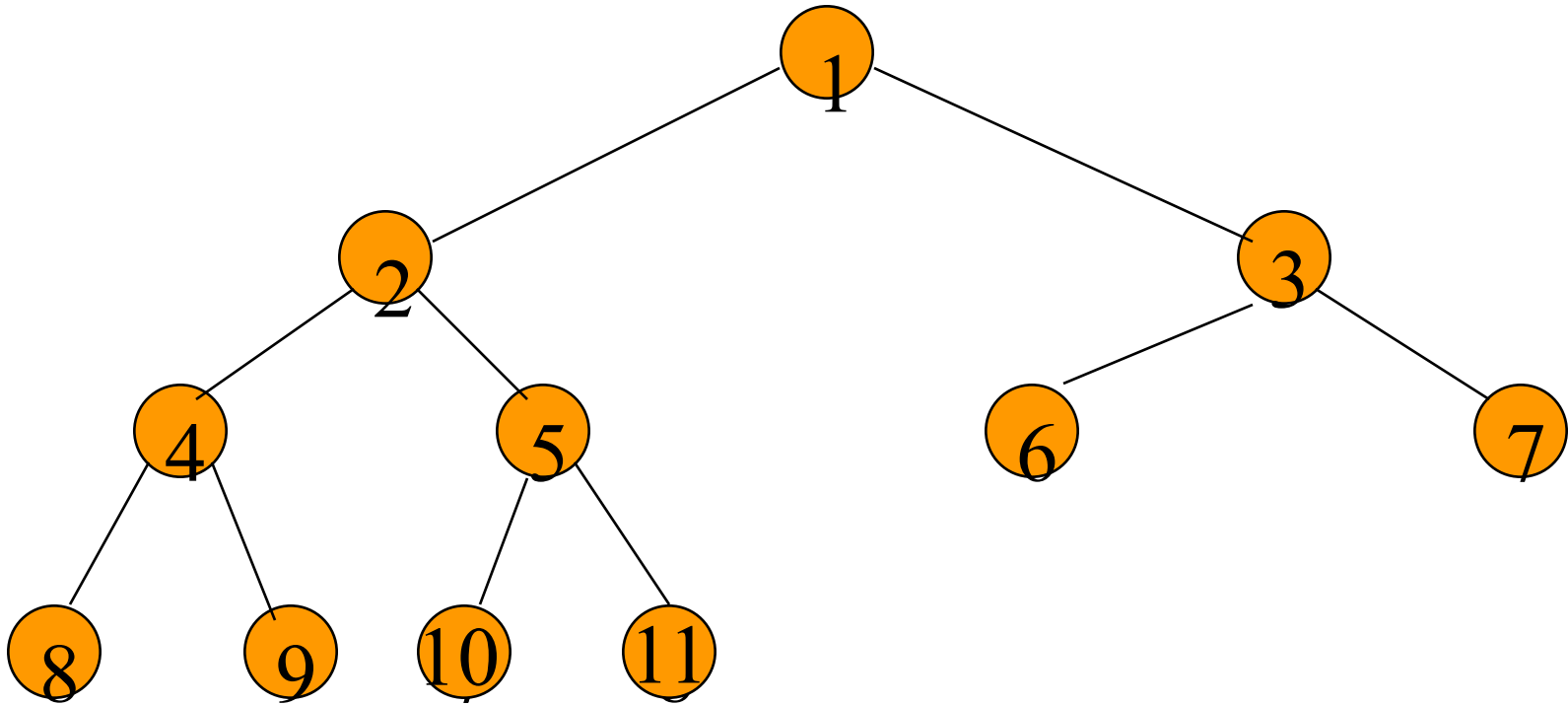
```
while (child <= heapSize)
{
    // set child to the larger child of currentNode
    if (child < heapSize && heap[child] < heap[child+1])
child++;

    // can we put lastE in currentNode?
    if (lastE >= heap[child]) break; // yes

    // no
    heap[currentNode] = heap[child]; // move child up
    currentNode = child; child *= 2; // move down a level
}
heap[currentNode] = lastE;
}
```

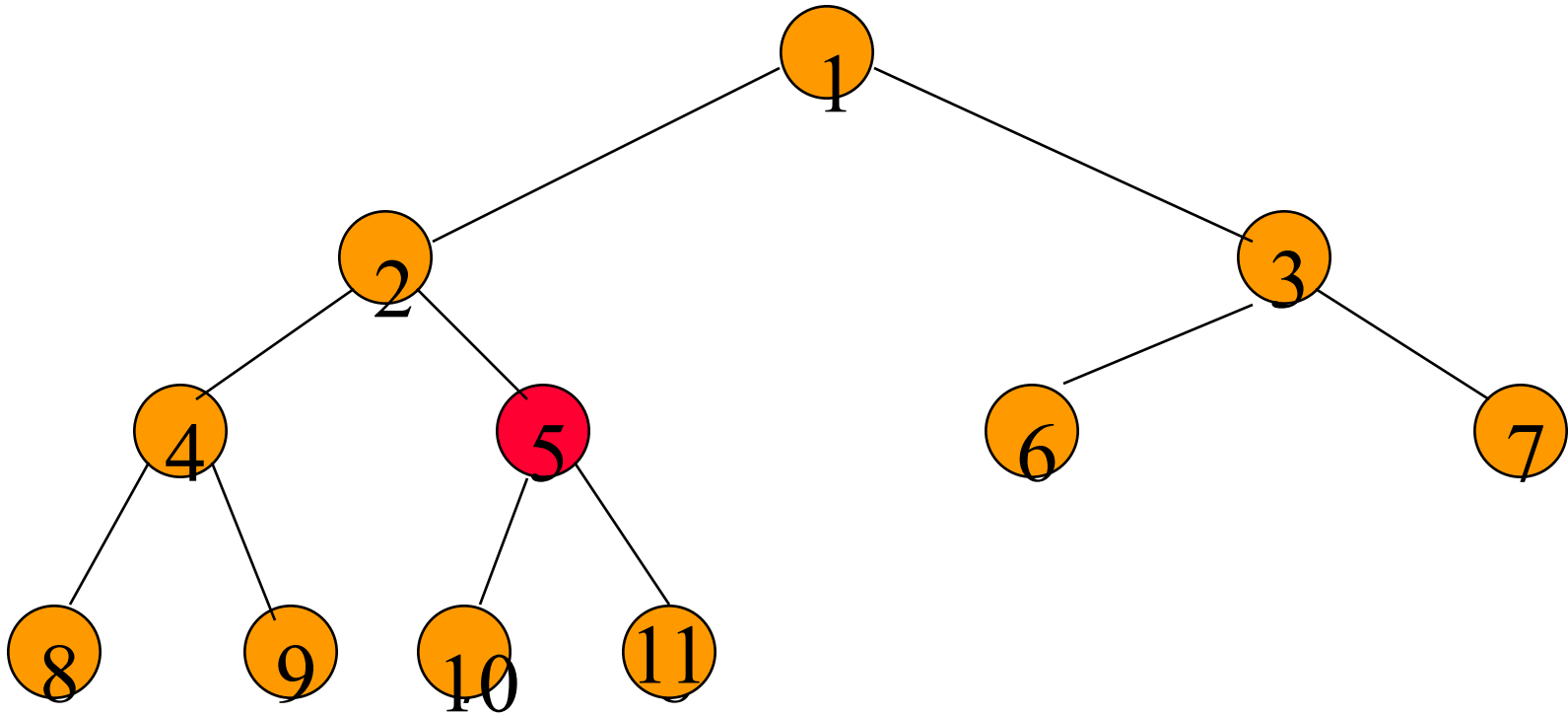
- **Exercises: P287-2, 3**

Initializing A Max Heap



input array = [-, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11]

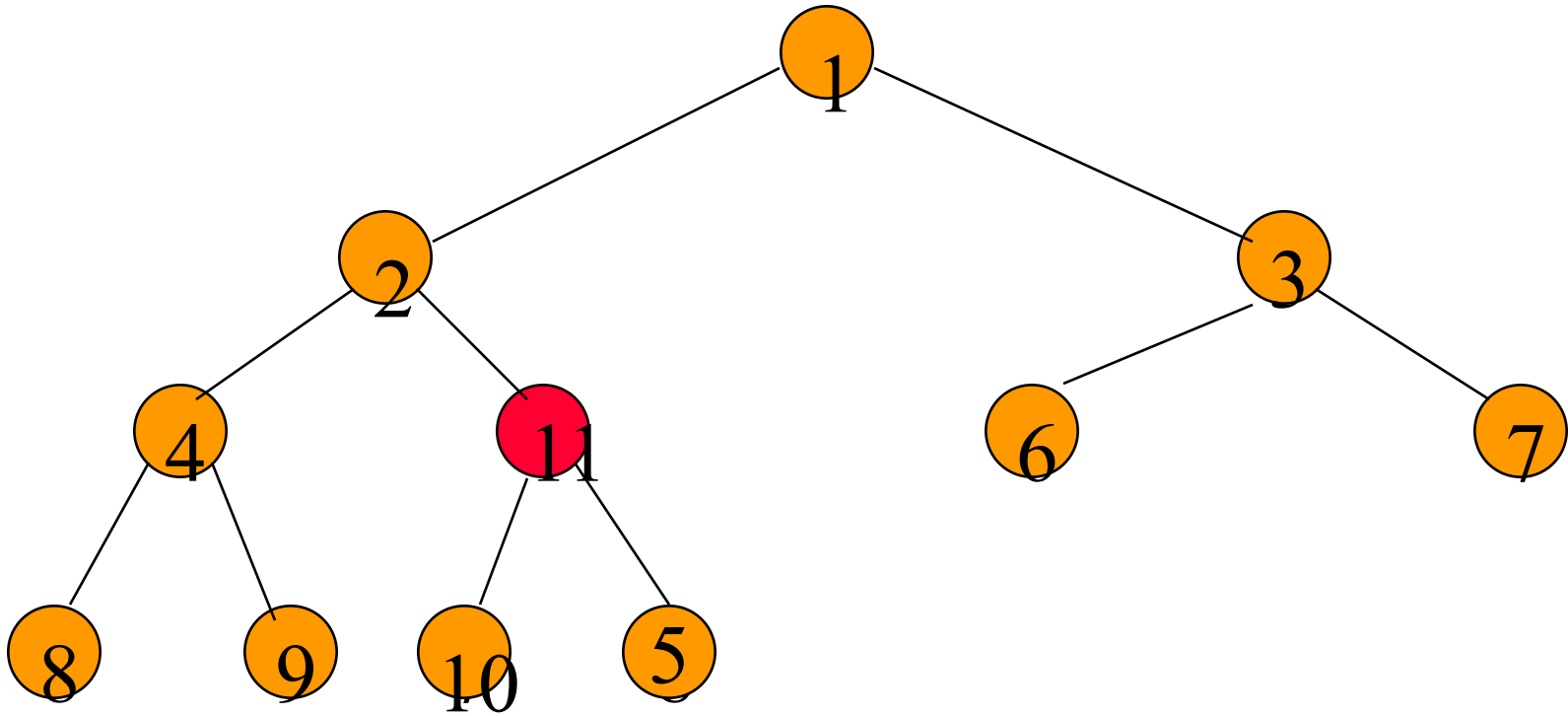
Initializing A Max Heap



Start at rightmost array position that has a child.

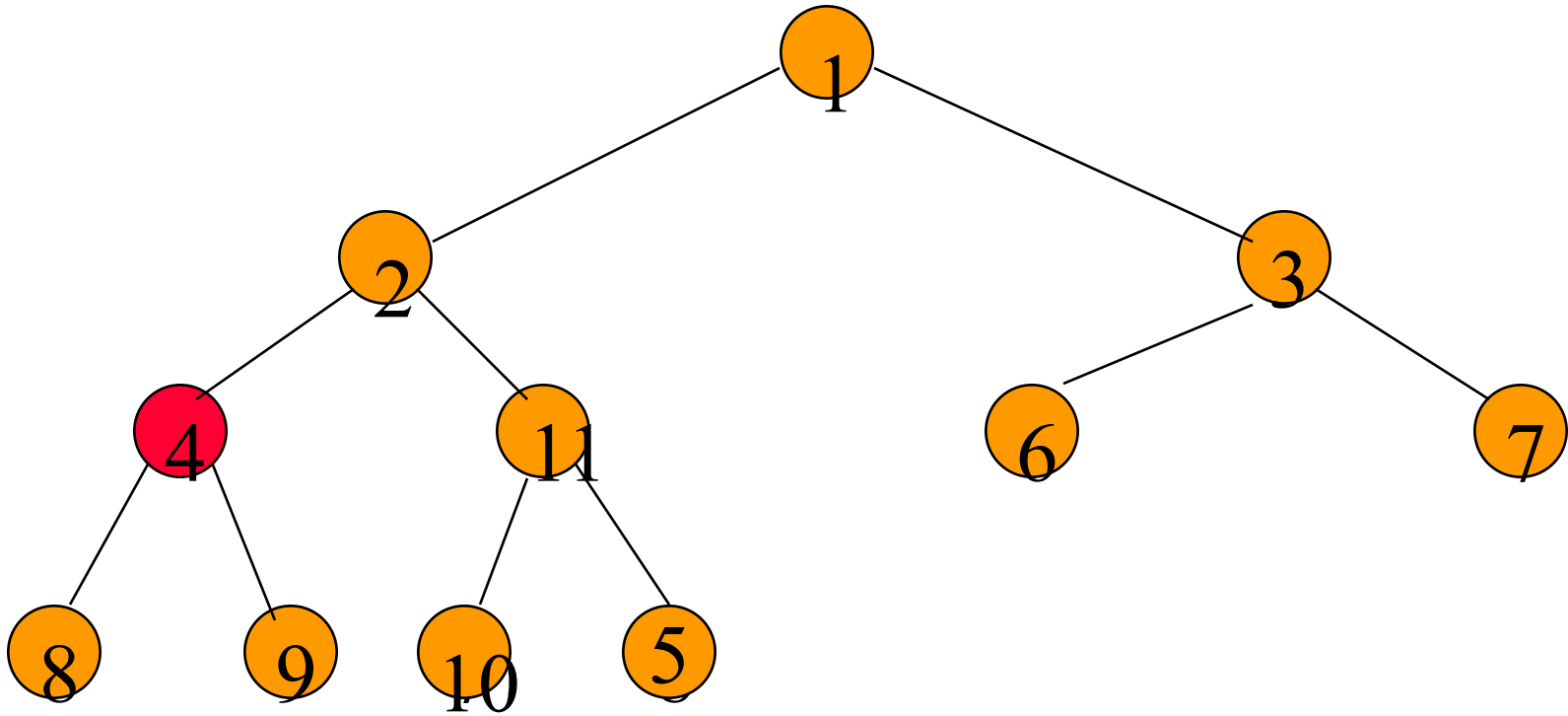
Index is $n/2$.

Initializing A Max Heap

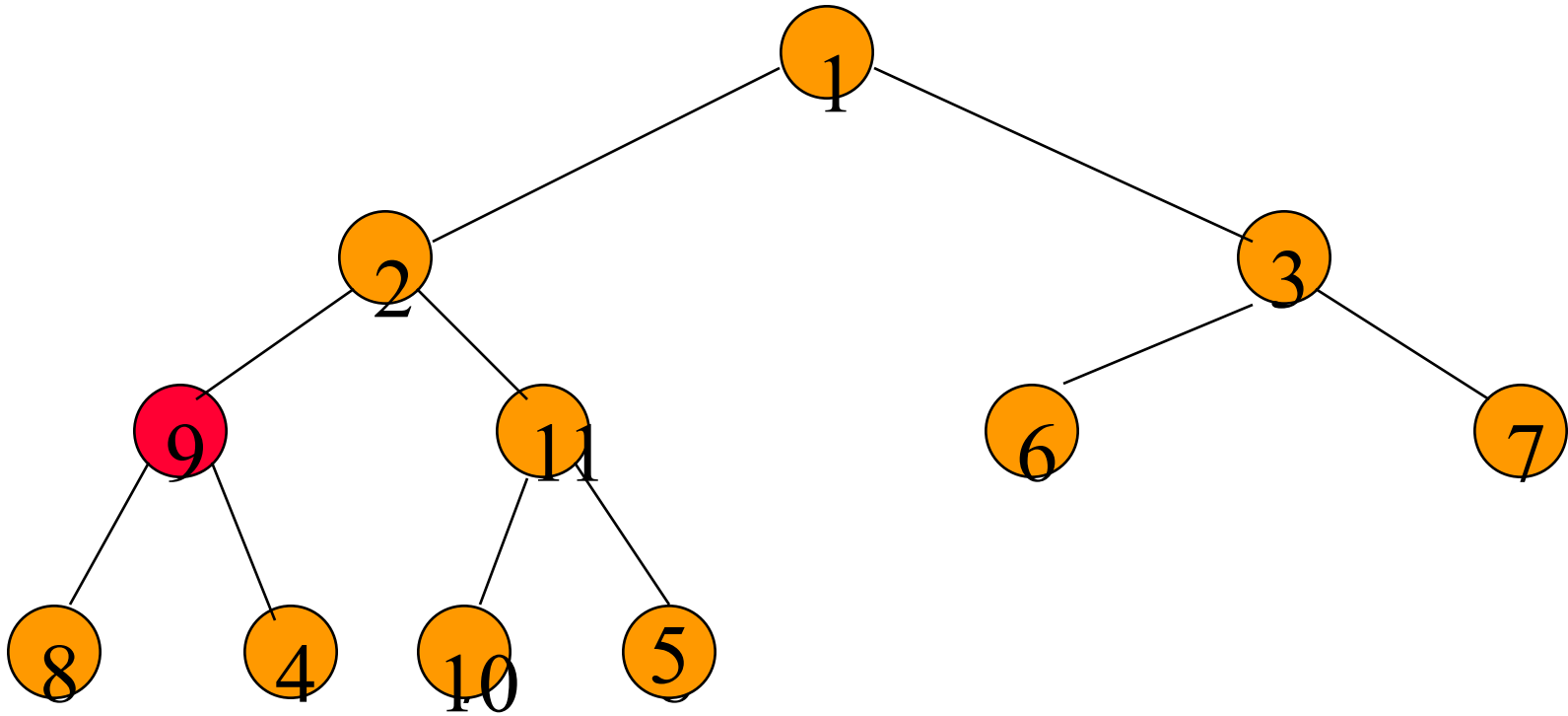


Move to next lower array position.

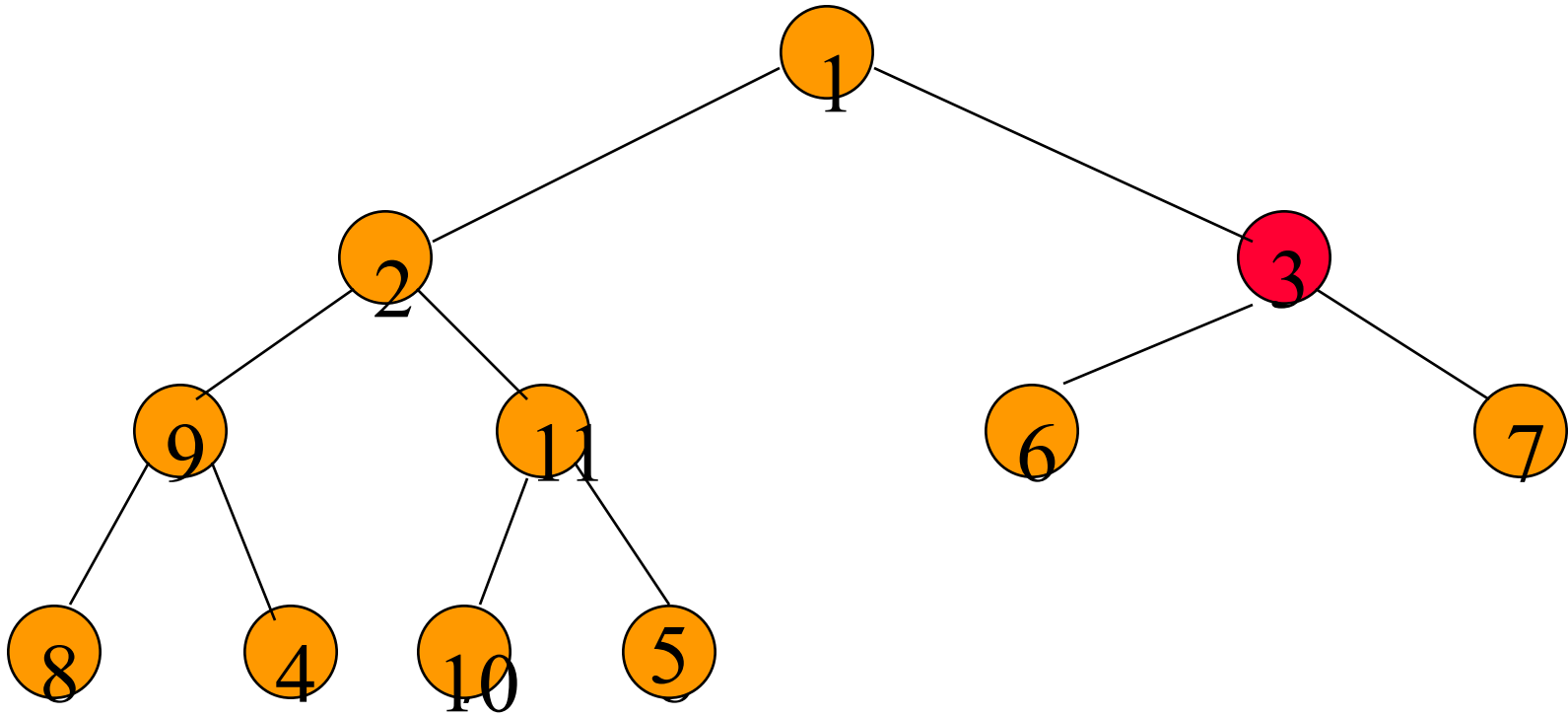
Initializing A Max Heap



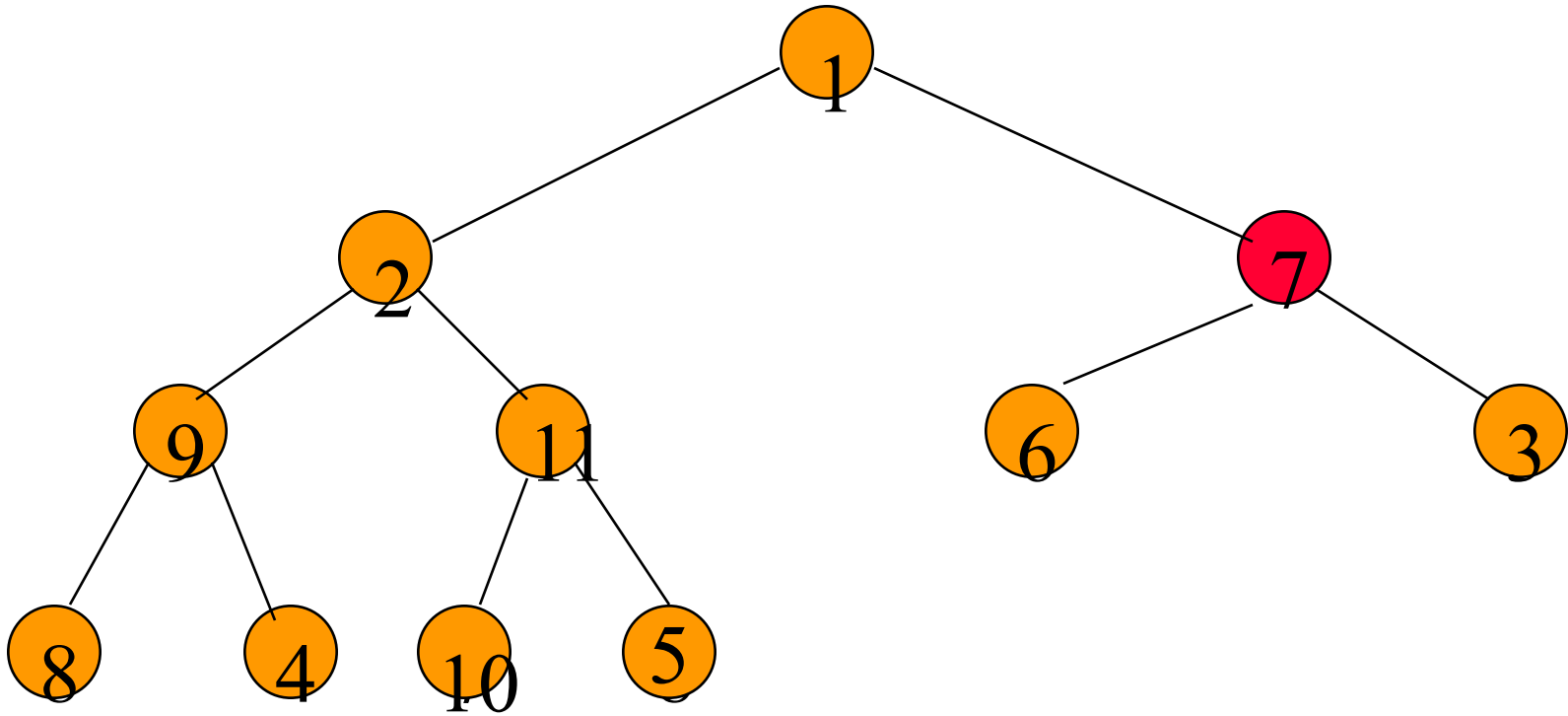
Initializing A Max Heap



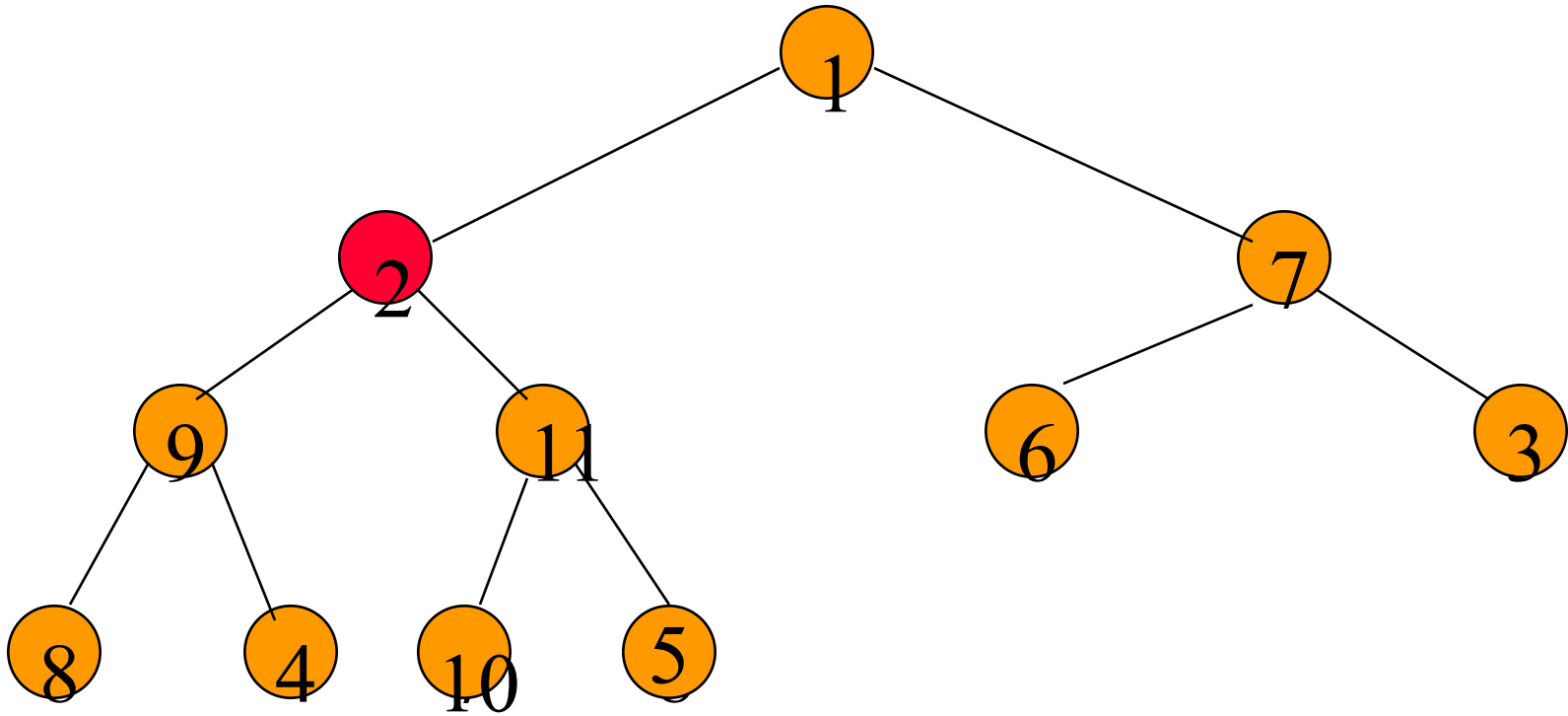
Initializing A Max Heap



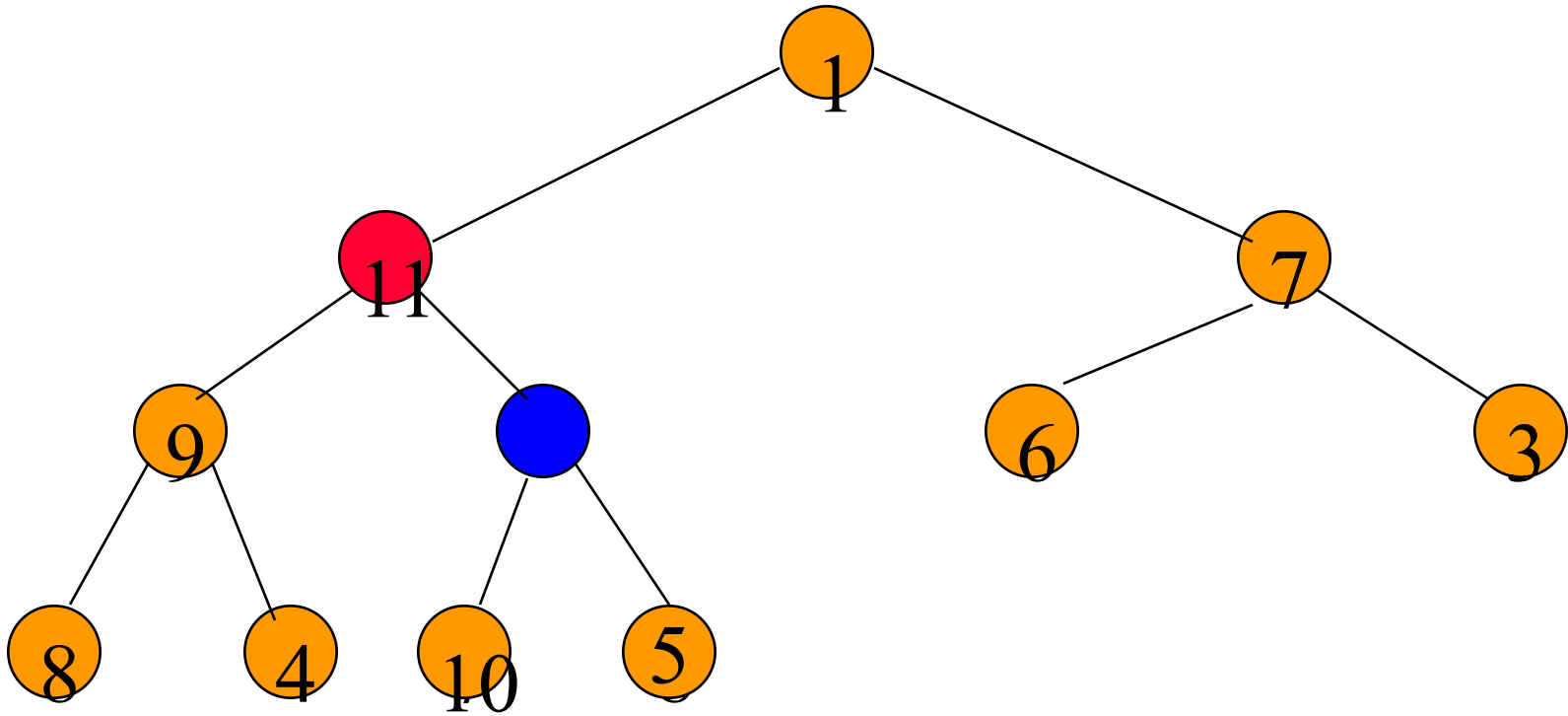
Initializing A Max Heap



Initializing A Max Heap

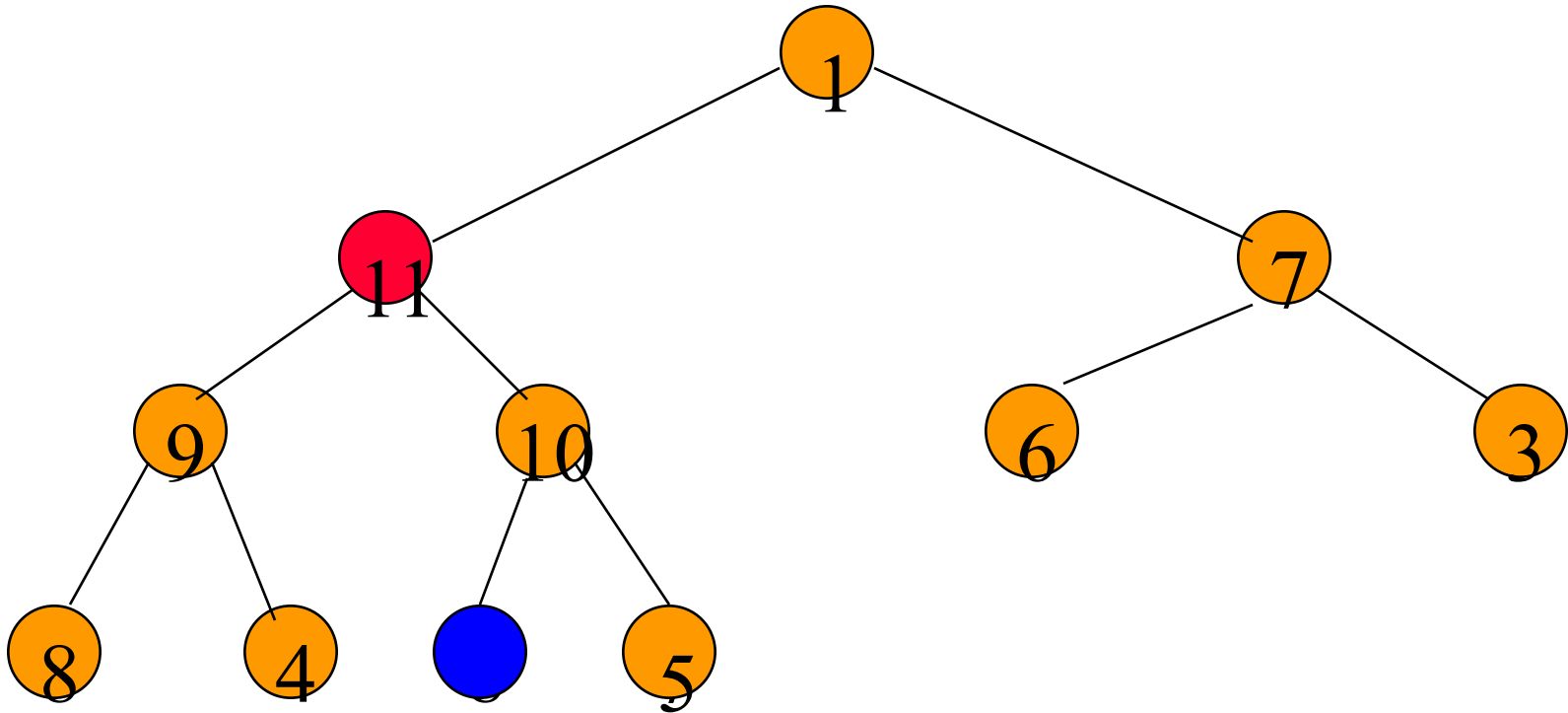


Initializing A Max Heap



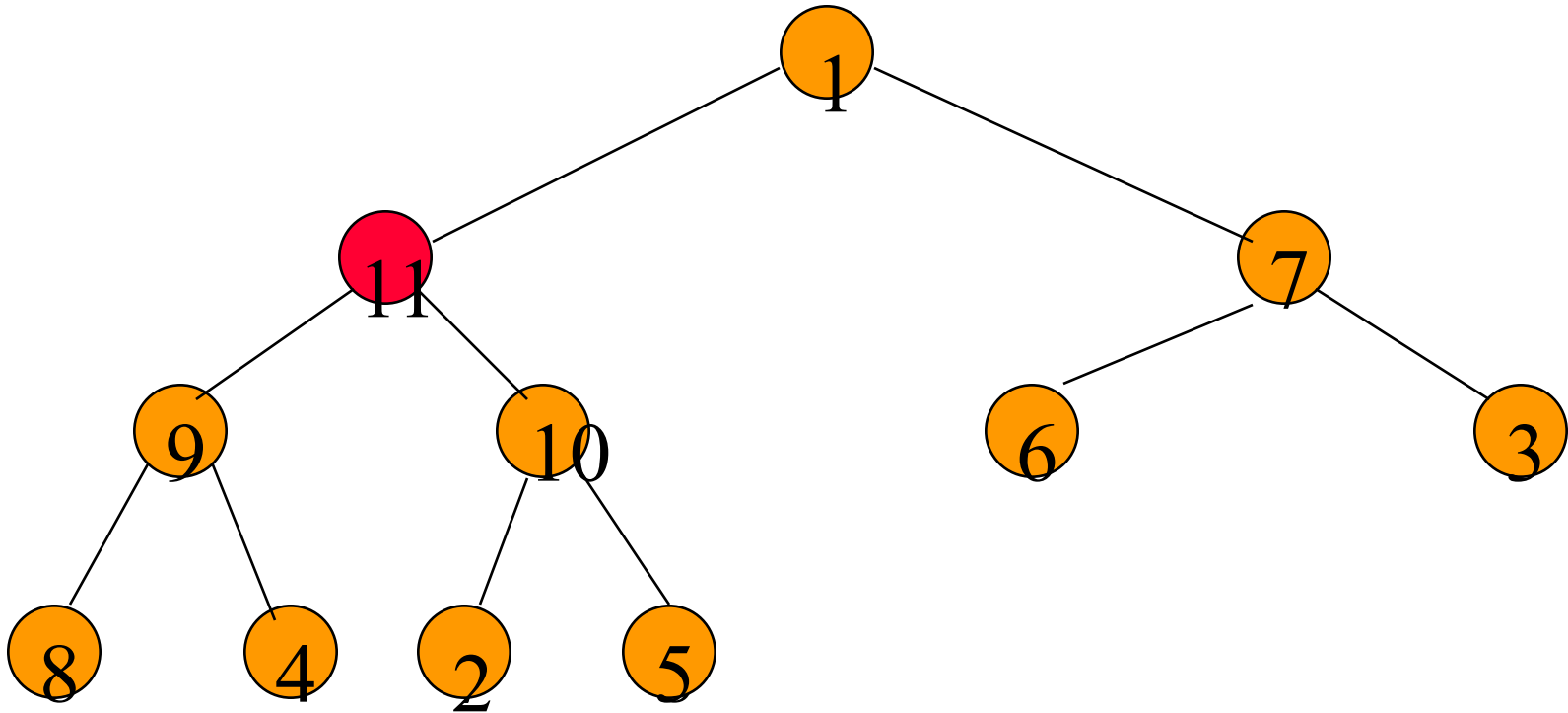
Find a home for 2.

Initializing A Max Heap



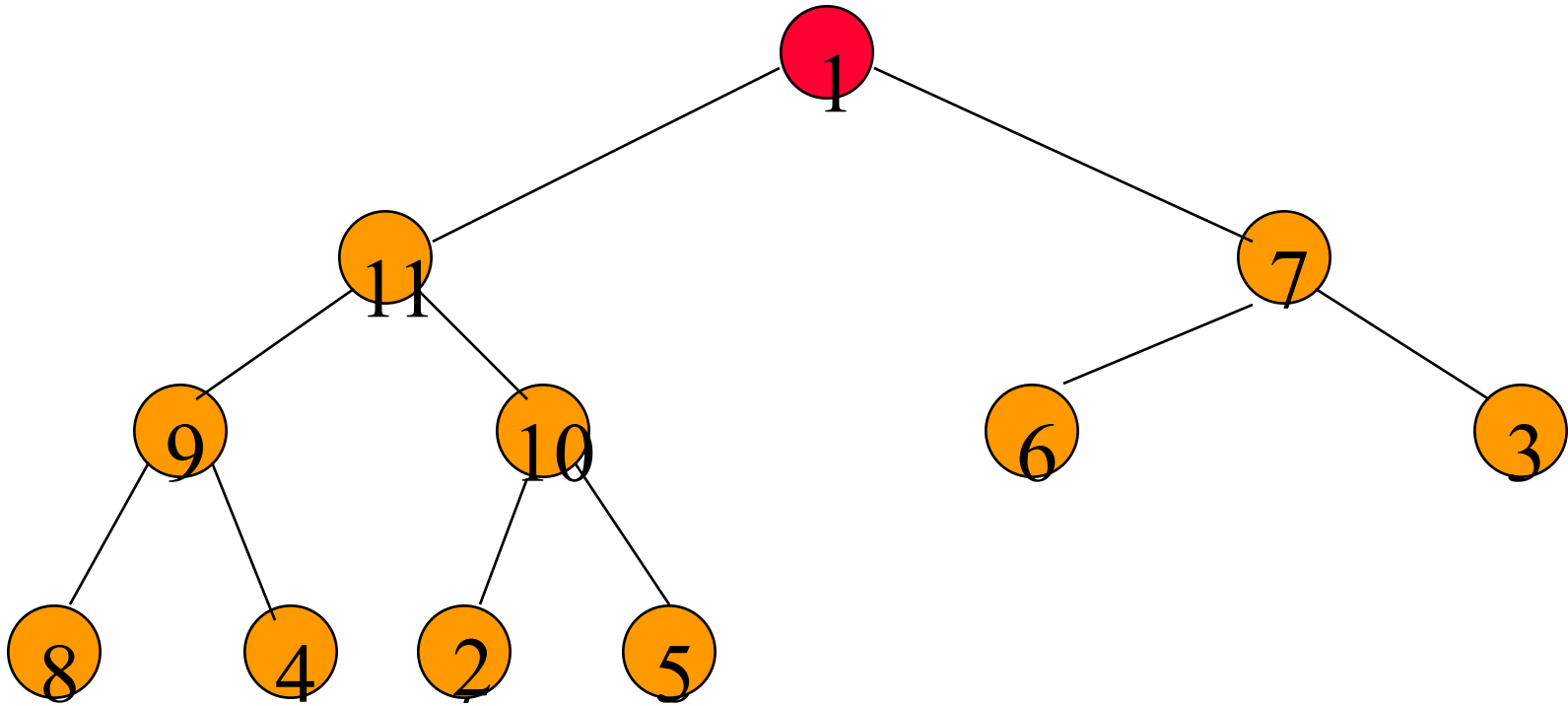
Find a home for 2.

Initializing A Max Heap



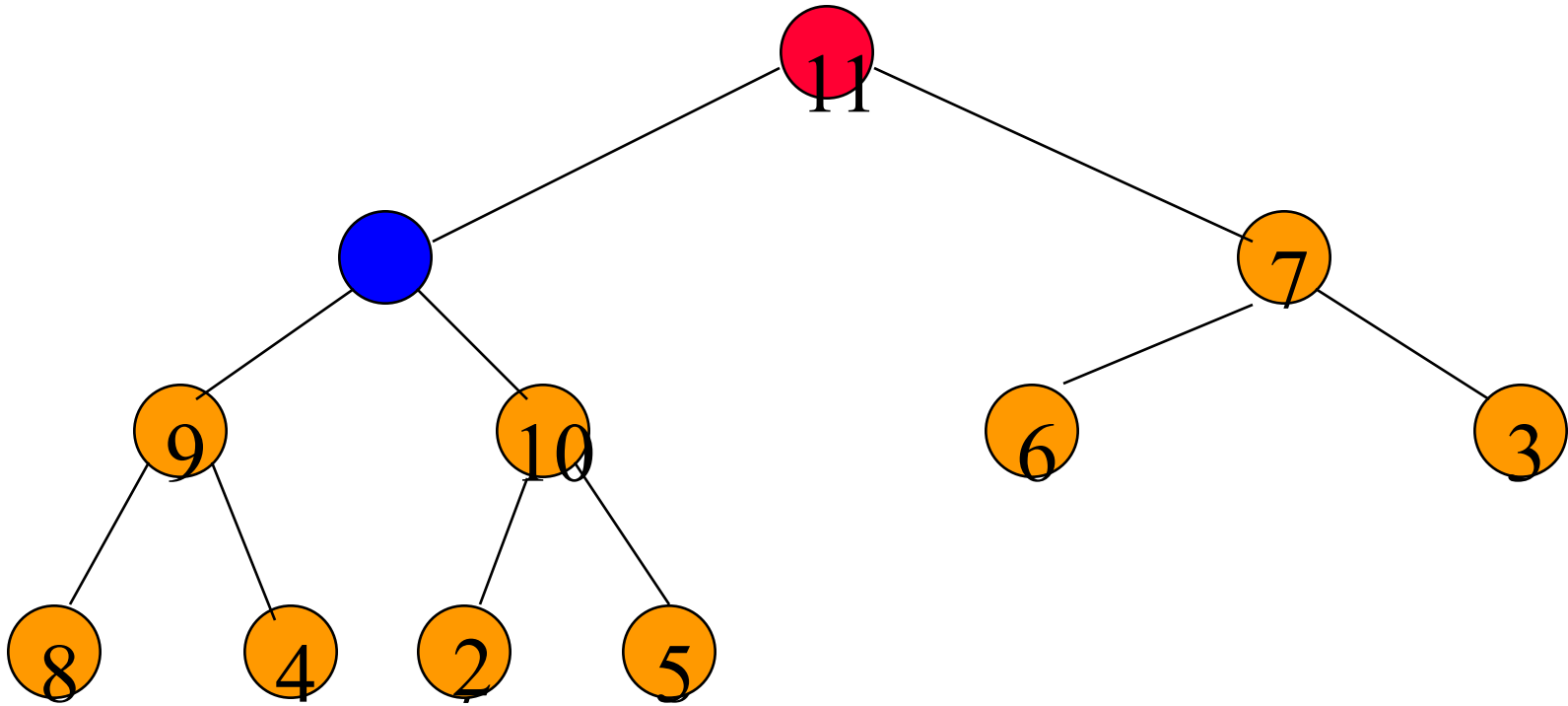
Done, move to next lower array position.

Initializing A Max Heap



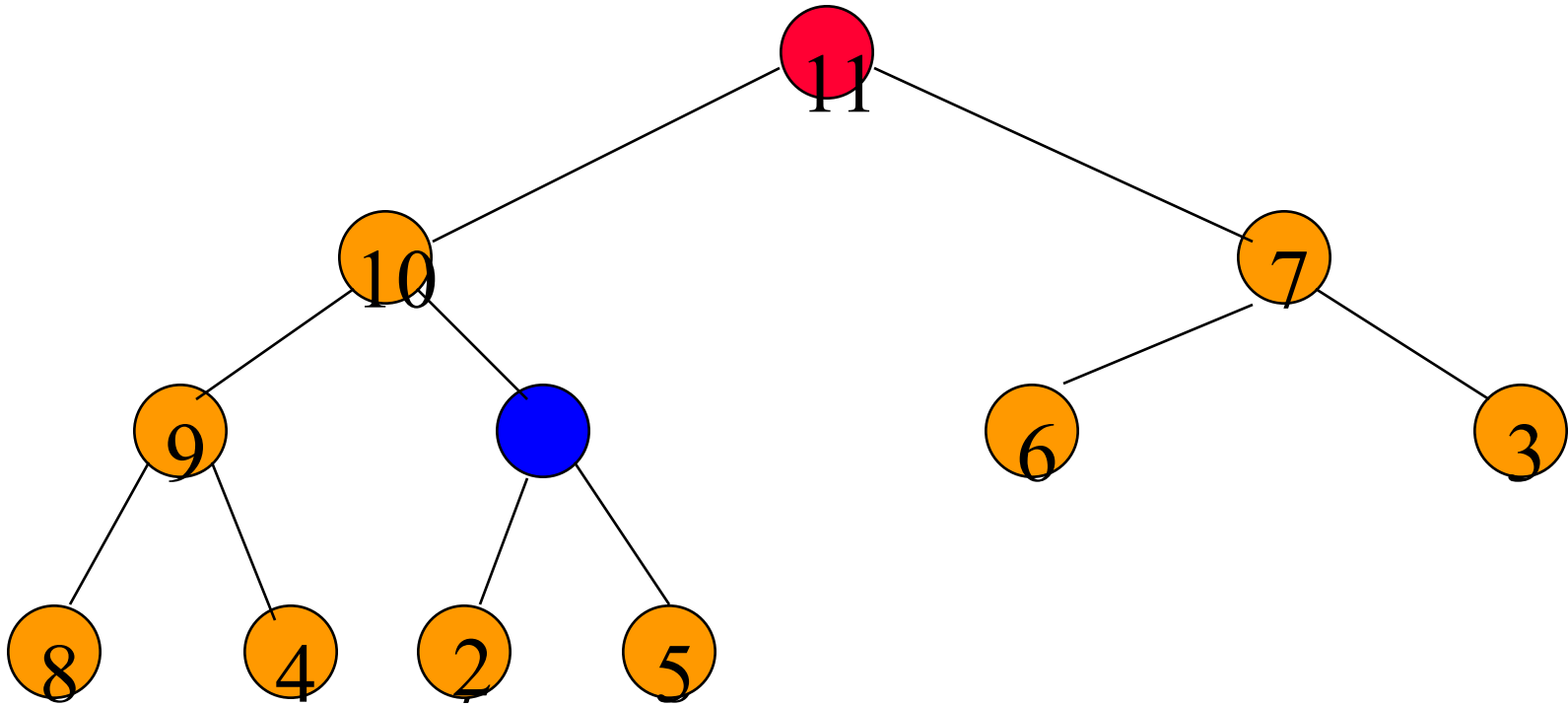
Find home for 1.

Initializing A Max Heap



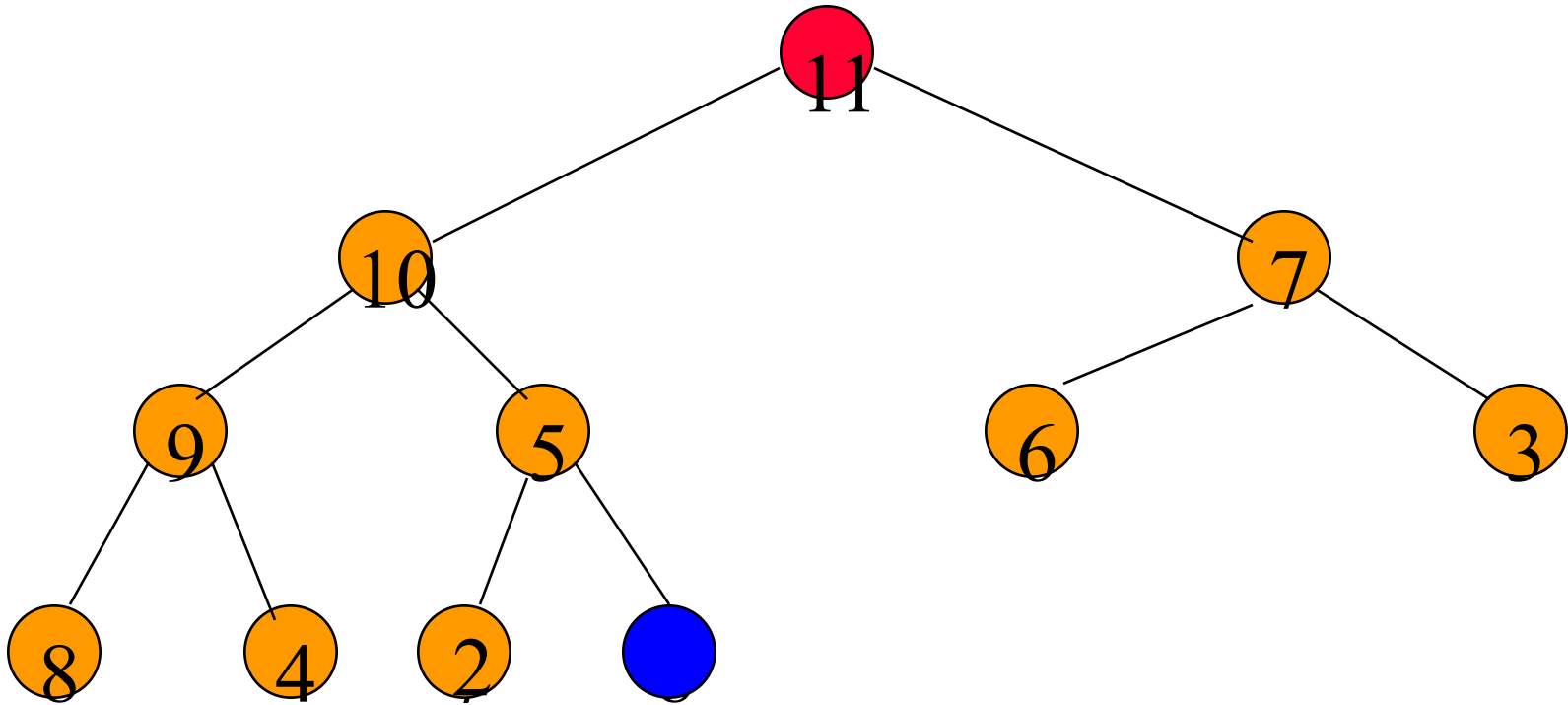
Find home for 1.

Initializing A Max Heap



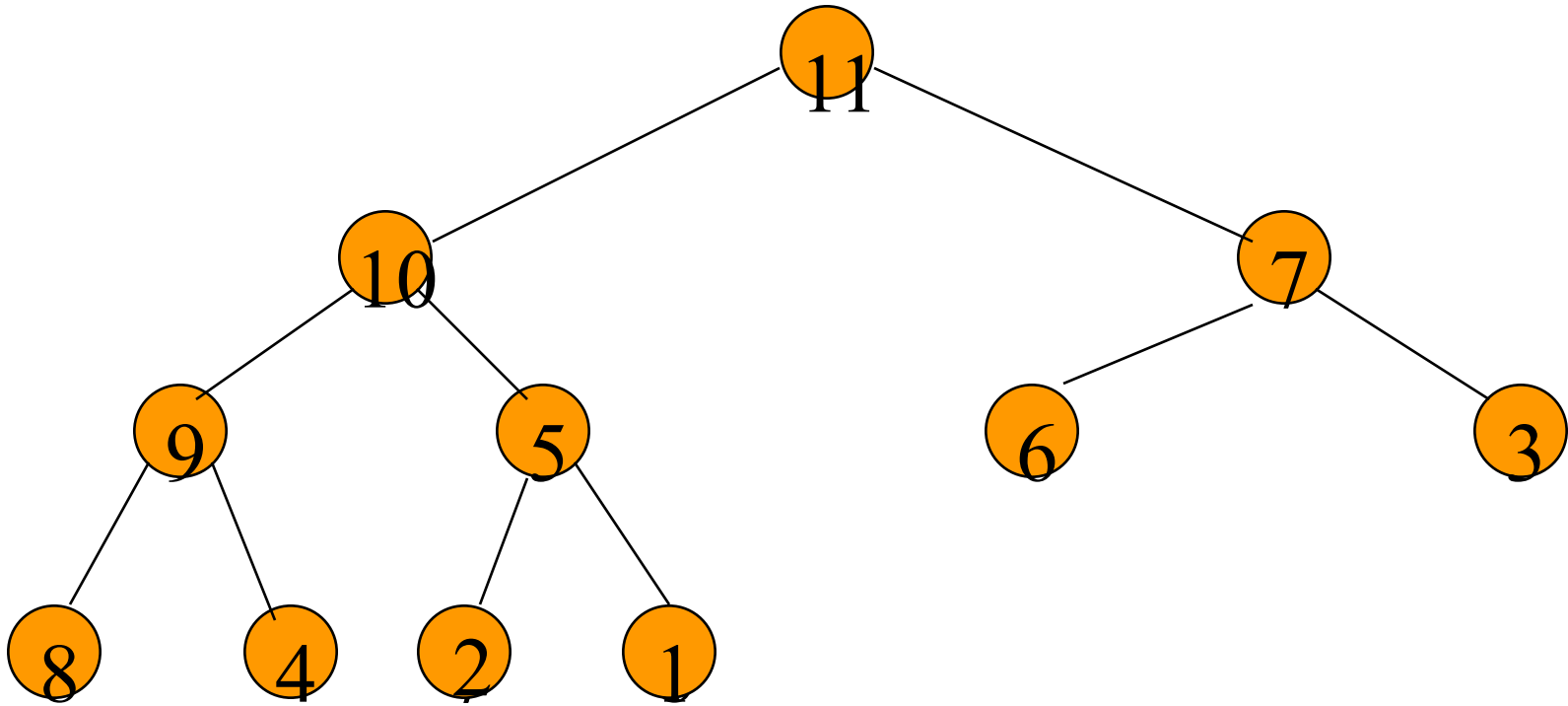
Find home for 1.

Initializing A Max Heap



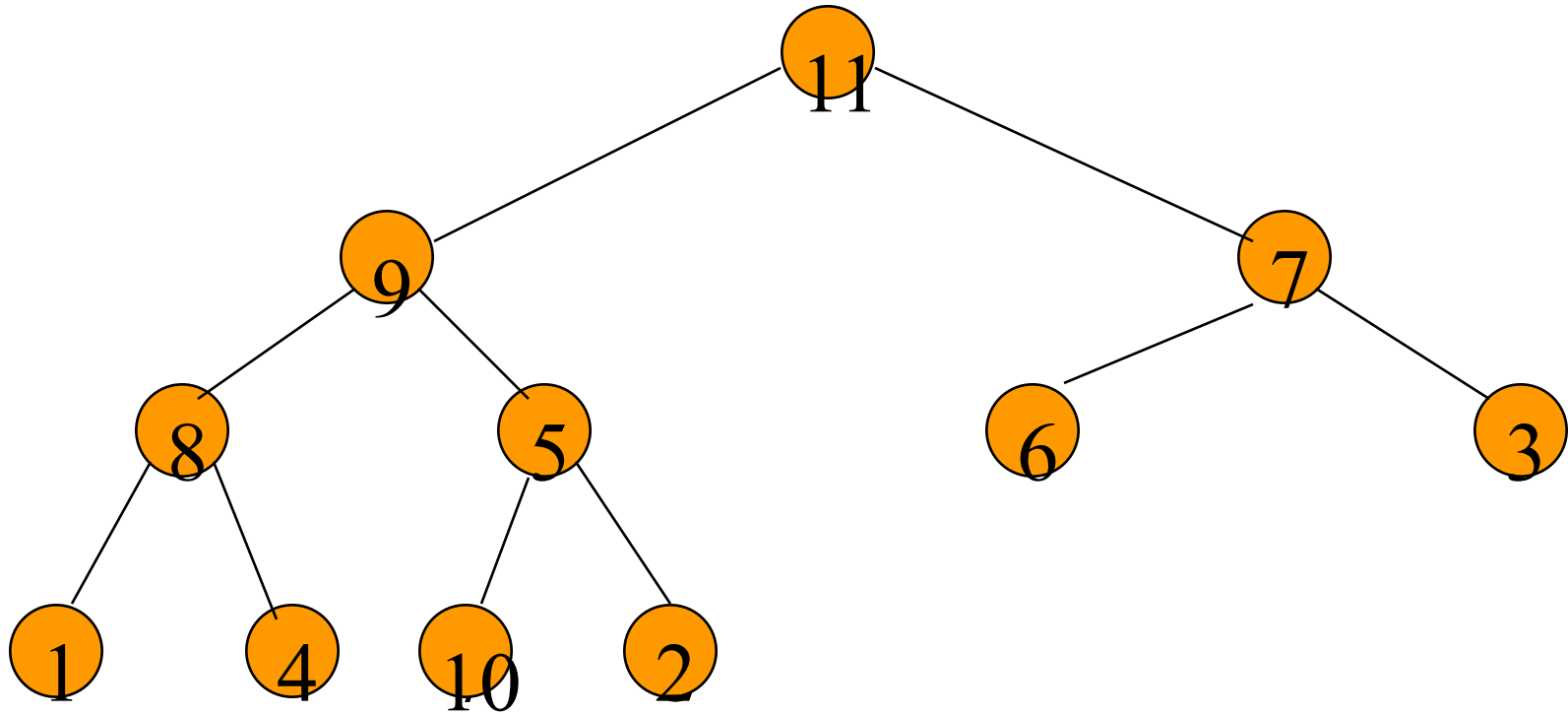
Find home for 1.

Initializing A Max Heap



Done.

Time Complexity



Height of heap = h .

Number of subtrees with root at level j is $\leq 2^{j-1}$.

Time for each subtree is $O(h-j+1)$.

Complexity



Time for level j subtrees is $\leq 2^{j-1}(h-j+1) = t(j)$.

Total time is $t(1) + t(2) + \dots + t(h-1) = O(n)$.

Programming:

Write an algorithm to initialize a Max Heap with C++.

Leftist Trees (optional)

Linked binary tree.

Can do everything a heap can do and in the same complexity.

- insert
- remove min (or max)
- initialize

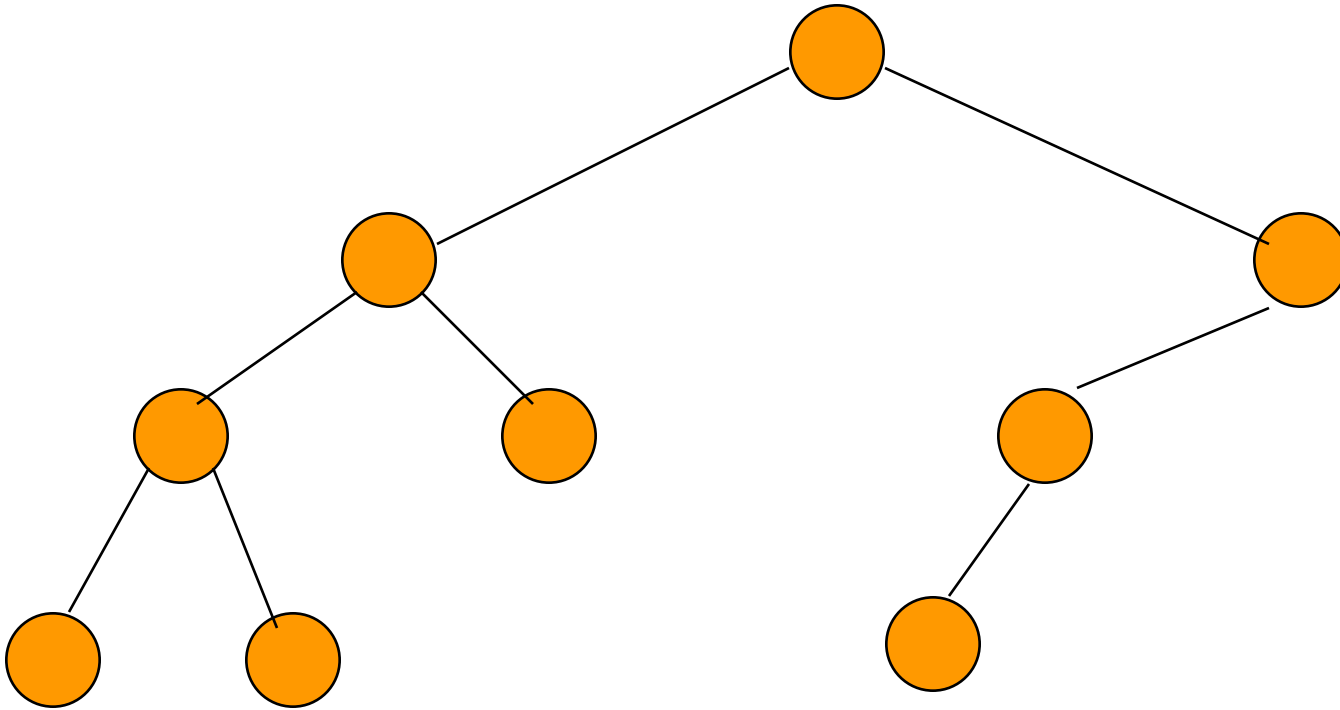
Can meld two leftist tree priority queues in $O(\log n)$ time.

Extended Binary Trees

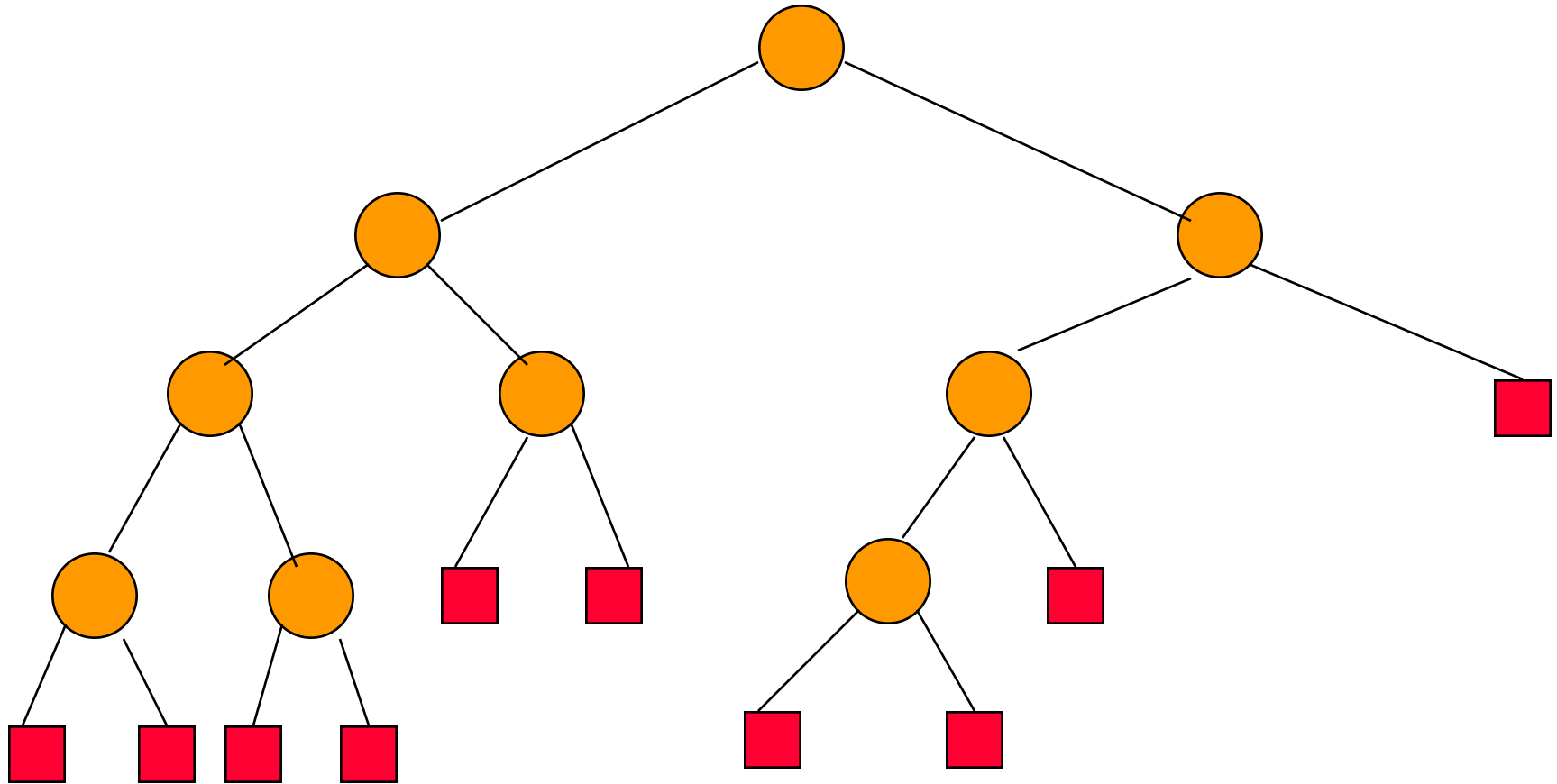
Start with any binary tree and add an external node wherever there is an empty subtree.

Result is an **extended** binary tree.

A Binary Tree



An Extended Binary Tree

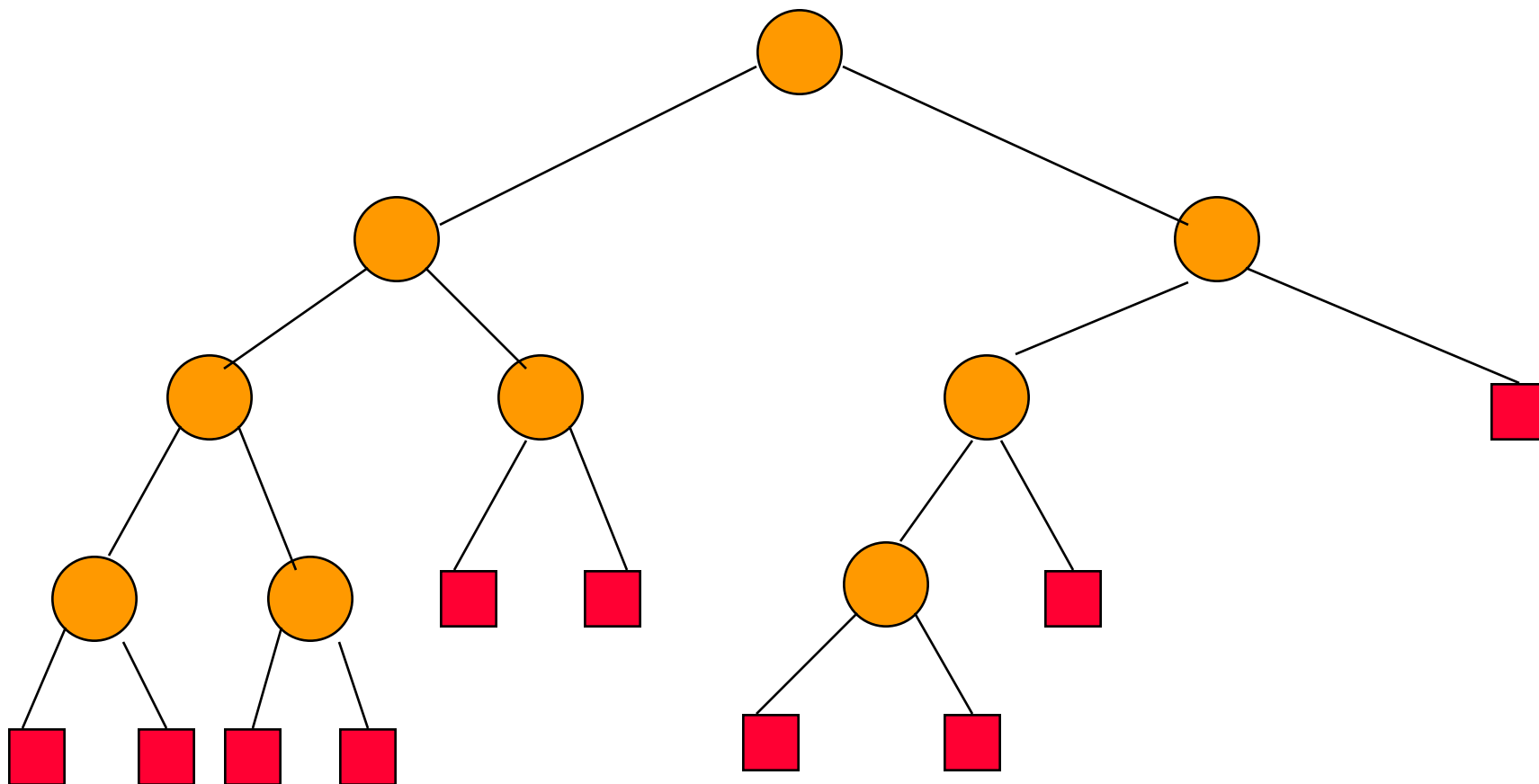


number of external nodes is $n+1$

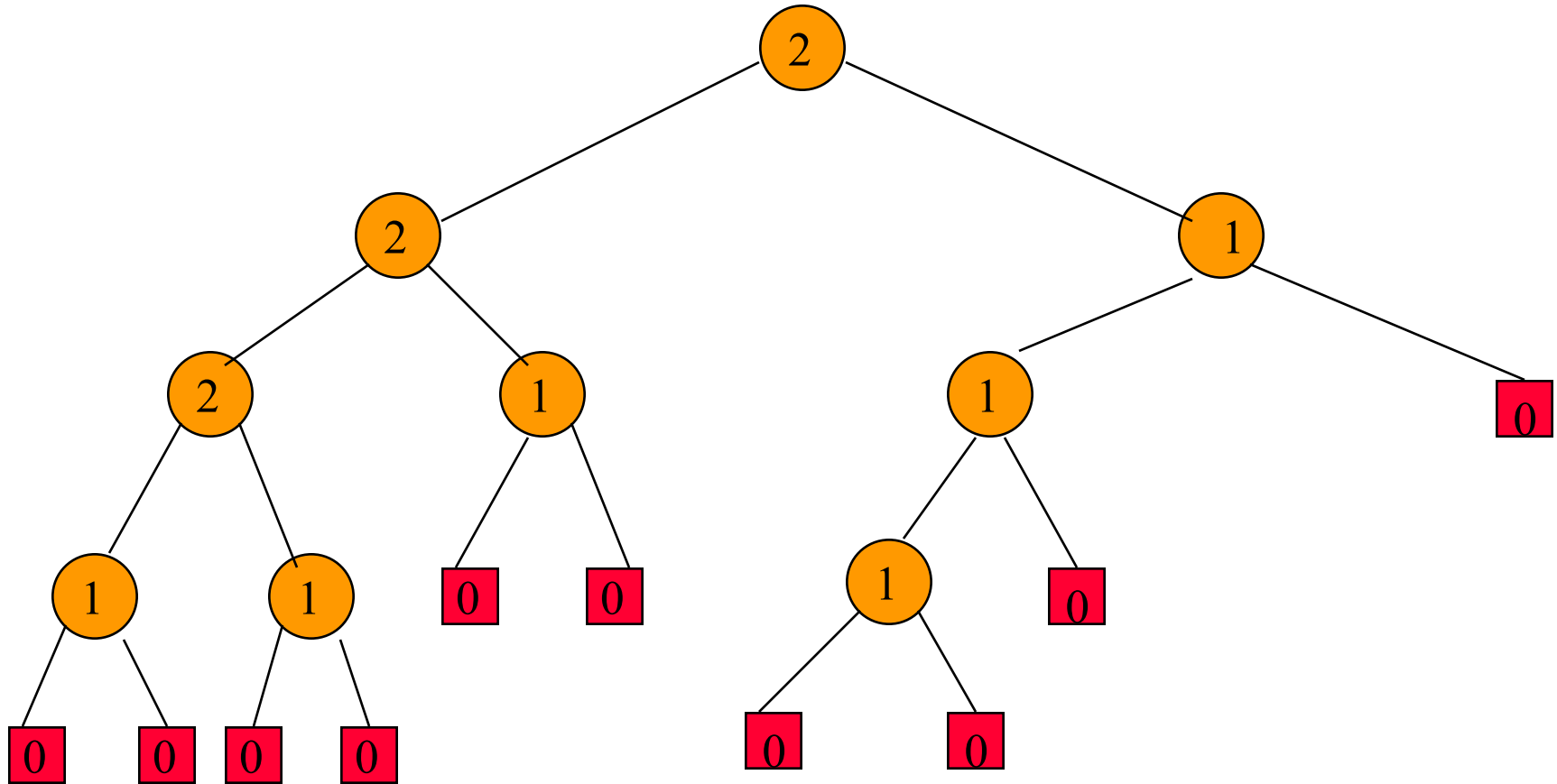
The Function $s()$

For any node x in an extended binary tree,
let $s(x)$ be the length of a shortest path
from x to an external node in the subtree
rooted at x .

S()



s() Values Example



Properties Of $s()$

If x is an external node, then $s(x) = 0$.

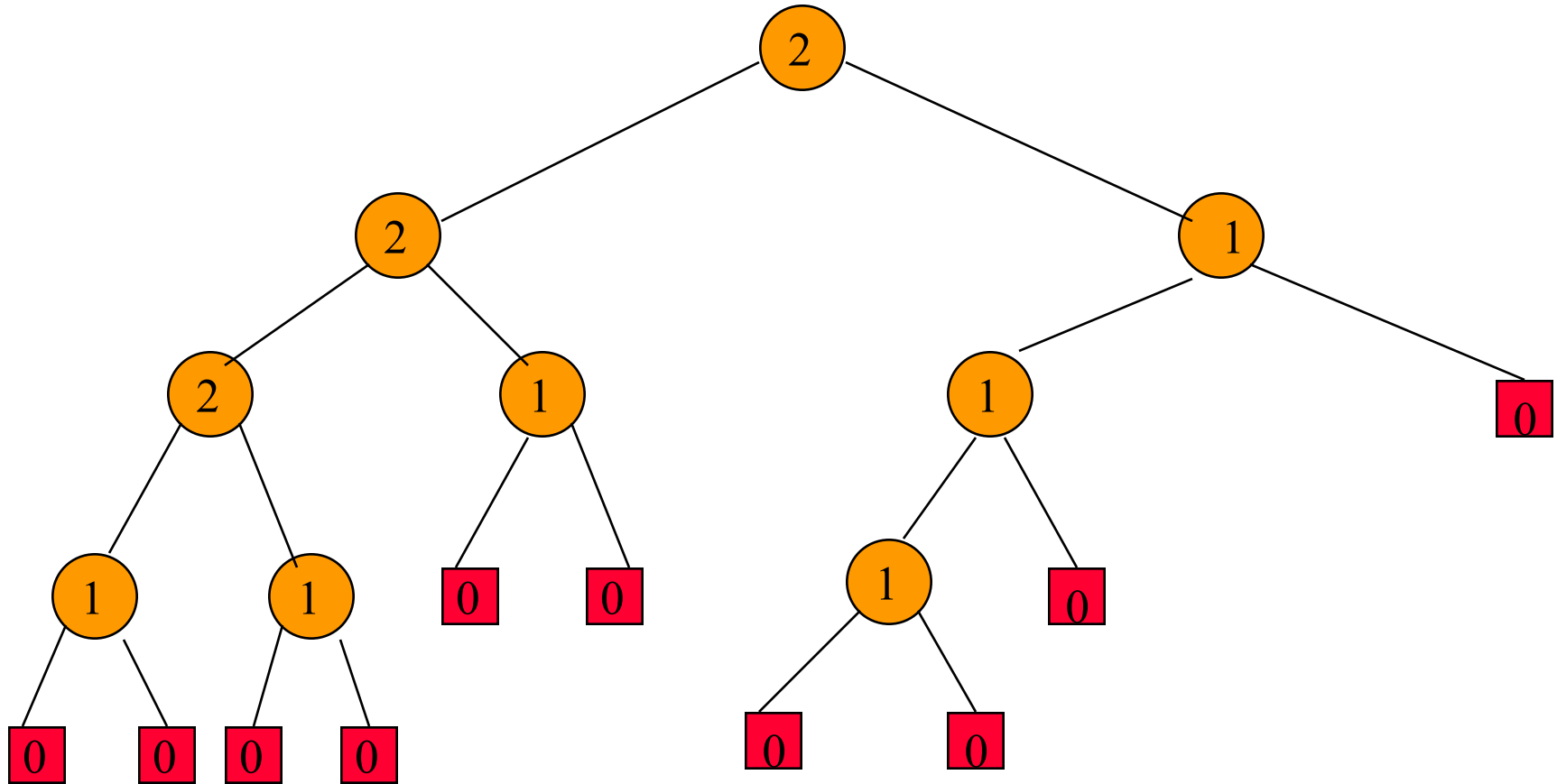
Otherwise,

$$s(x) = \min \{s(\text{leftChild}(x)), \\ s(\text{rightChild}(x))\} + 1$$

Height Biased Leftist Trees

A binary tree is a (height biased) leftist tree
iff for every internal node x ,
 $s(\text{leftChild}(x)) \geq s(\text{rightChild}(x))$

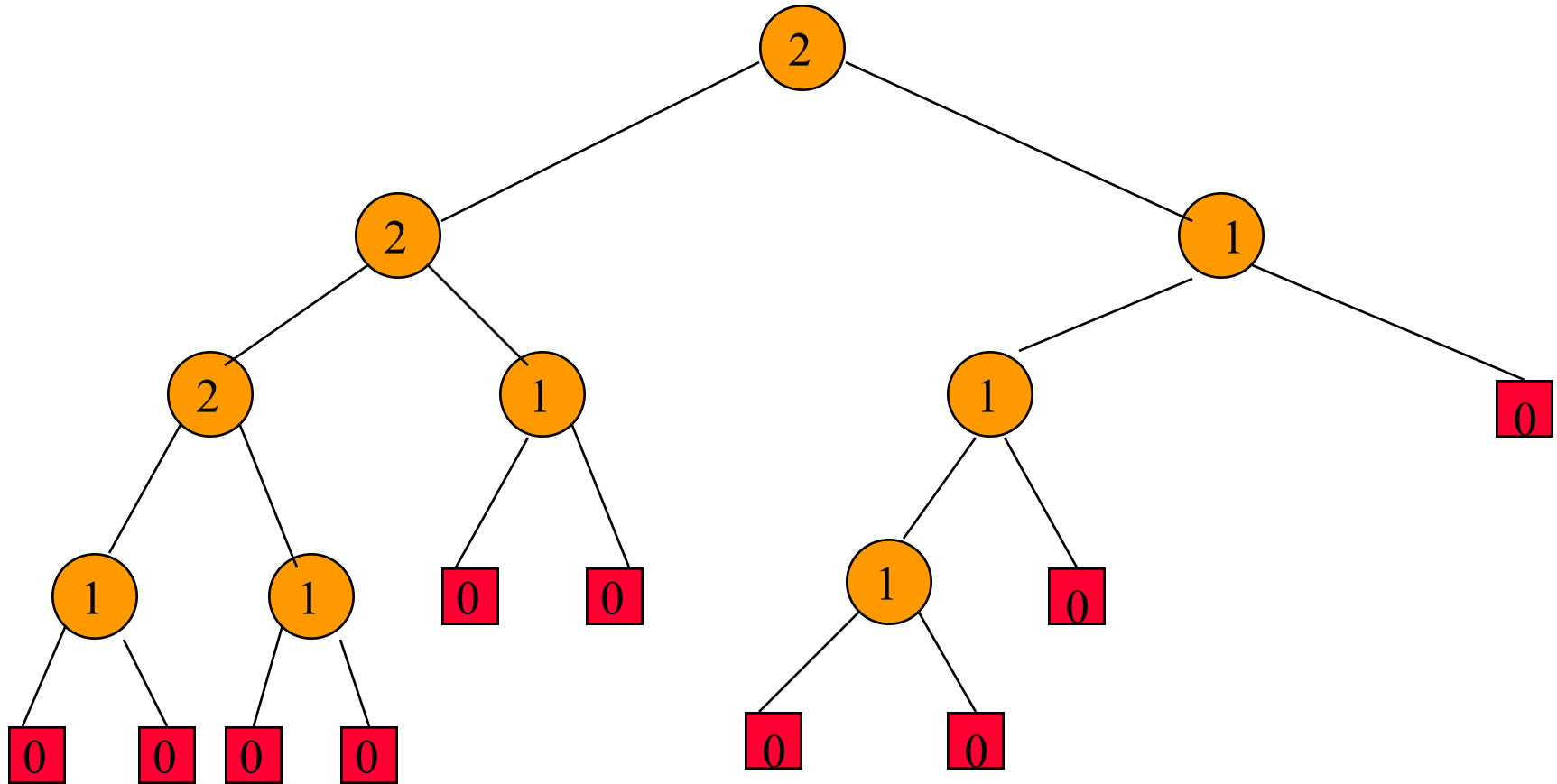
A Leftist Tree



Leftist Trees – Property 1

In a leftist tree, the rightmost path is a shortest root to external node path and the length of this path is $s(\text{root})$.

A Leftist Tree



Length of rightmost path is 2.

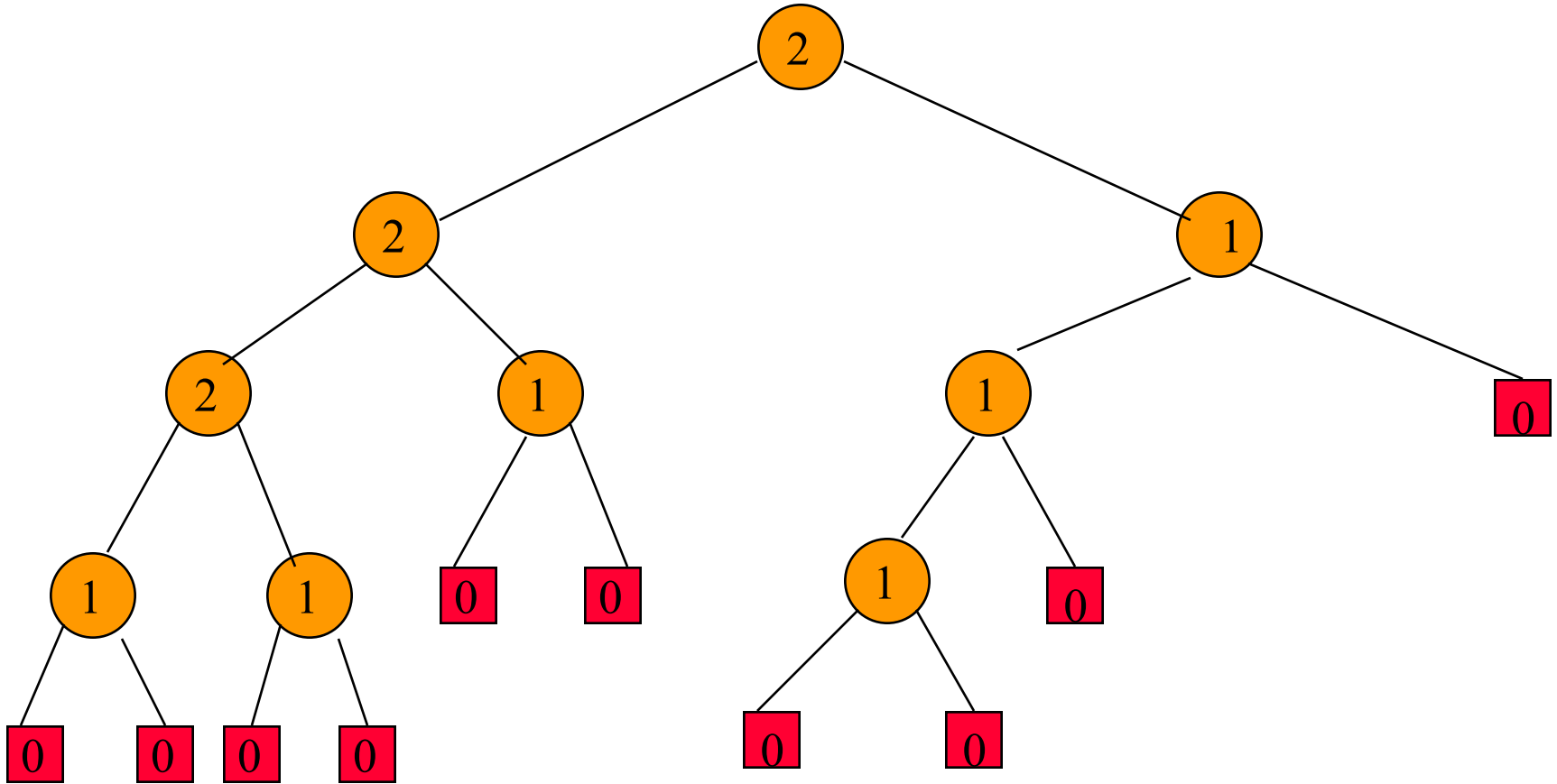
Leftist Trees—Property 2

The number of internal nodes is at least

$$2^{s(\text{root})} - 1$$

Because levels 1 through $s(\text{root})$ have no external nodes.

A Leftist Tree



Levels **1** and **2** have no external nodes.

Leftist Trees—Property 3

Length of rightmost path is $O(\log n)$, where n is the number of (internal) nodes in a leftist tree.

Property 2 $\Rightarrow$

$$\blacksquare n \geq 2^{s(\text{root})} - 1 \Rightarrow s(\text{root}) \leq \log_2(n+1)$$

Property 1 $\Rightarrow$ length of rightmost path is $s(\text{root})$.

Leftist Trees As Priority Queues

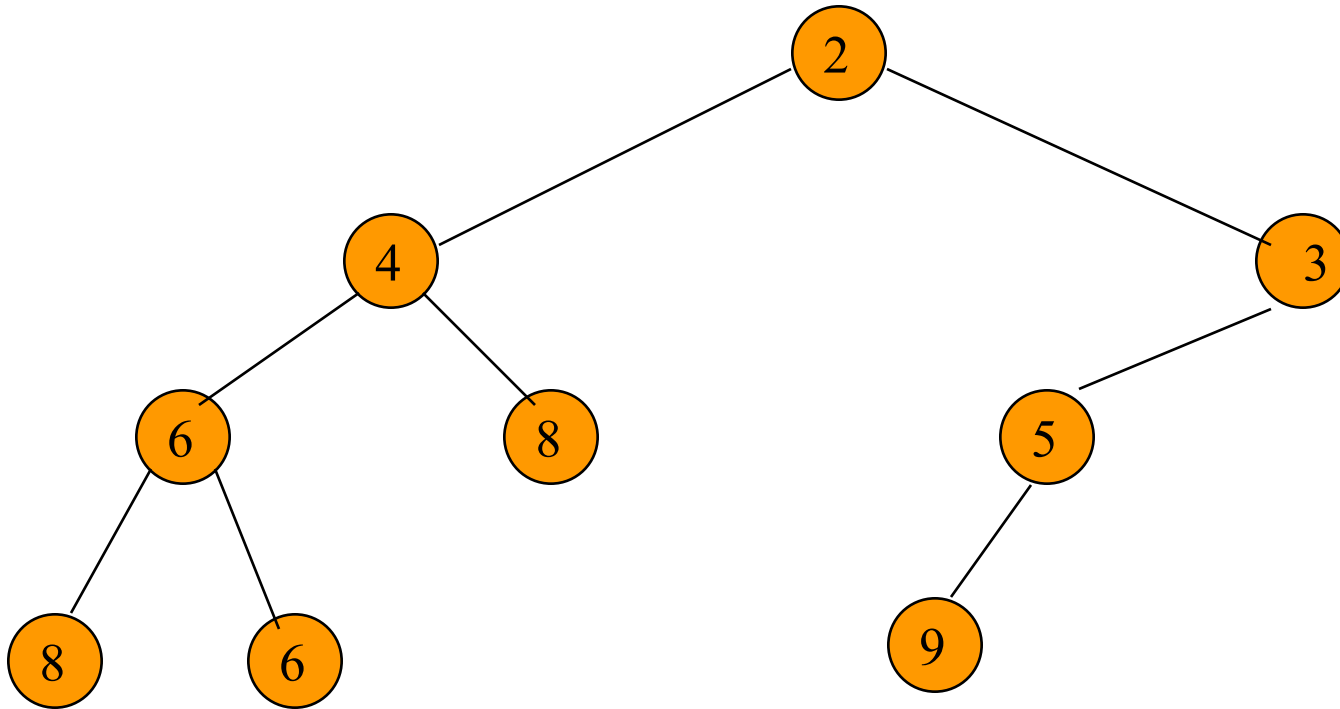
Min leftist tree ... leftist tree that is a min tree.

Used as a min priority queue.

Max leftist tree ... leftist tree that is a max tree.

Used as a max priority queue.

A Min Leftist Tree



Some Min Leftist Tree Operations

put

removeMin()

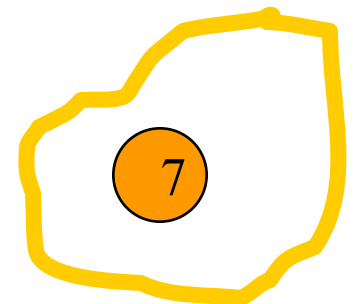
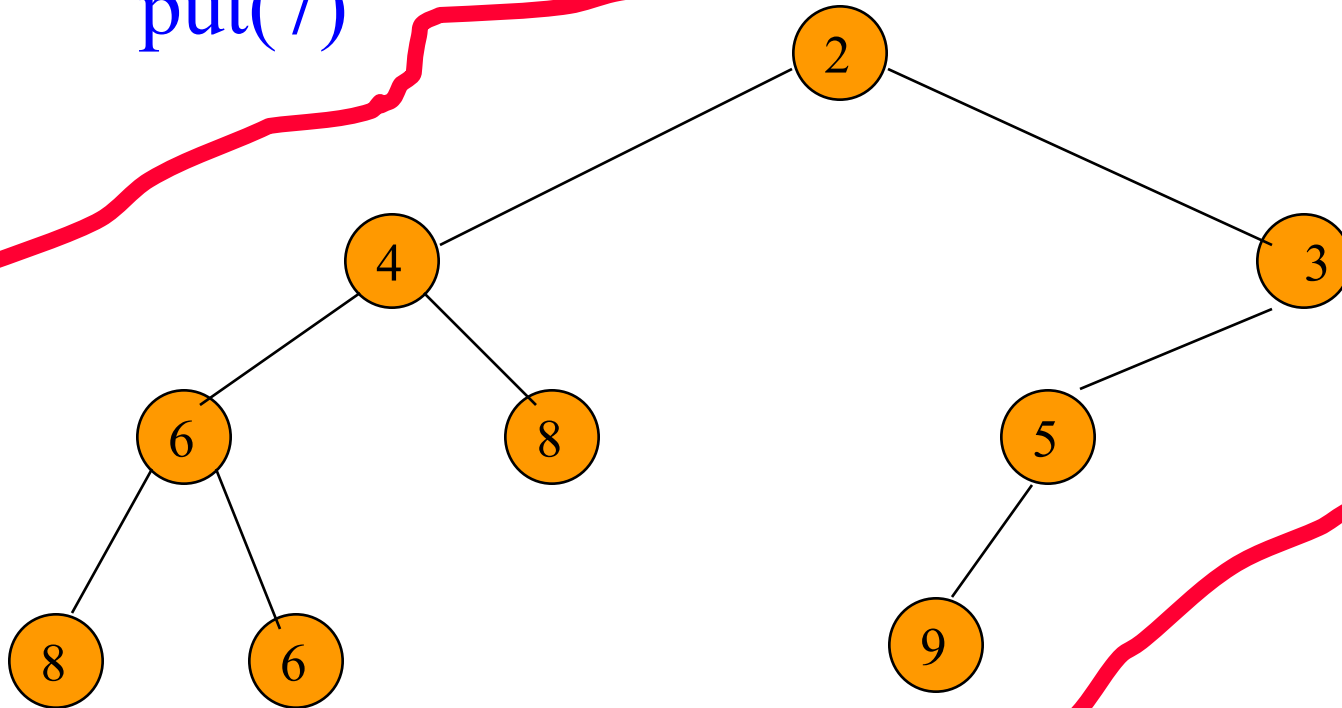
meld()

initialize()

put() and removeMin() use meld().

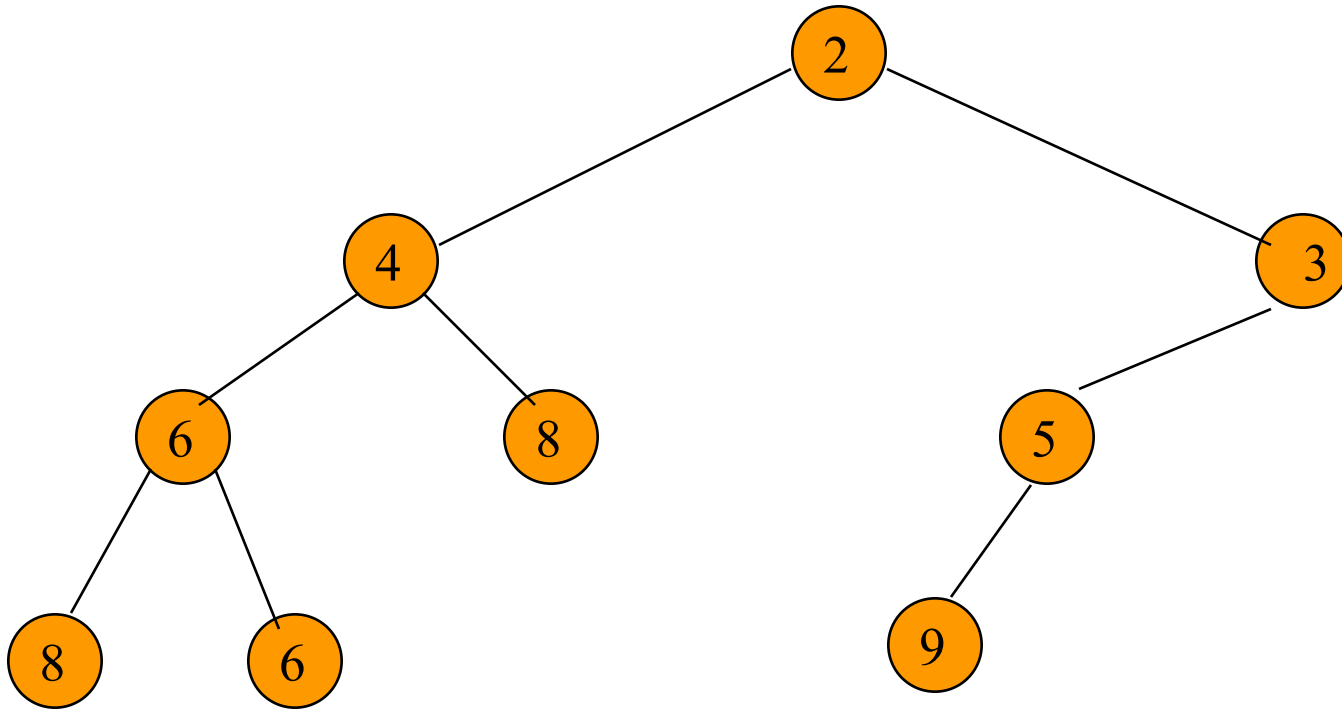
Put Operation

put(7)



Create a single node min leftist tree.
Meld the two min leftist trees.

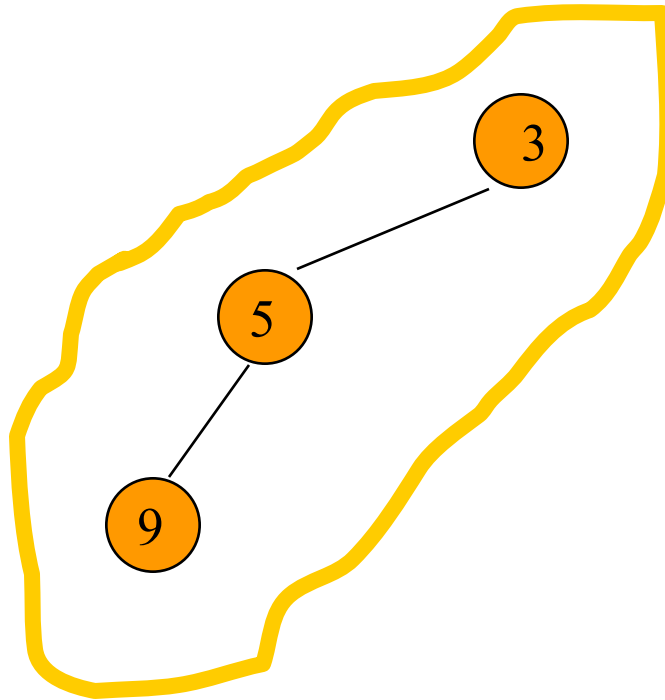
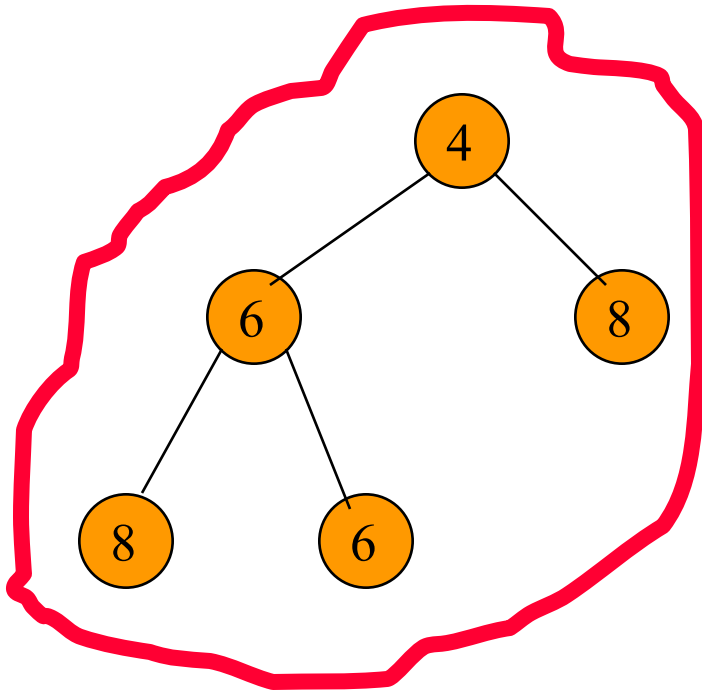
Remove Min



Remove the root.

Remove Min

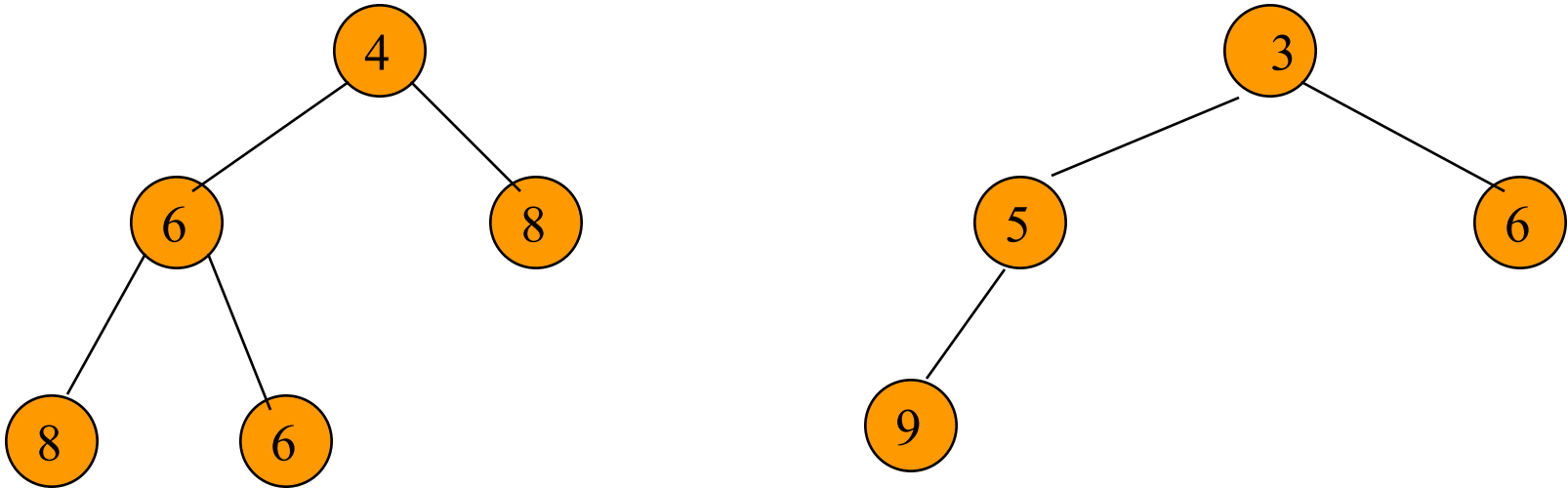
2



Remove the root.

Meld the two subtrees.

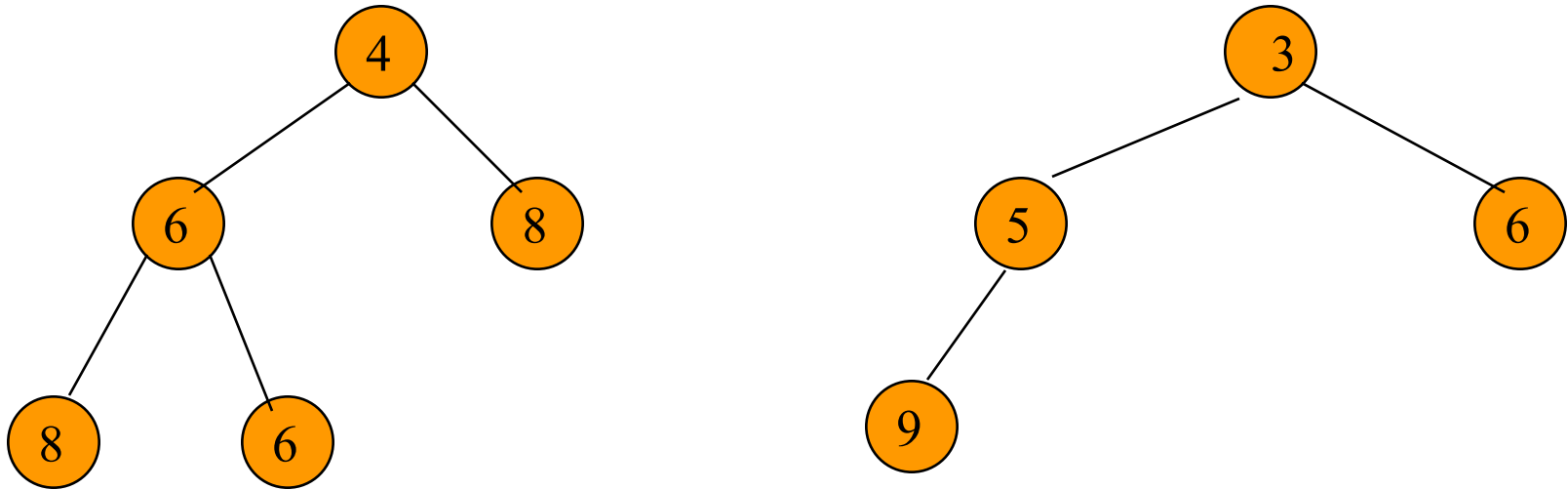
Meld Two Min Leftist Trees



HOW to get logarithmic performance?

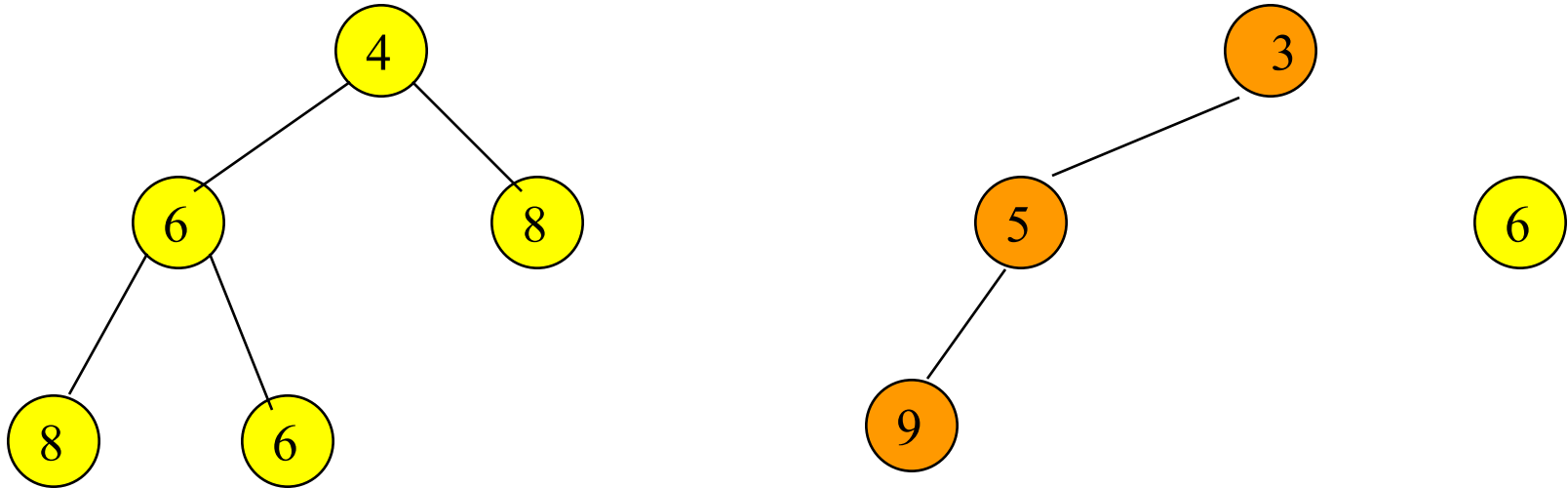
Traverse only the rightmost paths

Meld Two Min Leftist Trees



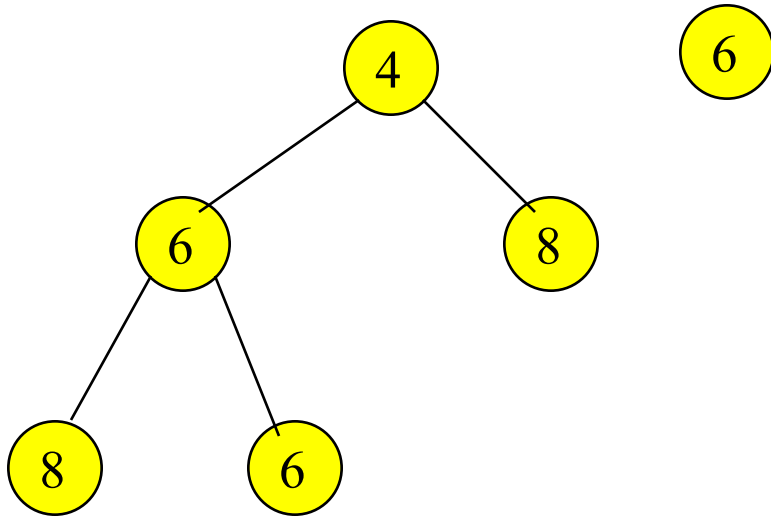
Meld right subtree of tree with smaller root and all of other tree.

Meld Two Min Leftist Trees



Meld right subtree of tree with smaller root and all of other tree.

Meld Two Min Leftist Trees



Meld right subtree of tree with smaller root and all of other tree.

Meld Two Min Leftist Trees



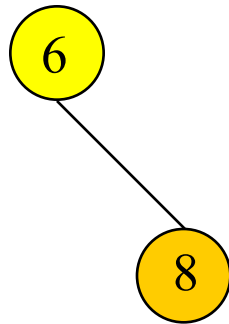
Meld right subtree of tree with smaller root and all of other tree.

Right subtree of **6** is empty. So, result of melding right subtree of tree with smaller root and other tree is the other tree.

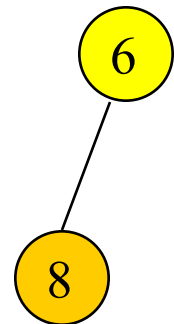
Meld Two Min Leftist Trees



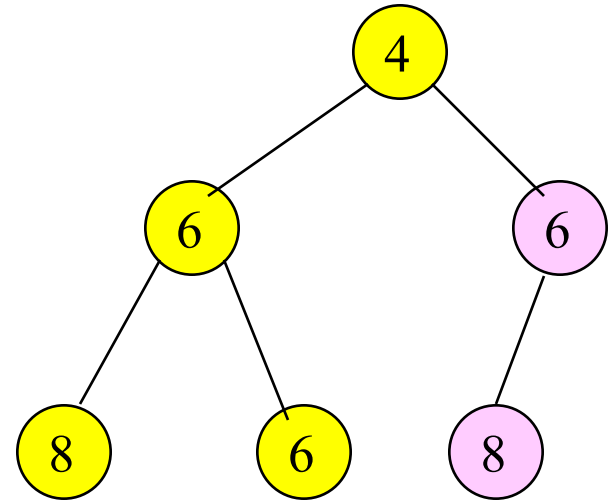
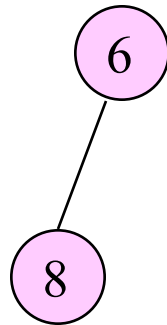
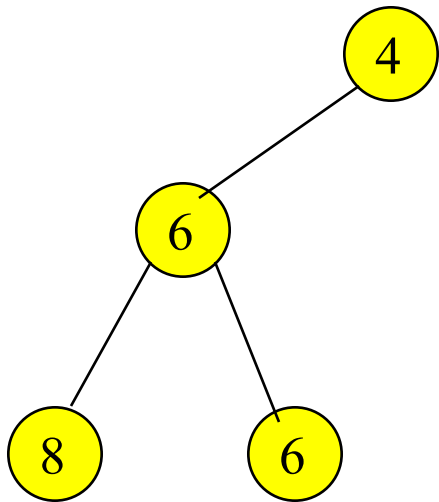
Make melded subtree right subtree of smaller root.



Swap left and right subtree if $s(\text{left}) < s(\text{right})$.



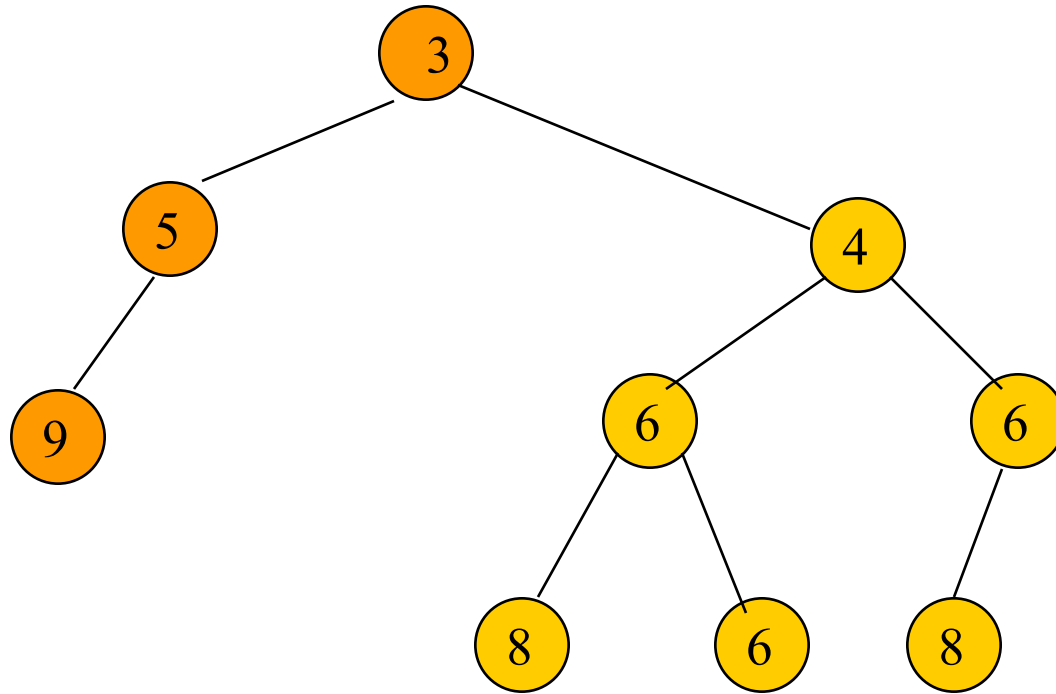
Meld Two Min Leftist Trees



Make melded subtree right subtree of smaller root.

Swap left and right subtree if $s(\text{left}) < s(\text{right})$.

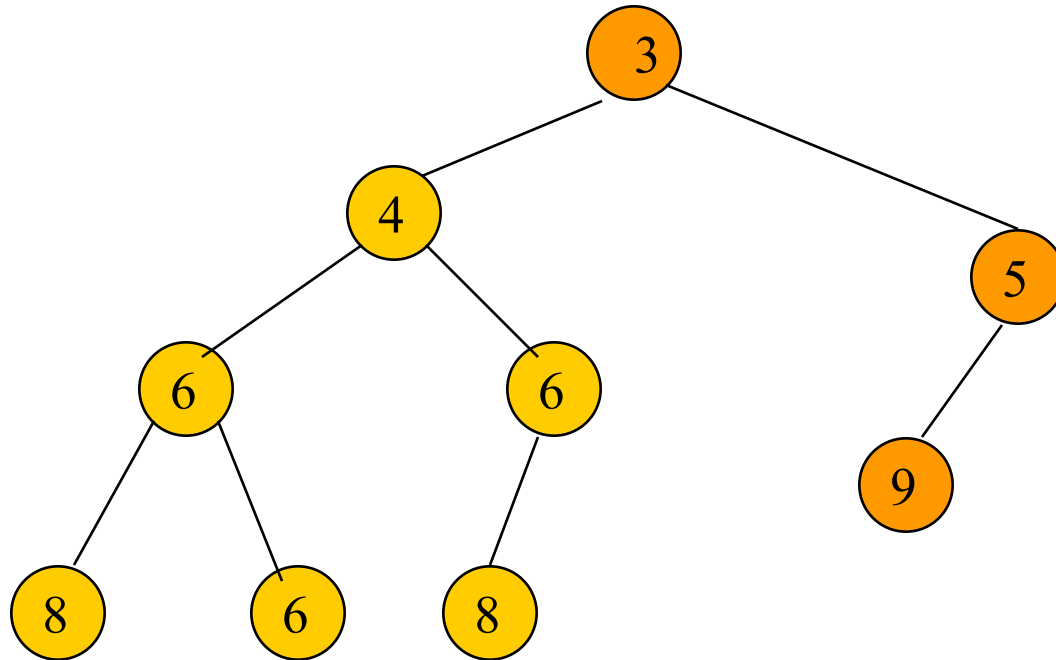
Meld Two Min Leftist Trees



Make melded subtree right subtree of smaller root.

Swap left and right subtree if $s(\text{left}) < s(\text{right})$.

Meld Two Min Leftist Trees

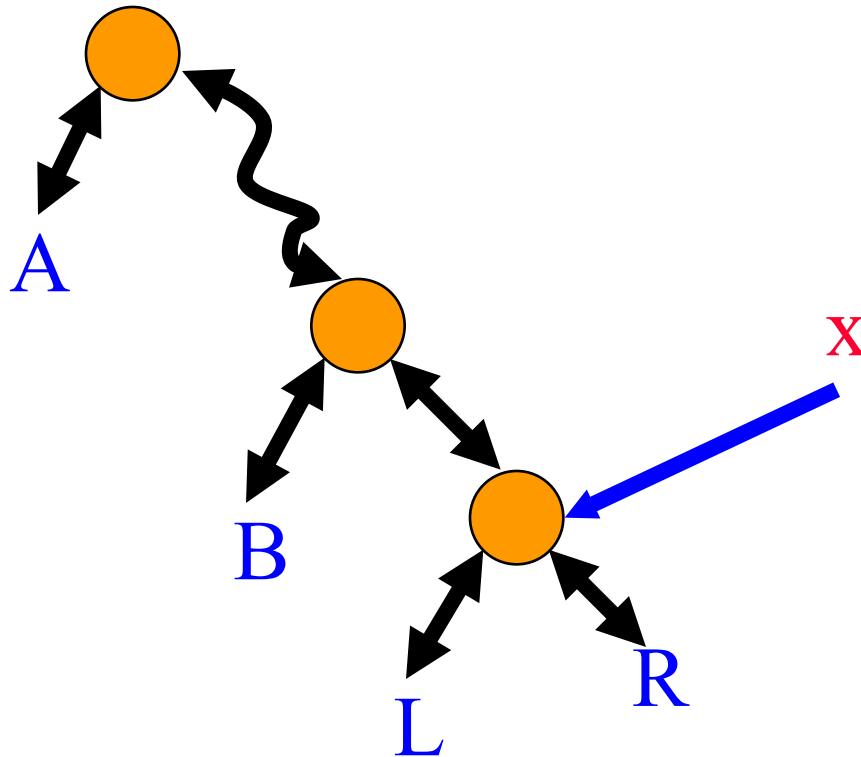


Initializing In $O(n)$ Time

- Create n single-node min leftist trees and place them in a FIFO queue.
- Repeatedly remove two min leftist trees from the FIFO queue, meld them, and put the resulting min leftist tree into the FIFO queue.
- The process terminates when only 1 min leftist tree remains in the FIFO queue.
- Analysis is the same as for heap initialization.

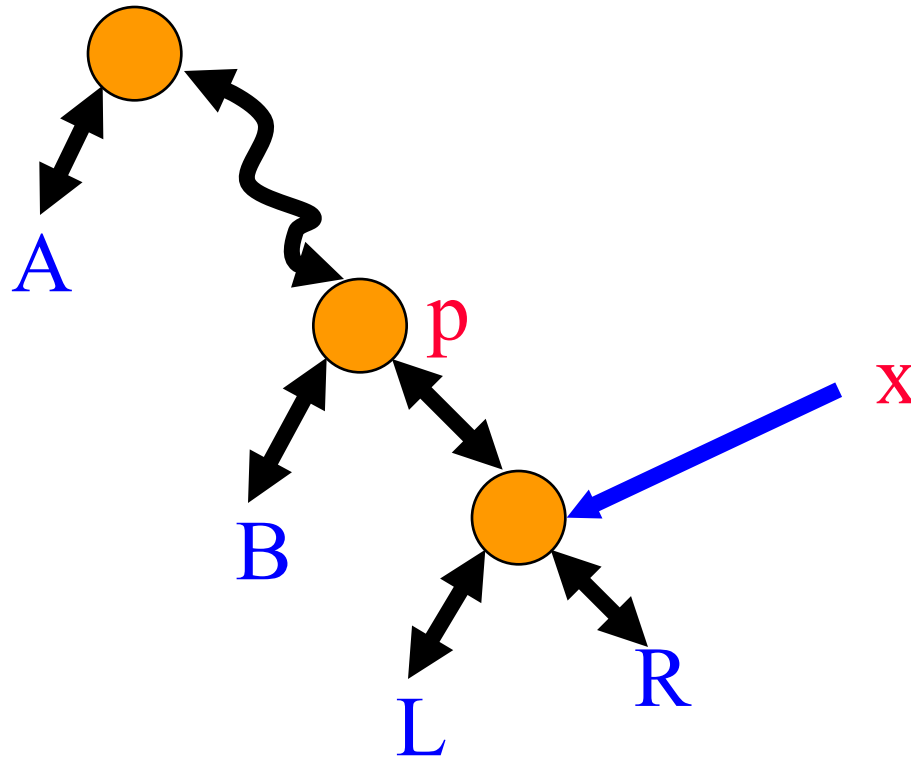
Arbitrary Remove

Remove element in node pointed at by **x**.



x = root $\Rightarrow$ remove min.

Arbitrary Remove, $x \neq \text{root}$

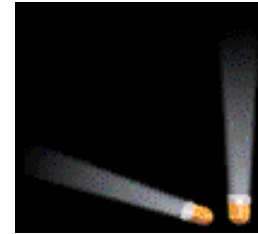
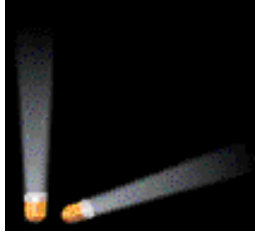


Make **L** right subtree of **p**.

Adjust **s** and leftist property on path from **p** to root.

Meld with **R**.

Selection Trees/ Tournament Trees



Polynomial addition (no equal items)

Merge: 2 sorted lists $\rightarrow$ one

2 items : smaller one selected

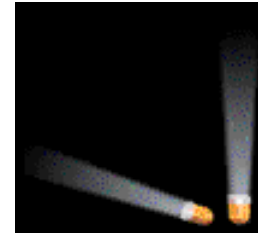
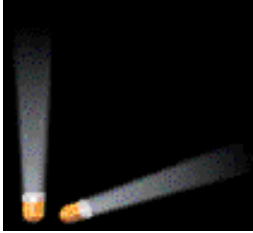
K polynomials?

Merge: k sorted lists $\rightarrow$ one

k items : the smallest one is selected

HOW?

Selection Trees/ Tournament Trees



Winner trees.

Loser Trees.

World Cup Knockout

16 teams

8 1/8 matches $\rightarrow$ 8 winners

4 1/4 matches $\rightarrow$ 4 winners

2 semifinal matches $\rightarrow$ 2 winners

1 final match $\rightarrow$ World Cup Championship

Winner Tree: A simulation

Winner Trees

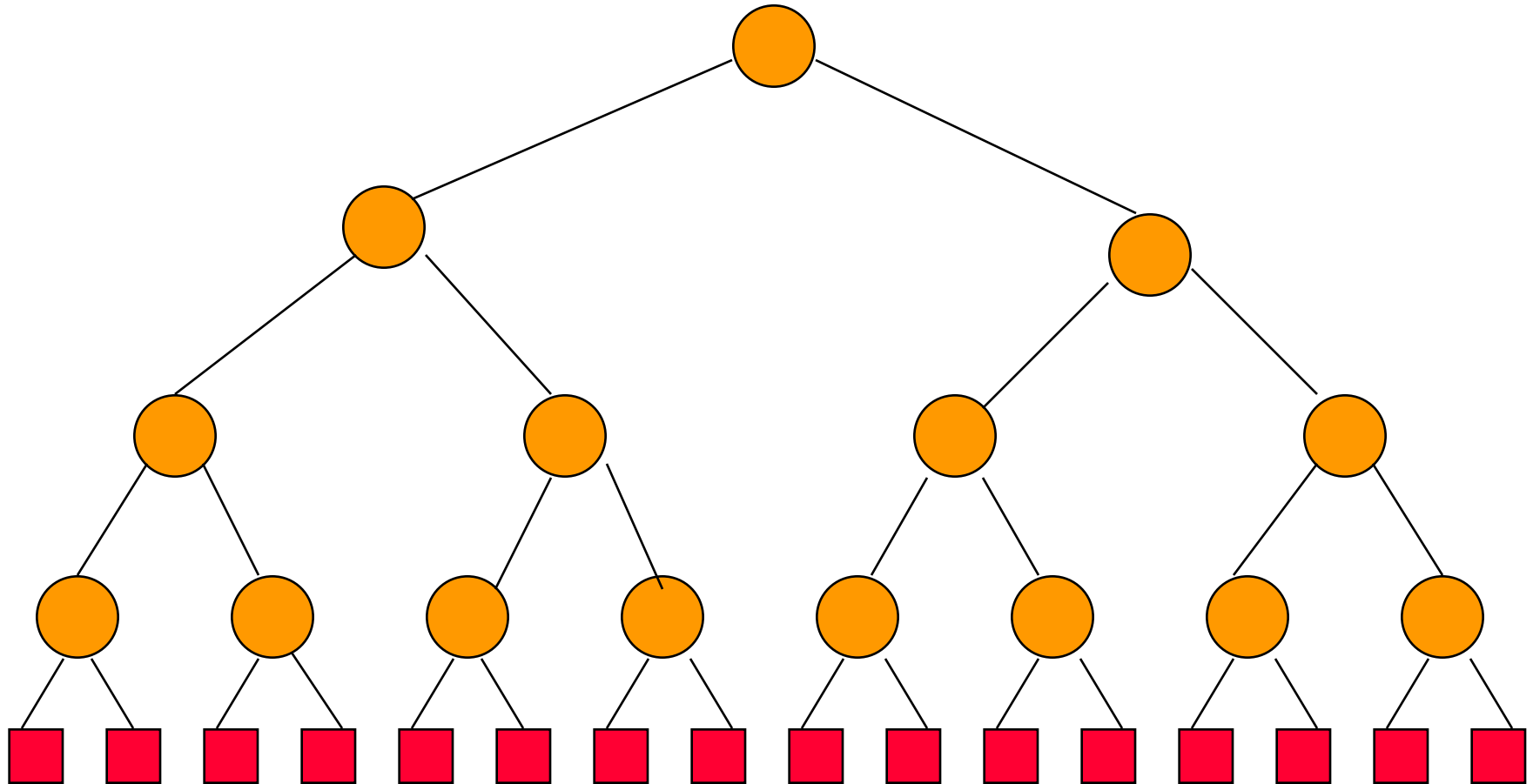
Complete binary tree with n external nodes and $n - 1$ internal nodes.

External nodes represent tournament players.

Each internal node represents a match played between its two children; the winner of the match is stored at the internal node.

Root has overall winner.

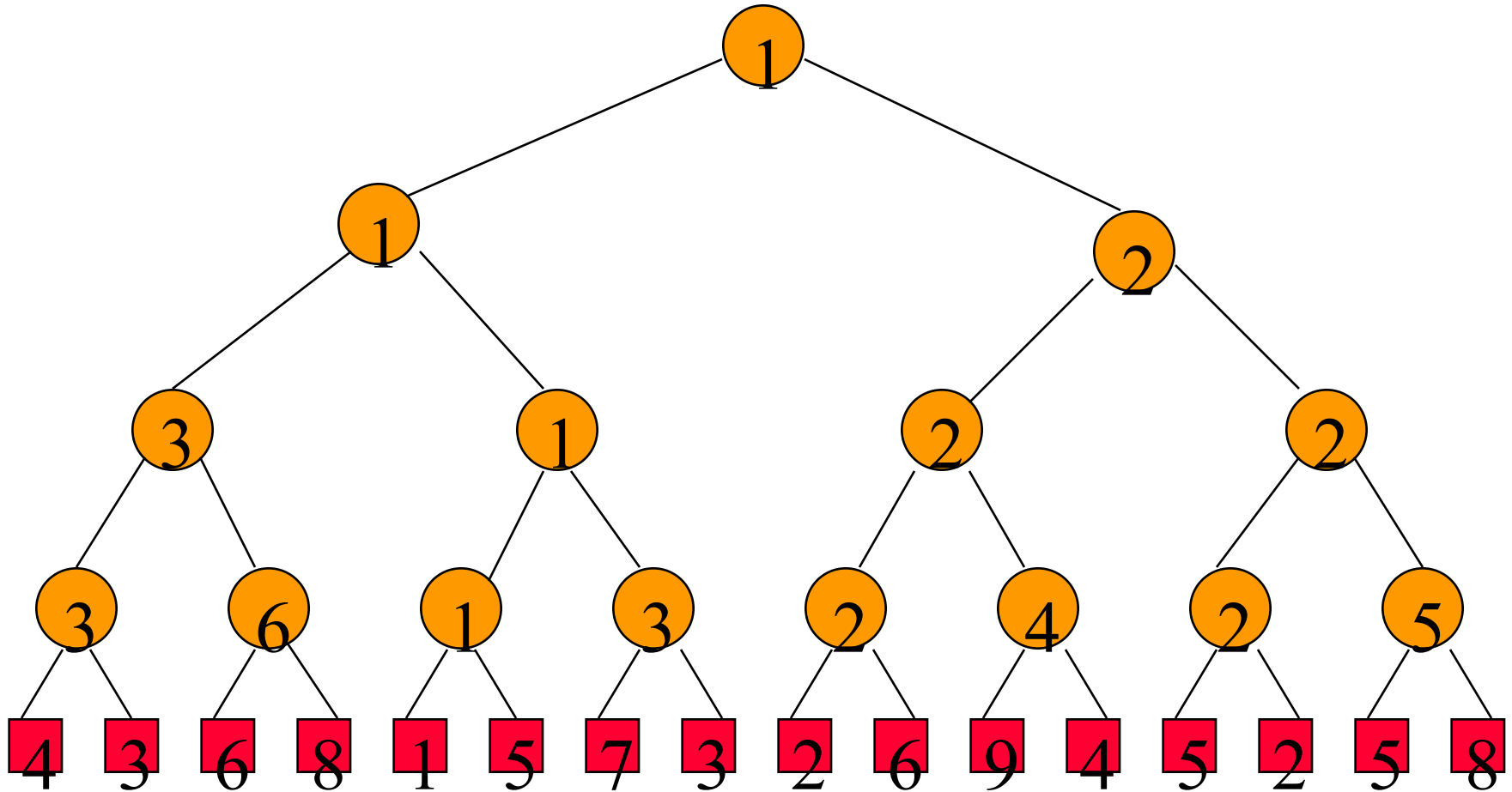
Winner Tree For 16 Players



 player

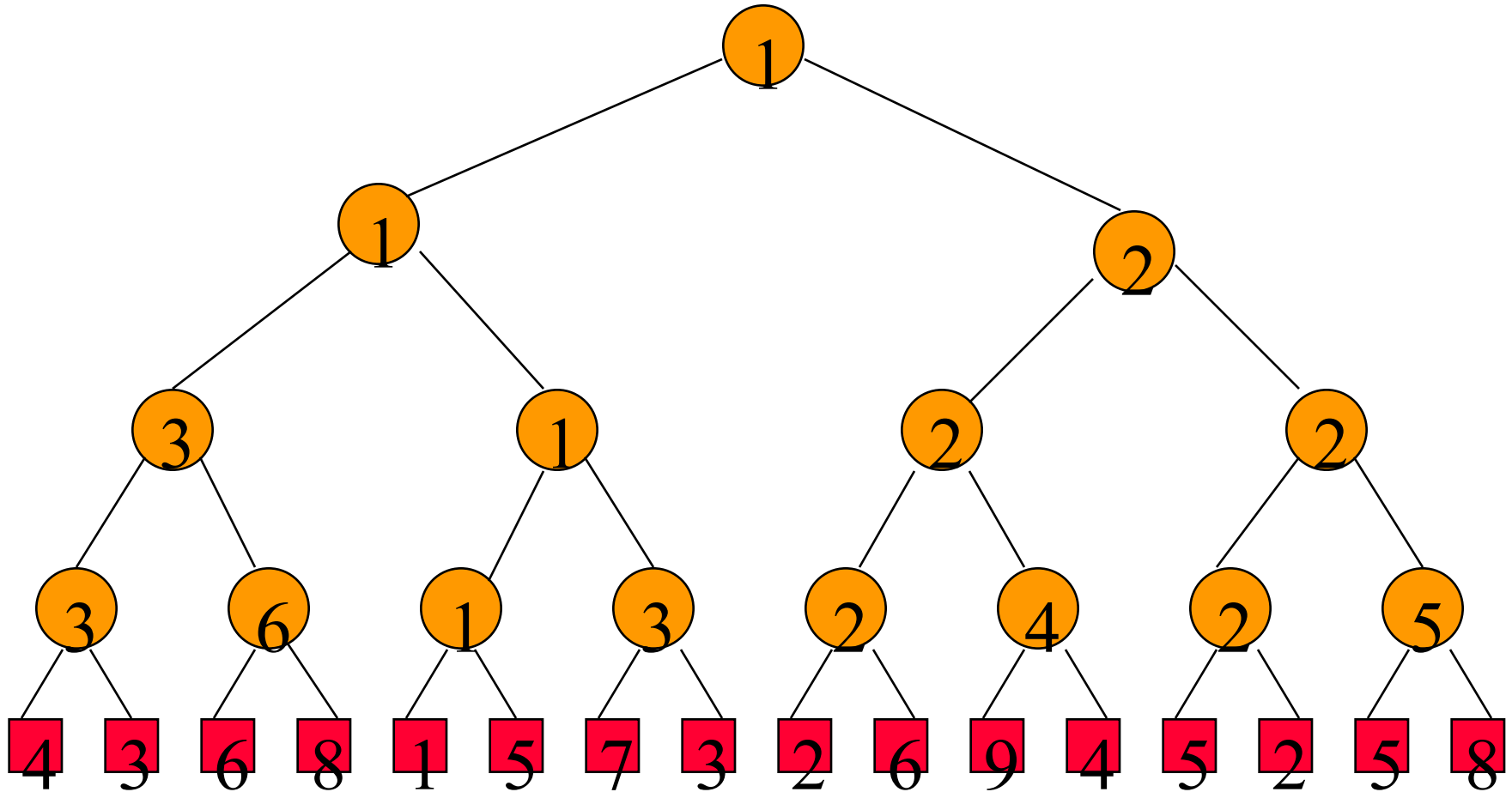
 match node

Winner Tree For 16 Players



Smaller element wins => min winner tree.

Winner Tree For 16 Players



height is $\log_2 n$ (excludes player level)

Complexity Of Initialize

- $O(1)$ time to play match at each match node.
- $n - 1$ match nodes.
- $O(n)$ time to initialize n player winner tree.

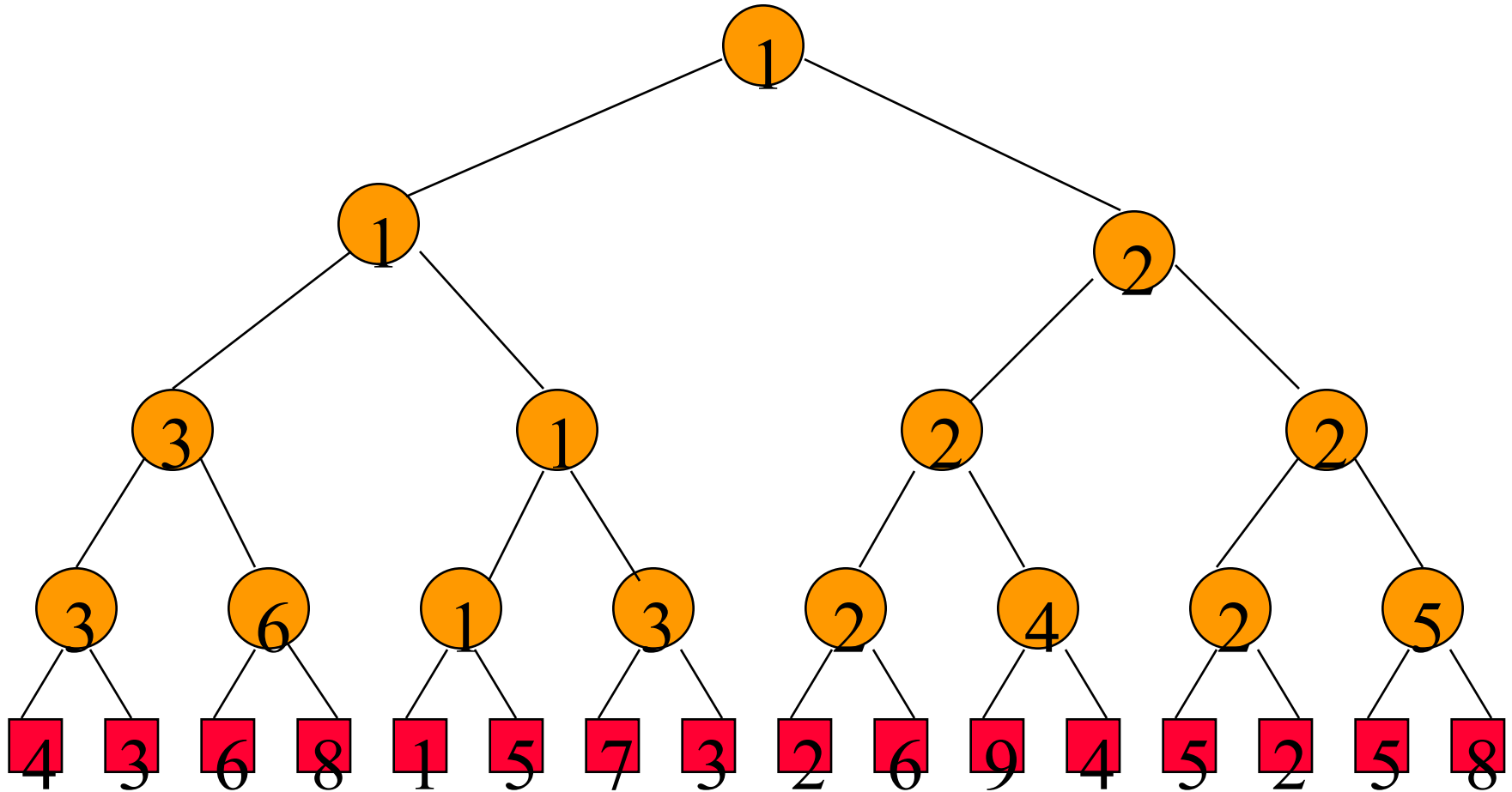
Applications

Sorting.

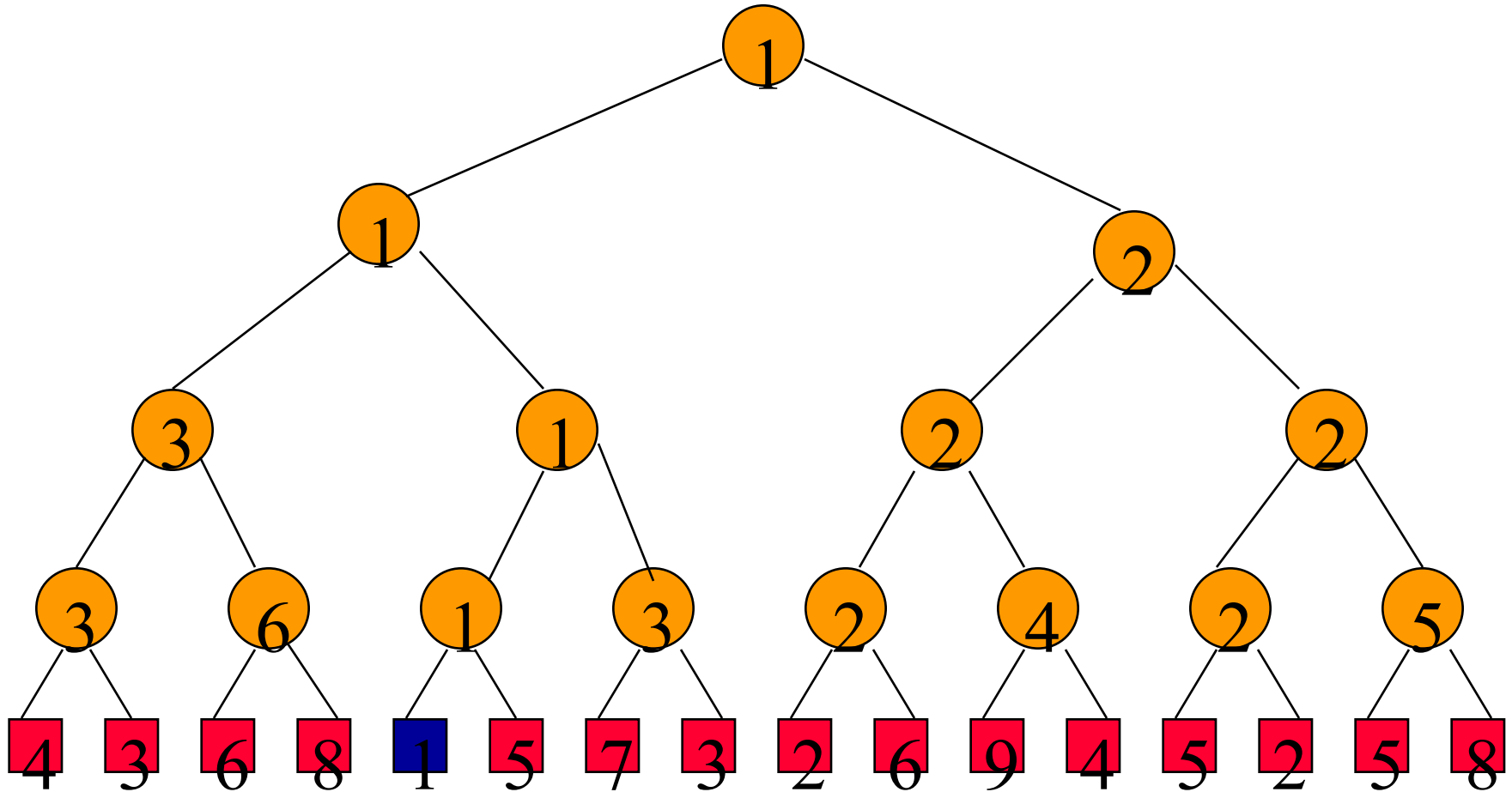
Put elements to be sorted into a winner tree.

Repeatedly extract the winner and replace by a large value.

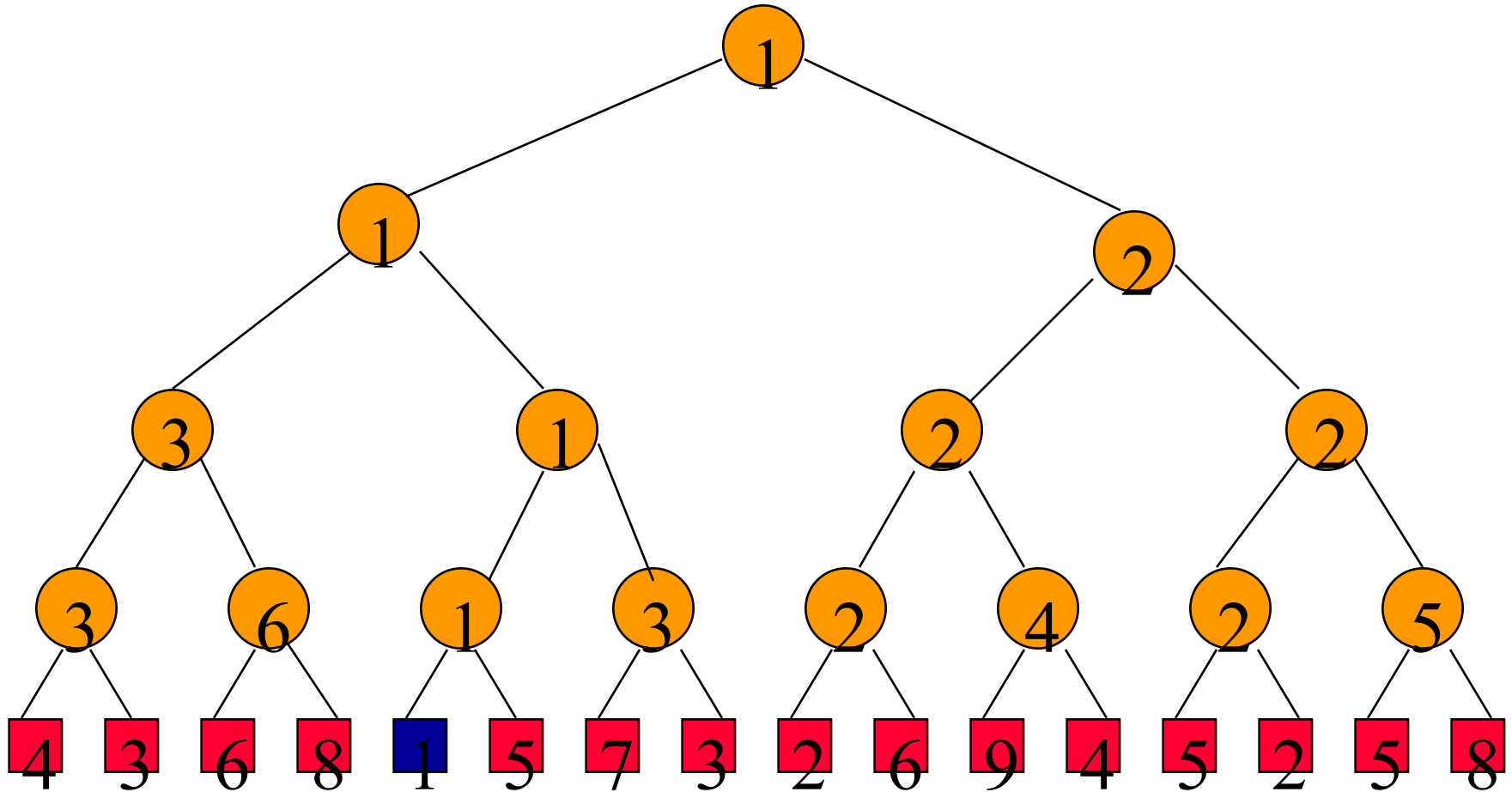
Sort 16 Numbers



Sort 16 Numbers

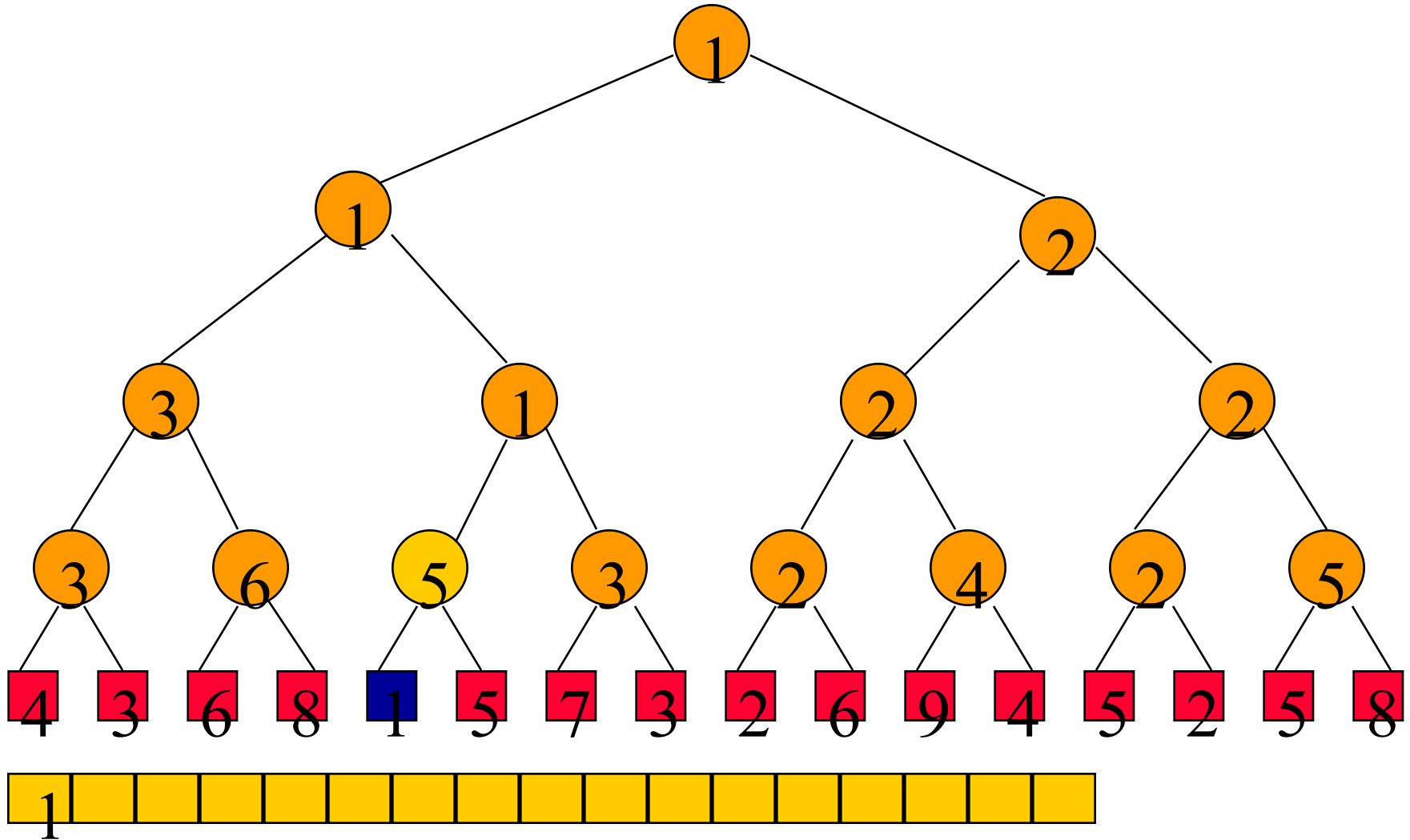


Sort 16 Numbers



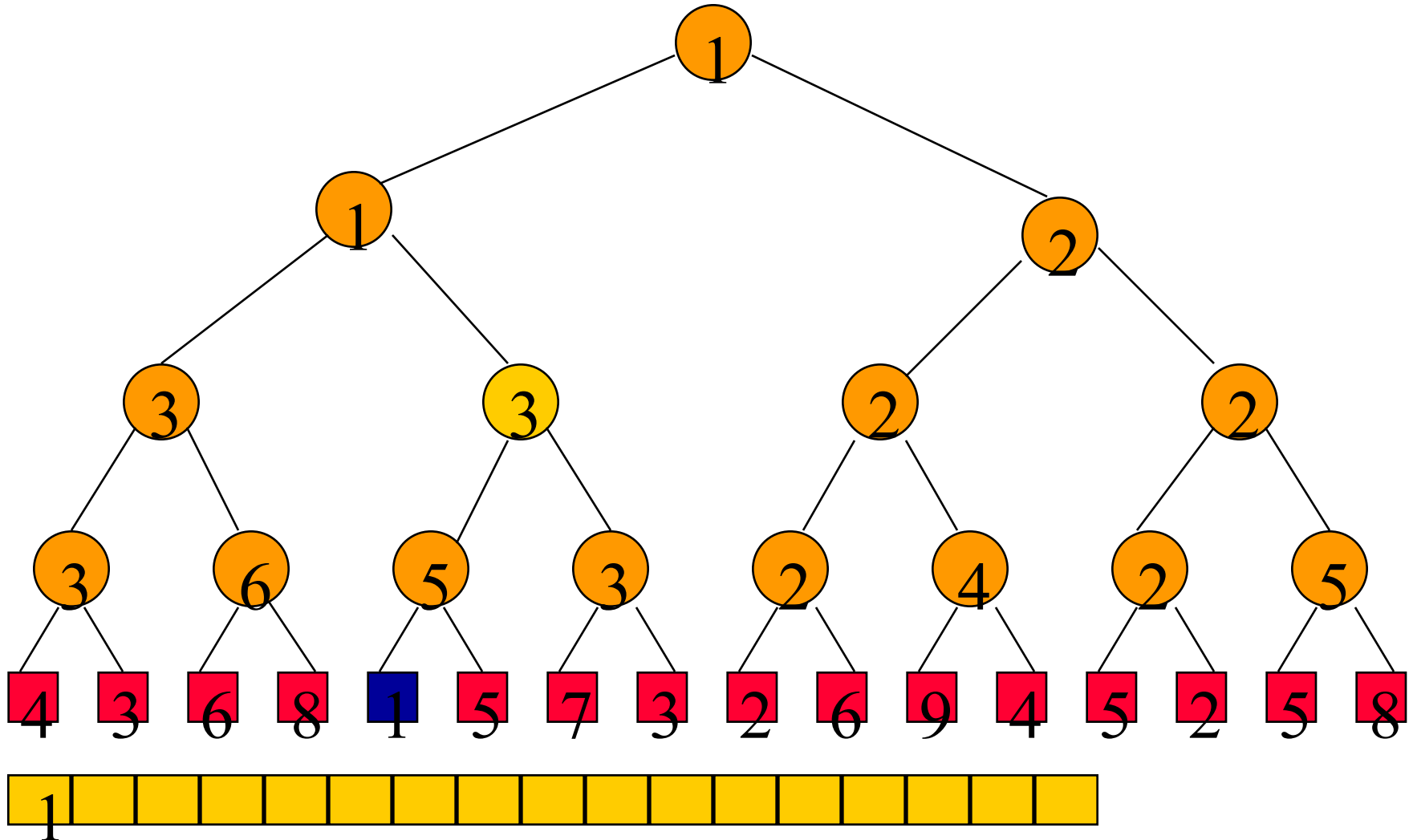
Sorted array.

Sort 16 Numbers



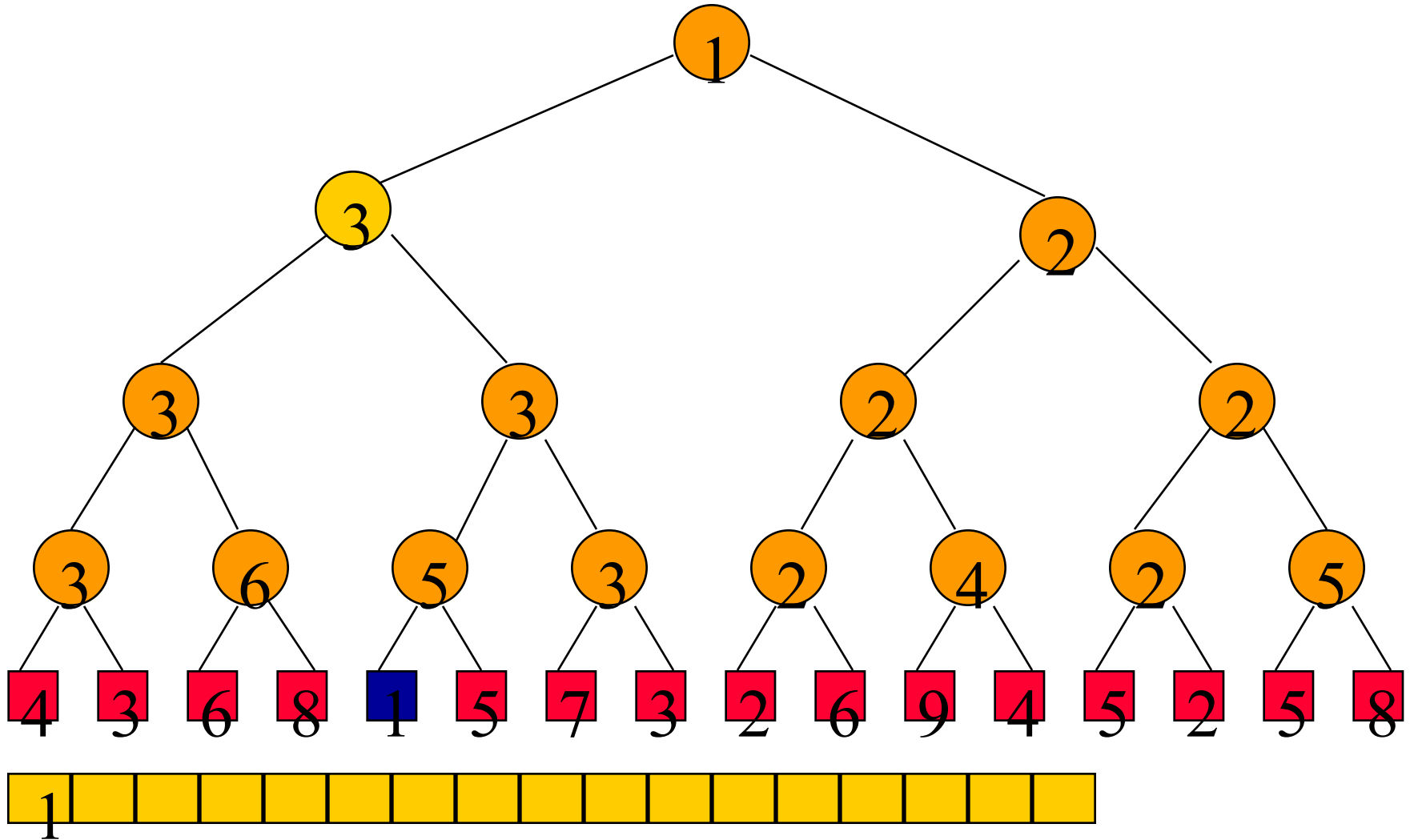
Sorted array.

Sort 16 Numbers

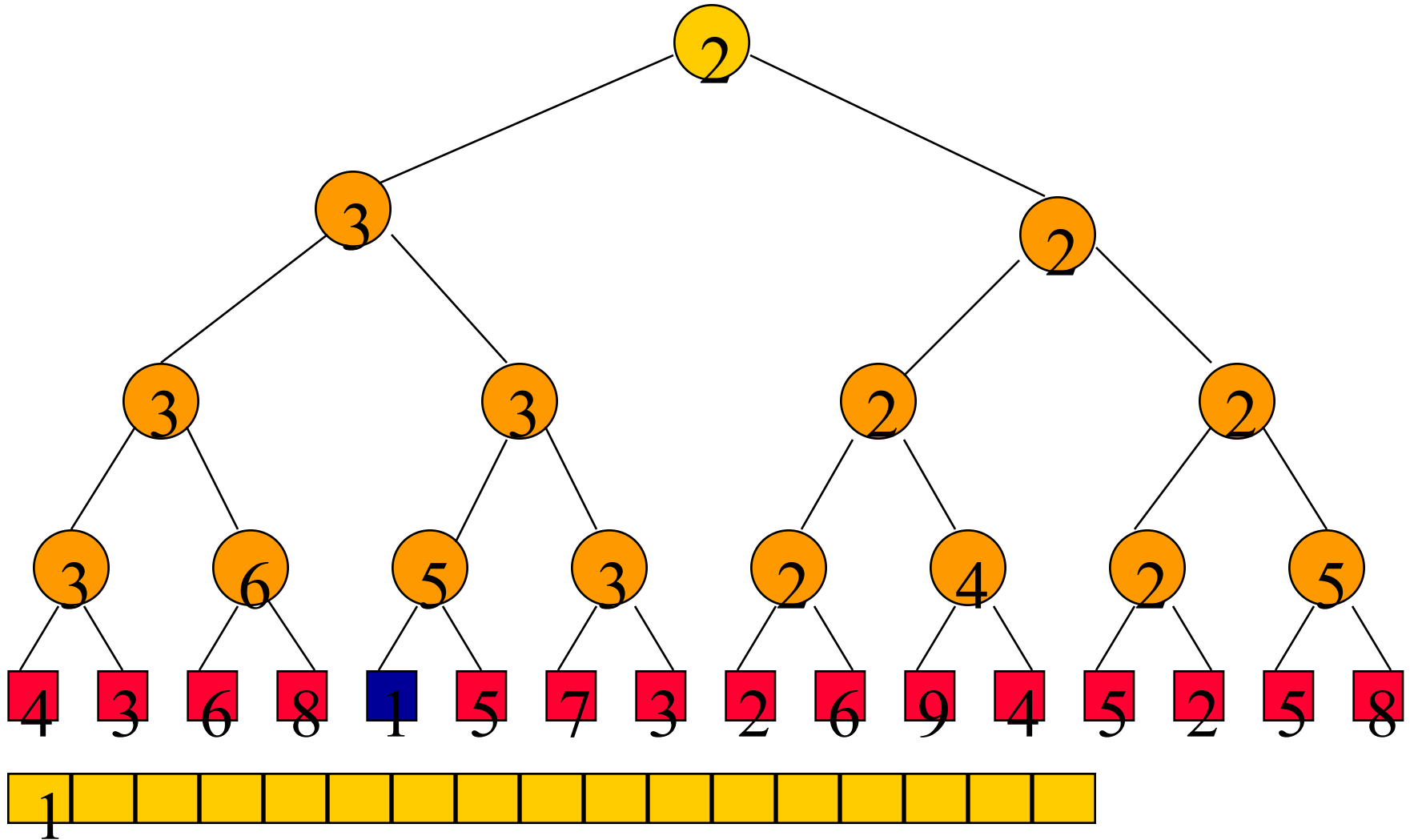


Sorted array.

Sort 16 Numbers

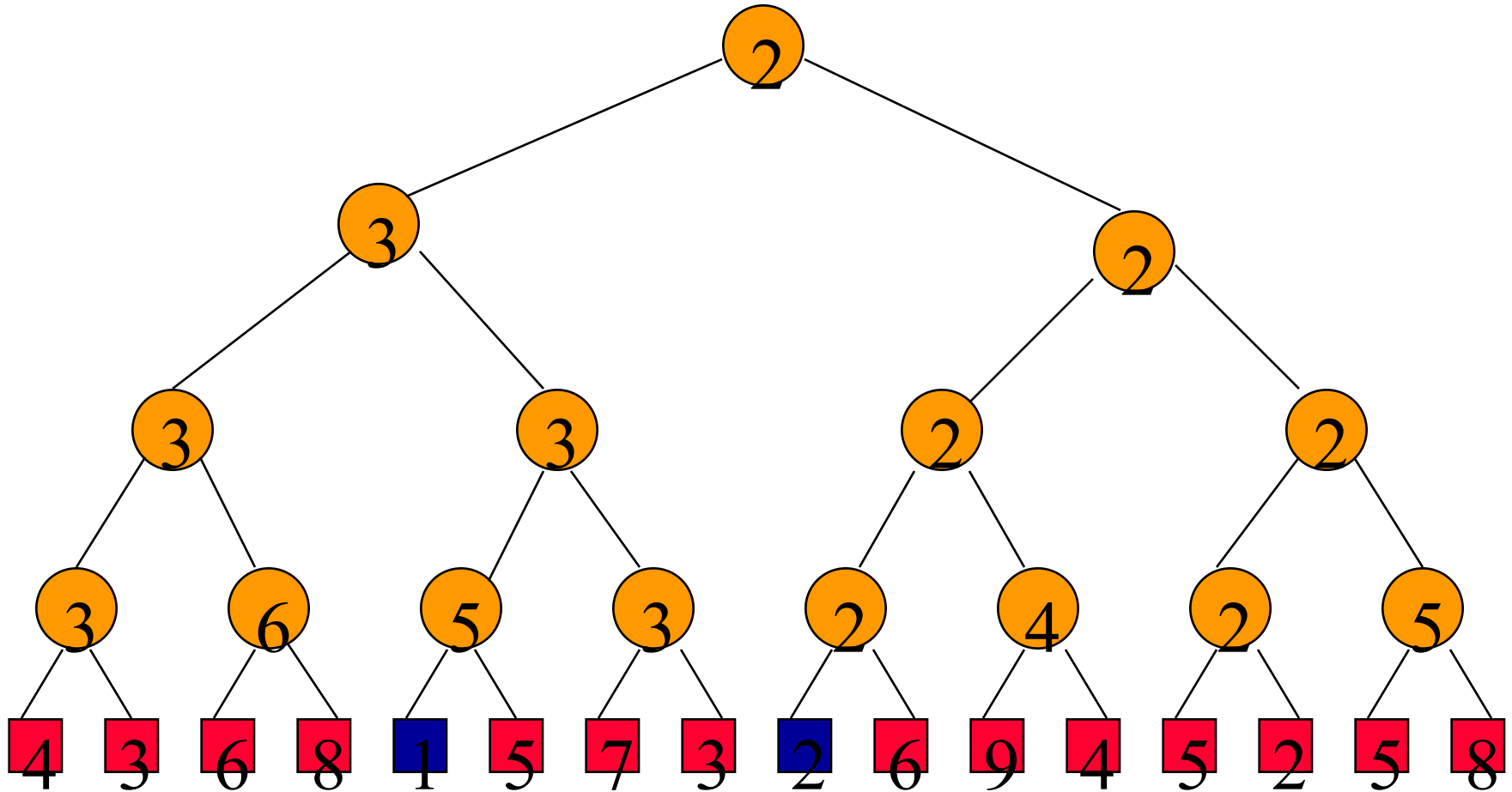


Sort 16 Numbers



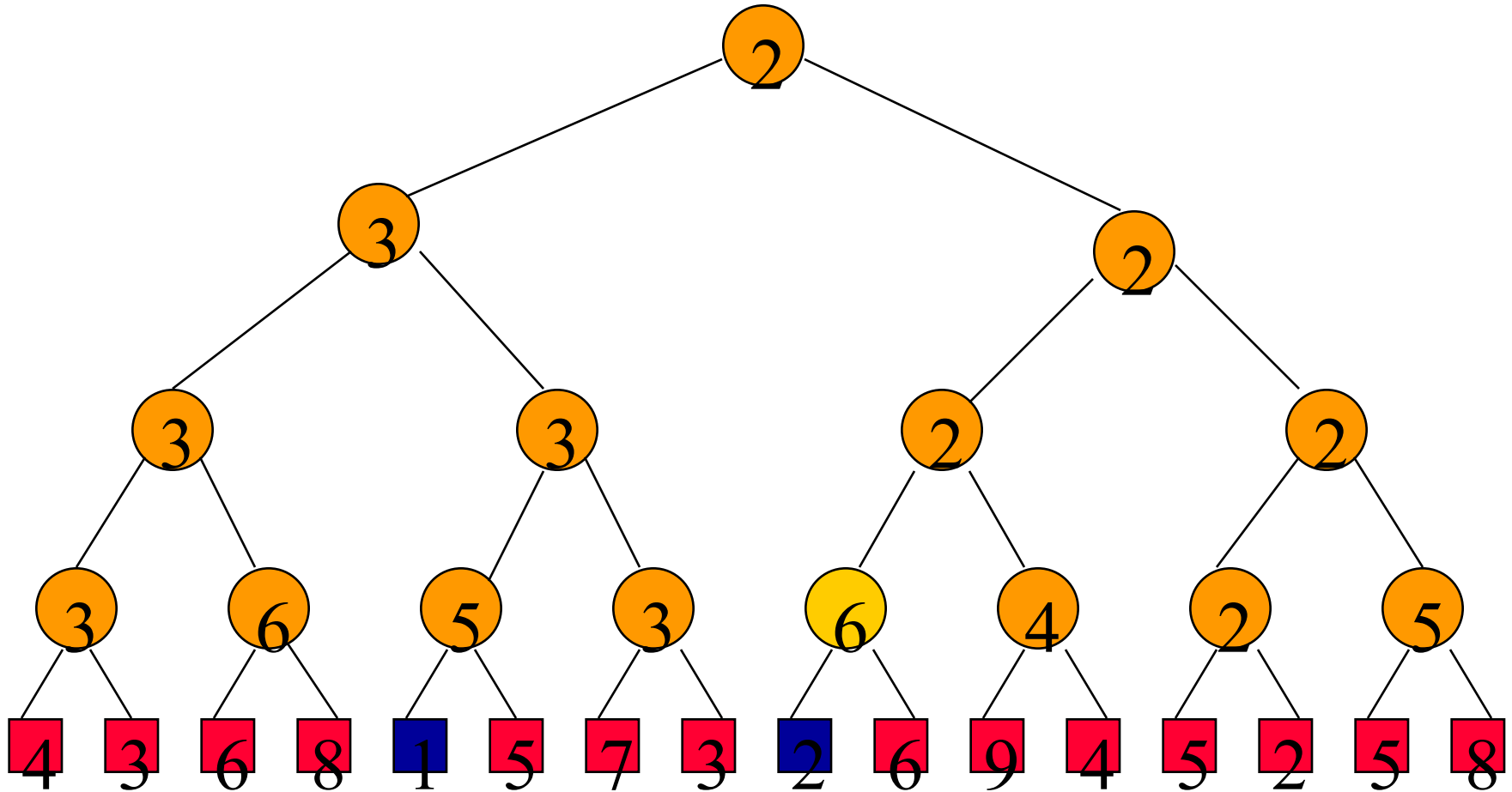
Sorted array.

Sort 16 Numbers



Sorted array.

Sort 16 Numbers

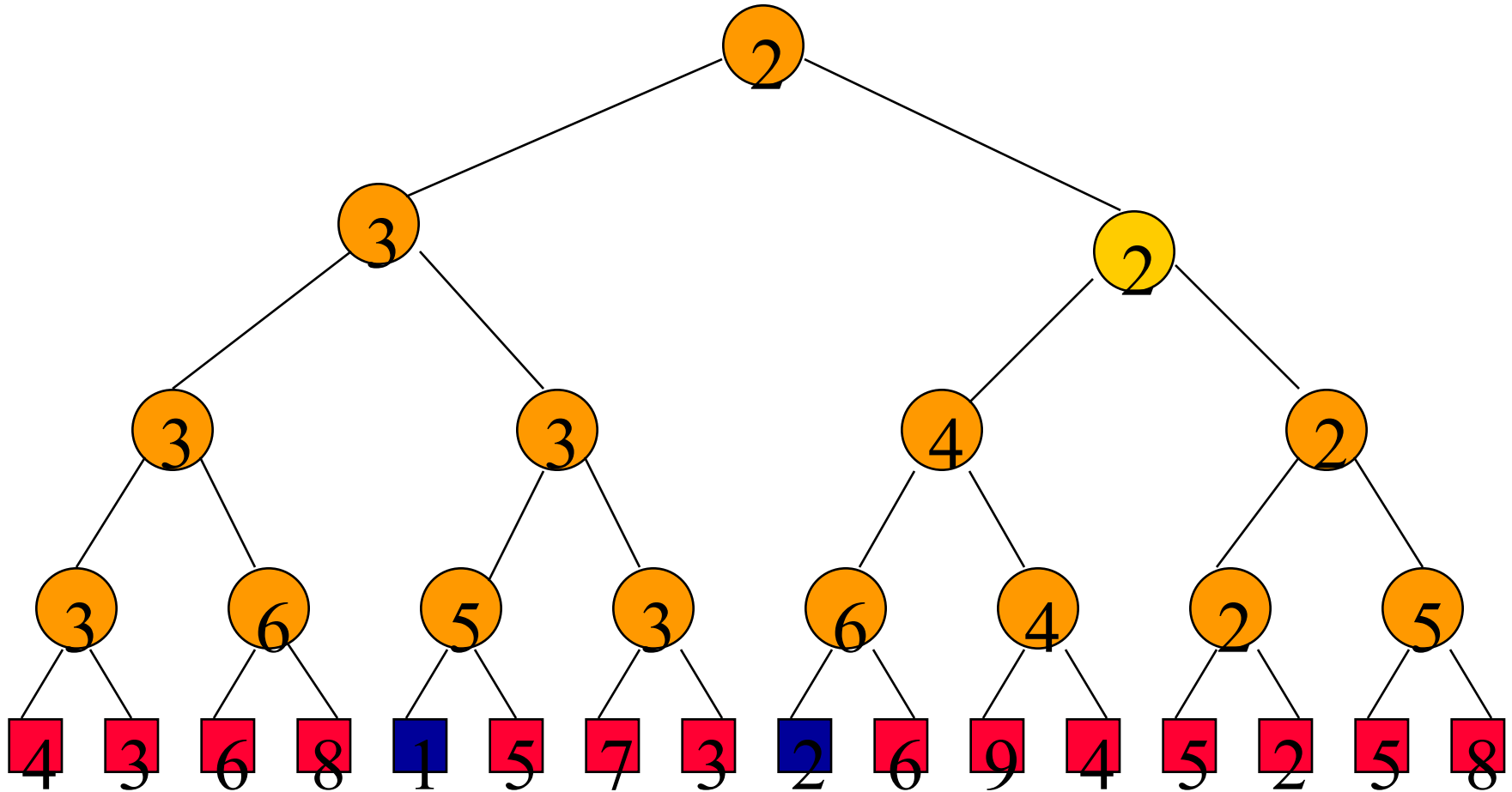


Sorted array.

The diagram illustrates a binary tree structure with 16 leaf nodes. The root node is 2. The tree is divided into two main branches. The left branch has a root of 3, which branches into 3 and 3. The right branch has a root of 2, which branches into 4 and 2. The leaf nodes are represented by colored squares: red for most, blue for the first node in the second branch from the left, and yellow for the first node in the third branch from the left. Below the tree is a horizontal bar with 16 segments, the first two of which are labeled 1 and 2.

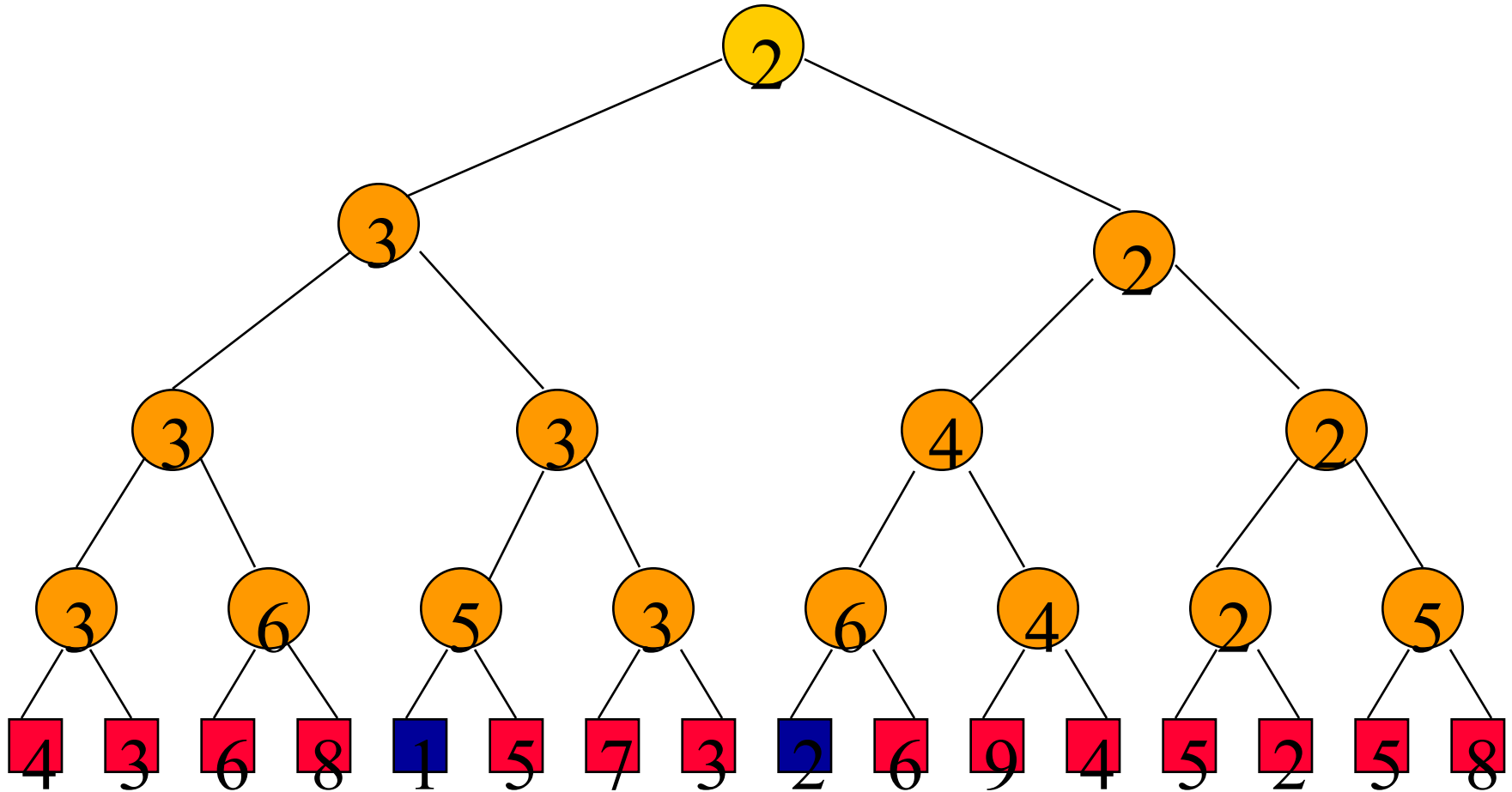
Sorted array.

Sort 16 Numbers



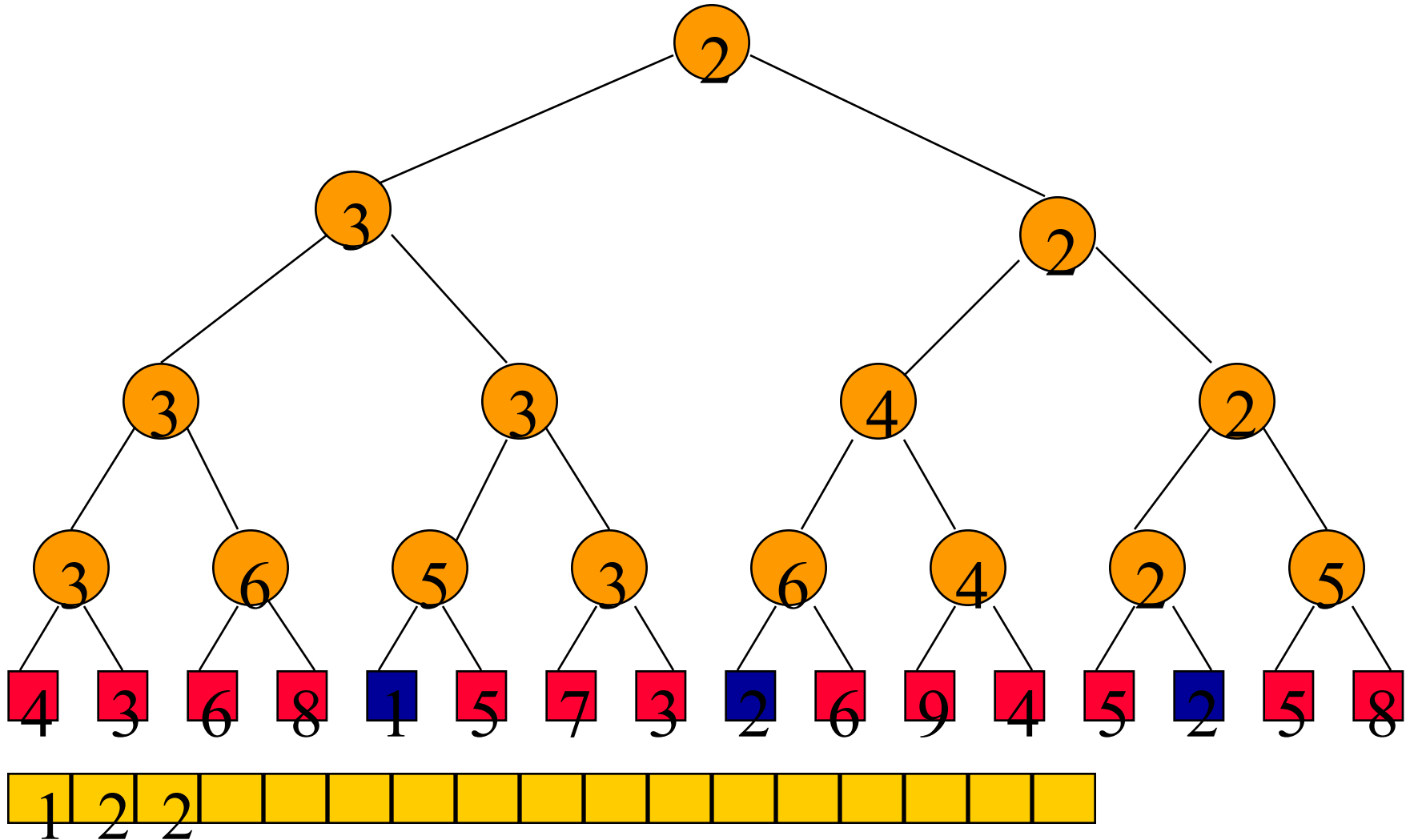
Sorted array.

Sort 16 Numbers



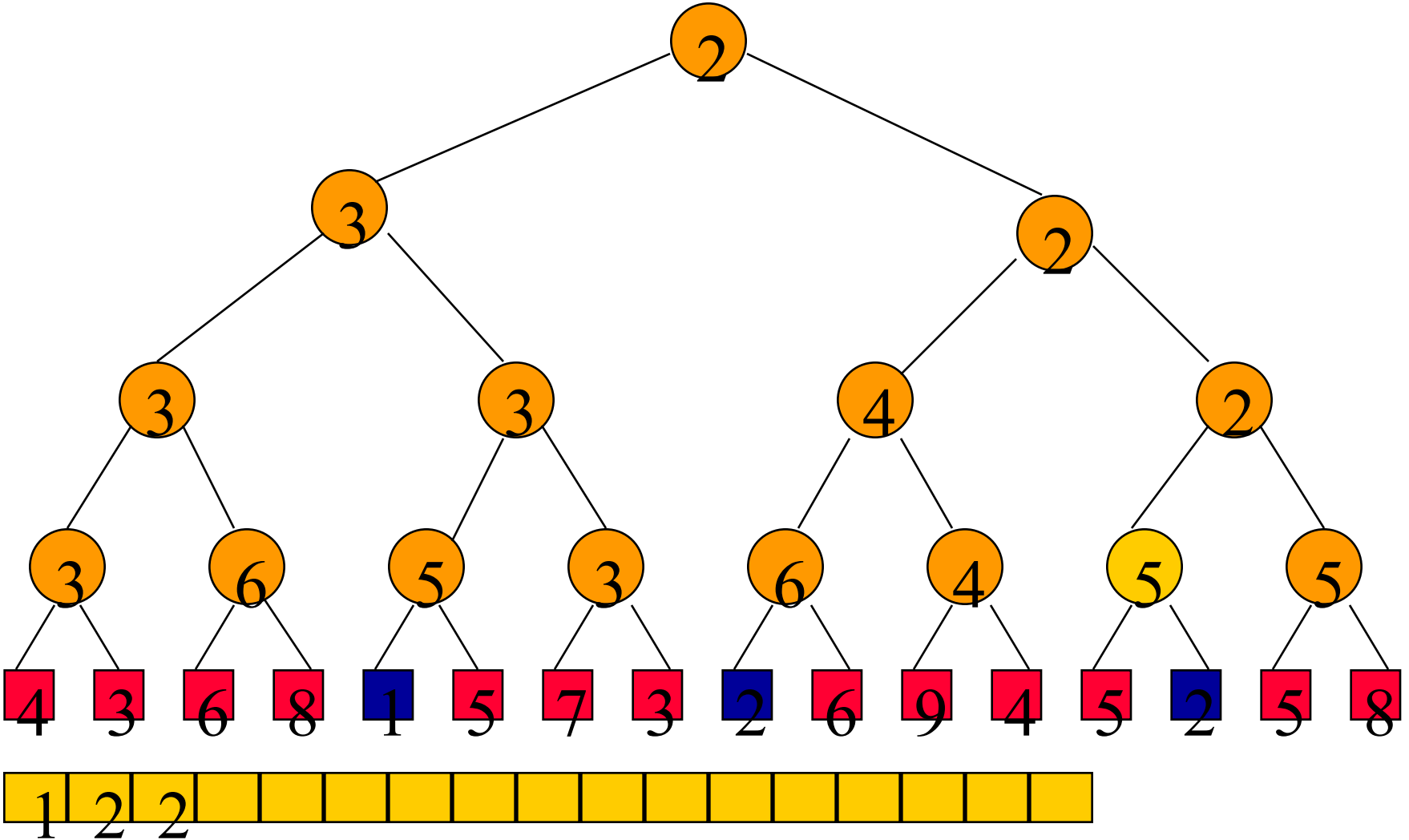
Sorted array.

Sort 16 Numbers



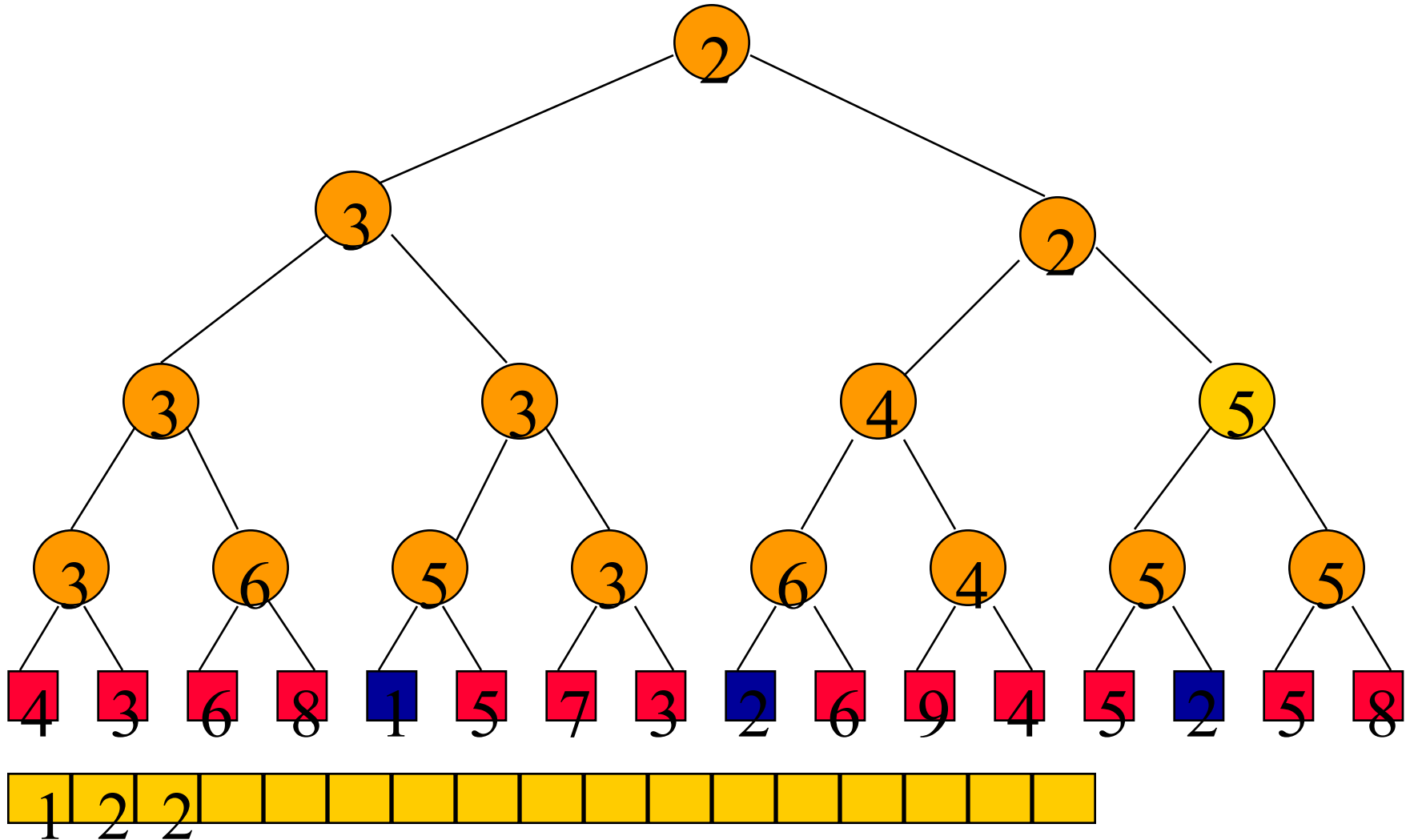
Sorted array.

Sort 16 Numbers



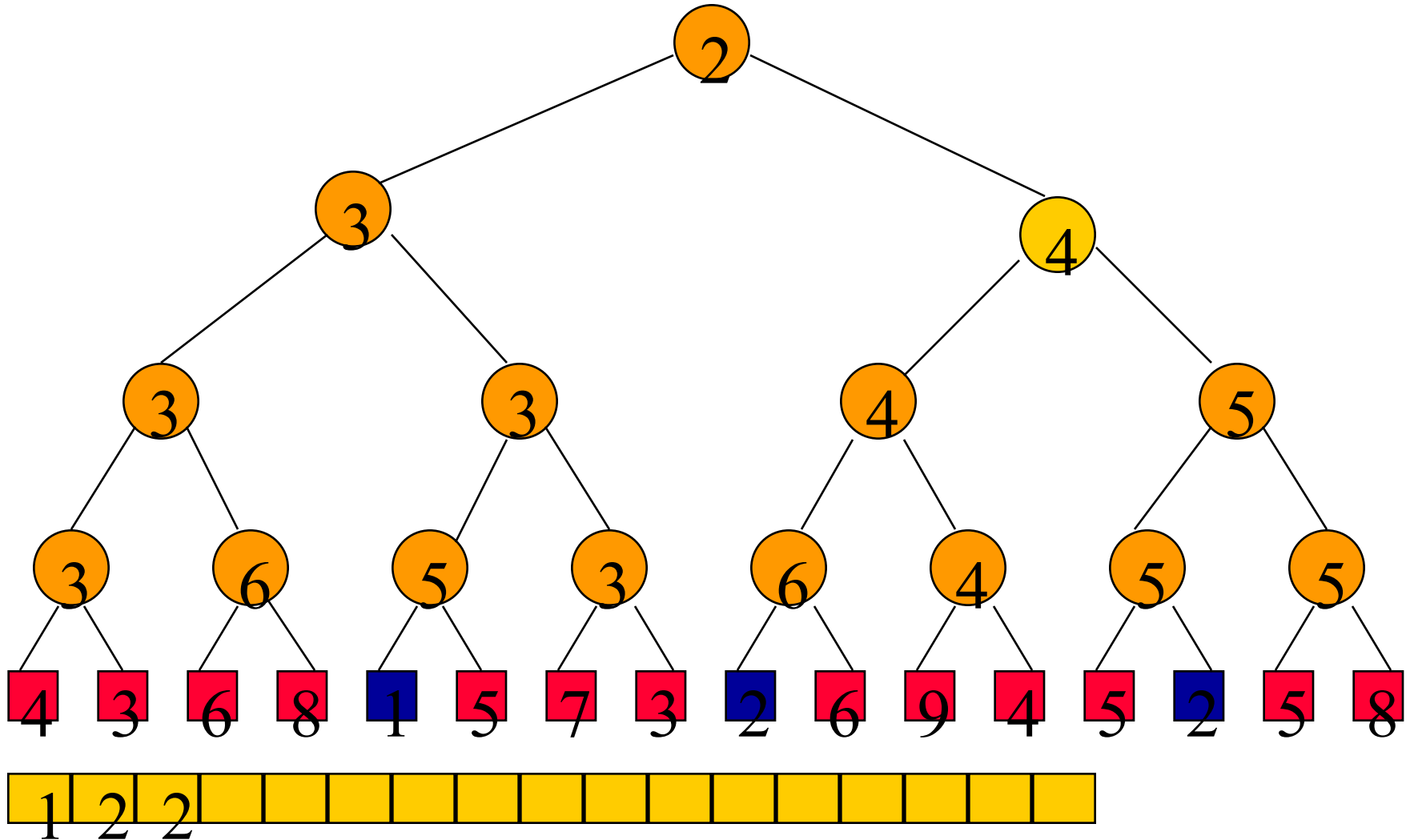
Sorted array.

Sort 16 Numbers



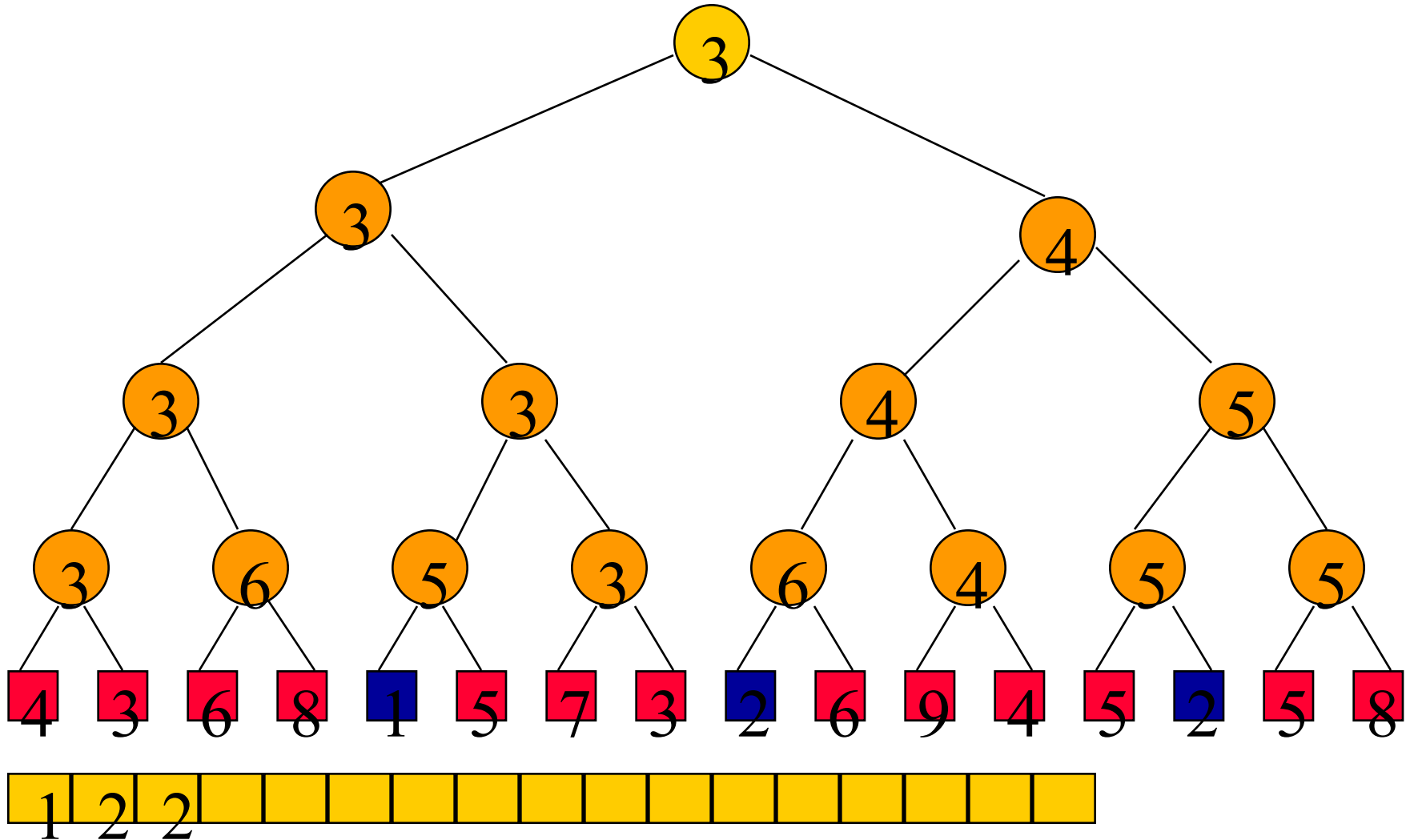
Sorted array.

Sort 16 Numbers



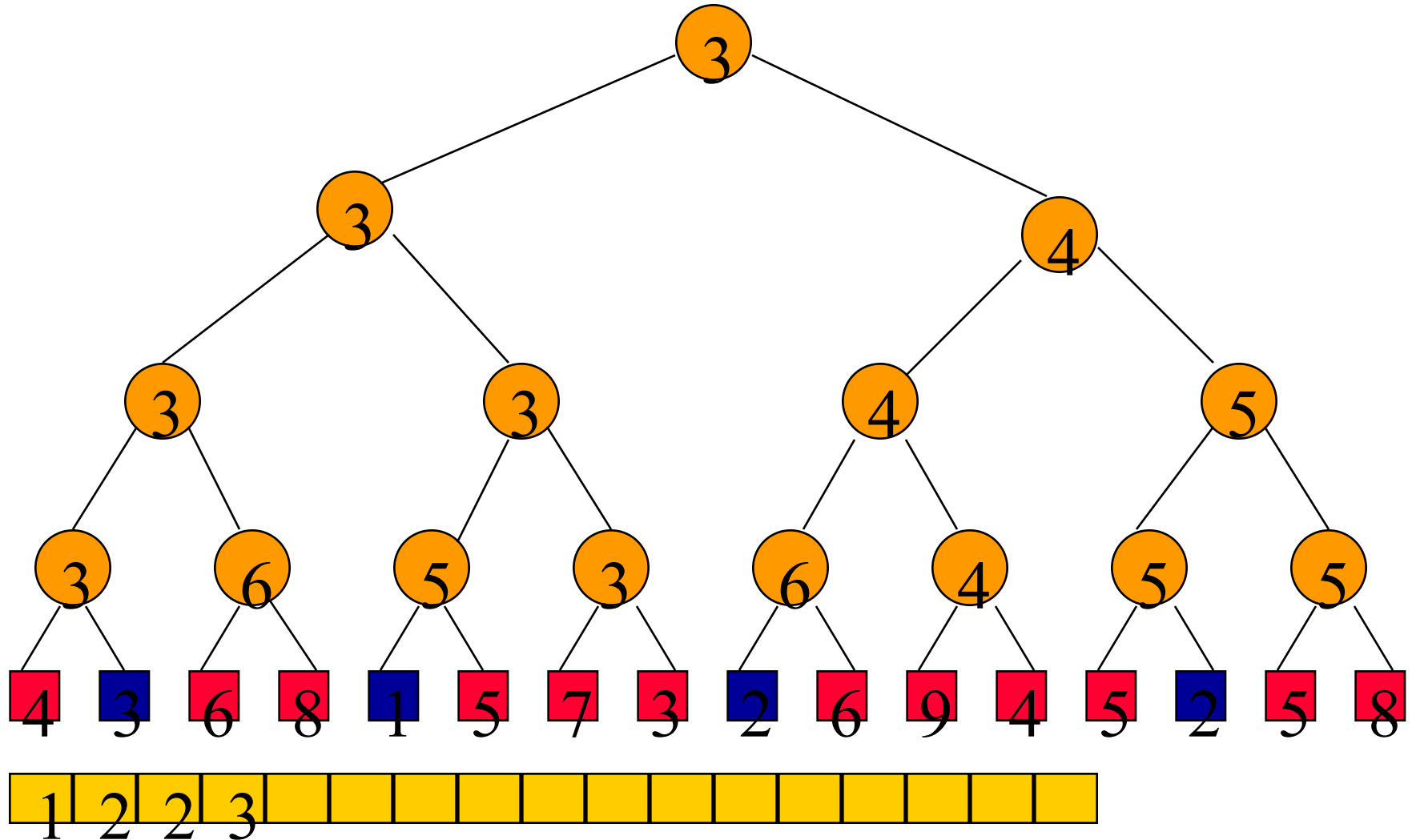
Sorted array.

Sort 16 Numbers



Sorted array.

Sort 16 Numbers



Sorted array.

Time To Sort

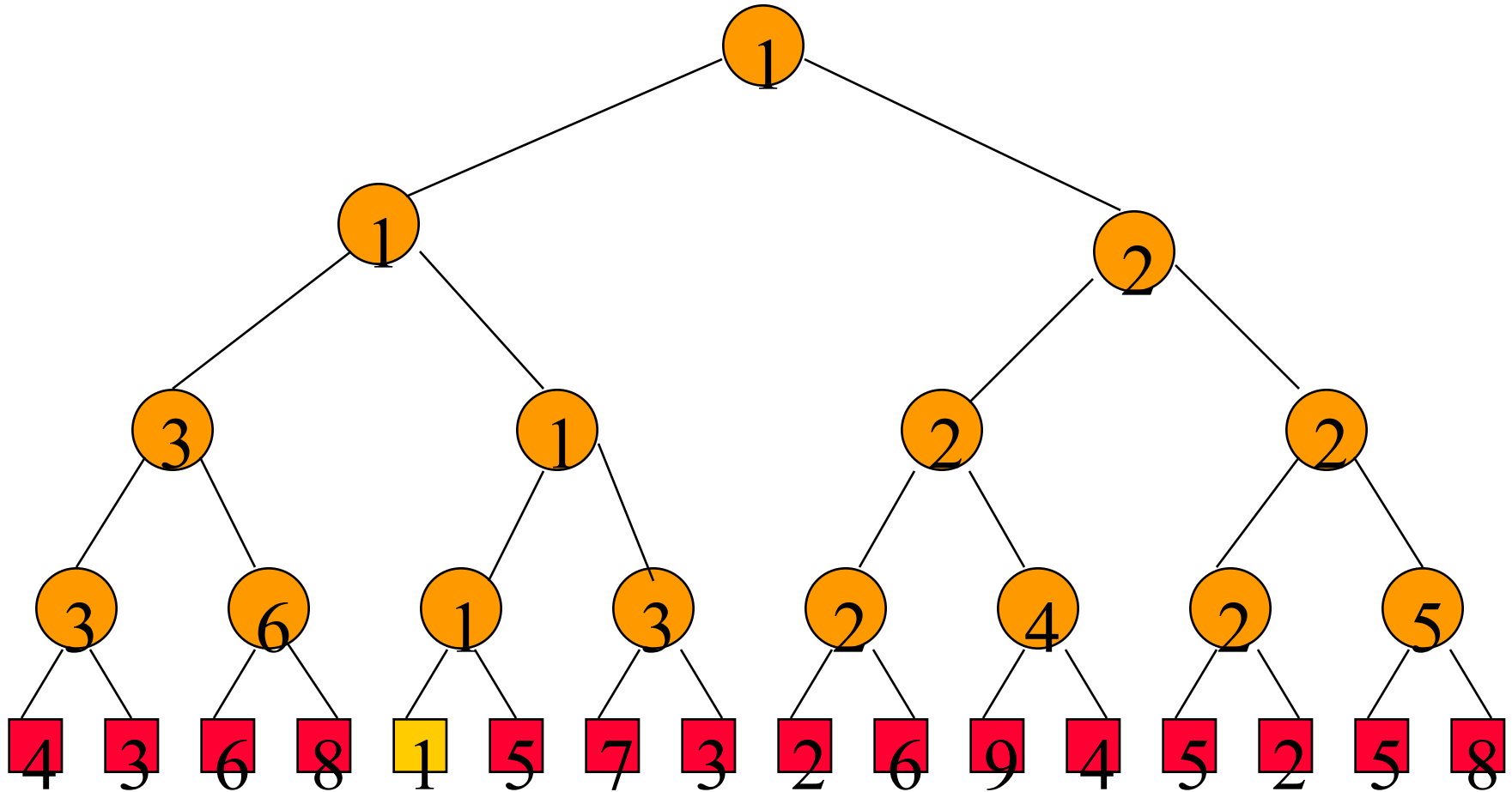


- Initialize winner tree.
 - $O(n)$ time
- Remove winner and replay.
 - $O(\log n)$ time
- Remove winner and replay n times.
 - $O(n \log n)$ time
- Total sort time is $O(n \log n)$.

Winner Tree Operations

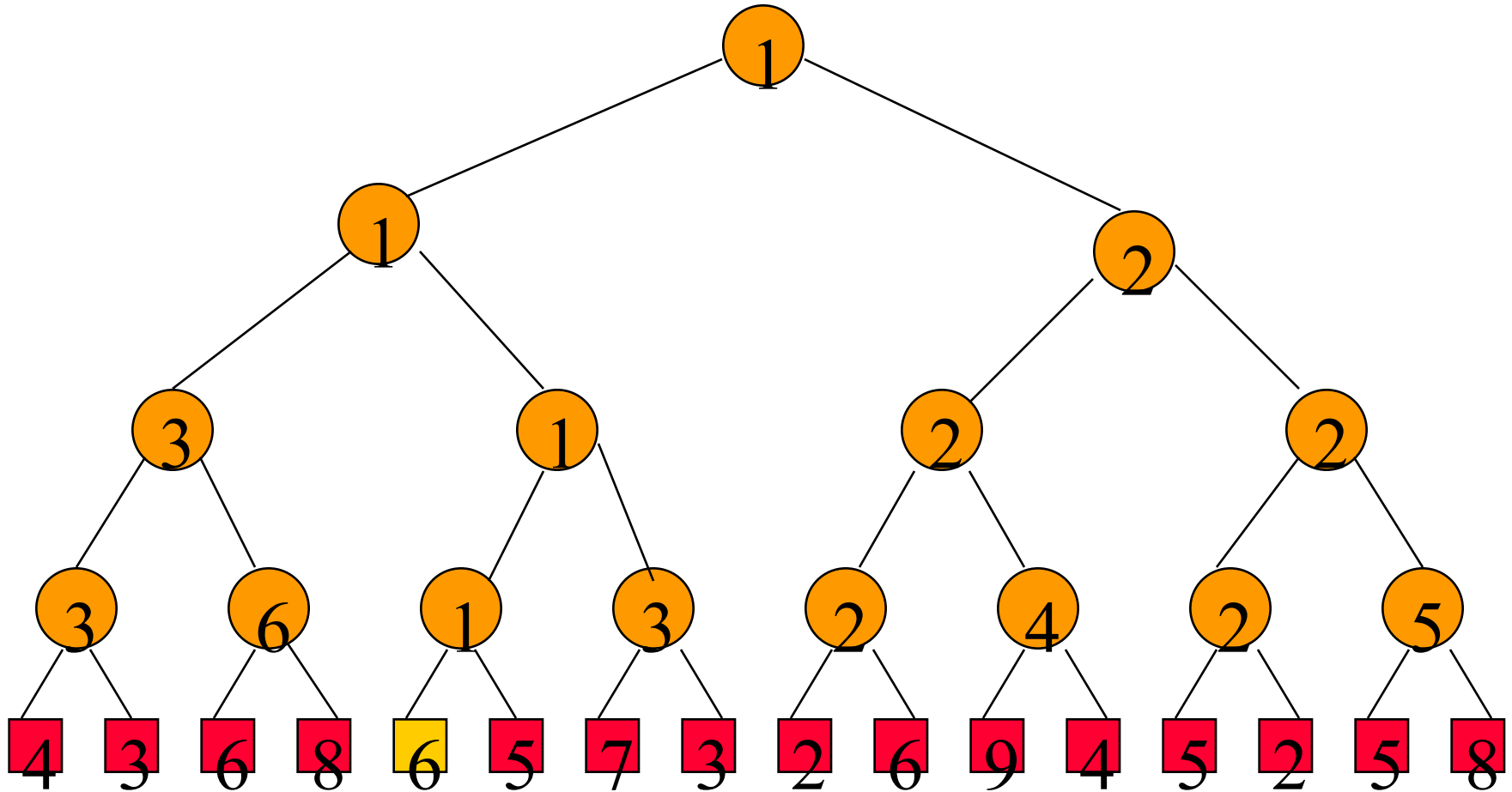
- Initialize
 - $O(n)$ time
- Get winner
 - $O(1)$ time
- Remove/replace winner and replay
 - $O(\log n)$ time
 - more precisely $\Theta(\log n)$

Replace Winner And Replay



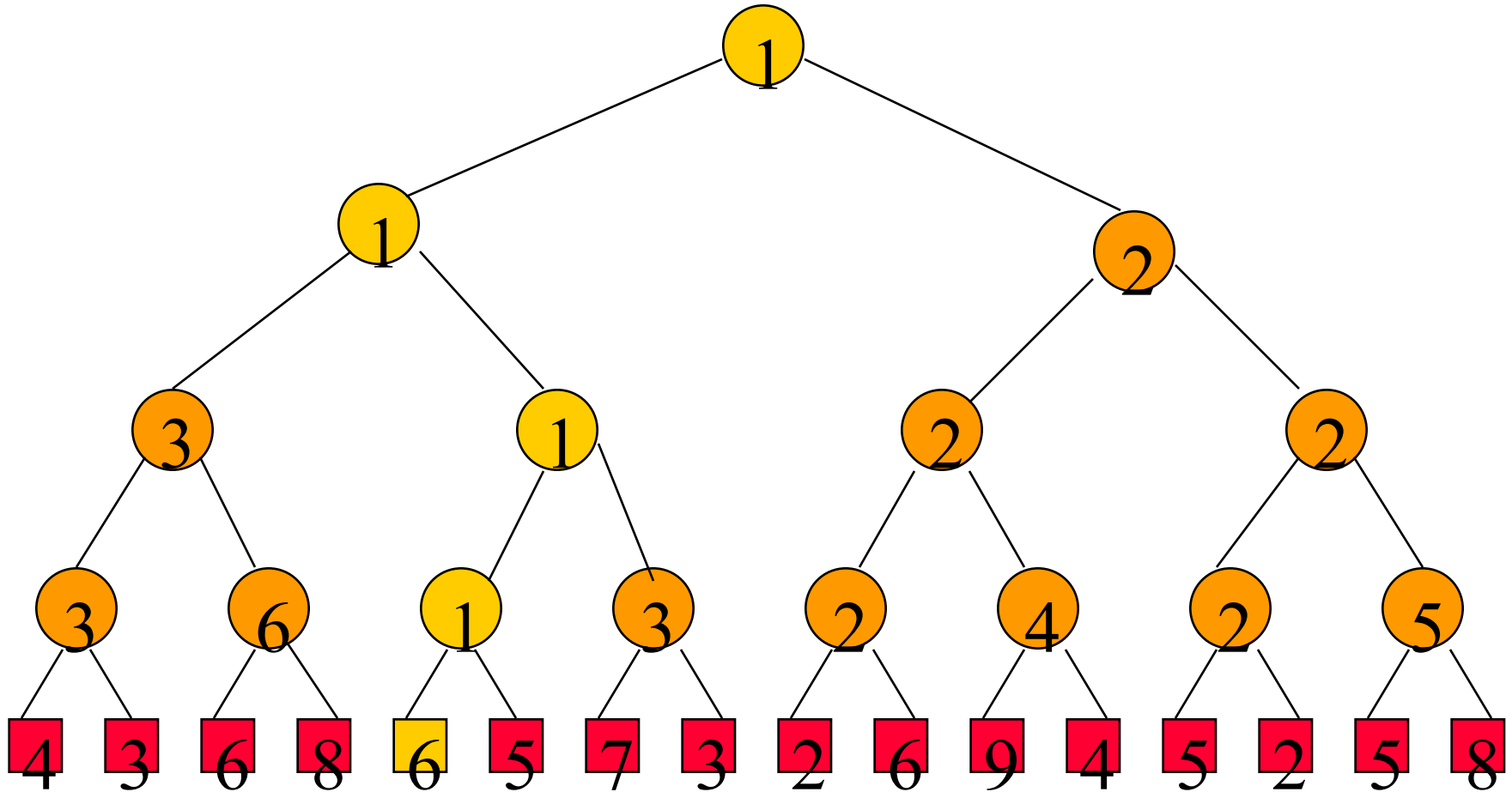
Replace winner with 6.

Replace Winner And Replay



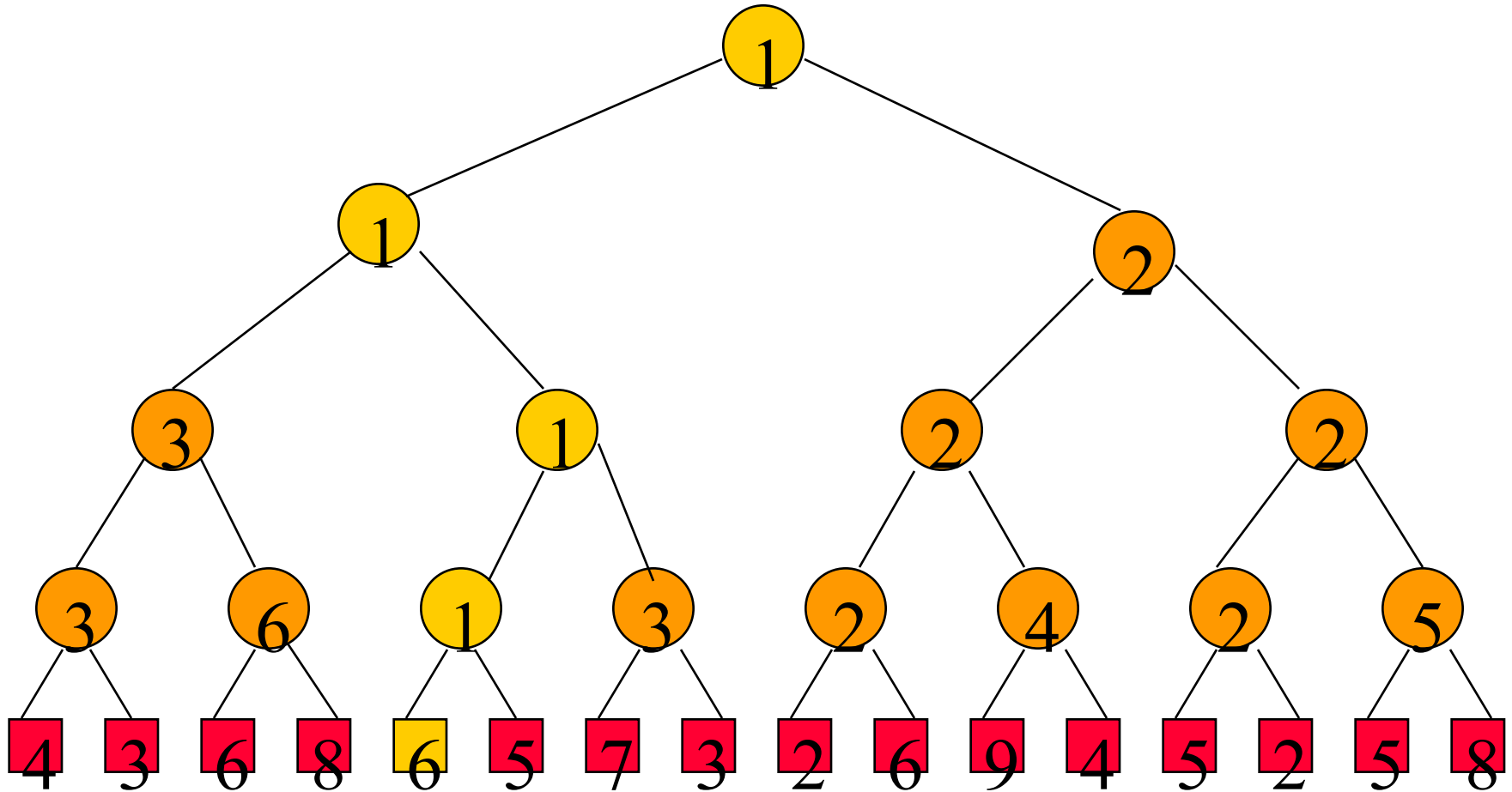
Replay matches on path to root.

Replace Winner And Replay



Replay matches on path to root.

Replace Winner And Replay

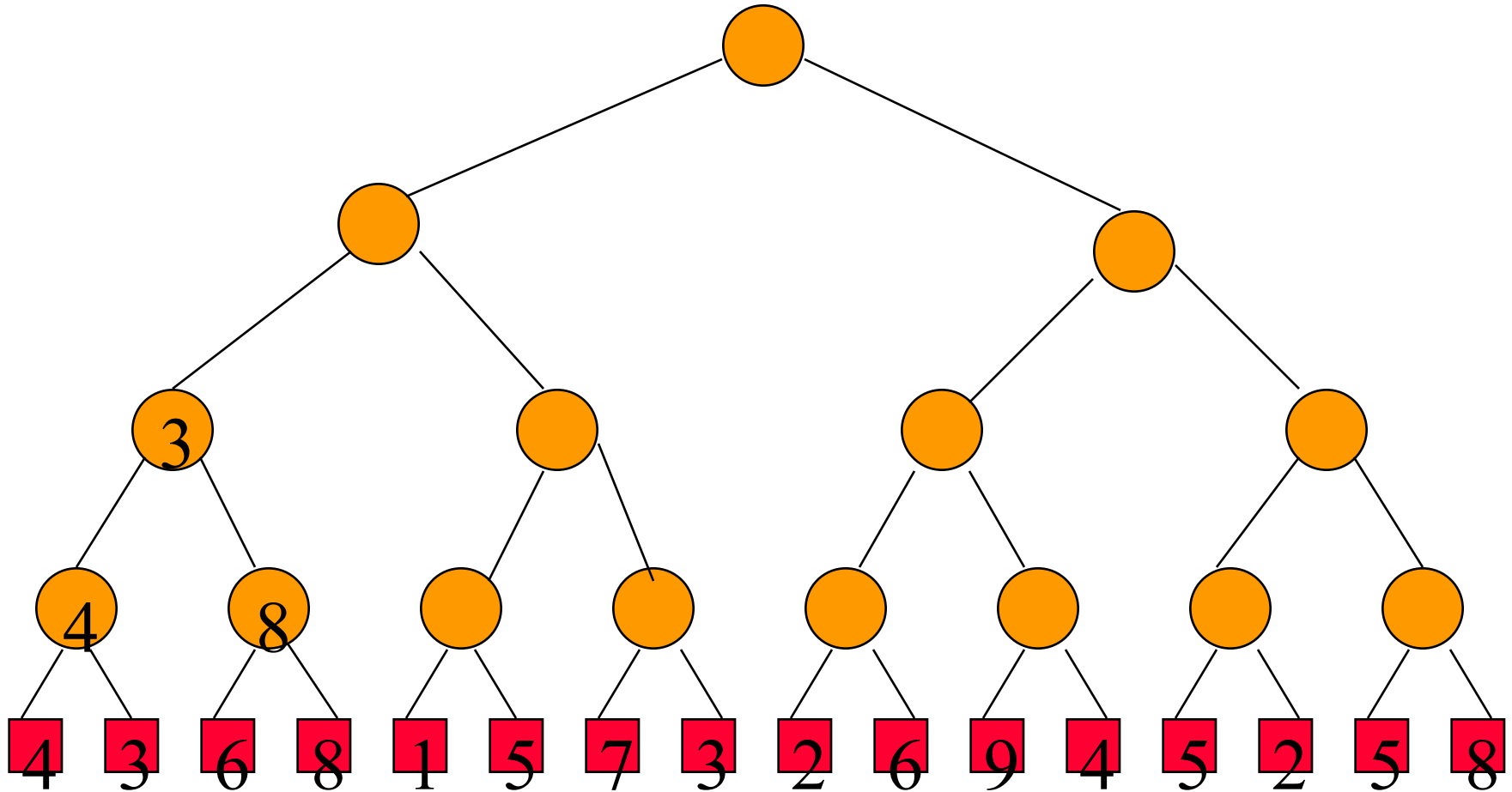


Opponent is player who lost last match played at this node.

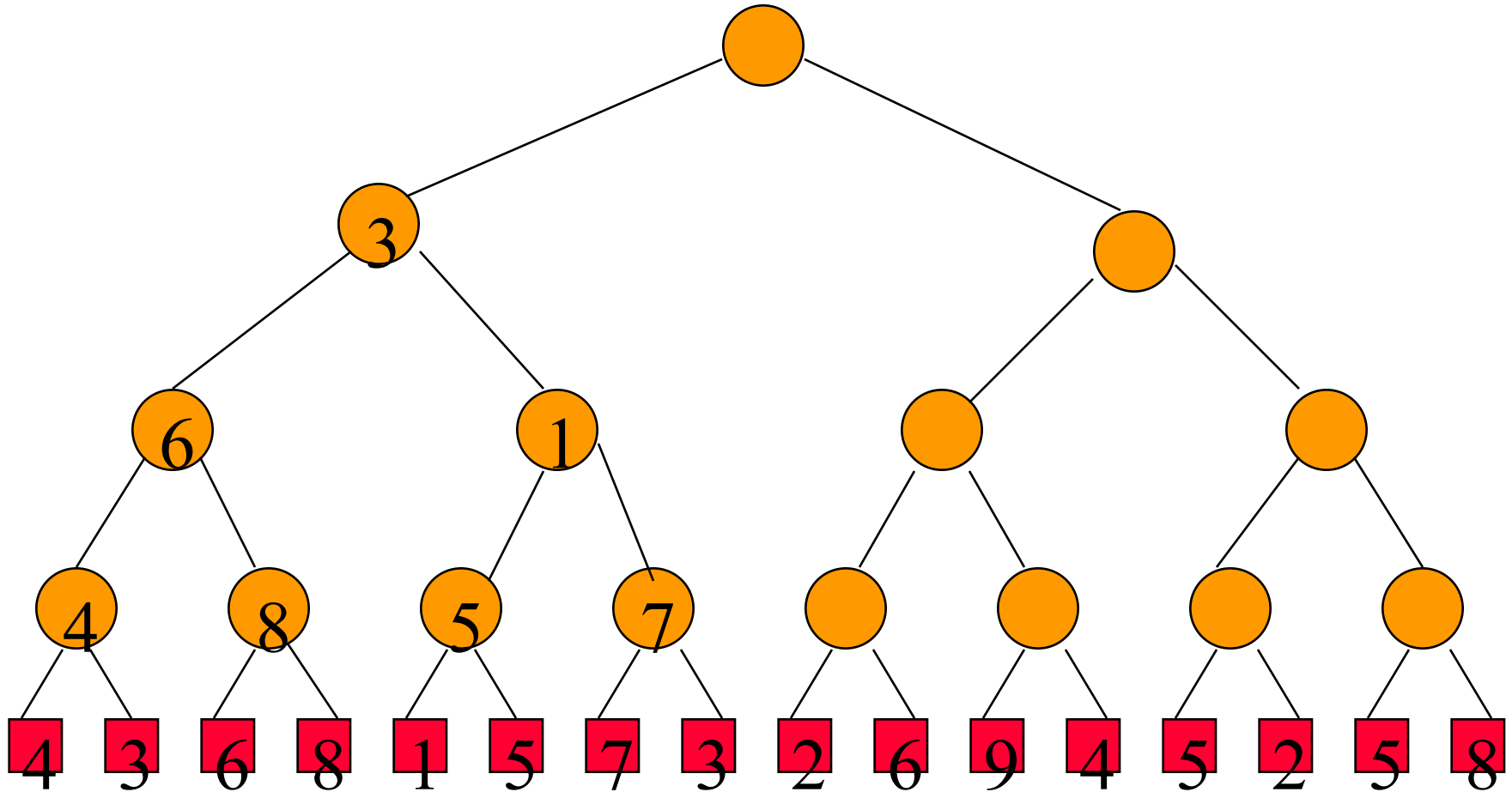
Loser Tree

Each match node stores the match loser rather than the match winner.

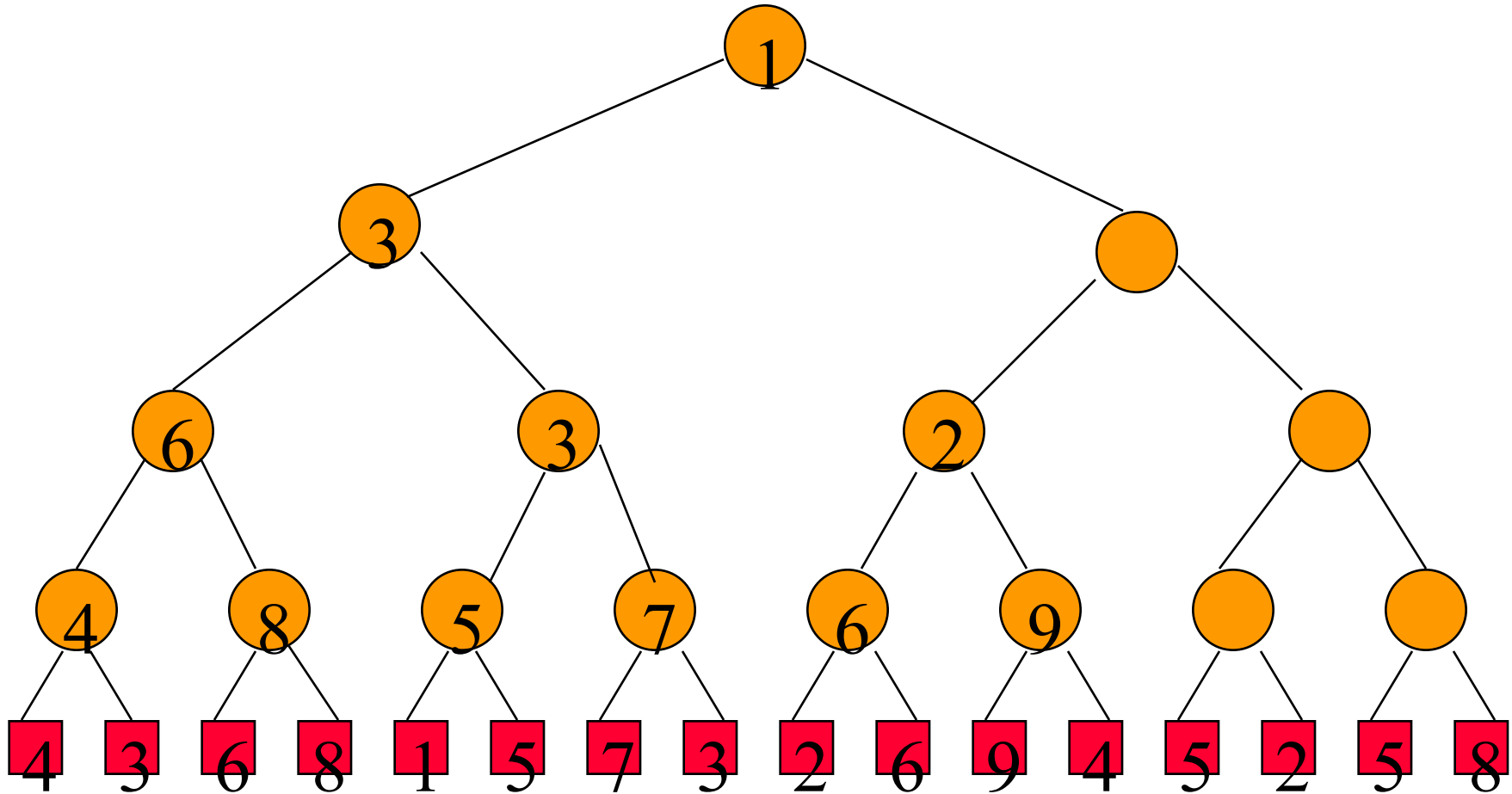
Min Loser Tree For 16 Players



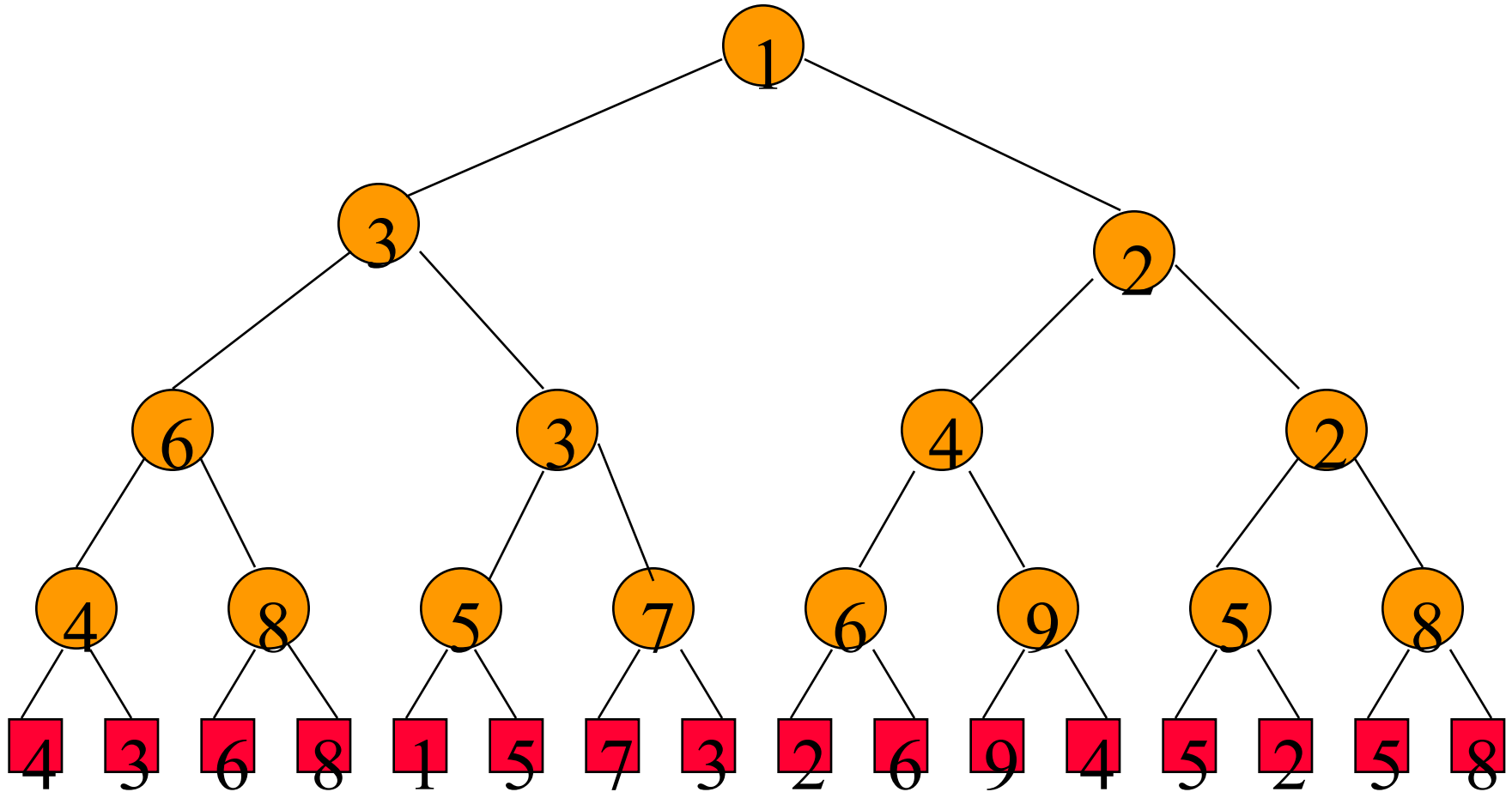
Min Loser Tree For 16 Players



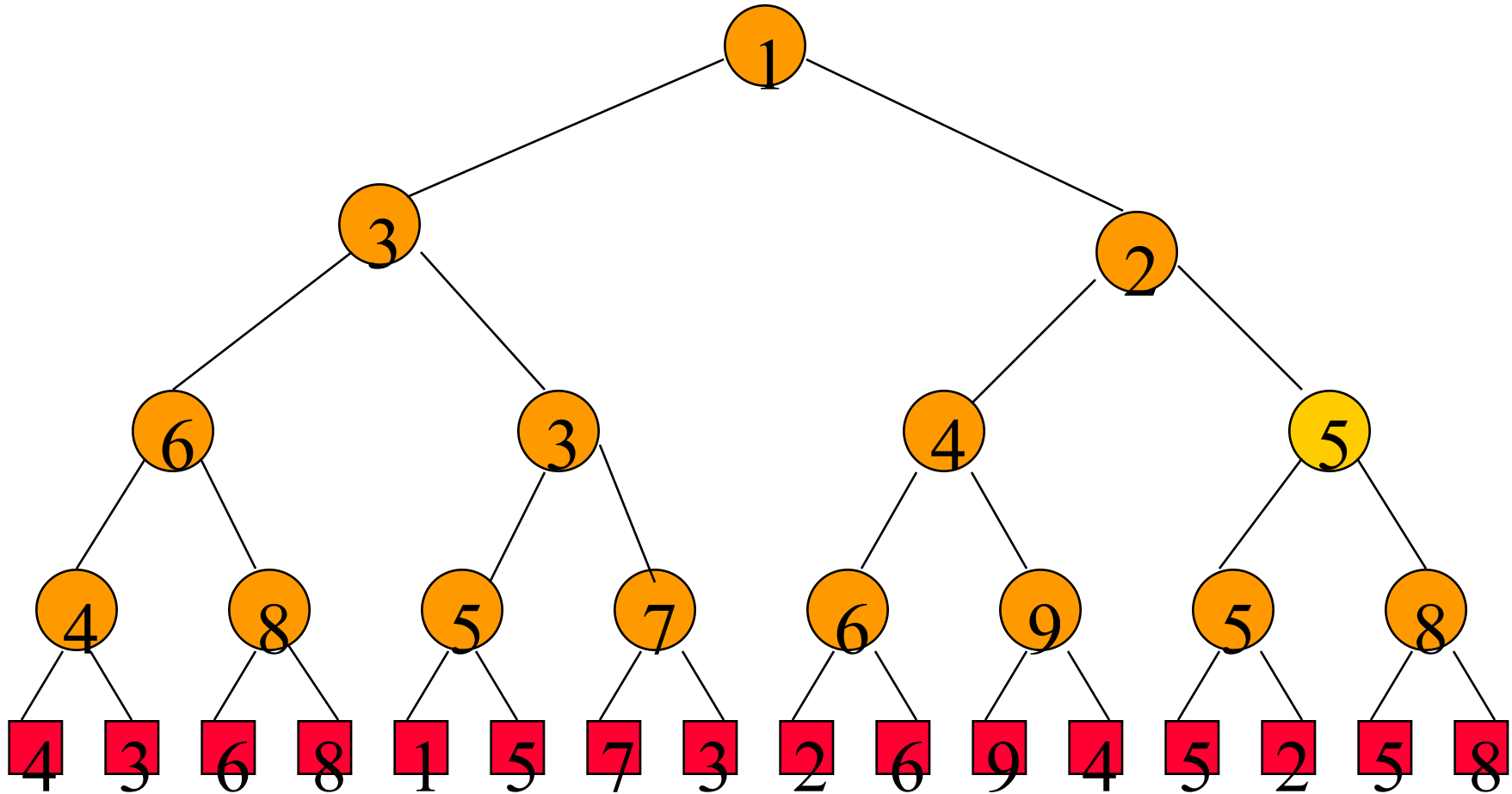
Min Loser Tree For 16 Players



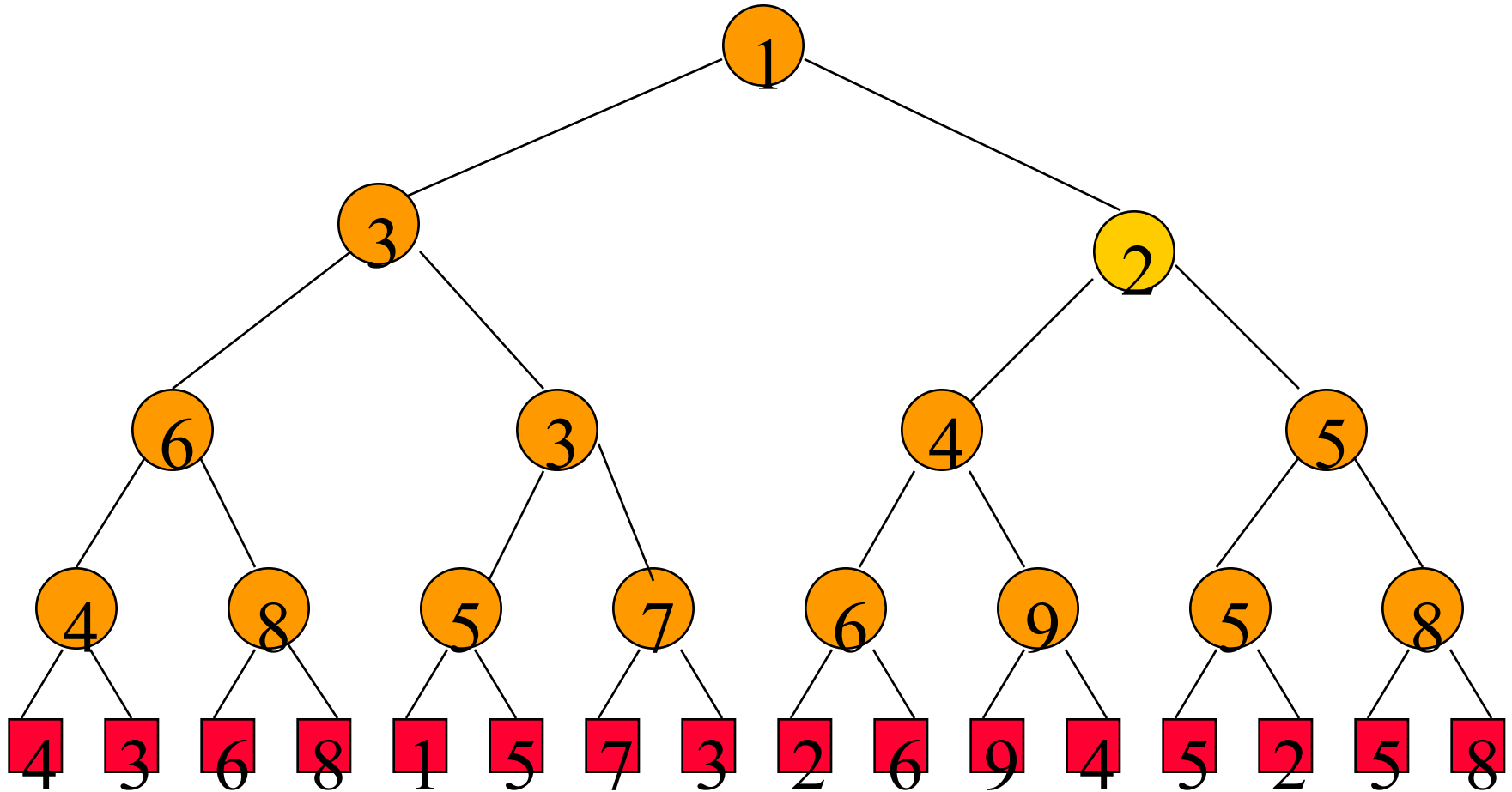
Min Loser Tree For 16 Players



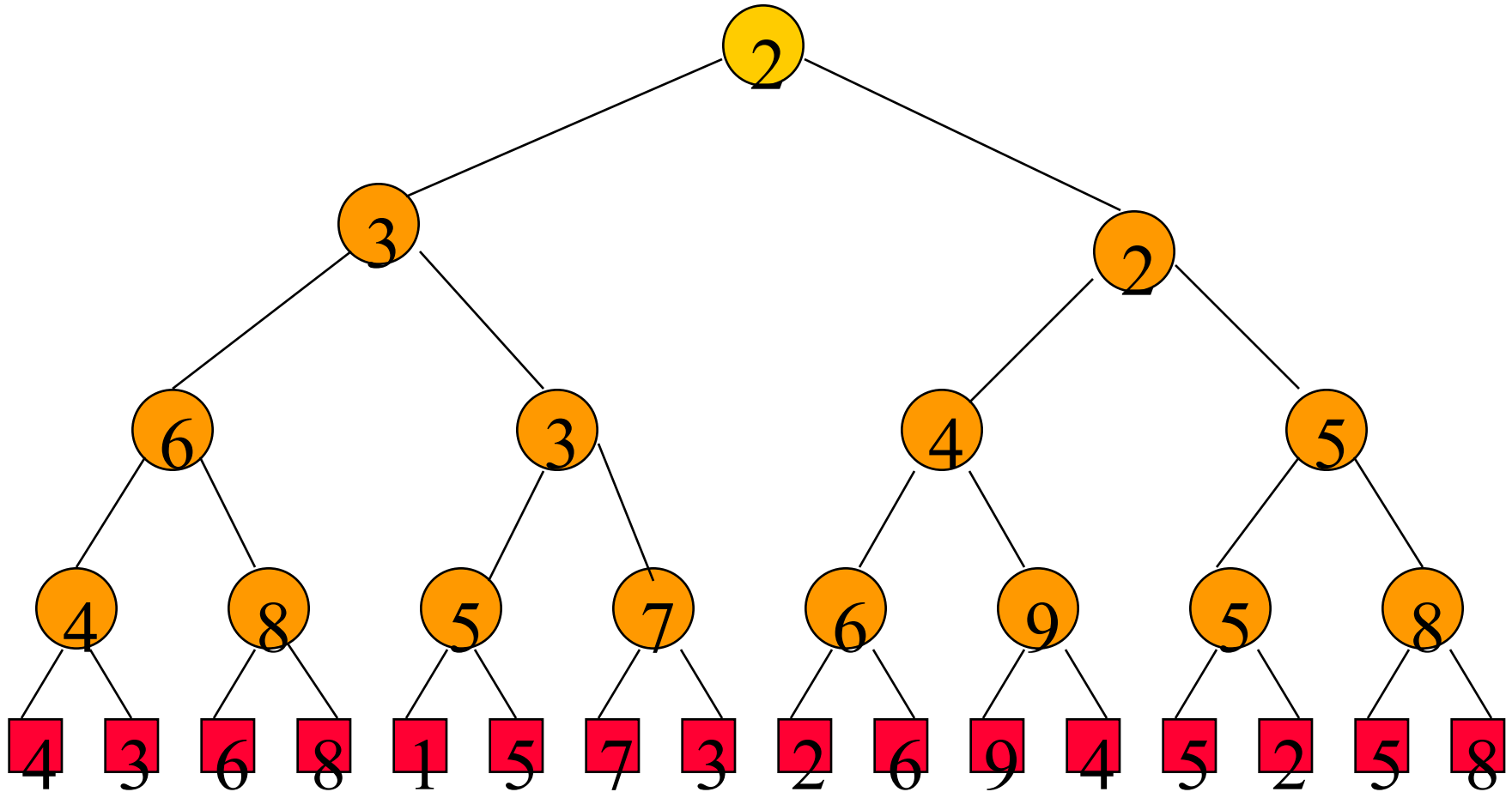
Min Loser Tree For 16 Players

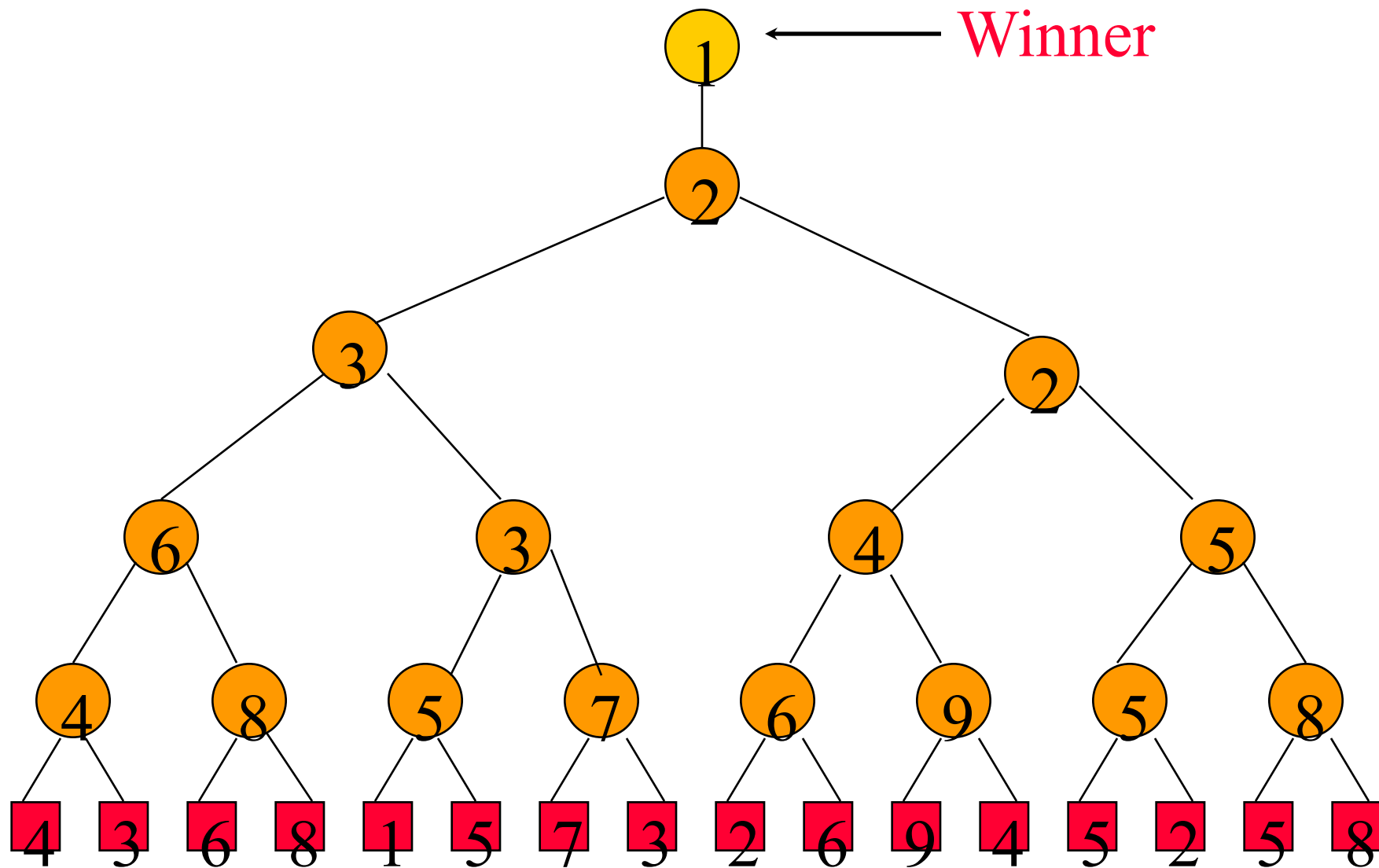


Min Loser Tree For 16 Players



Min Loser Tree For 16 Players

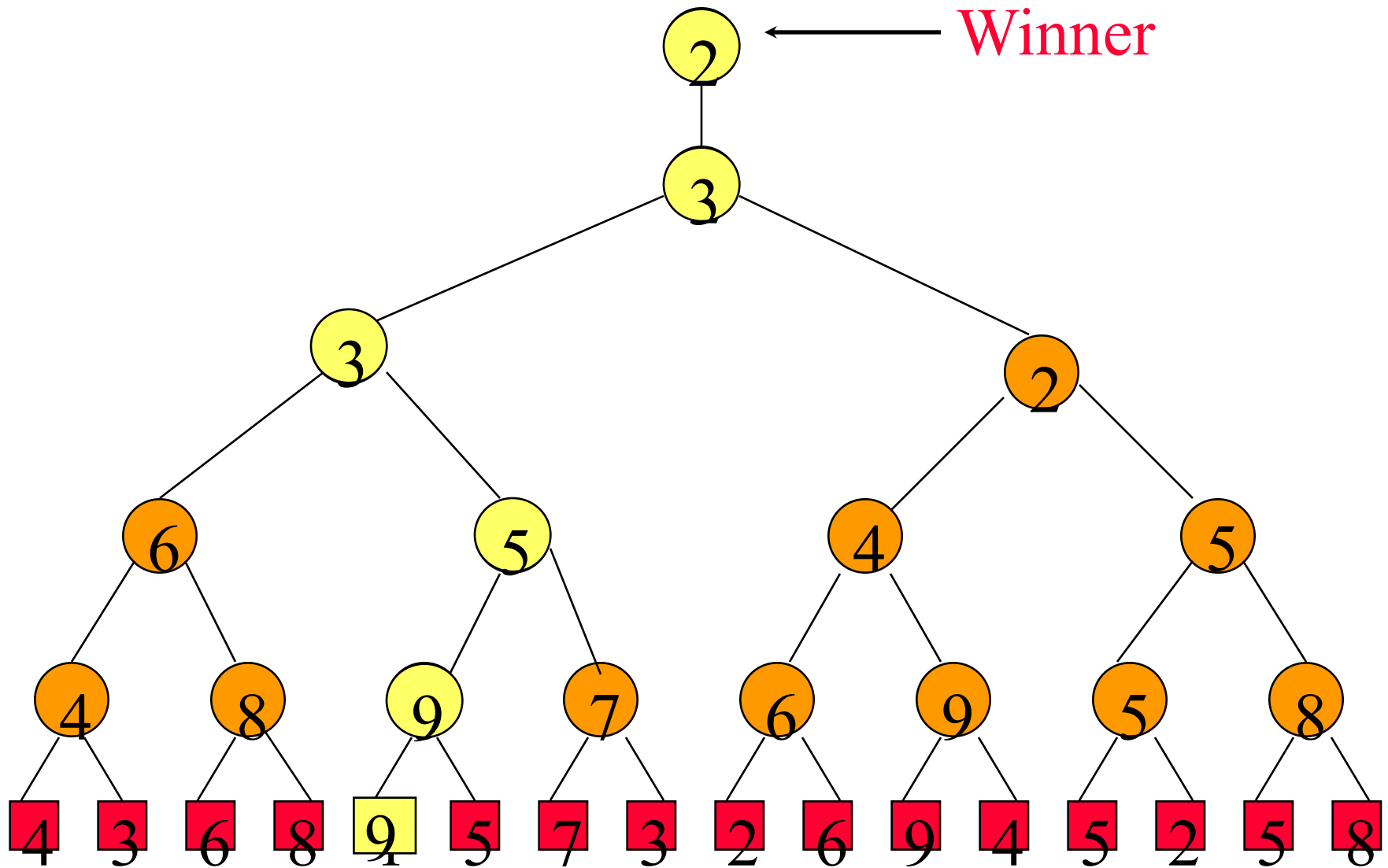




Complexity Of Loser Tree Initialize



- One match at each match node.
- One store of a left child winner.
- Total time is $O(n)$.
- More precisely $\Theta(n)$.



Replace winner with 9 and replay matches.

Complexity Of Replay



- One match at each level that has a match node.
- $O(\log n)$
- More precisely $\Theta(\log n)$.

- **class** LoserTree {
- **public:**
- LoserTree(int k);
- **void** Build();
- ...
- **private:**
- **int** k;
- **int** *l;
- Rec *buf;
- **int** getKey(int i);
- **int** getIndex(int i);
- };

- `LoserTree::LoserTree(int k) {`
- `l = new int[k];`
- `buf = new Rec[k];`
- `}`

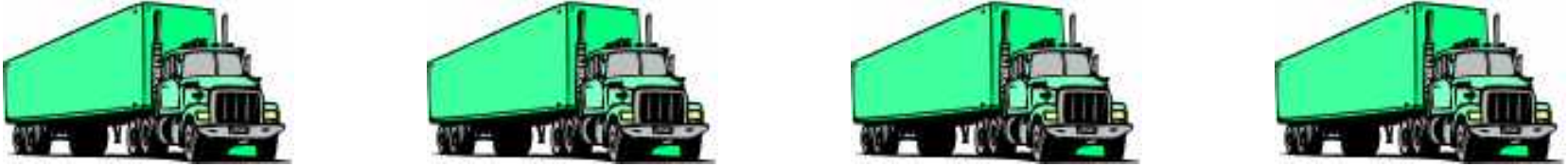
- **void** LoserTree::Build() {
- **int** i;
- **for** (i = k - 1; i > 0; i--)
- **if** (getKey(2*i) > getKey(2*i + 1))
- l[i] = getIndex(2*i + 1);
- **else** l[i] = getIndex(2*i);
- l[0] = l[1];
- **for** (i = 1; i < k; i++)
- **if** (l[i] == getIndex(2*i) l[i] = getIndex(2*i + 1);
- **else** l[i] = getIndex(2*i);
- }

- **int** LoserTree::getKey(**int** i) {
- **if** (i < k) **return** buf[l[i]].key; **else return**
buf[i - k].key;
- }
- **int** LoserTree::getIndex(**int** i) {
- **if** (i < k) **return** l[i]; **else return** (i - k);
- }

More Tournament Tree Applications

- k -way merging of runs during an external merge sort
- Truck loading

Truck Loading



- n packages to be loaded into trucks
- each package has a weight
- each truck has a capacity of c tons
- minimize number of trucks

Truck Loading

$n = 5$ packages

weights $[2, 5, 6, 3, 4]$

truck capacity $c = 10$

Load packages from left to right. If a package doesn't fit into current truck, start loading a new truck.

Truck Loading

$n = 5$ packages

weights $[2, 5, 6, 3, 4]$

truck capacity $c = 10$

truck1 $= [2, 5]$

truck2 $= [6, 3]$

truck3 $= [4]$

uses 3 trucks when 2 trucks suffice

Truck Loading

$n = 5$ packages

weights $[2, 5, 6, 3, 4]$

truck capacity $c = 10$

truck1 $= [2, 5, 3]$

truck2 $= [6, 4]$

Bin Packing

- n items to be packed into bins
- each item has a size
- each bin has a capacity of c
- minimize number of bins

Bin Packing

Truck loading is same as bin packing.

Truck is a bin that is to be packed (loaded).

Package is an item/element.

Bin packing to minimize number of bins is NP-hard.

Several fast heuristics have been proposed.

Bin Packing Heuristics

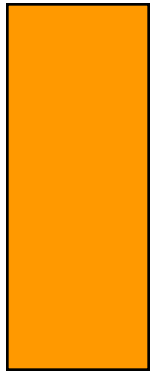
- First Fit.
 - Bins are arranged in left to right order.
 - Items are packed one at a time in given order.
 - Current item is packed into leftmost bin into which it fits.
 - If there is no bin into which current item fits, start a new bin.

First Fit

$n = 4$

weights = [4, 7, 3, 6]

capacity = 10



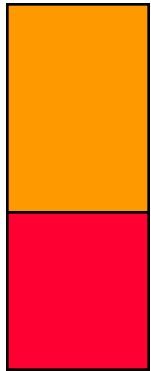
Pack red item into first bin.

First Fit

$n = 4$

weights = [4, 7, 3, 6]

capacity = 10



Pack blue item next.

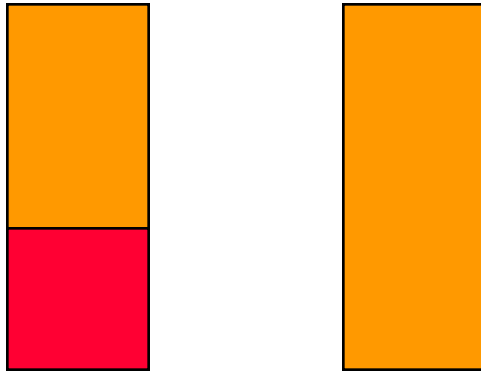
Doesn't fit, so start a new bin.

First Fit

$n = 4$

weights = [4, 7, 3, 6]

capacity = 10

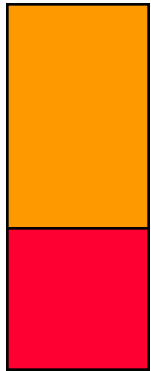


First Fit

$n = 4$

weights = [4, 7, 3, 6]

capacity = 10



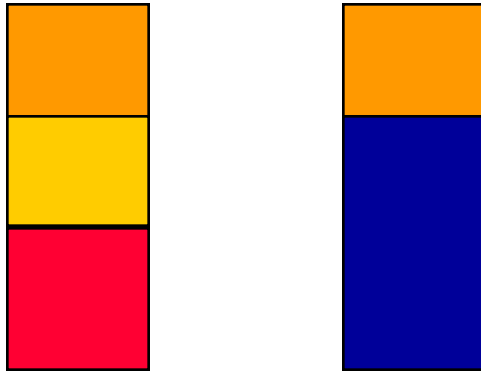
Pack yellow item into first bin.

First Fit

$n = 4$

weights = [4, 7, 3, 6]

capacity = 10



Pack green item.

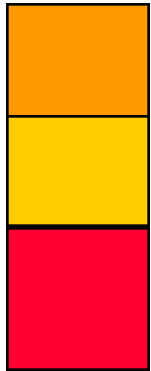
Need a new bin.

First Fit

$n = 4$

weights = [4, 7, 3, 6]

capacity = 10



Not optimal.

2 bins suffice.

Bin Packing Heuristics

- First Fit Decreasing.
 - Items are sorted into decreasing order.
 - Then first fit is applied.

Bin Packing Heuristics

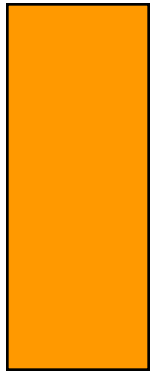
- Best Fit.
 - Items are packed one at a time in given order.
 - To determine the bin for an item, first determine set **S** of bins into which the item fits.
 - If **S is empty**, then start a new bin and put item into this new bin.
 - Otherwise, pack into bin of **S** that has least available capacity.

Best Fit Example

$n = 4$

weights = **[4, 7, 3, 6]**

capacity = 10



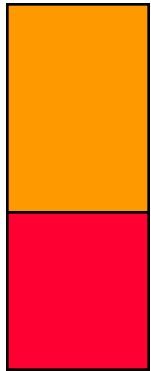
Pack red item into first bin.

Best Fit

$n = 4$

weights = **[4, 7, 3, 6]**

capacity = 10



Pack blue item next.

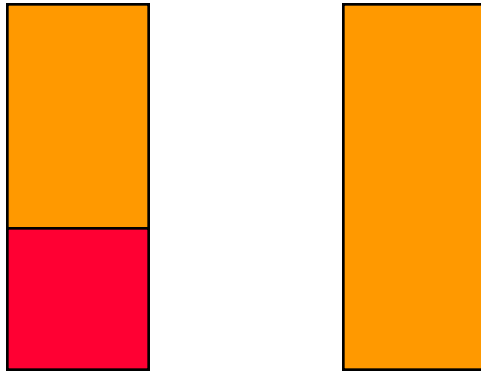
Doesn't fit, so start a new bin.

Best Fit

$n = 4$

weights = **[4, 7, 3, 6]**

capacity = 10

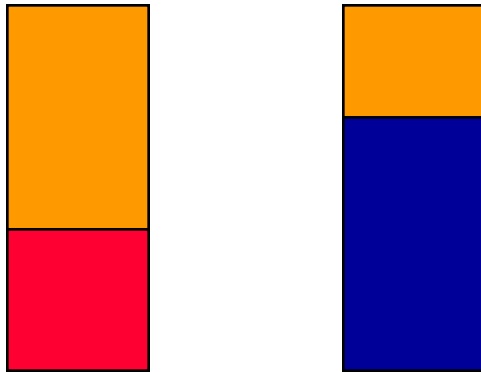


Best Fit

$n = 4$

weights = **[4, 7, 3, 6]**

capacity = 10



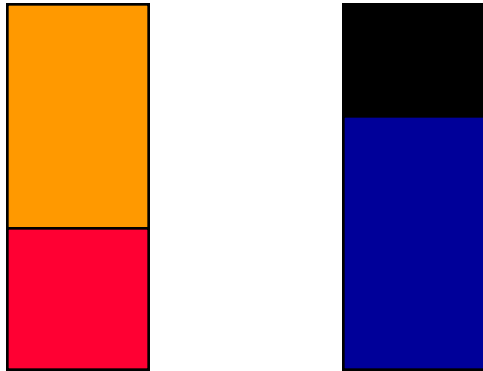
**Pack yellow item into
second bin.**

Best Fit

$n = 4$

weights = **[4, 7, 3, 6]**

capacity = 10



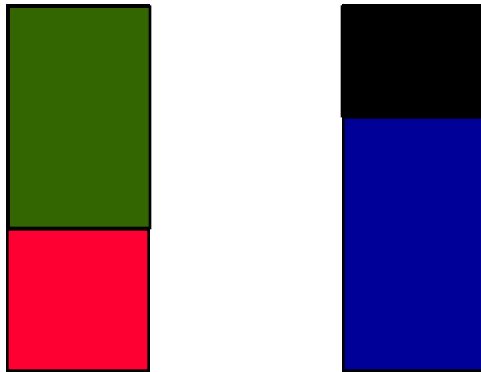
**Pack green item
into first bin.**

Best Fit

$n = 4$

weights = **[4, 7, 3, 6]**

capacity = 10



**Optimal
packing.**

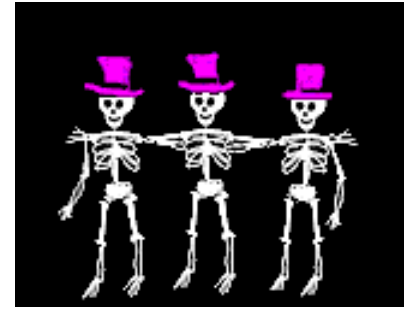
Implementation Of Best Fit

- Use a dynamic dictionary in which the elements are of the form (available capacity, bin index).
- Pack an item whose requirement is s .
 - Find a bin with smallest available capacity $\geq s$.
 - Reduce available capacity of this bin by s .
 - May be done by removing old pair and inserting new one.
 - If no such bin, start a new bin.
 - Insert a new pair into the dictionary.

Bin Packing Heuristics

- Best Fit Decreasing.
 - Items are sorted into decreasing order.
 - Then best fit is applied.

Performance



- For first fit and best fit:

$$\text{Heuristic Bins} \leq (17/10)(\text{Minimum Bins}) + 2$$

- For first fit decreasing and best fit decreasing:

$$\text{Heuristic Bins} \leq (11/9)(\text{Minimum Bins}) + 4$$

Complexity Of First Fit



Use a max tournament tree in which the players are **n** bins and the value of a player is the available capacity in the bin.

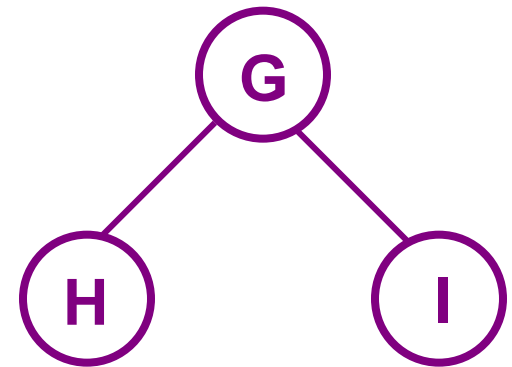
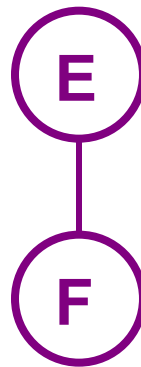
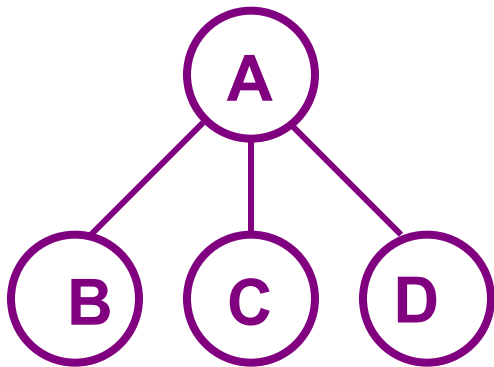
$O(n \log n)$, where **n** is the number of items.

- **Exercises: P301-1,4**

Forests



- Definition:
 - A forest is a set of $n \geq 0$ disjoint trees



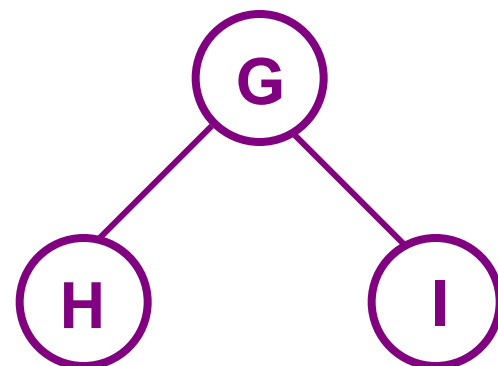
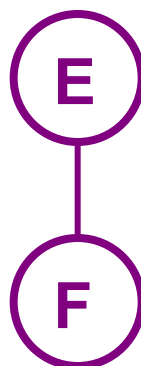
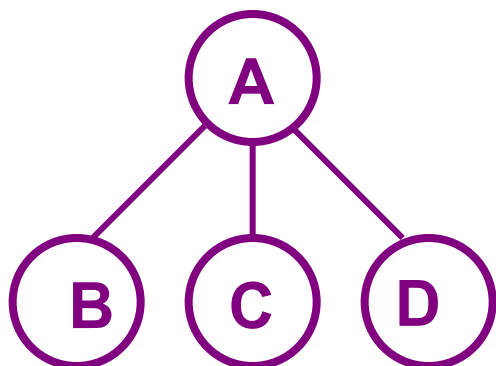
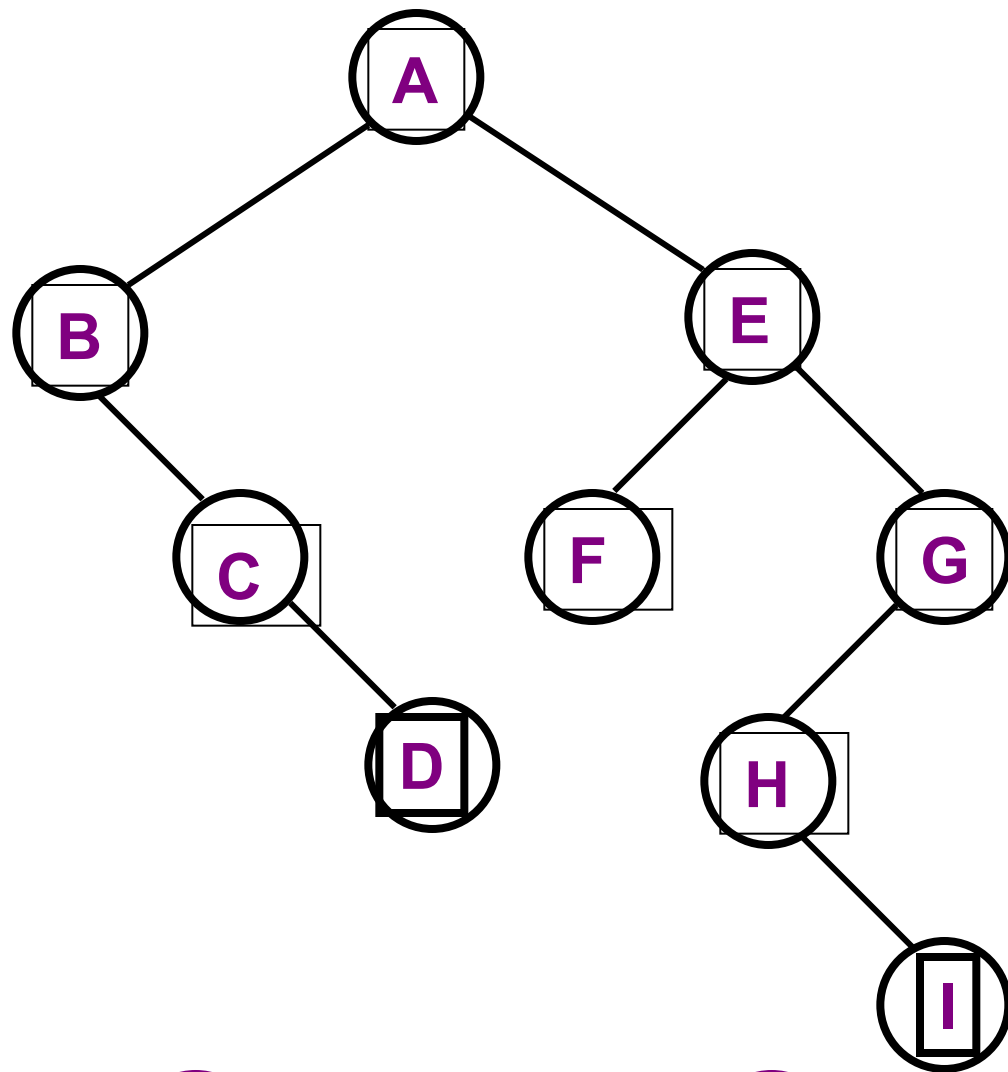
Transforming a Forest into a Binary Tree

Definition:

If $T_1, \dots, T_n$ is a forest of trees, then the binary tree corresponding to it, denoted by $B(T_1, \dots, T_n)$,

(1) is empty if $n=0$

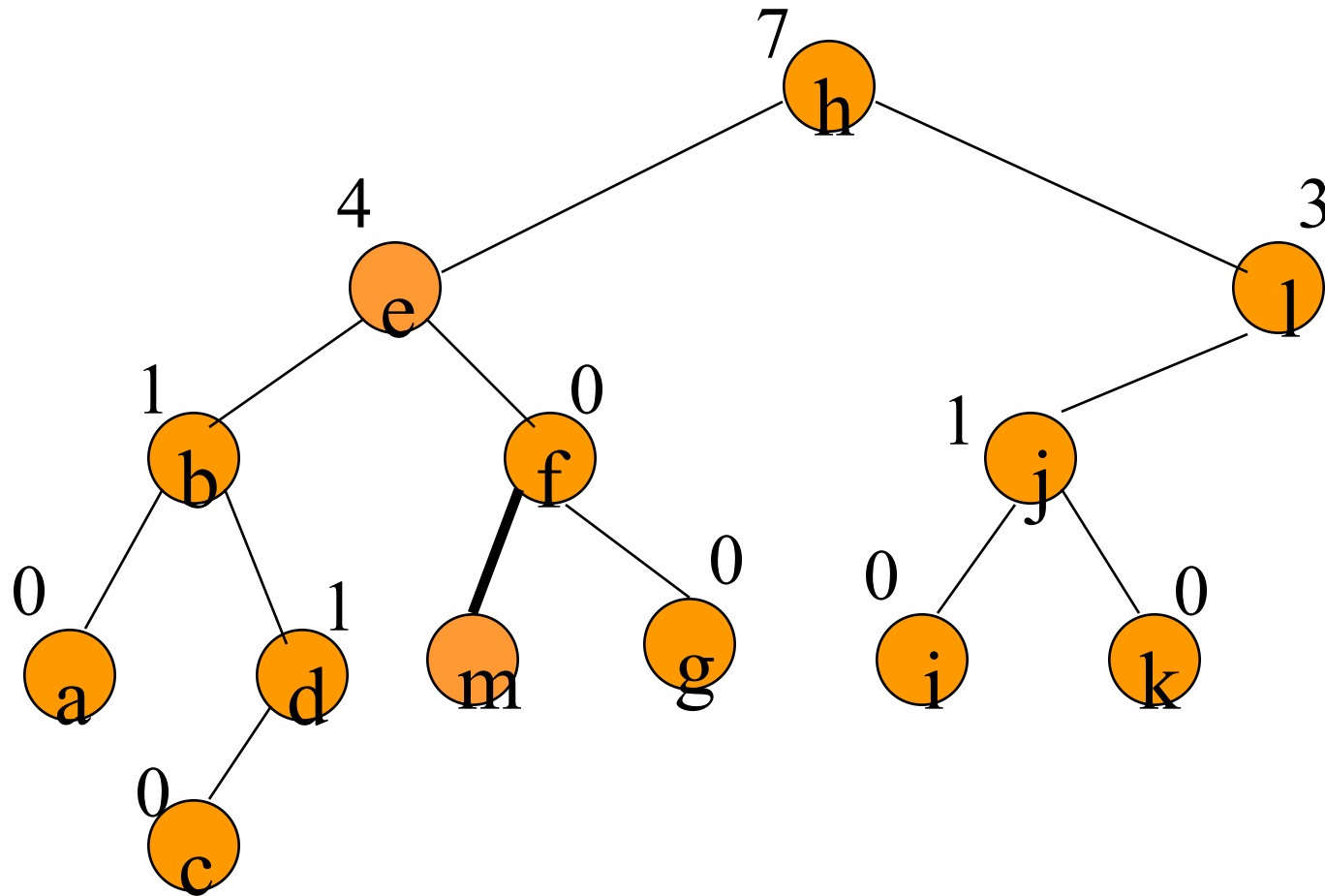
(2) has root equal to $\text{root}(T_1)$; has left subtree equal to $B(T_{11}, \dots, T_{1m})$, where $T_{11}, \dots, T_{1m}$ are the subtrees of $\text{root}(T_1)$; and has right subtree $B(T_2, \dots, T_n)$.



Transforming a Forest into a Binary Tree

- Left child $\rightarrow$ first child
- Right child $\rightarrow$ Sibling

Transform a Binary Tree to a Forest

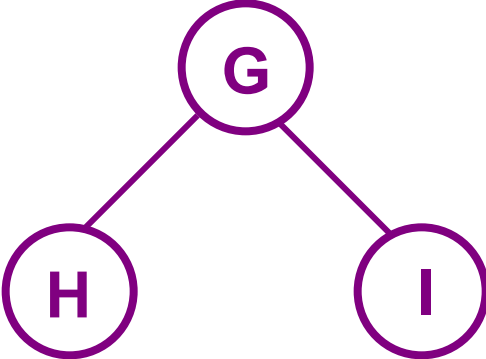
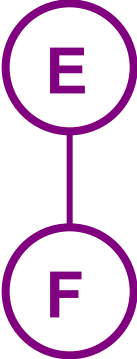
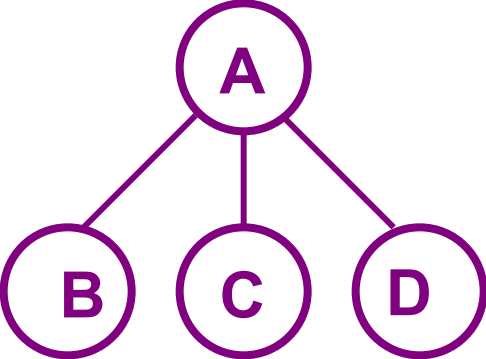


Forest Traversals

Let T be the corresponding binary tree of a forest F .

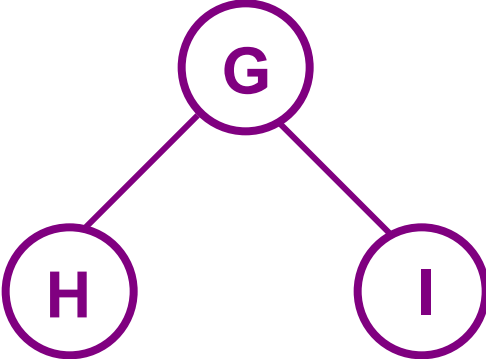
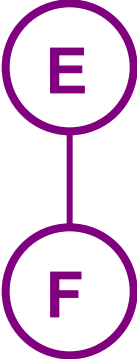
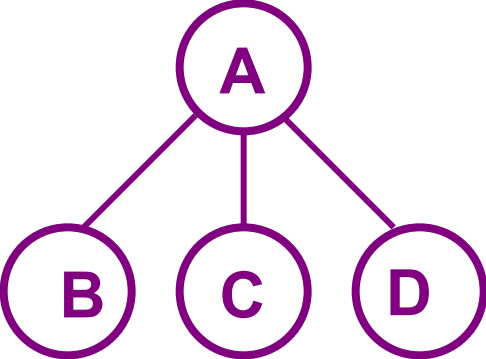
Visiting the nodes of F in **forest preorder** is defined as:

- (1) If F is empty then return.
- (2) Visit the root of the first tree of F .
- (3) Traverse the subtrees of the first tree in forest preorder.
- (4) Traverse the remaining trees of F in forest preorder.



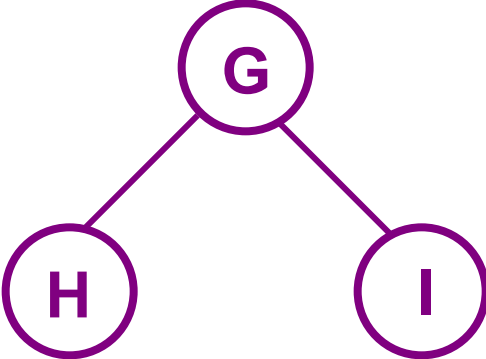
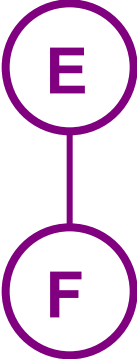
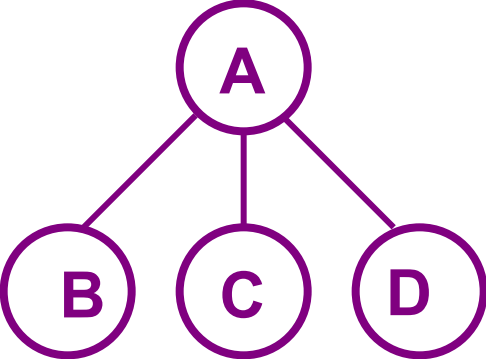
Visiting the nodes of F in forest **inorder** is defined as:

- (1) If F is empty then return.
- (2) Traverse the subtrees of the first tree in forest **inorder**.
- (3) Visit the root of the first tree of F .
- (4) Traverse the remaining trees of F in forest **inorder**.



Visiting the nodes of F in forest **postorder** is defined as:

- (1) If F is empty then return.
- (2) Traverse the subtrees of the first tree in forest **postorder**.
- (3) Traverse the remaining trees of F in forest **postorder**.
- (4) Visit the root of the first tree of F .

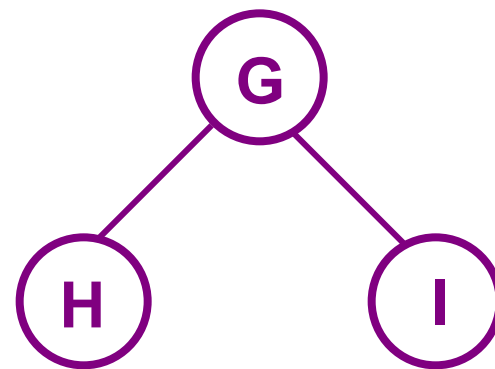
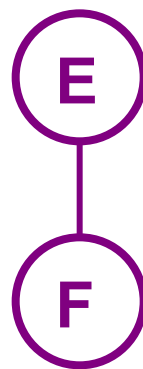
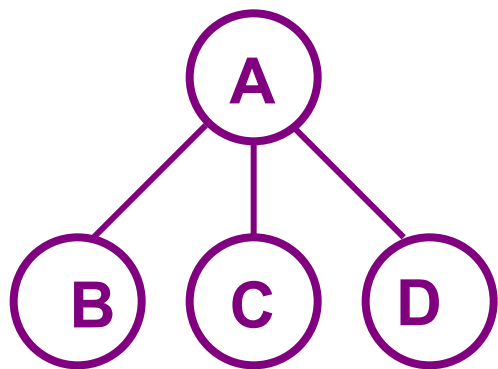


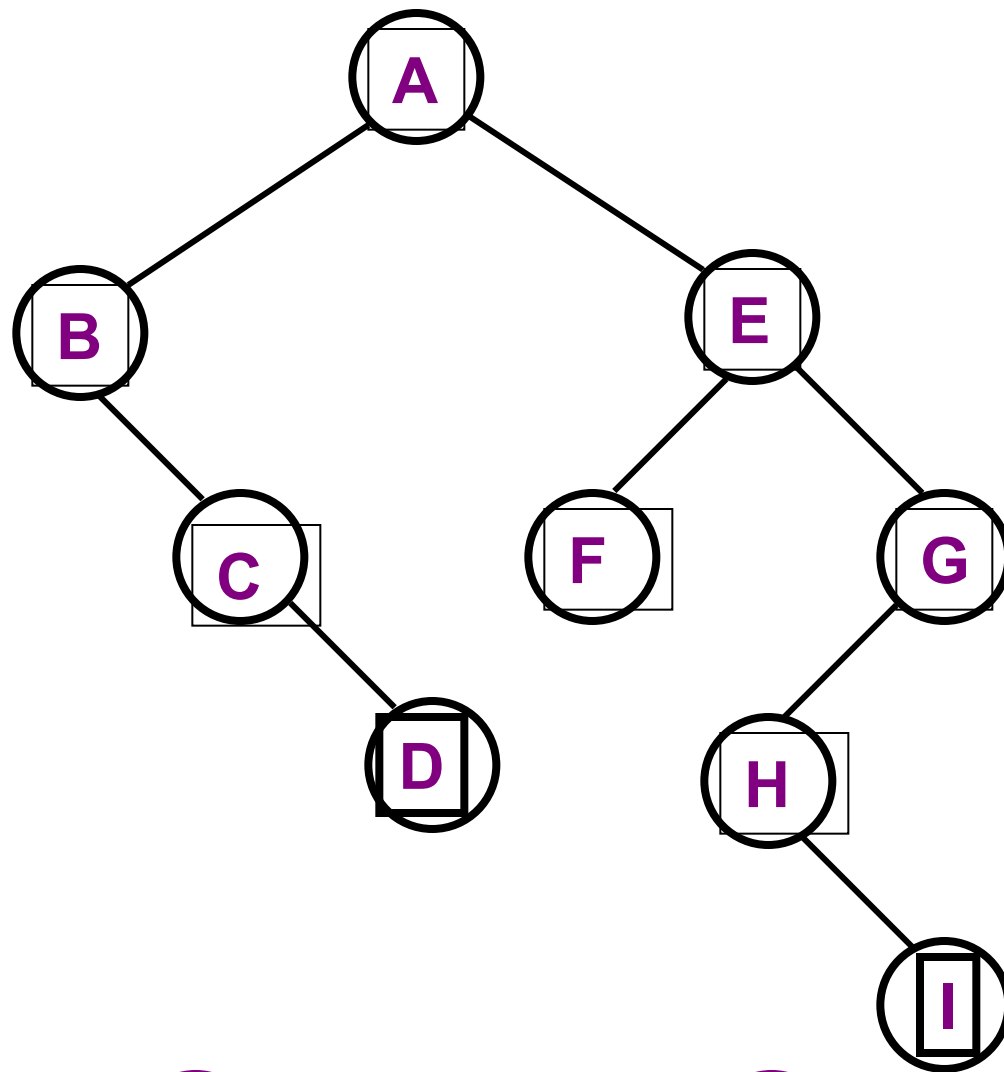
In level-order traversal of F

Nodes are visited by level

Beginning with the roots of each trees in F

Within each level, from left to right.

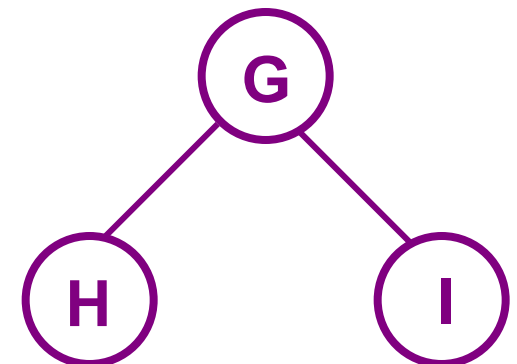
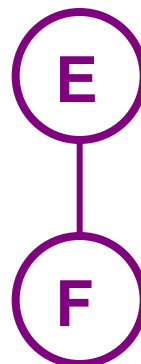
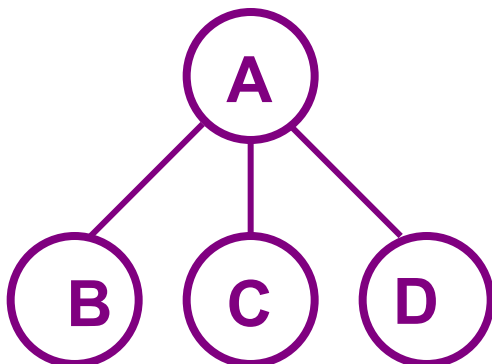




PreOrder

InOrder

PostOrder



Exercises: P304-3.

Set Presentation



- Assume:
- Elements of the sets are the numbers 0, 1, 2, ..., $n-1$ (might be thought as indices).
- For any two sets $S_i, S_j, i \neq j, S_i \cap S_j = \emptyset$.
- Operations:
 - Disjoint set union $S_i \cup S_j$.
 - Find(i)---find the set containing i .

Union-Find Problem



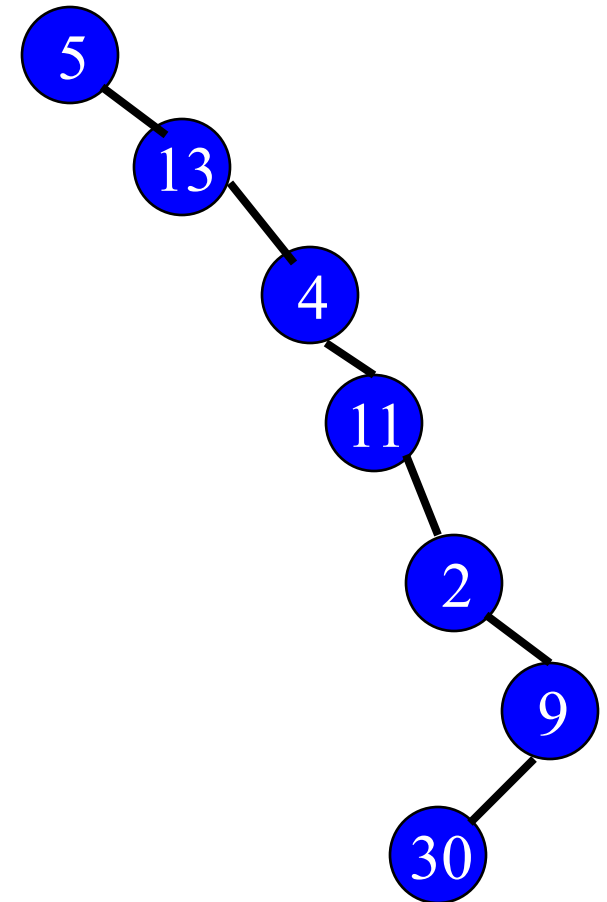
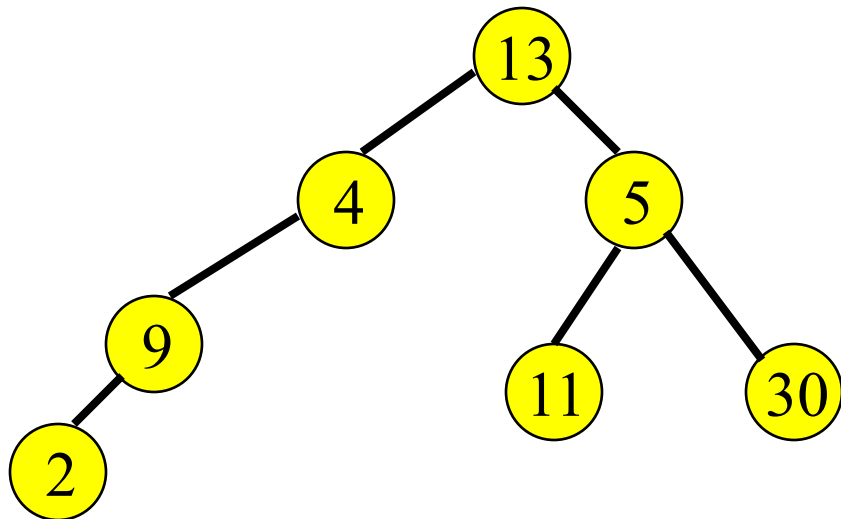
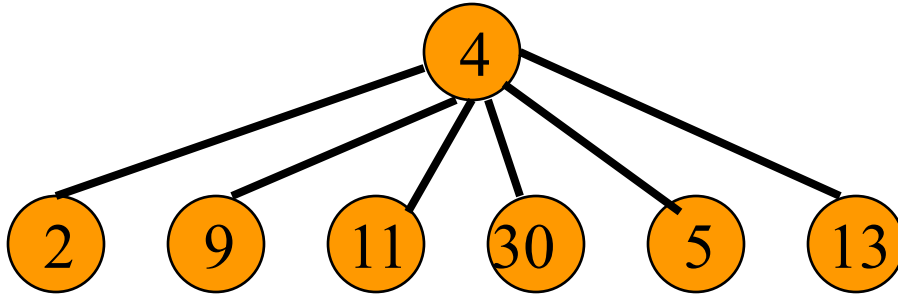
- A union operation combines two sets into one.
 - Each of the n elements is in exactly one set at any time.
- A find operation identifies the set that contains a particular element.

Using Arrays And Chains

- Array
 - Union
 - Find
- Chains
 - Union
 - Find

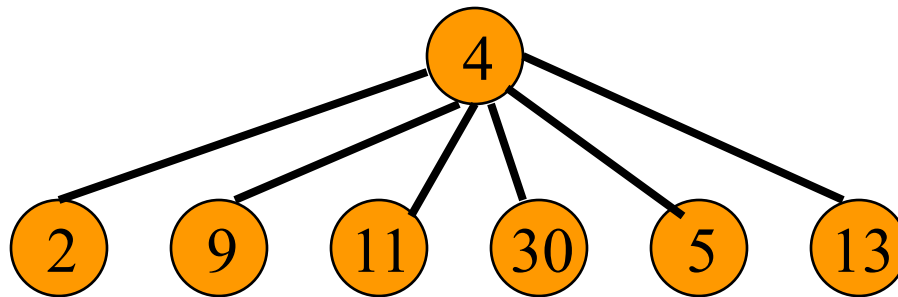
A Set As A Tree

- $S = \{2, 4, 5, 9, 11, 13, 30\}$
- Some possible tree representations:



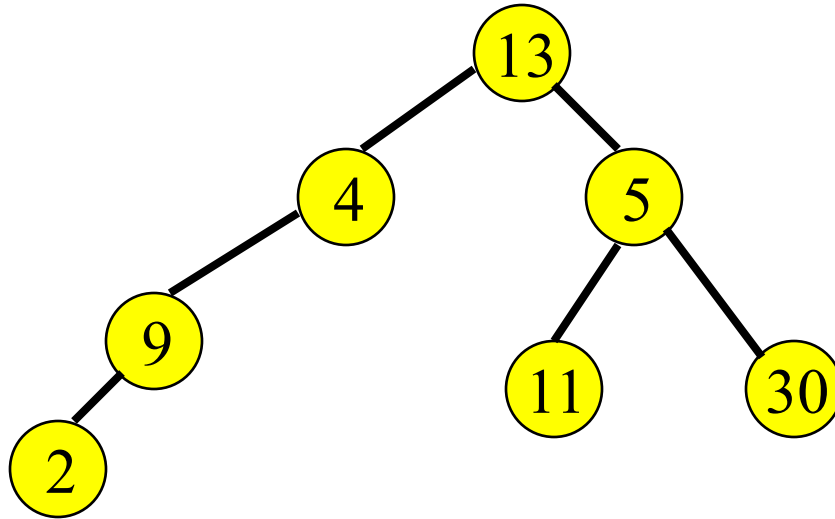
Result Of A Find Operation

- $\text{find}(i)$ is to identify the set that contains element i .
- In most applications of the union-find problem, the user does not provide set identifiers.
- The requirement is that $\text{find}(i)$ and $\text{find}(j)$ return the same value iff elements i and j are in the same set.



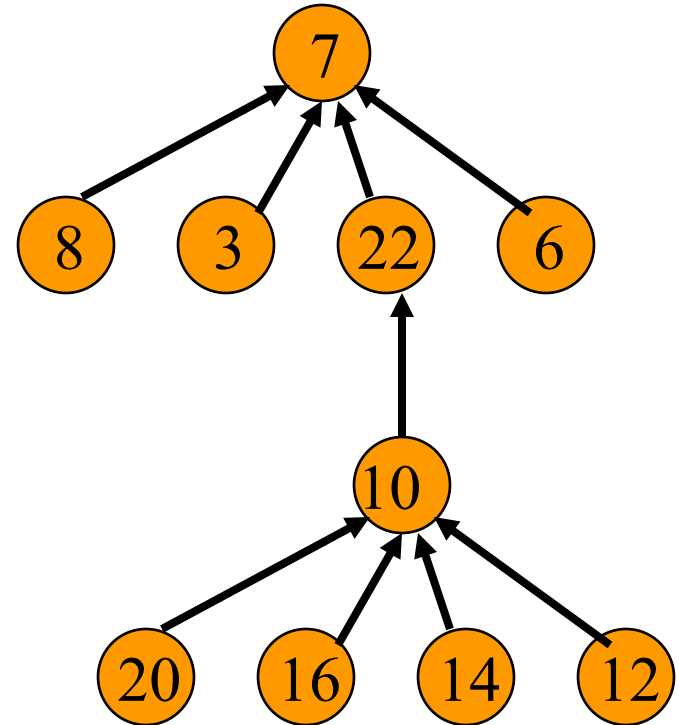
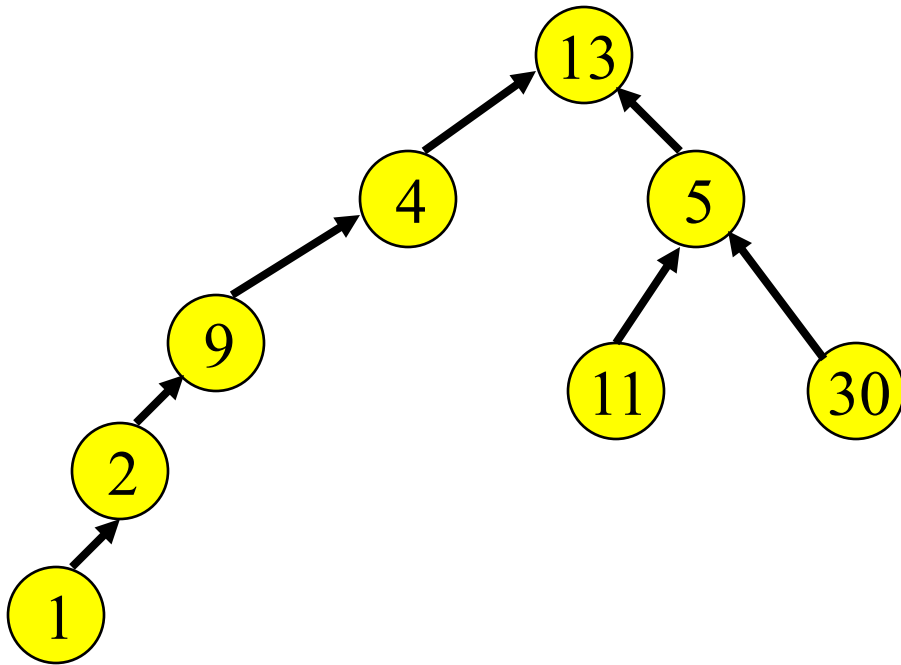
$\text{find}(i)$ will return the element that is in the tree root.

Strategy For find(i)



- Tree traversal from the root? $O(n)$
- Start at the node that represents element **i** and climb up the tree until the root is reached.
- Return the element in the root.
- To climb the tree, each node must have a parent pointer.

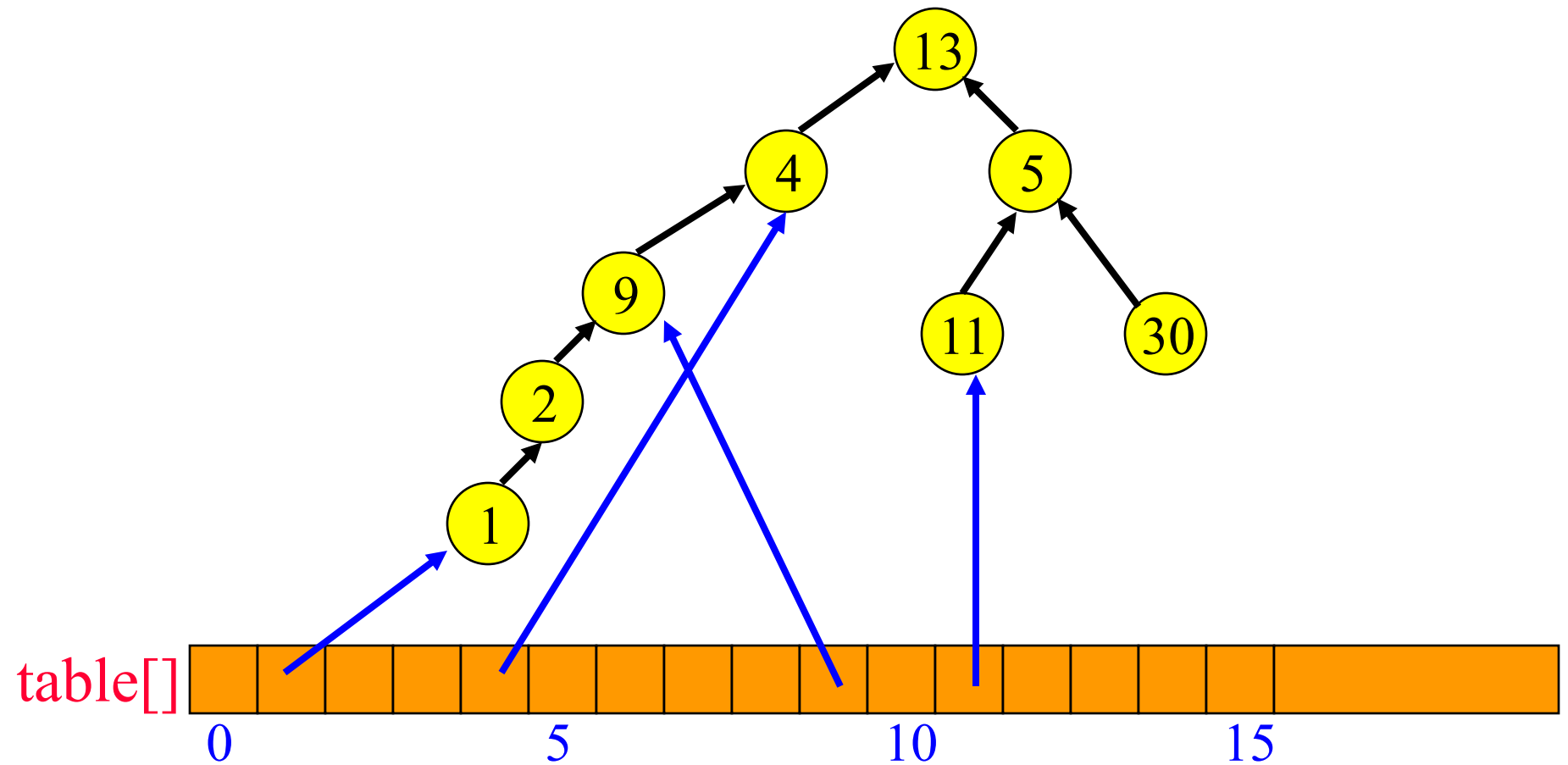
Trees With Parent Pointers



Possible Node Structure

- Use nodes that have two fields: **element** and **parent**.
 - Use an array **table[]** such that **table[i]** is a pointer to the node whose element is **i**.
 - To do a **find(i)** operation, start at the node given by **table[i]** and follow parent fields until a node whose parent field is null is reached.
 - Return element in this root node.

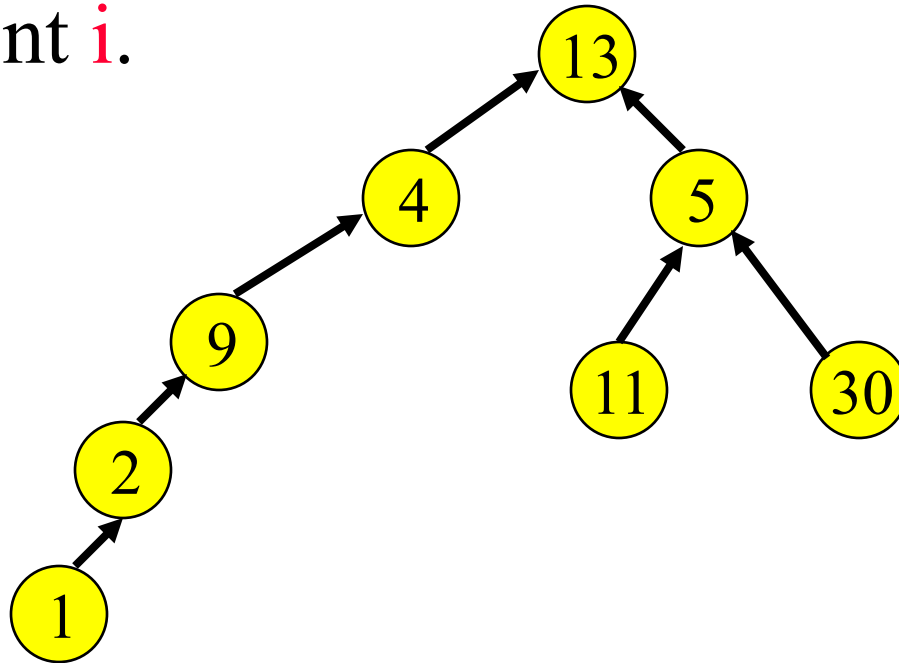
Example



(Only some table entries are shown.)

Better Representation

- Use an integer array `parent[]` such that `parent[i]` is the element that is the parent of element `i`.



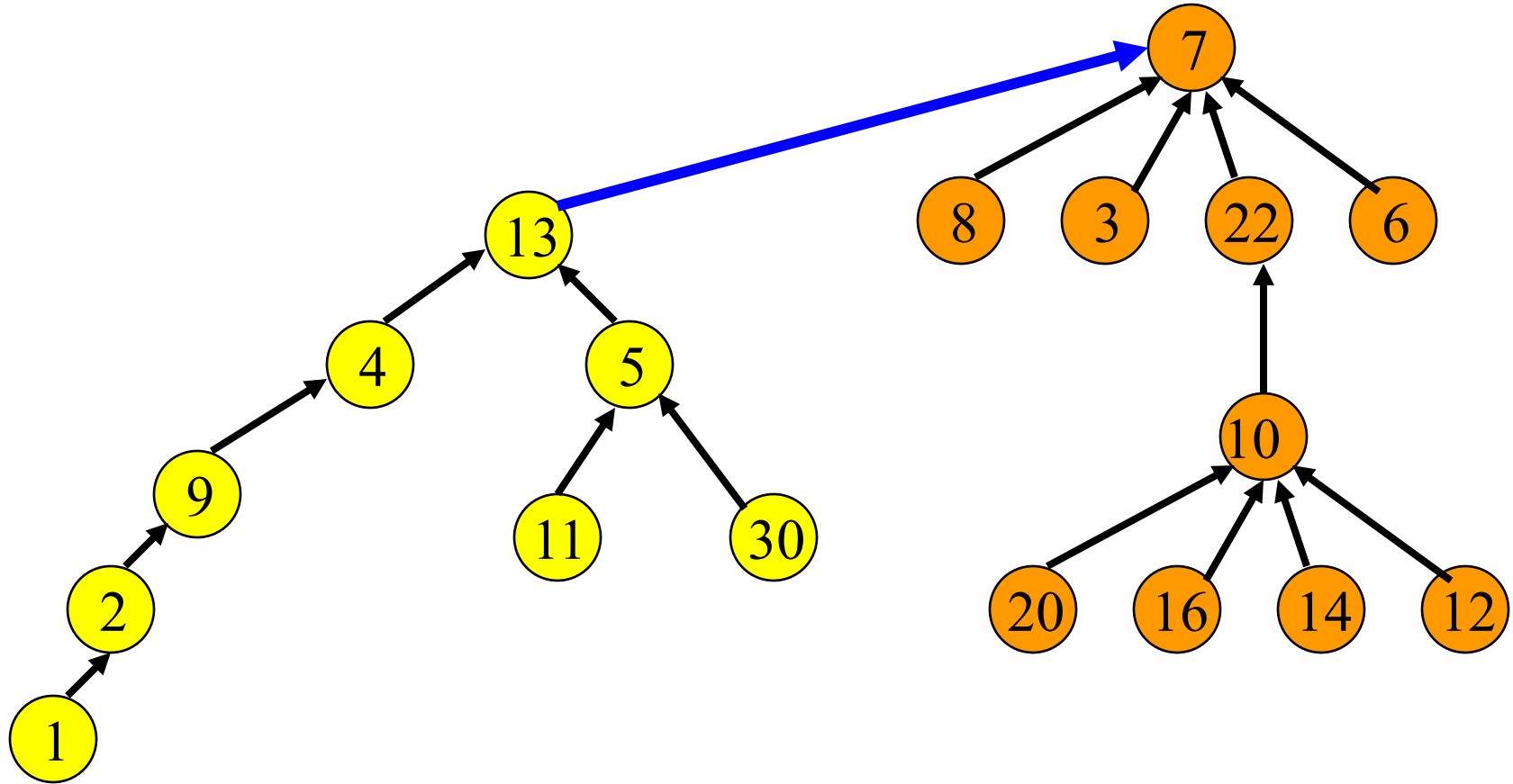
parent[]		2	9		13	13				4		5		0			
	0				5					10				15			

- **class** Sets {
- **public:**
- // Set operations
- **private:**
- **int** *parent;
- **int** n; // number of set elements
- };
- Sets::Sets (**int** numberOfElements)
- {
- **if** (numberOfElements < 2) **throw** “Must have at least 2 elements.”;
- n=numberOfElements;
- parent=**new int**[n];
- fill(parent, parent+n, -1);
- }

Union Operation

- $\text{union}(i, j)$
 - i and j are the roots of two different trees, $i \neq j$.
- To unite the trees, make one tree a subtree of the other.
 - $\text{parent}[j] = i$

Union Example



- $\text{union}(7, 13)$

The Simple Find Method

```
int Sets::SimpleFind (int i)
{ //find the root of the tree containing element i.
  while (parent[i]>=0)
    i=parent[i];
  return i;
}
```

The Simple Union Method

```
void Sets::SimpleUnion (int j, int i)
{ // Replace the disjoint sets with roots i and j,
  i!=j with their
    // union
    parent[i] = j;
}
```


Time Complexity Of union()

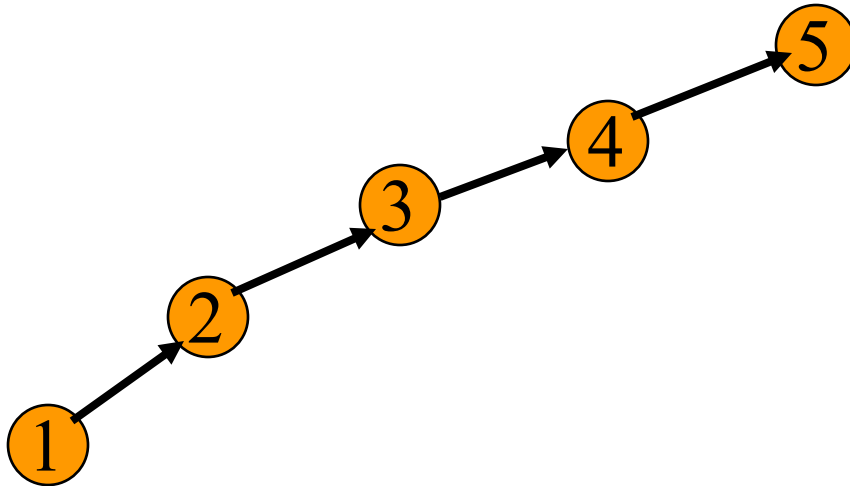


- $O(1)$

Time Complexity of find()



- Tree height may equal number of elements in tree.
 - $\text{union}(2,1), \text{union}(3,2), \text{union}(4,3), \text{union}(5,4)\dots$



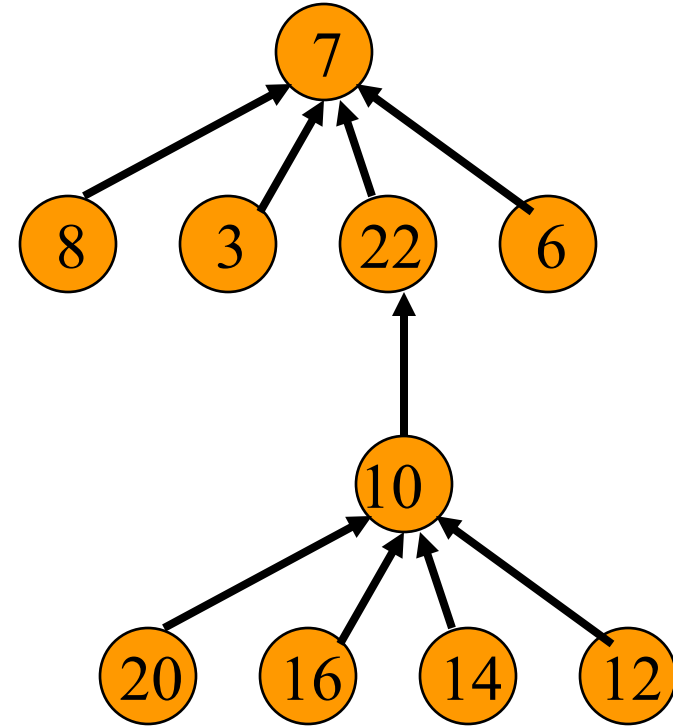
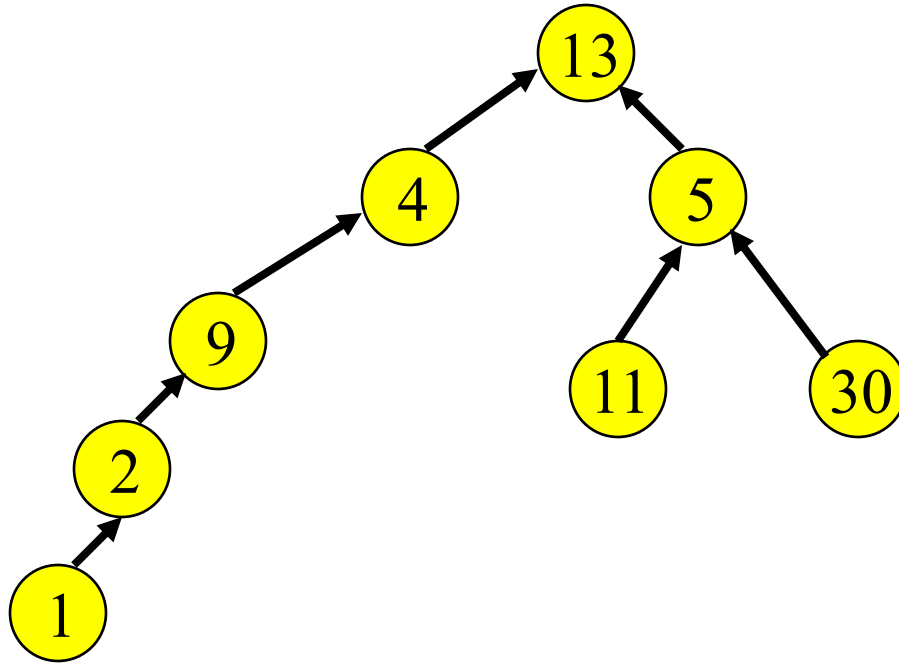
So complexity is $O(u)$.

u Unions and f Find Operations



- $O(u + uf) = O(uf)$
- Time to initialize $\text{parent}[i] = 0$ for all i is $O(n)$.
- Total time is $O(n + uf)$.

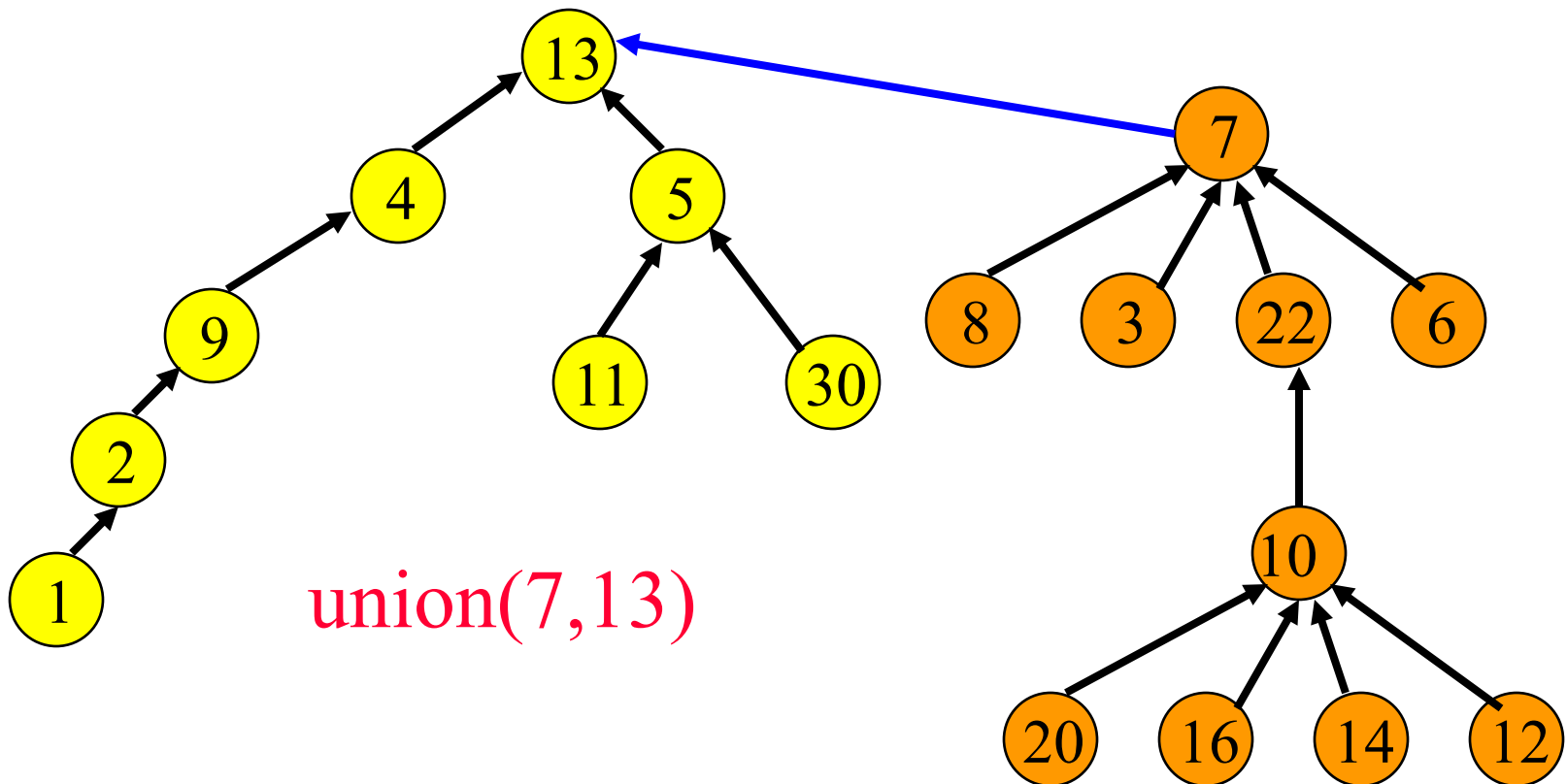
Smart Union Strategies



- $\text{union}(7, 13)$
- Which tree should become a subtree of the other?

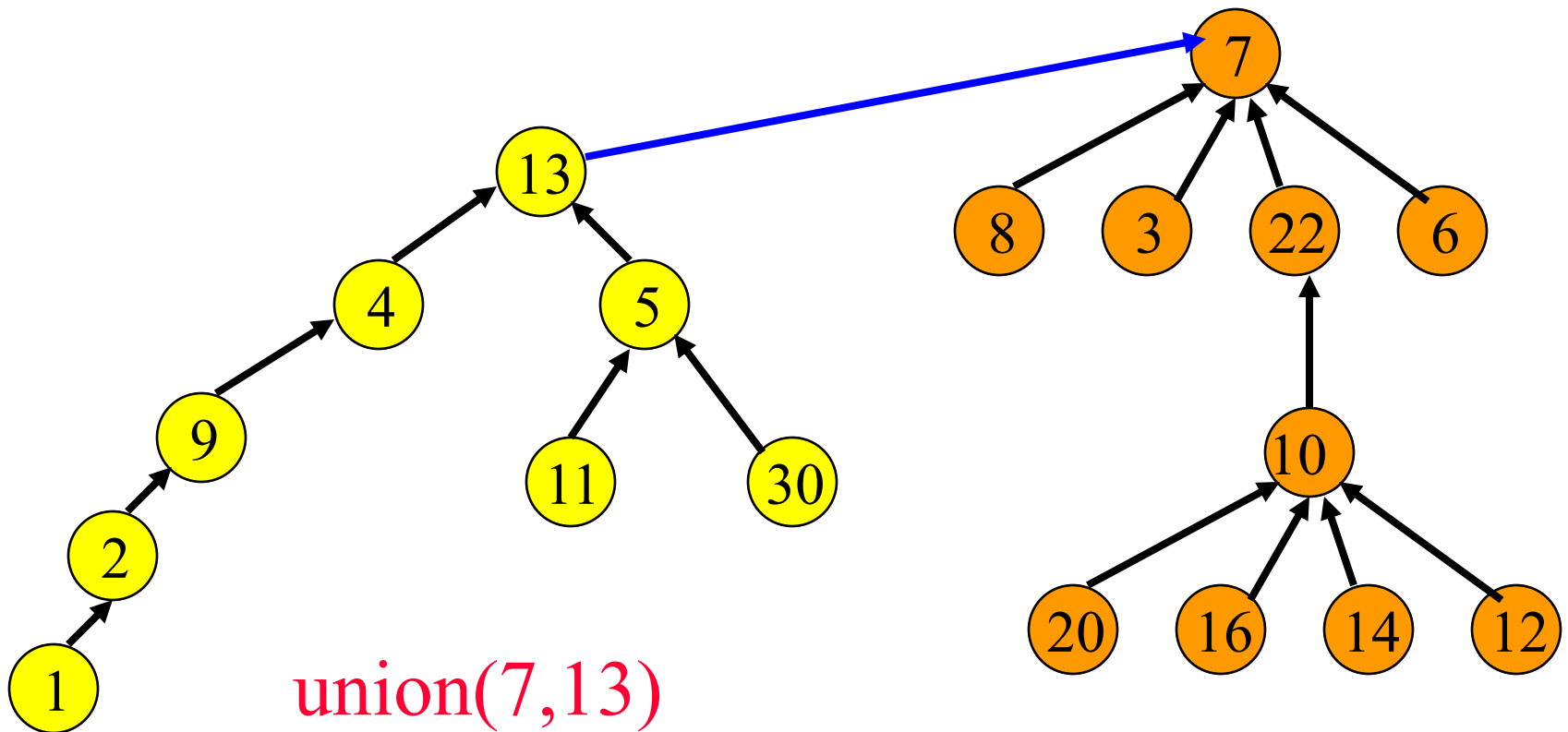
Height Rule

- Make tree with smaller height a subtree of the other tree.



Weight Rule

- Make tree with fewer number of elements a subtree of the other tree.



Implementation

- Root of each tree must record either its height or the number of elements in the tree.
- When a union is done using the height rule, the height increases only when two trees of equal height are united.
- When the weight rule is used, the weight of the new tree is the sum of the weights of the trees that are united.

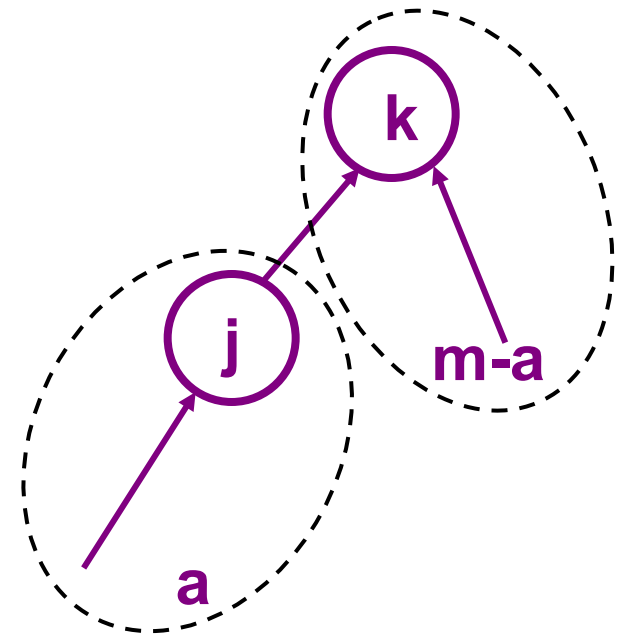
```
void Sets::WeightedUnion (int i, int j)
{ // Union sets with roots i and j,  $i \neq j$ , weighting rule
  //  $\text{parent}[i] = -\text{count}[i]$  and  $\text{parent}[j] = -\text{count}[j]$ 
  int temp = parent[i] + parent[j];
  if (parent[i] > parent[j]) { // i has fewer nodes
    parent[i] = j;
    parent[j] = temp;
  }
  else { // j has fewer nodes
    parent[j] = i;
    parent[i] = temp;
  }
}
```


Height Of A Tree

- Suppose we start with single element trees and perform unions using either the height or the weight rule.
- **Lemma 5.5** The height of a tree with m elements is at most $\text{floor}(\log_2 m) + 1$.

- **Lemma 5.5** The height of a tree with m elements is at most $\text{floor}(\log_2 m) + 1$.
- **Proof by induction:**
 - $m = 1$, it is true.
 - Assume it is true for all trees with $i \leq m-1$ nodes.
 - For $i = m$, let T be a tree with m nodes created by **WeightedUnion**.
 - Consider the last union performed,
 - **Union(k, j).**

Let **a** be the number of nodes in tree **j** and **m-a** that in tree **k**. without loss of generality, assume $1 \leq a \leq m/2$. Then the height of T is either **the same as that of k** or is **1 + that of j**.



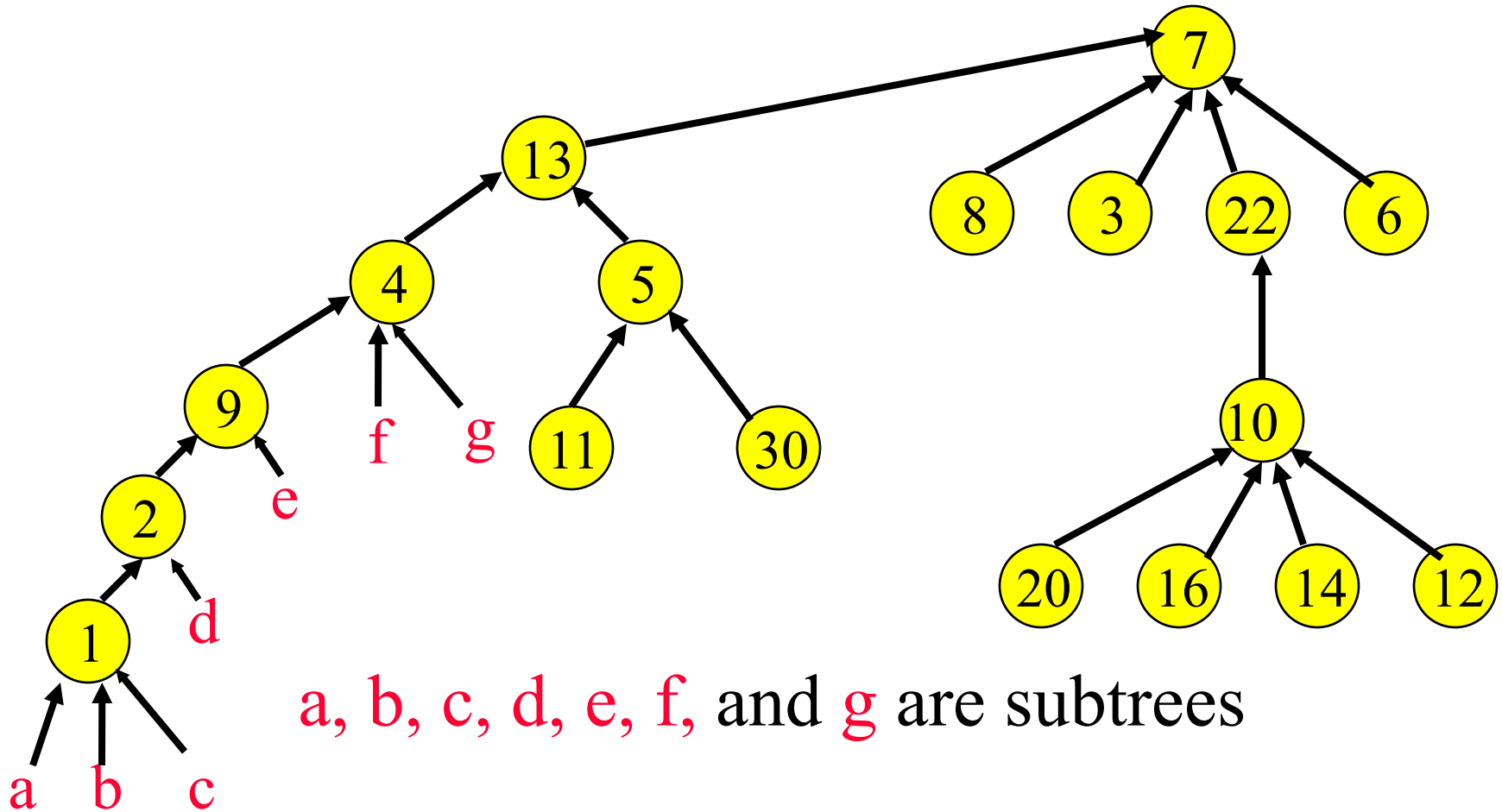
$$m-a \geq m/2 \geq a$$

If the former is the case, the height of $T \leq \lfloor \log_2 (m-a) \rfloor + 1 \leq \lfloor \log_2 m \rfloor + 1$.

If the latter is the case, the height of $T \leq \lfloor \log_2 a \rfloor + 2 \leq \lfloor \log_2 m/2 \rfloor + 2 \leq \lfloor \log_2 m \rfloor + 1$.

- **The time to process a find is at most $O(\log n)$ in a tree of n nodes**
- **If an intermixed sequence of $u-1$ union and f find is to be done**
- **The worst case time is $O(u + f \log u)$.**

Sprucing Up The Find Method



- `find(1)`
- Do additional work to make future finds easier.

Path Collapsing/Path Compaction

Further improvement in the find algorithm.

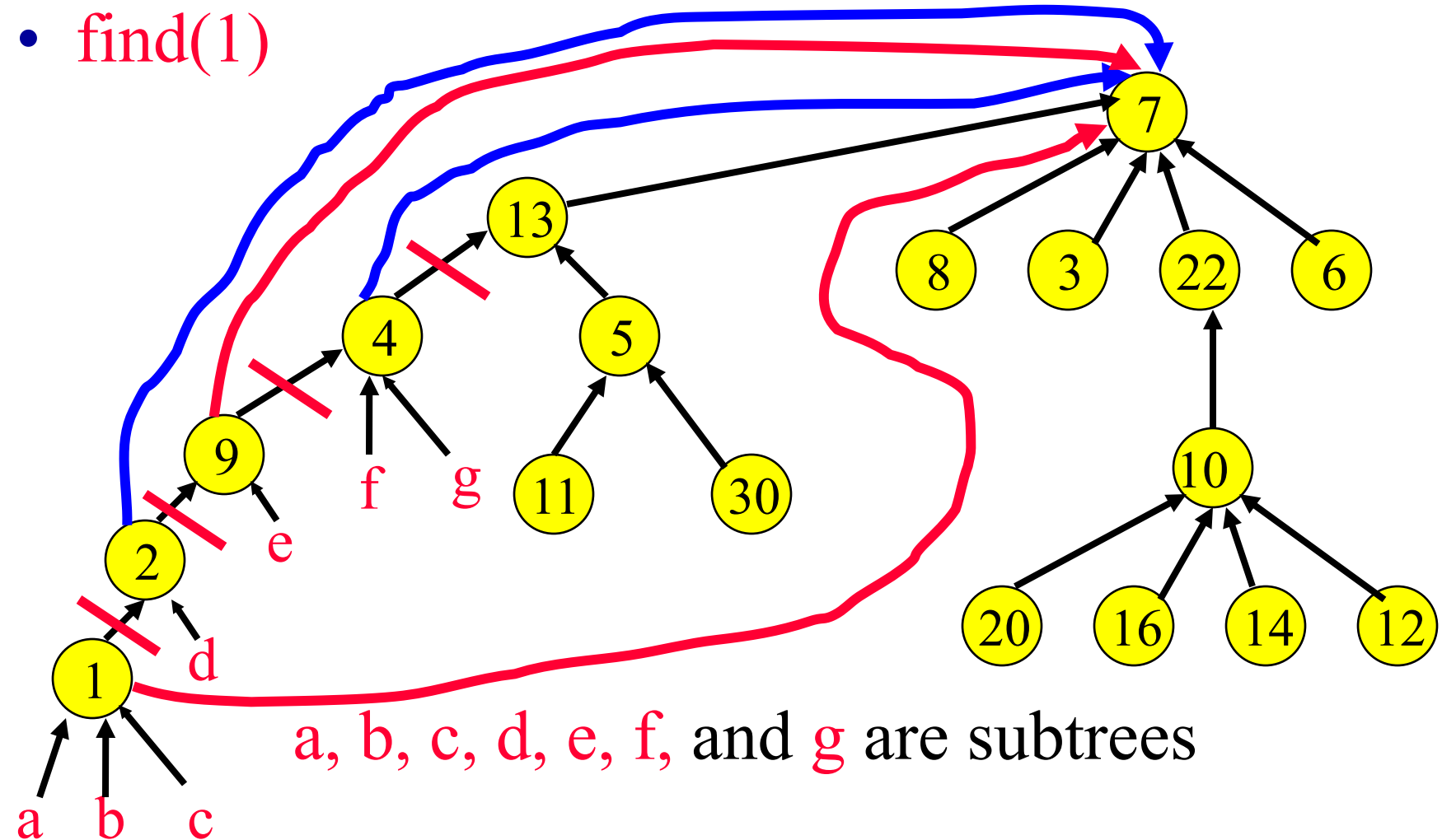
Definition [Collapsing rule] :

If j is a node on the path from i to its root and $\text{parent}[j] \neq \text{root}(i)$,

then set $\text{parent}[j]$ to $\text{root}(i)$.

Path Collapsing/Path Compaction

- Make all nodes on find path point to tree root.
- **find(1)**

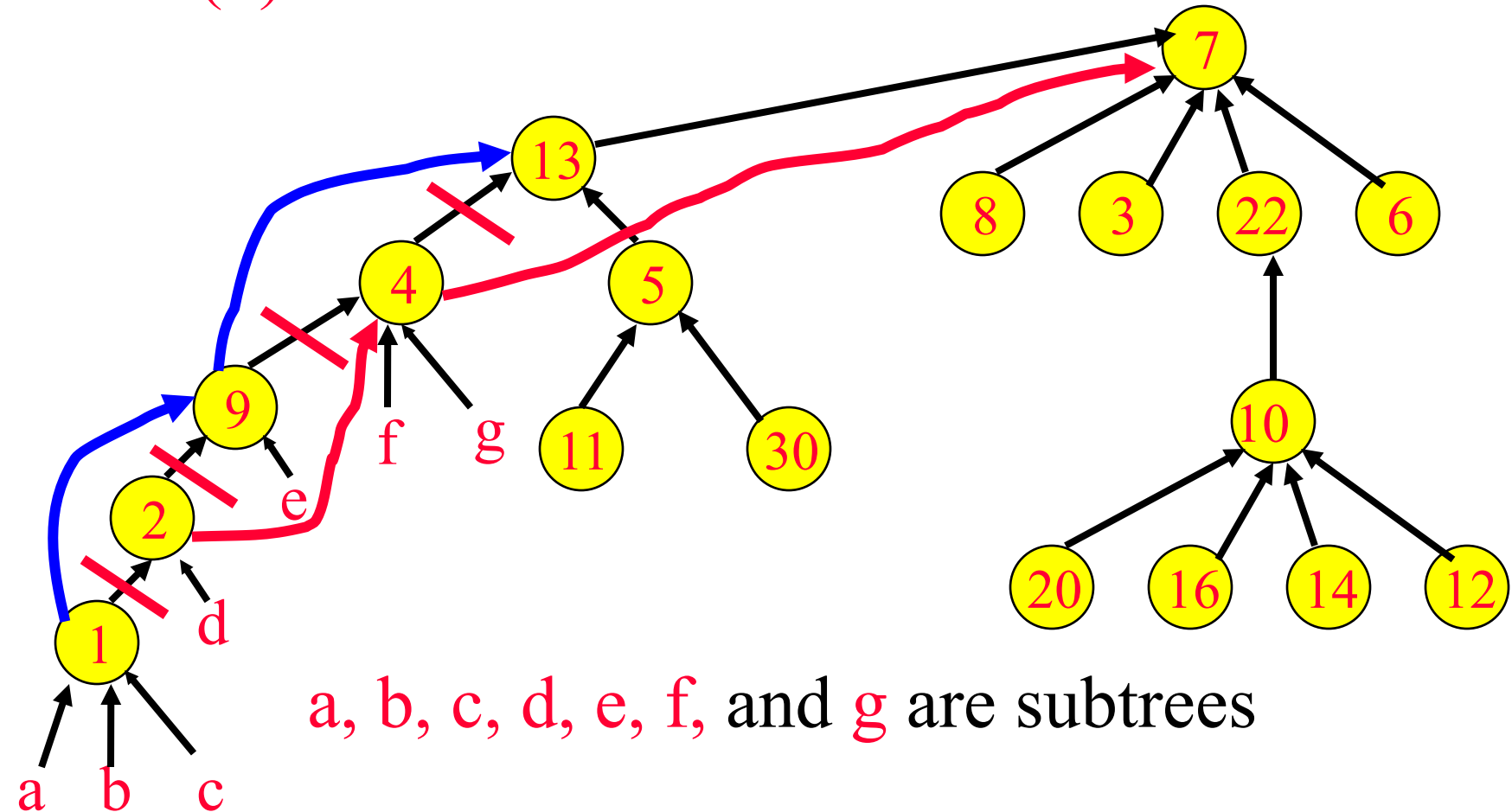


Makes two passes up the tree.

```
int Sets::CollapsingFind (int i )  
{ // Find the root of tree containing element i. Use the  
  // collapsing rule to collapse all nodes from i to the root.  
  // find the root  
  for (int r = i; parent[r] >= 0; r = parent[r]);  
  while ( i != r ) {  
    int s = parent[i];  
    parent[i] = r;  
    i = s;  
  }  
  return r;  
}
```


Path Splitting

- Nodes on find path point to former grandparent.
- **find(1)**

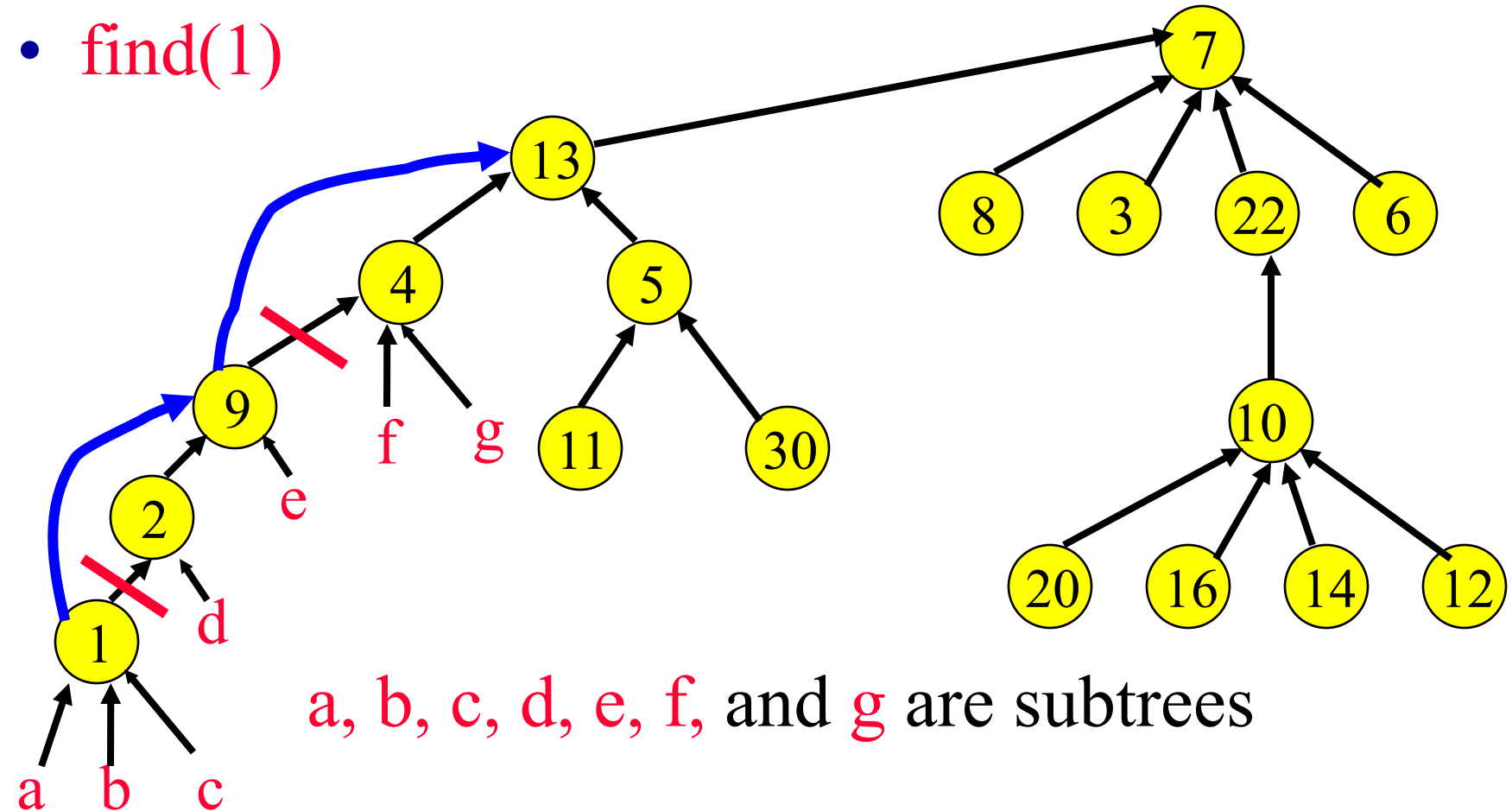


a, b, c, d, e, f, and g are subtrees

Makes only one pass up the tree.

Path Halving

- Parent pointer in every other node on find path is changed to former grandparent.
- **find(1)**



Changes half as many pointers.

Application to Equivalent Classes

equivalence classes $\Leftrightarrow$ disjoint sets

Initially, $\text{parent}[i] = -1, 0 \leq i \leq n-1$.

To process $i \equiv j$,

Let $x = \text{find}(i), y = \text{find}(j)$ --- 2 finds

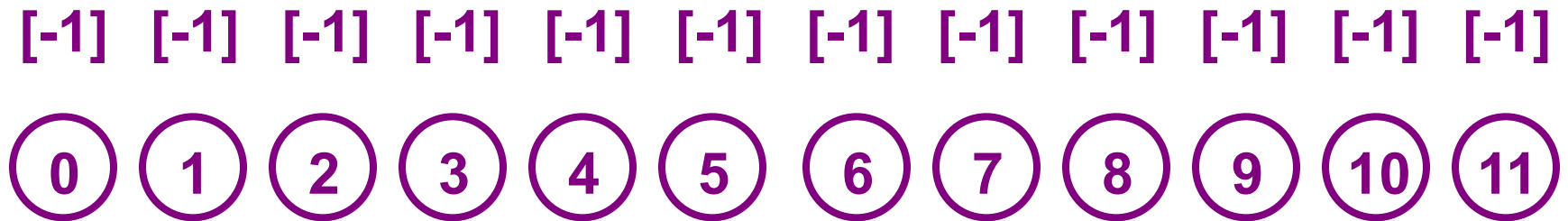
If $x \neq y$ then $\text{union}(x, y)$ --- at most 1 union

Thus if we have n elements and m equivalence pairs, we needs $2m$ finds and $\min \{n-1, m\}$ unions. The total time is $O(n+2m \alpha(n+2m, n))$.

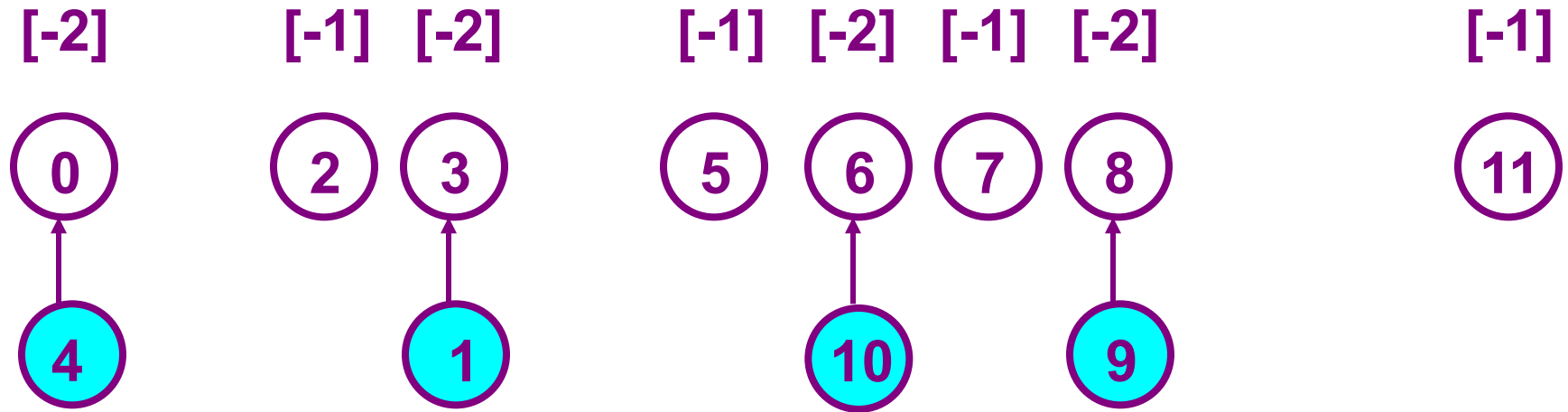
Example:

$n = 12$, process equivalence pairs:

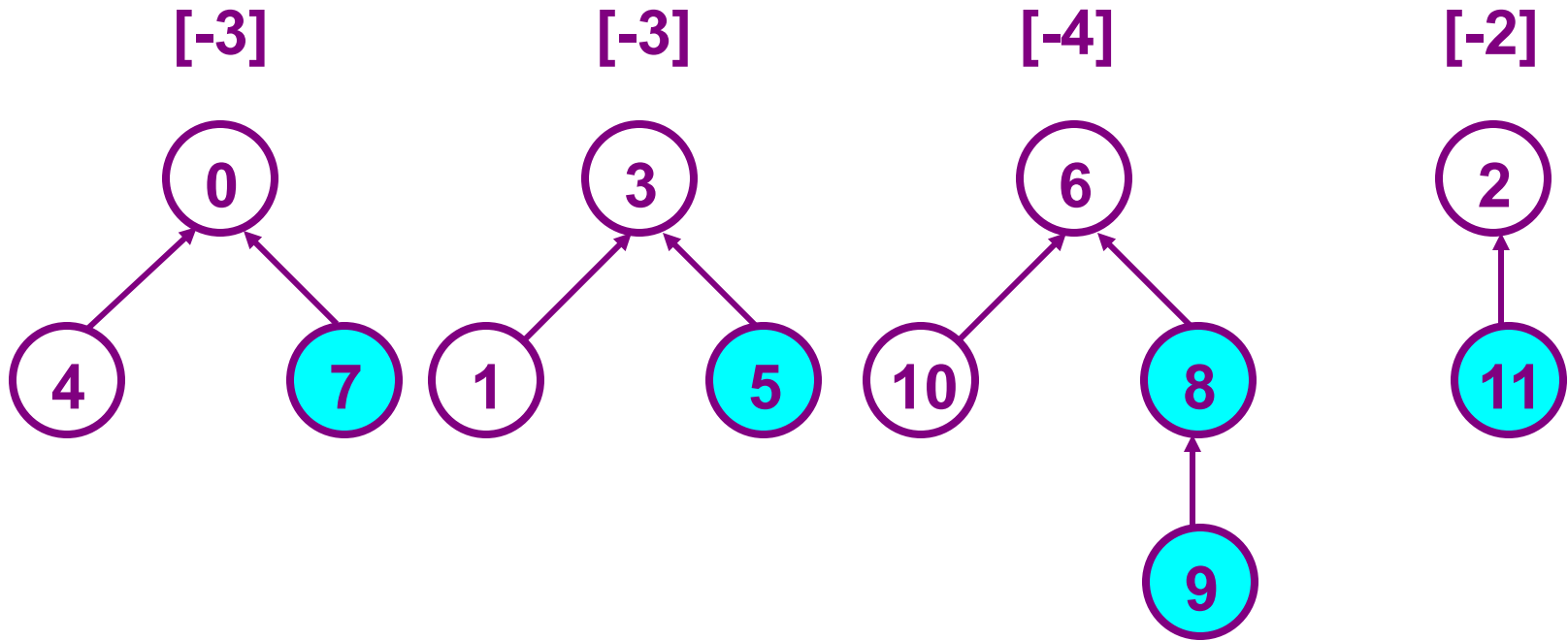
**$0 \equiv 4, 3 \equiv 1, 6 \equiv 10, 8 \equiv 9, 7 \equiv 4, 6 \equiv 8, 3 \equiv 5, 2 \equiv 11,$
 $11 \equiv 0$**



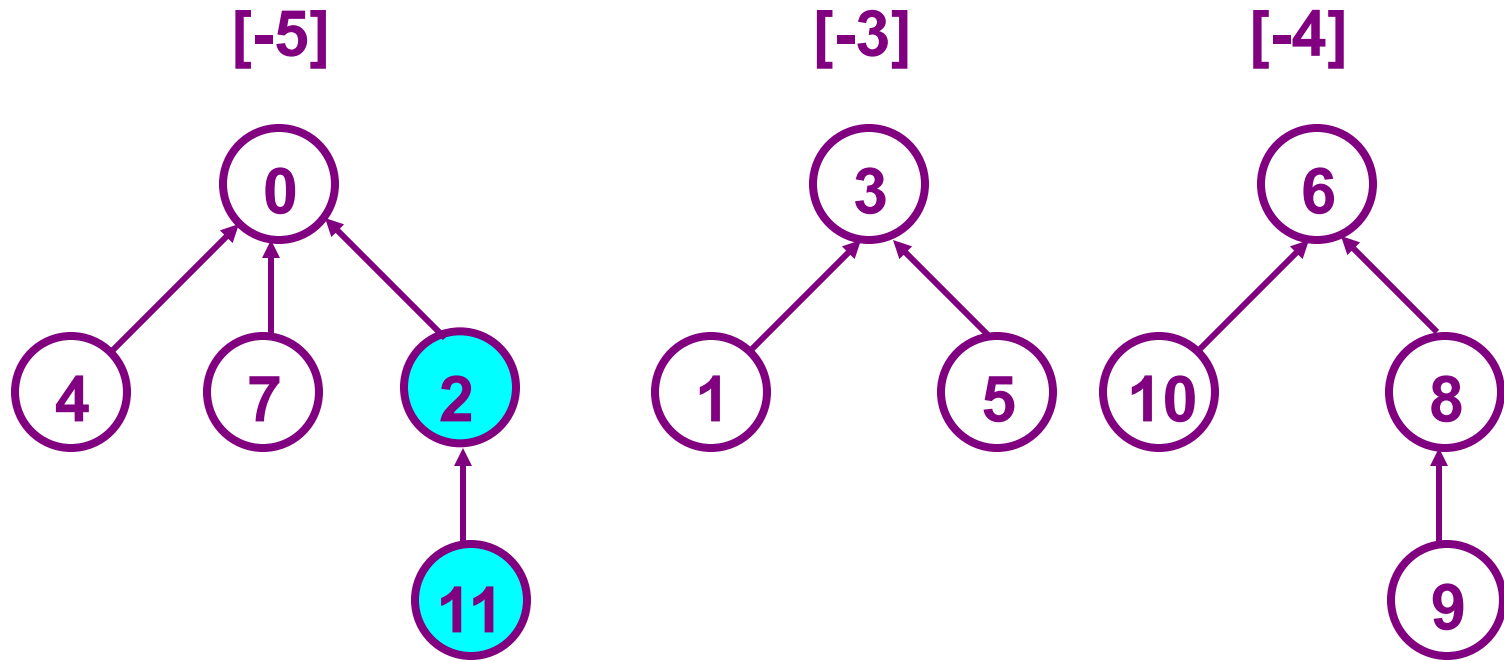
(a) Initial trees



(b) After processing $0 \equiv 4$, $3 \equiv 1$, $6 \equiv 10$, and $8 \equiv 9$



(c) After processing $7 \equiv 4$, $6 \equiv 8$, $3 \equiv 5$, and $2 \equiv 11$



(d) After processing $11 \equiv 0$

Thinking

- Application
 - Coding: msg $\rightarrow$ 101011100001
- What we have
 - Dictionary of Message: n words
 - Every word w_i has an average frequency f_i
- Requirement
 - For an message, minimize its code length

Problem formulation

- n elements
- Each element has a length l_i
- Each element has a frequency f_i
- Binary coding
 - unfixed length
- Wanted:
 - Larger $f_i \rightarrow$ smaller l_i
 - $\text{Min}(\Sigma(l_i * f_i))$

Huffman Tree

- Binary tree with n leaves
- Frequency: leaf value
- Coding length
 - Distance from the root
- Wanted:
 - Larger $f_i \rightarrow$ deeper level
 - $\text{Min}(\sum(l_i * f_i))$
- Word coding.....

Huffman Tree

- ADT
- Algorithm
 - Tree Construction
 - Coding
 - decoding

Building a Tree

Scan the original text

Eerie eyes seen near lake.

- What characters are present?

E e r i space
y s n a r l k .



Building a Tree

Scan the original text

Eerie eyes seen near lake.

- What is the frequency of each character in the text?

Char Freq. Char Freq. Char Freq.

E	1	y	1	k	1
e	8	s	2	.	1
r	2	n	2		
i	1	a	2		
space	4	l	1		

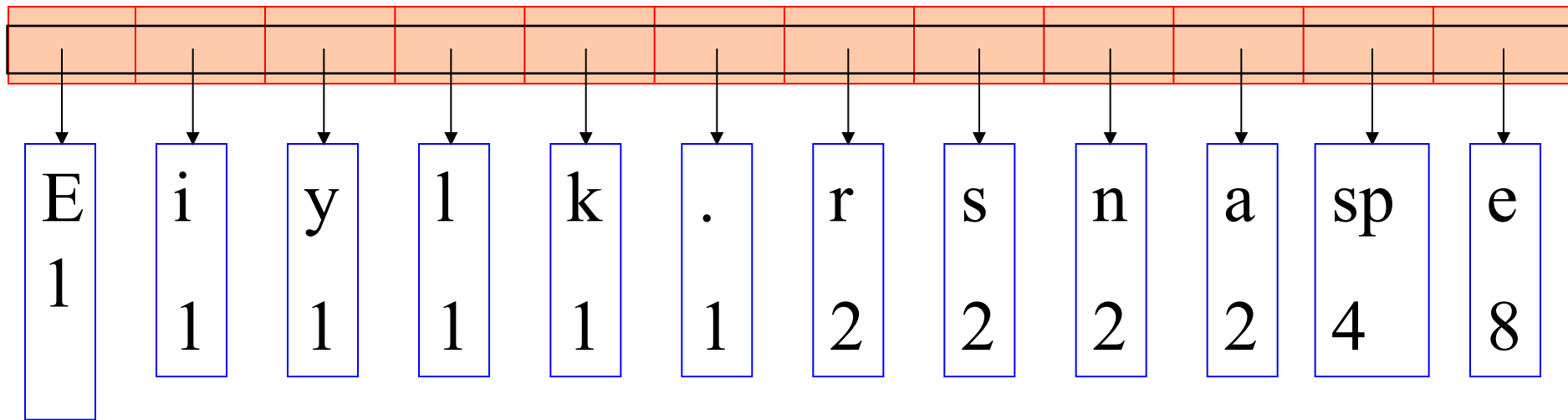
Building a Tree

Prioritize characters

- Create binary tree nodes with character and frequency of each character
- Place nodes in a priority queue
 - The lower the occurrence, the higher the priority in the queue

Building a Tree

- The queue after inserting all nodes

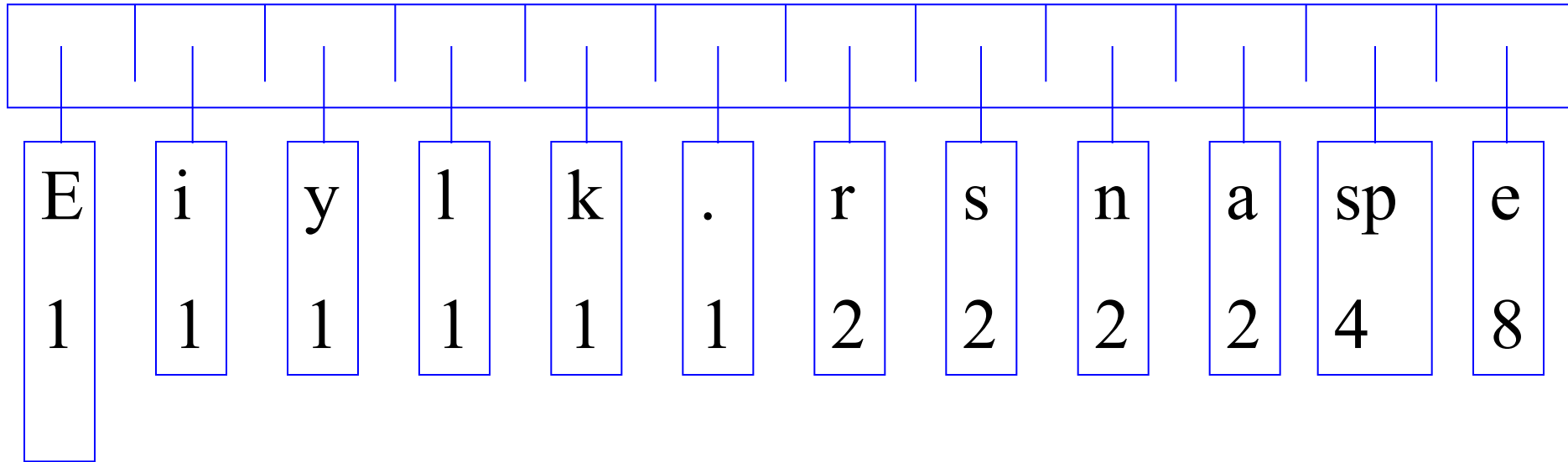


- Null Pointers are not shown

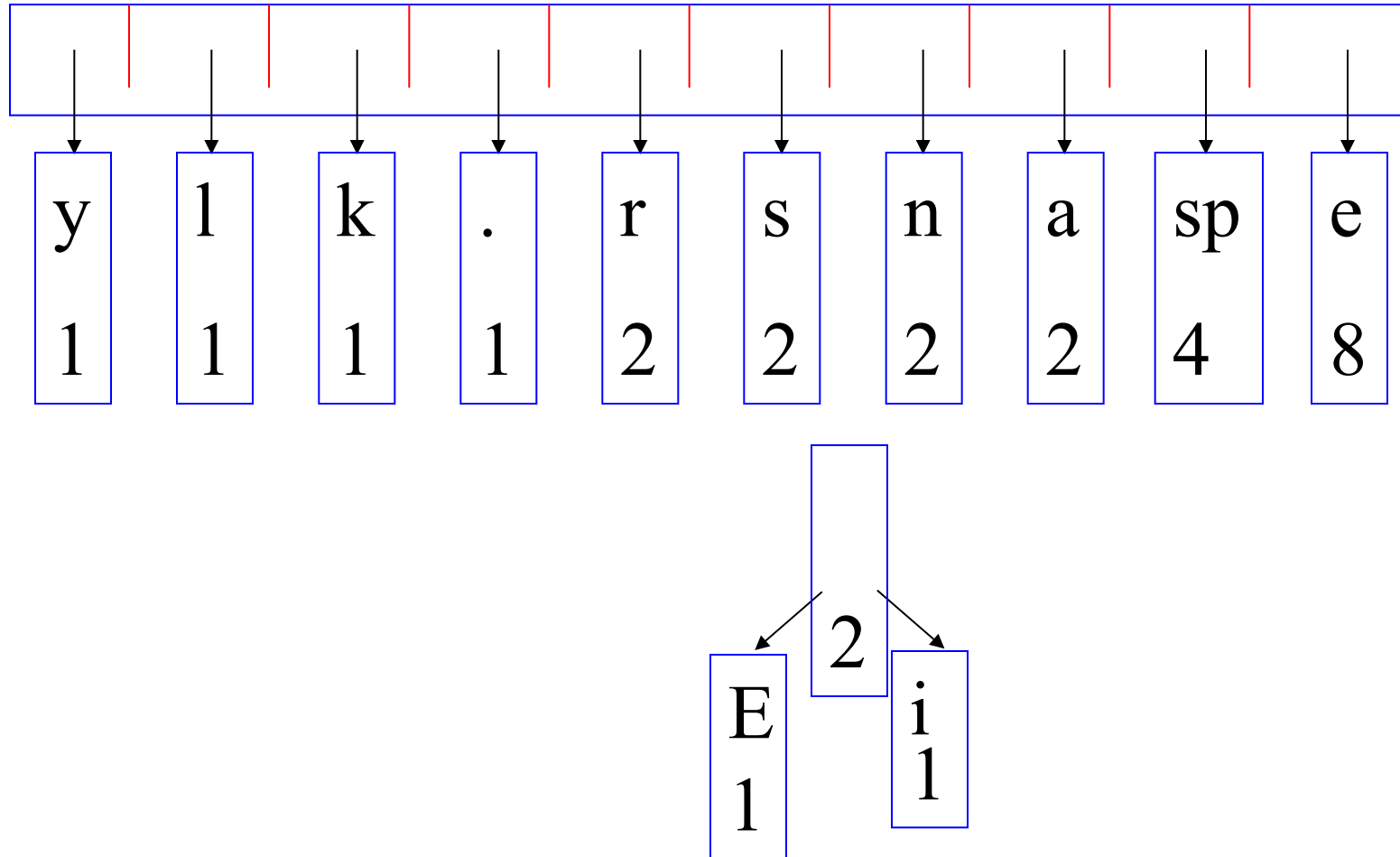
Building a Tree

- While priority queue contains two or more nodes
 - Create new node
 - Dequeue node and make it left subtree
 - Dequeue next node and make it right subtree
 - Frequency of new node equals sum of frequency of left and right children
 - Enqueue new node back into queue

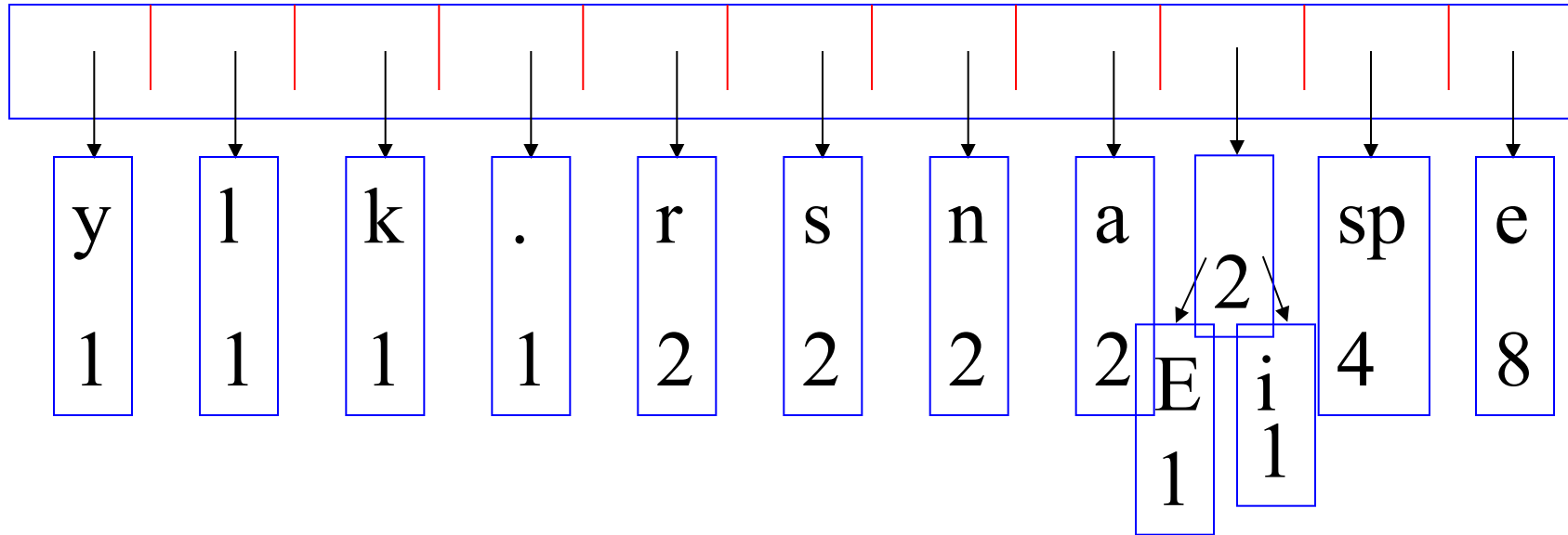
Building a Tree



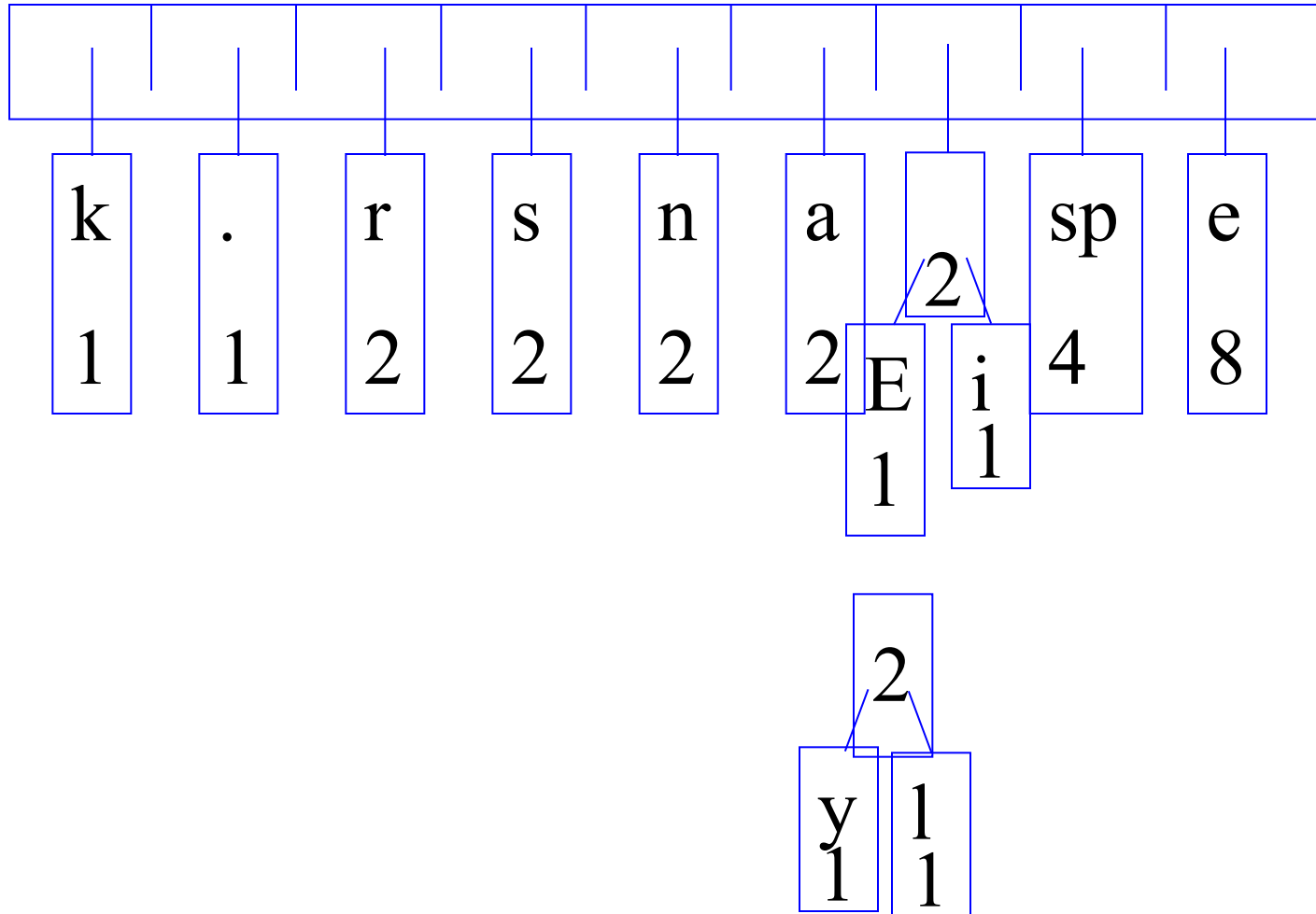
Building a Tree



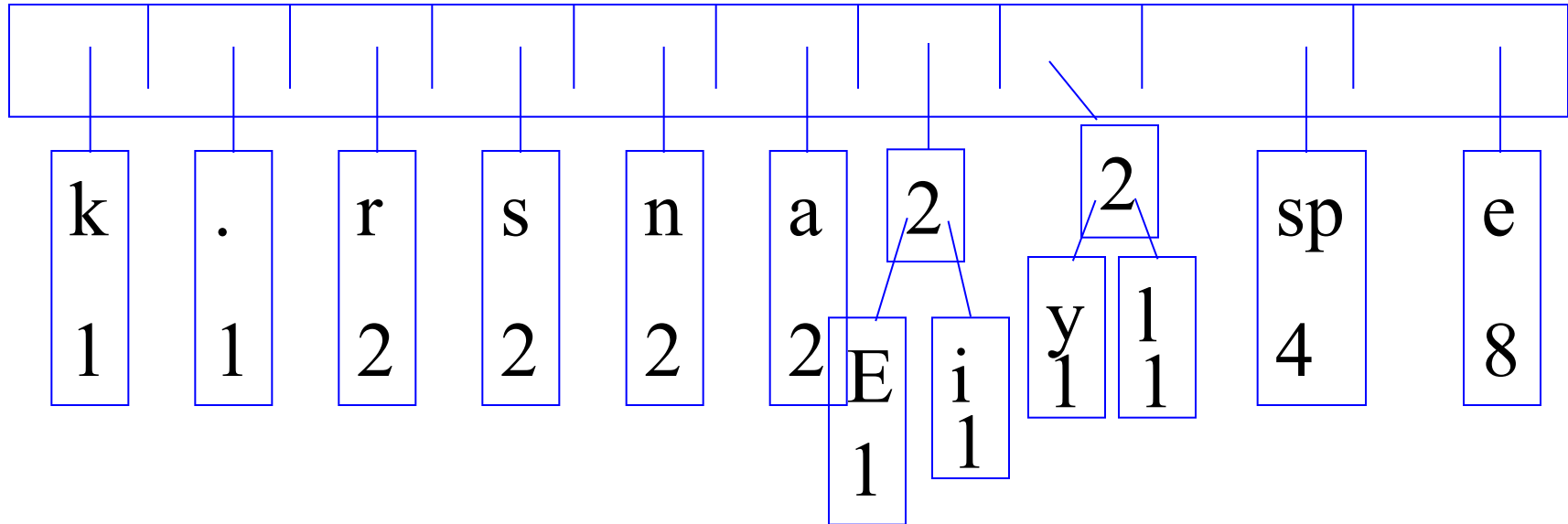
Building a Tree



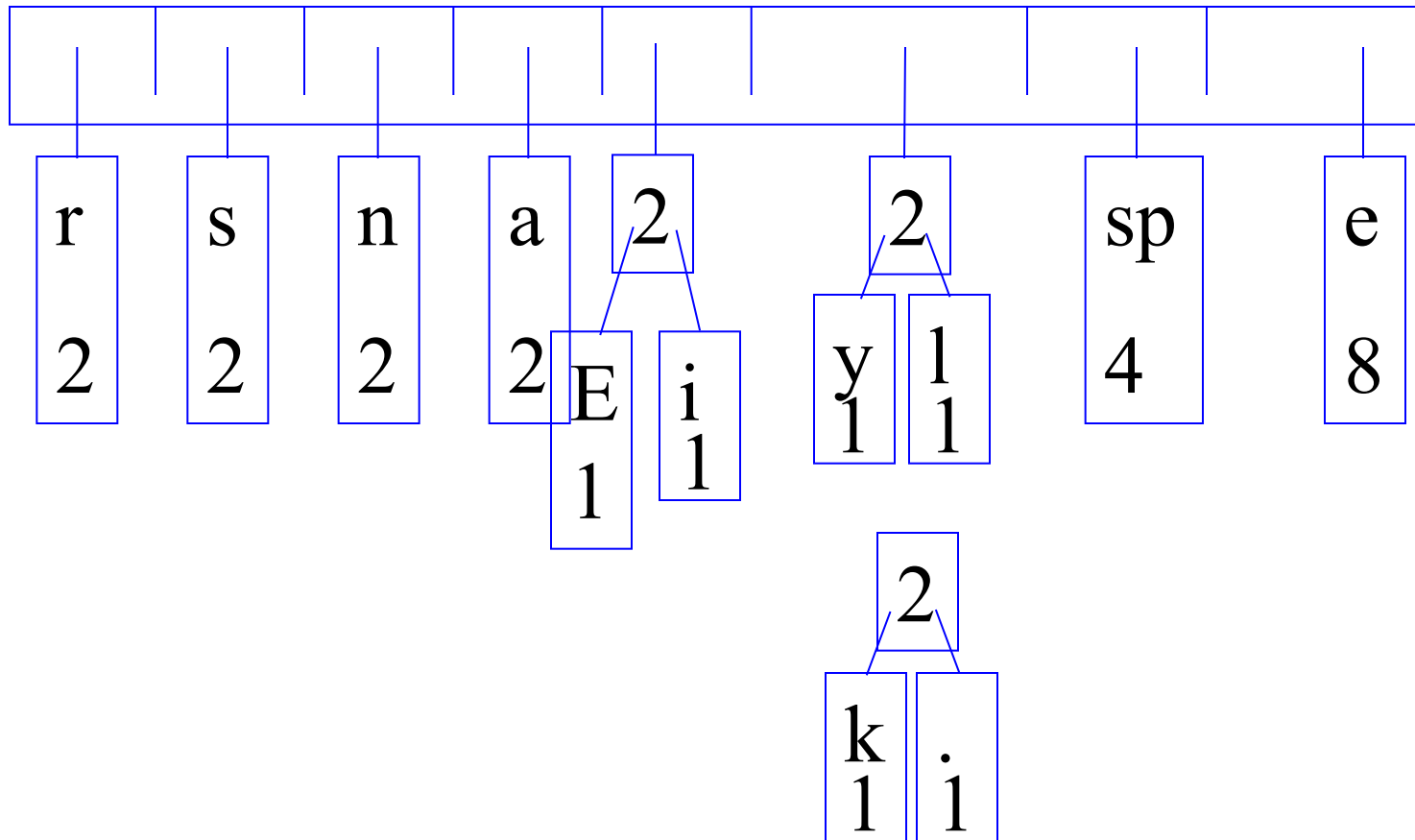
Building a Tree



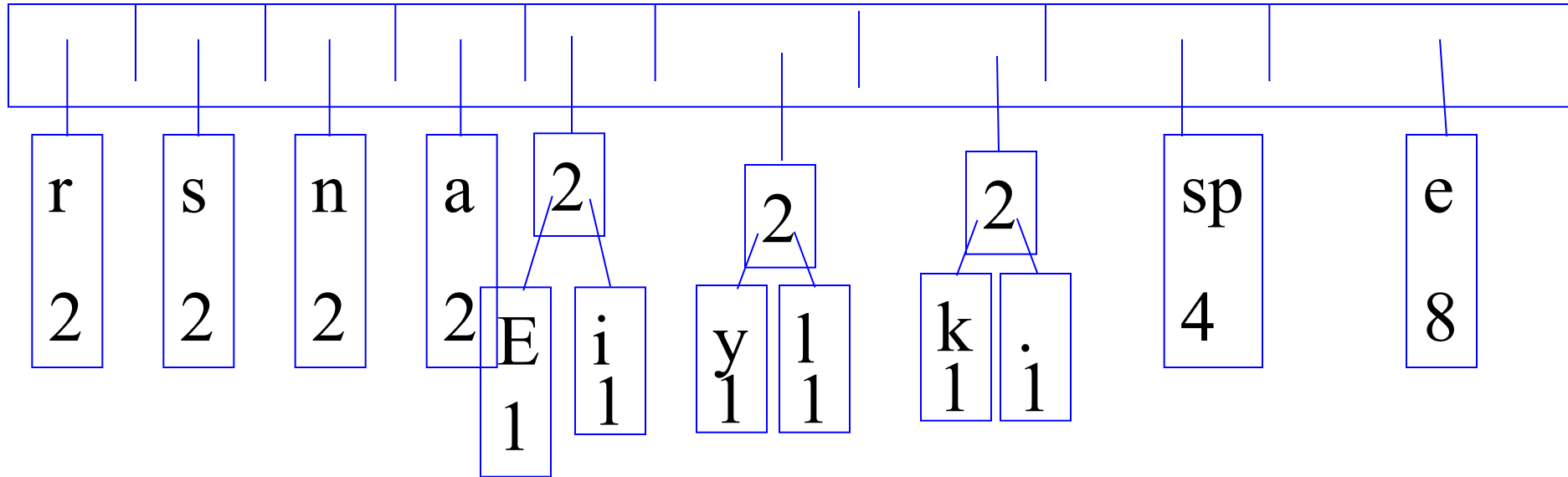
Building a Tree



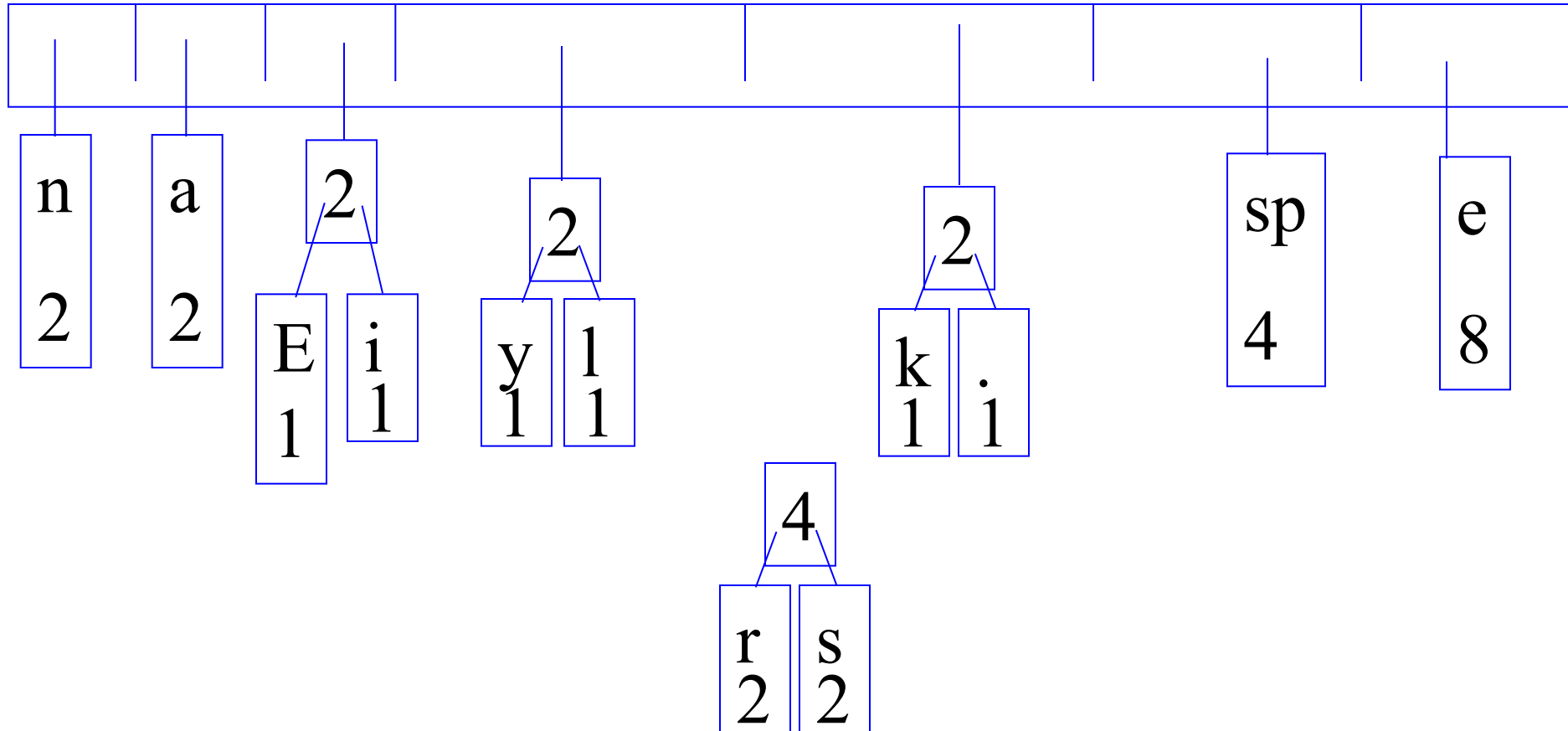
Building a Tree



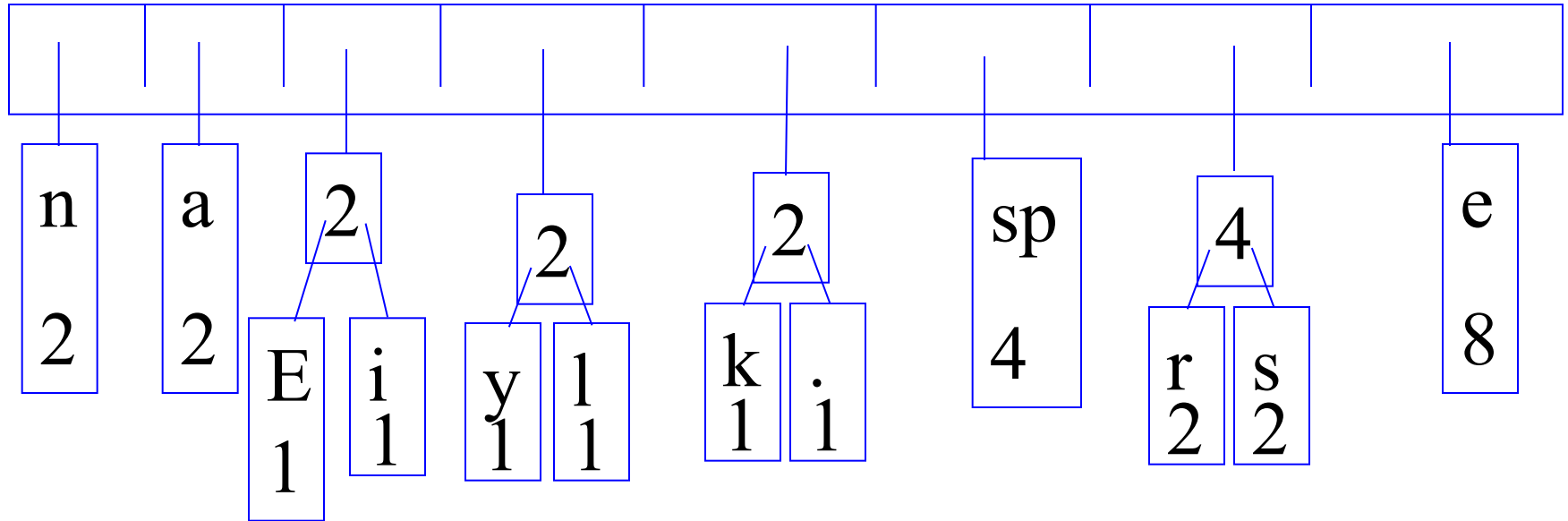
Building a Tree



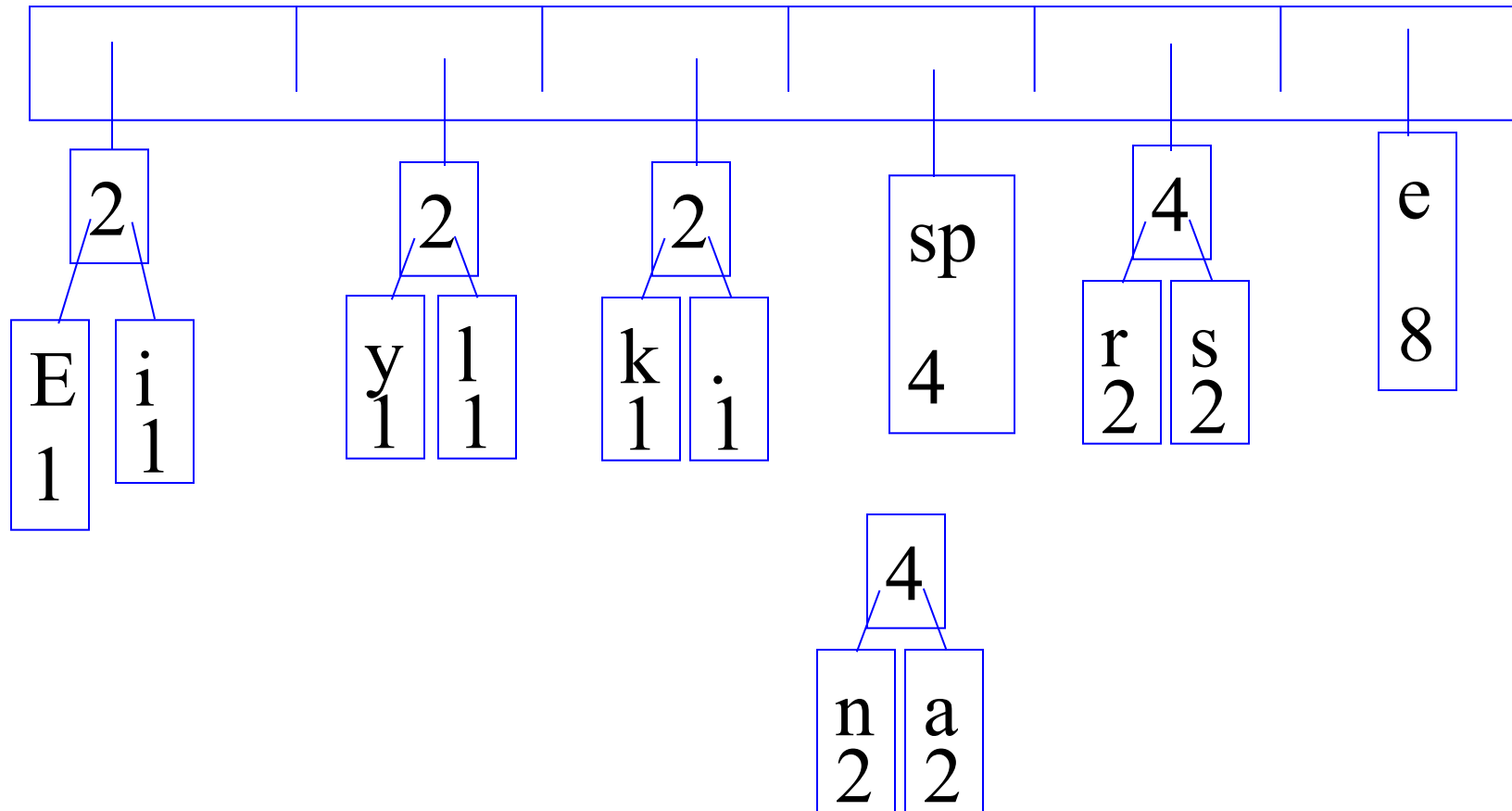
Building a Tree



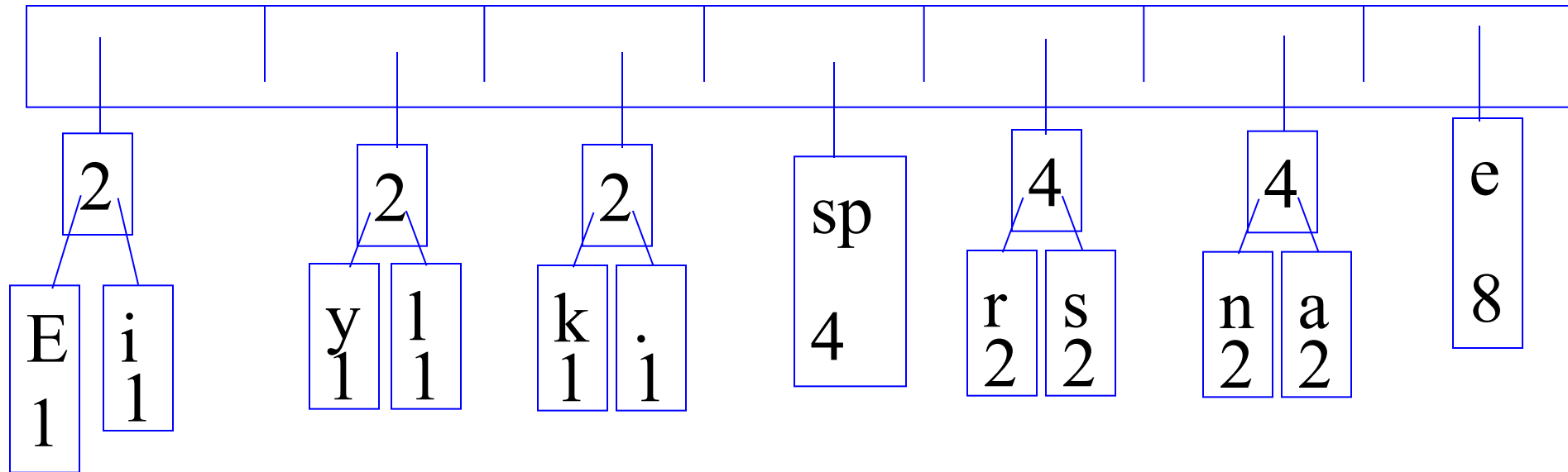
Building a Tree



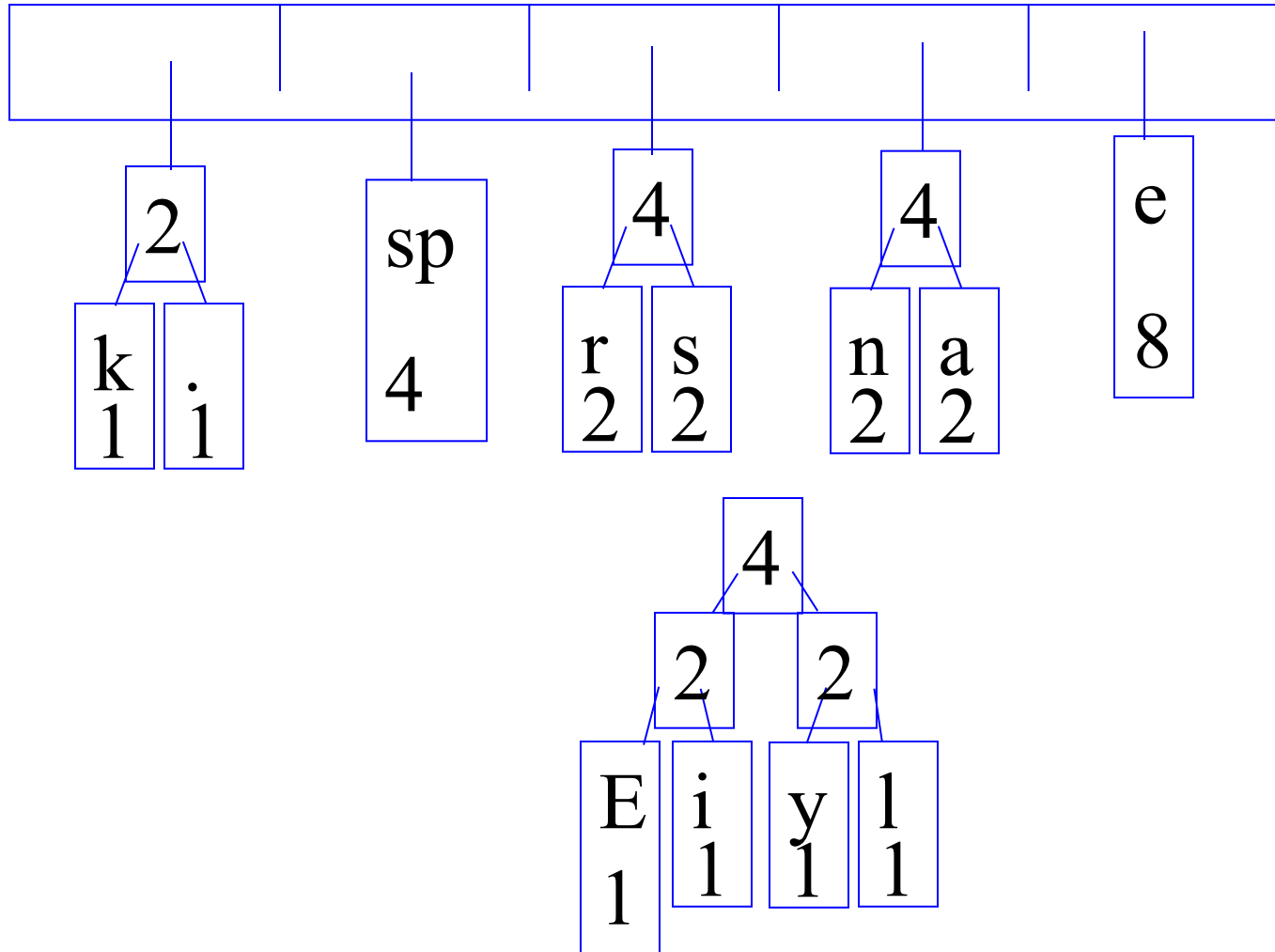
Building a Tree



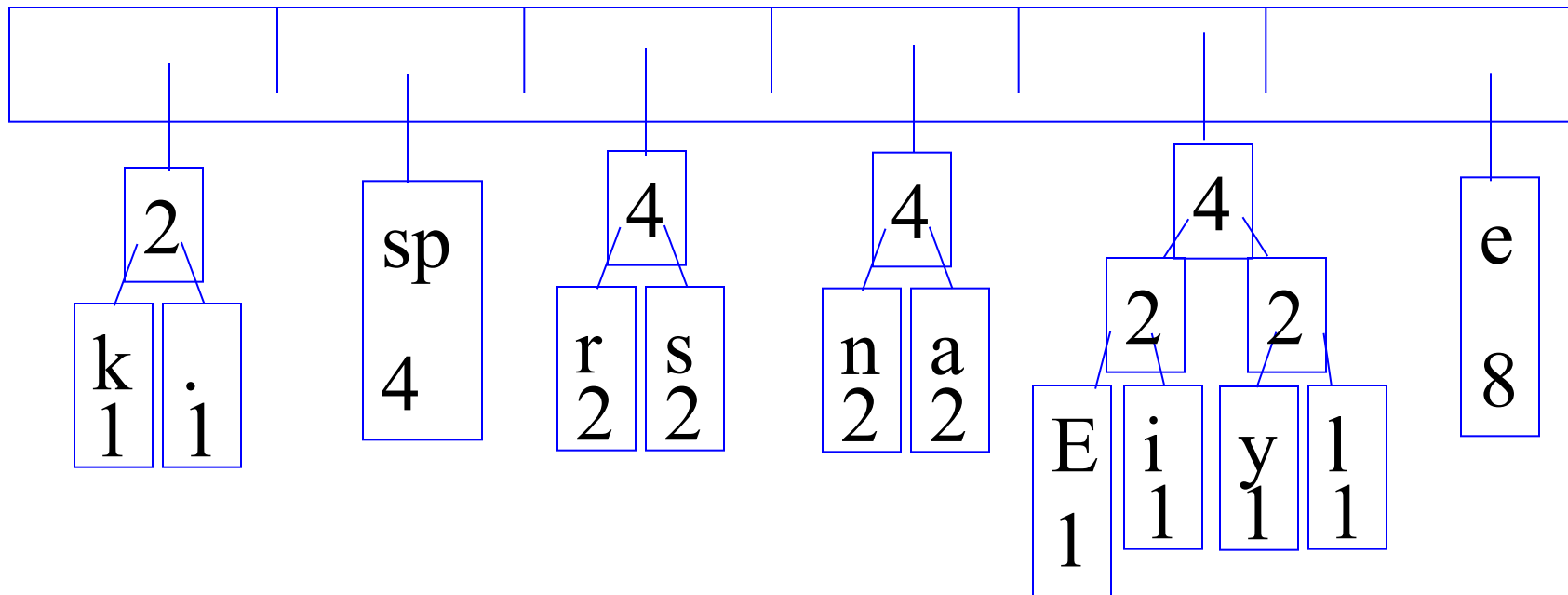
Building a Tree



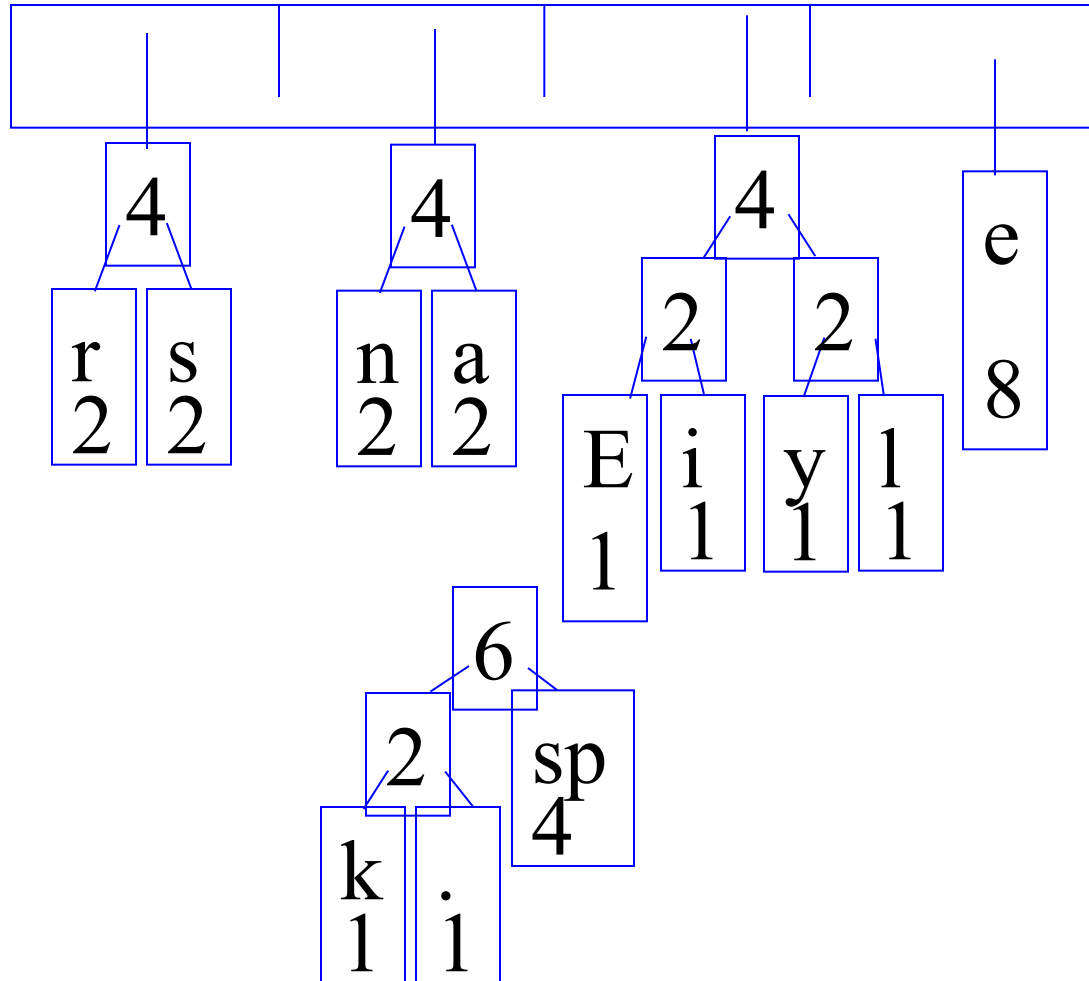
Building a Tree



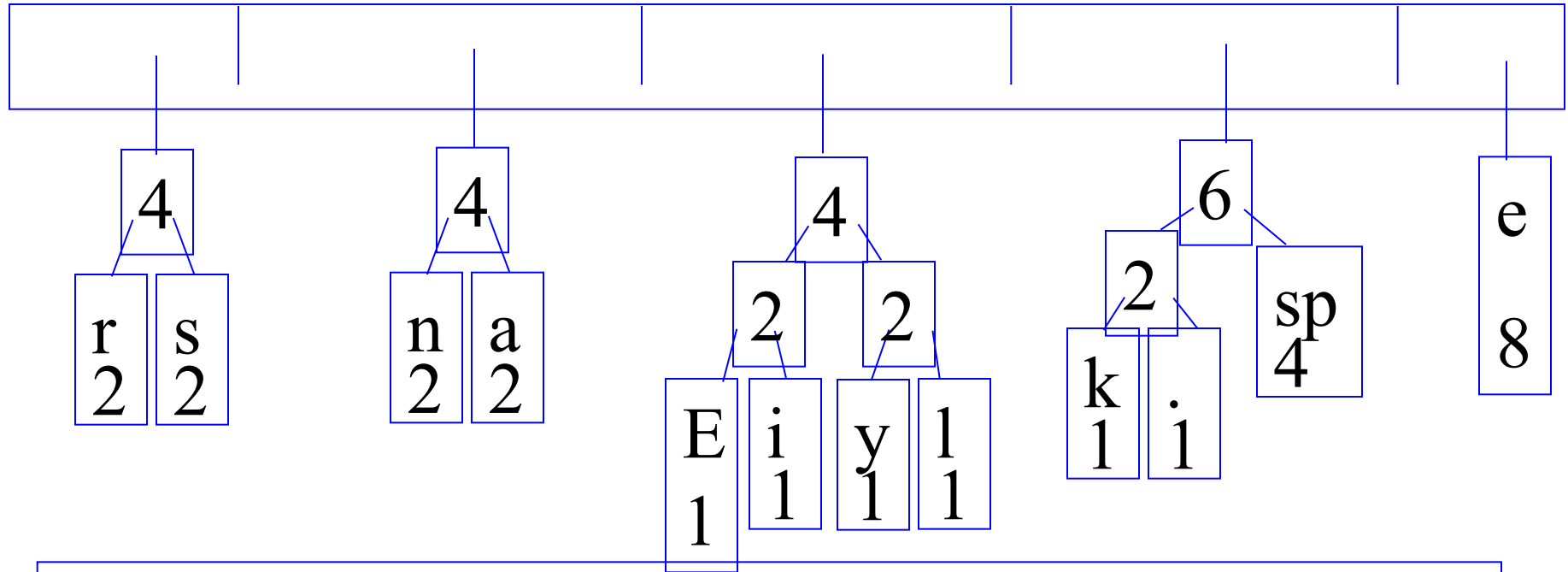
Building a Tree



Building a Tree

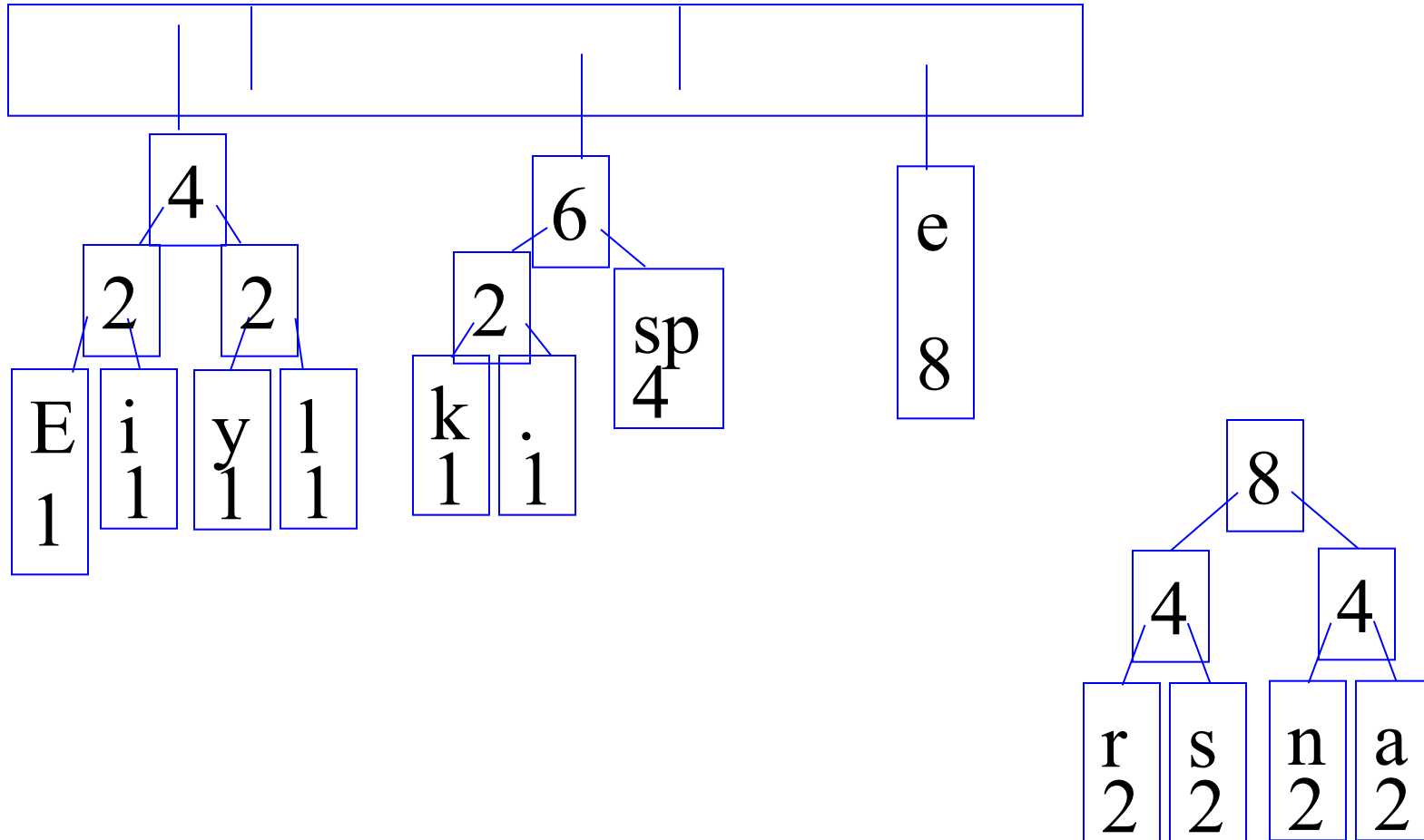


Building a Tree

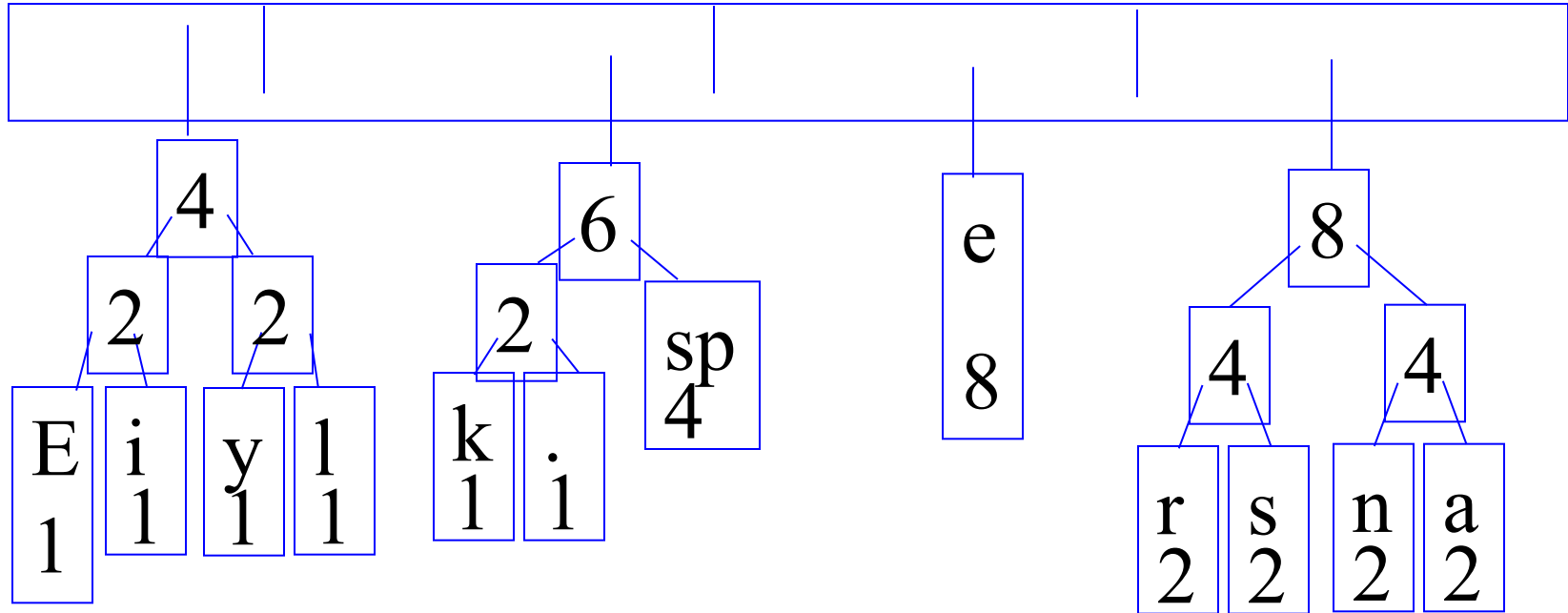


What is happening to the characters with a low number of occurrences?

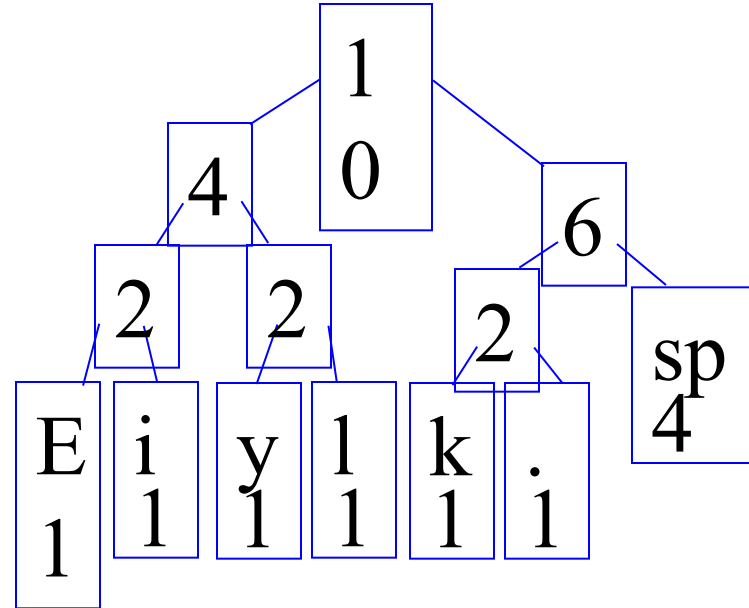
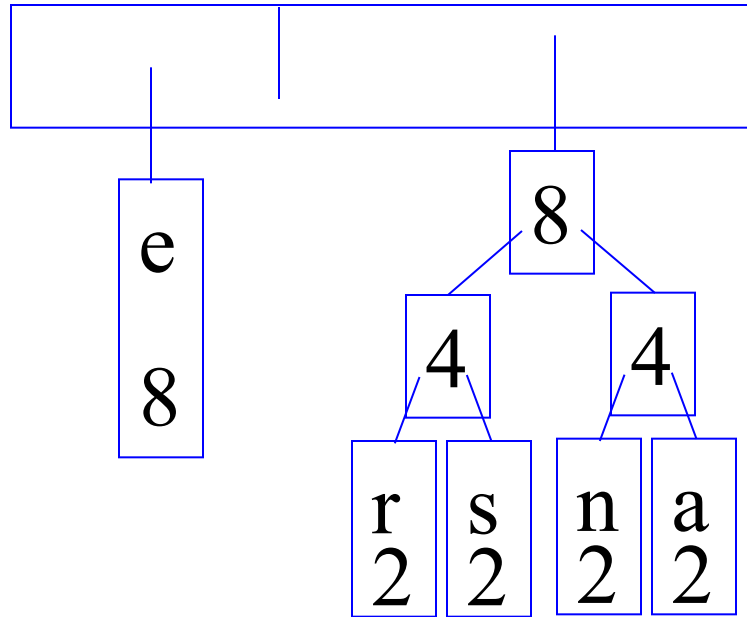
Building a Tree



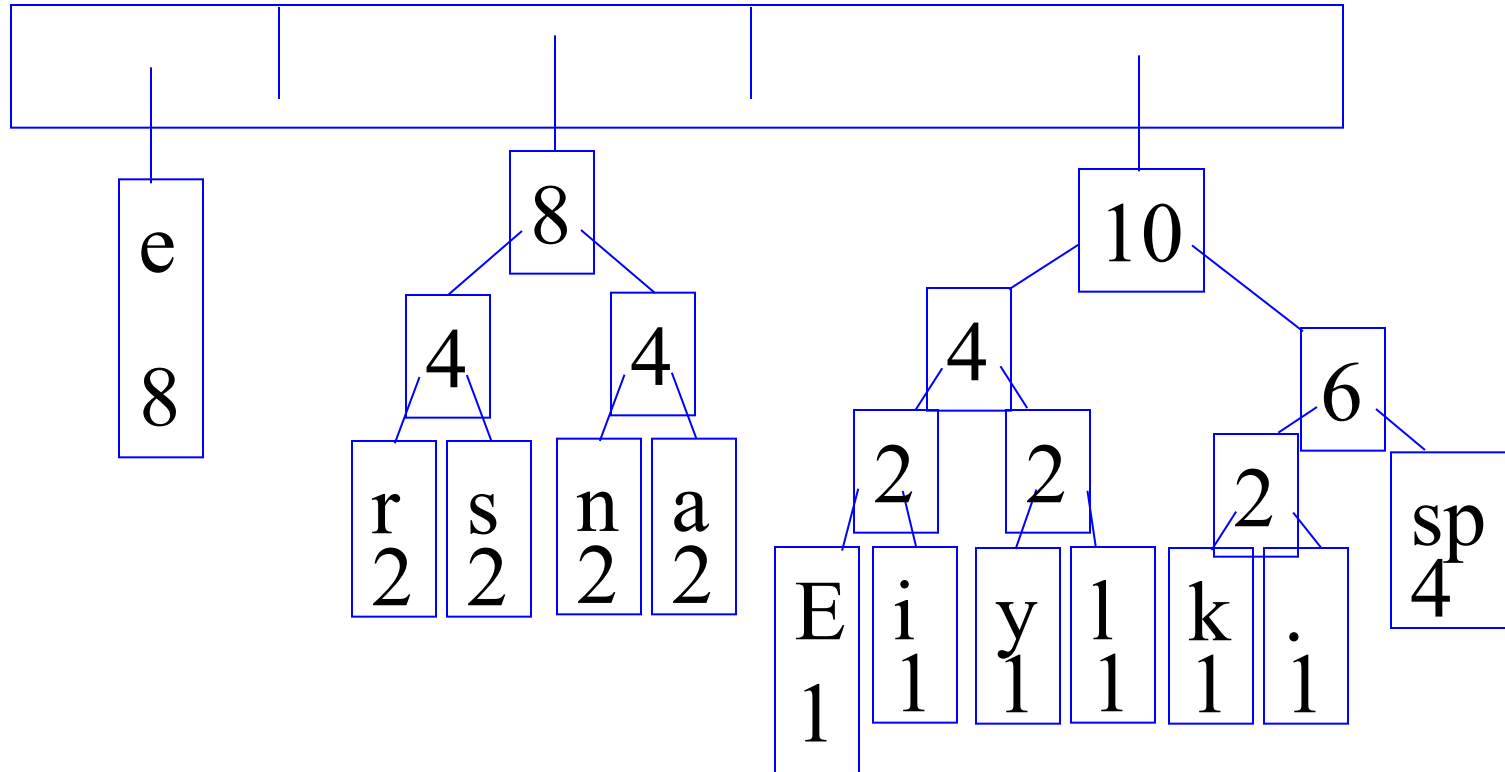
Building a Tree



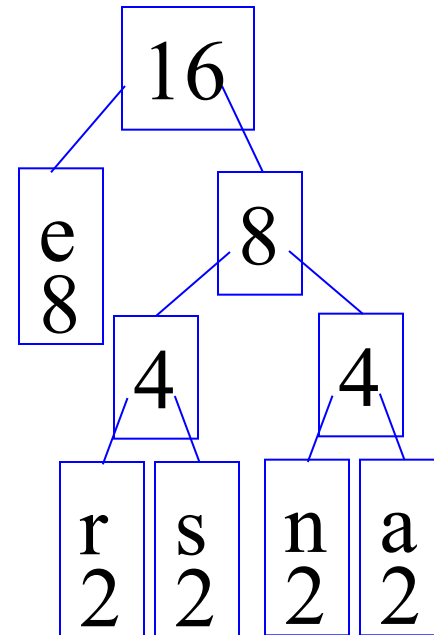
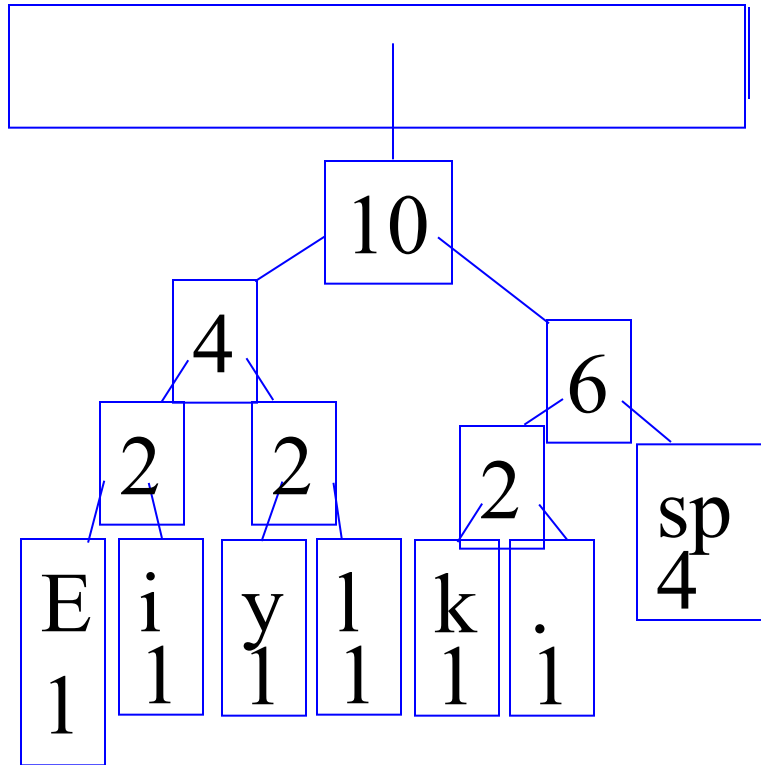
Building a Tree



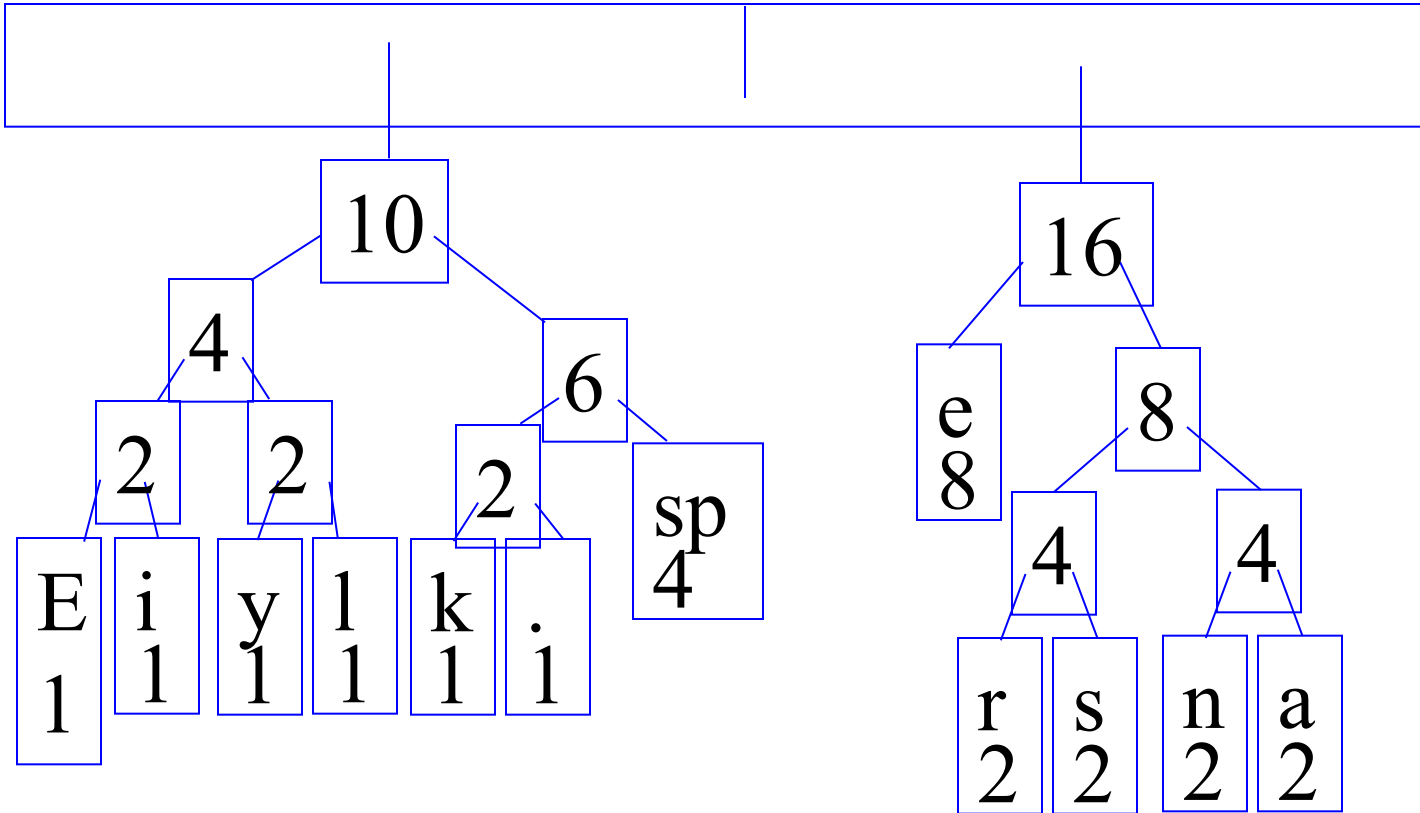
Building a Tree



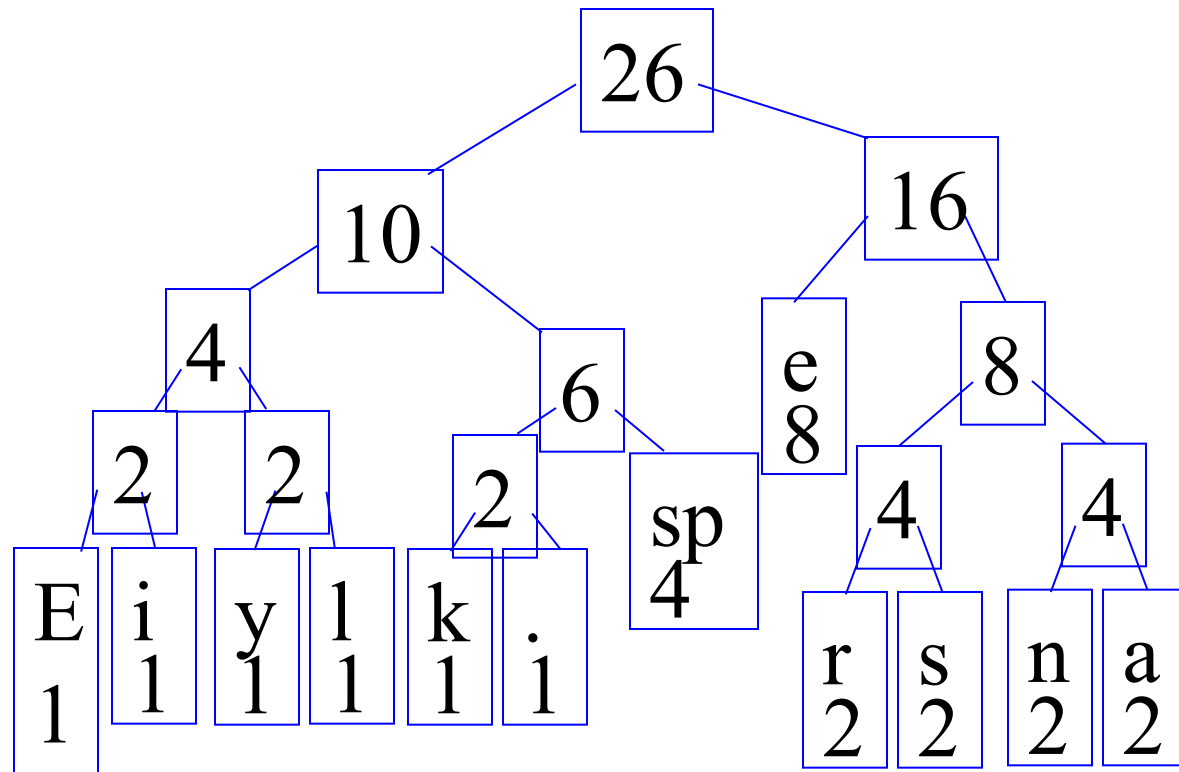
Building a Tree



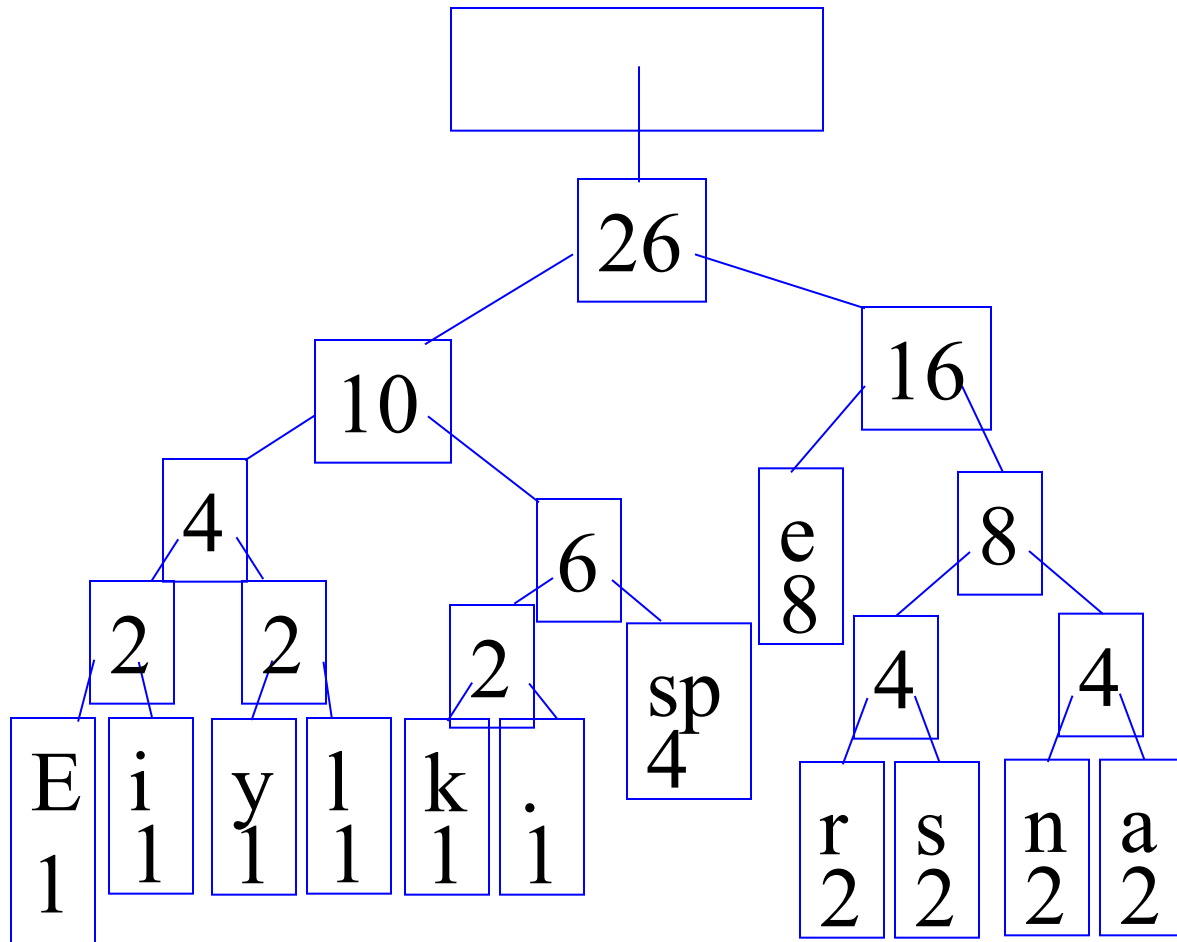
Building a Tree



Building a Tree



Building a Tree



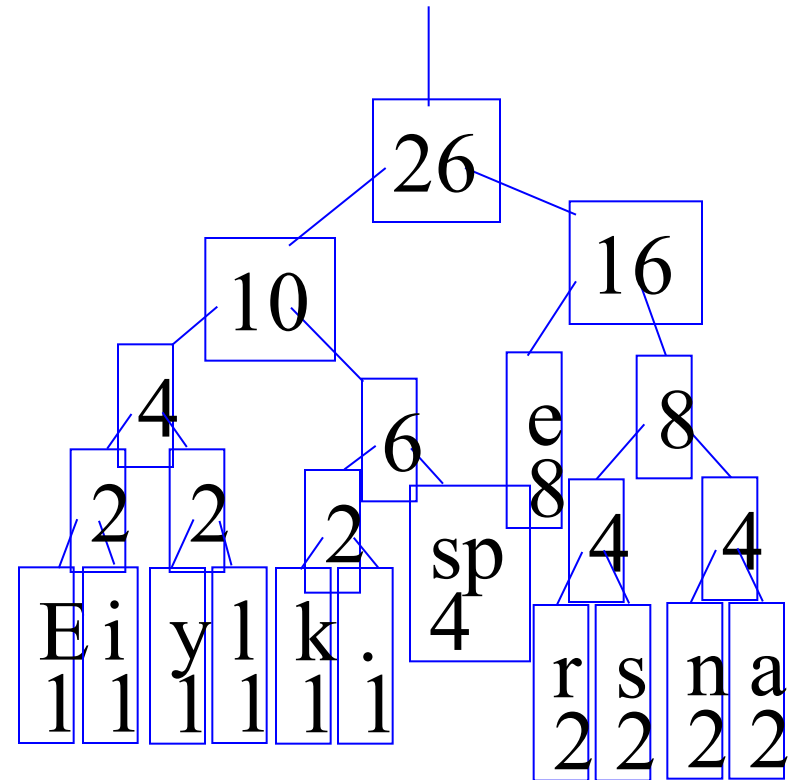
After enqueueing this node there is only one node left in priority queue.

Building a Tree

Dequeue the single node left in the queue.

This tree contains the new code words for each character.

Frequency of root node should equal number of characters in text.



Eerie eyes seen near lake. ✓ 26 characters

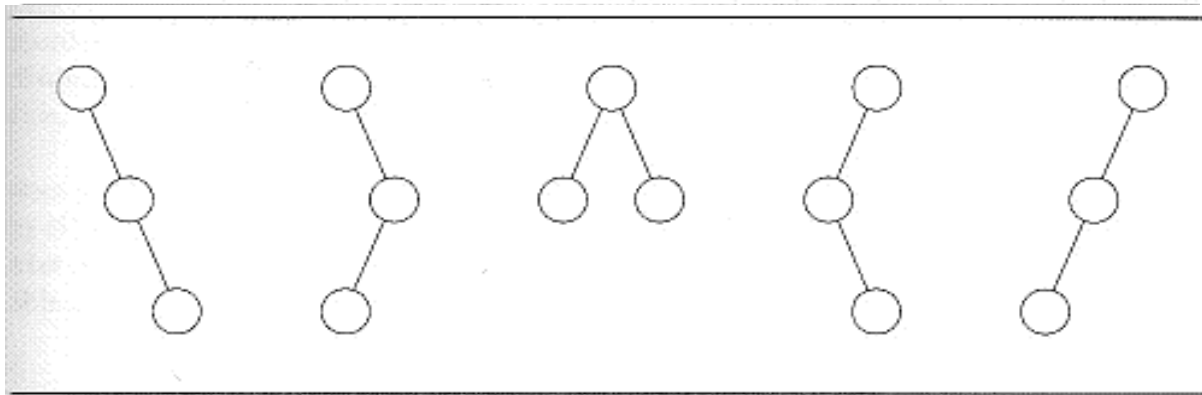
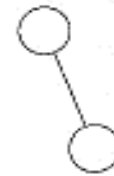
- Write Path Splitting /Path Halving algorithms.
- **Huffman Tree: Coding, Decoding**
- **Exercises: P316-3**

Counting Binary trees(1/10)

- Distinct Binary Trees :
 - If $n=0$ or $n=1$, there is only one binary tree.
 - If $n=2$ and $n=3$,



and

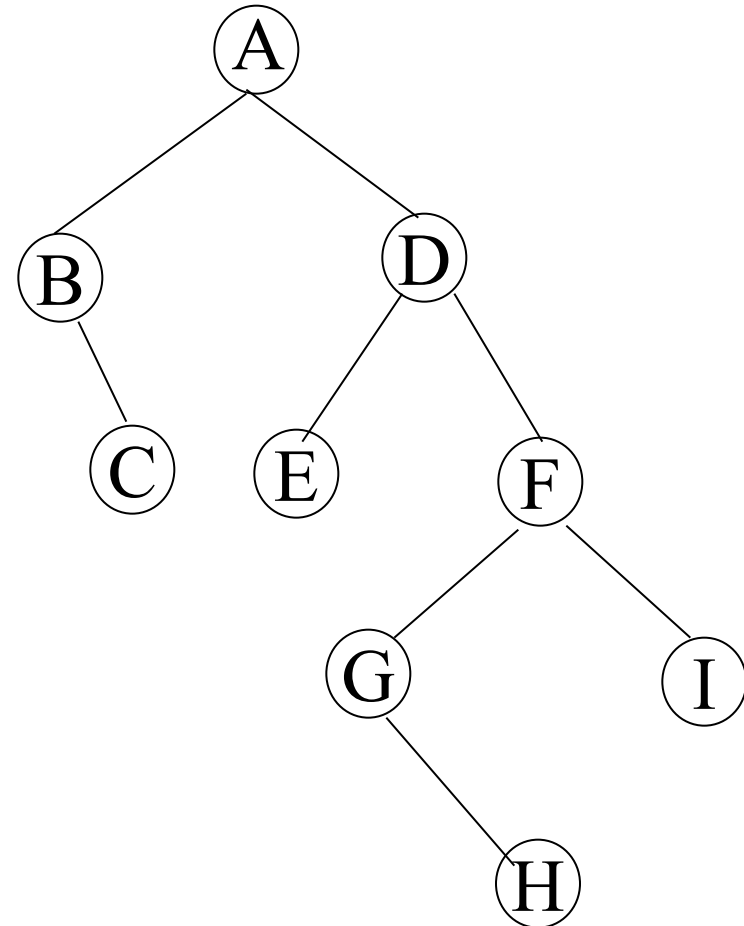
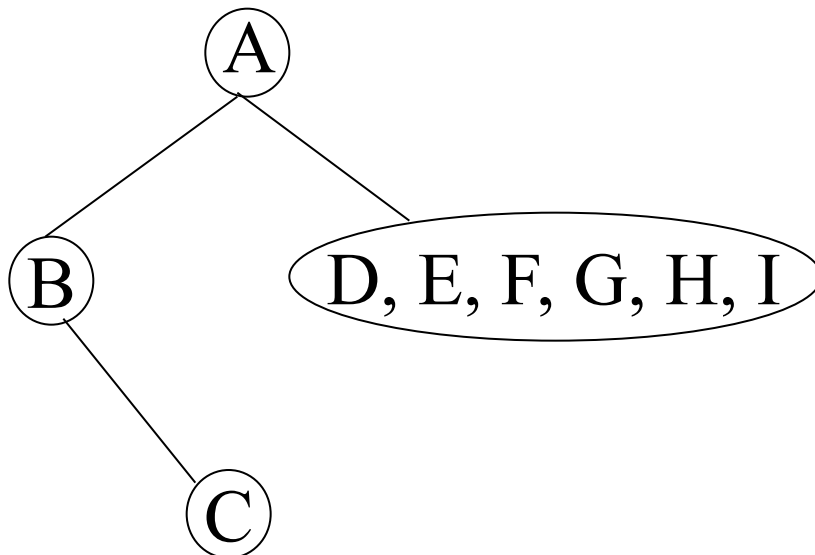
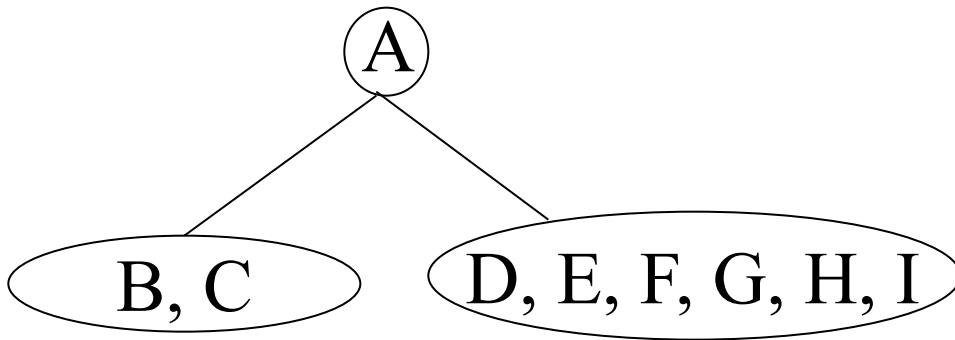


Counting Binary trees(2/10)

- Stack Permutations

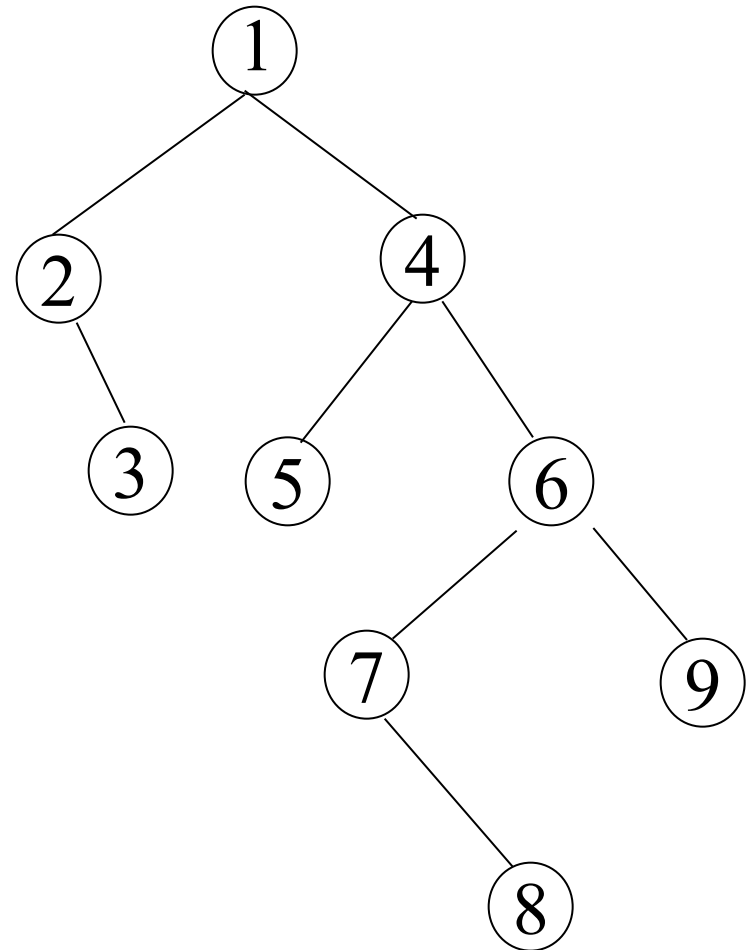
preorder: A B C D E F G H I

inorder: B C A E D G H F I



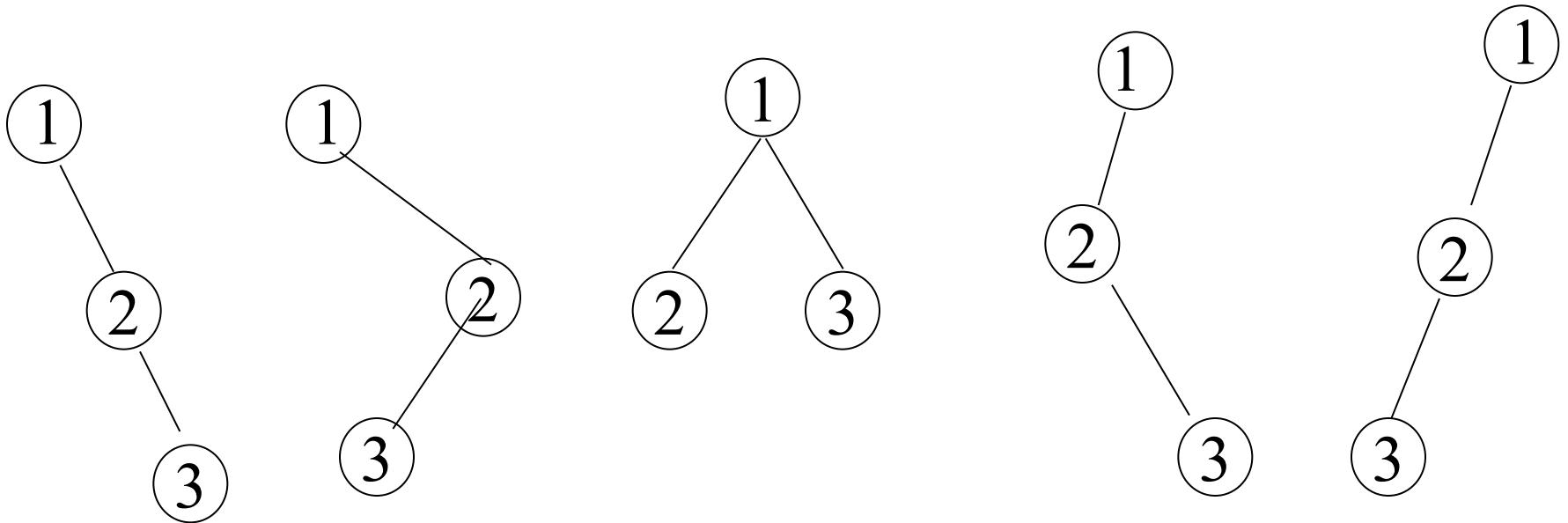
Counting Binary trees(3/10)

- Preorder permutation is 1,2...,9, and its inorder permutation is 2,3,1,5,4,7,8,6,9.



Counting Binary trees(4/10)

- If we start with the numbers 1,2,3, then the possible permutations obtainable by a stack are:
(1,2,3) (1,3,2) (2,1,3) (2,3,1) (3,2,1)
- Obtaining(3,1,2) is impossible.



Counting Binary trees(5/10)

- Matrix Multiplication

- Suppose that we wish to compute the product of n matrices:

$$M_1 * M_2 \dots * M_n$$

- Since matrix multiplication is associative, we can perform these multiplications in any order. We would like to know how many different ways we can perform these multiplications . For example, If $n = 3$, there are two possibilities:

$$(M_1 * M_2) * M_3$$

$$M_1 * (M_2 * M_3)$$

Counting Binary trees(6/10)

- Let b_n be the number of different ways to compute the product of n matrices. Then $b_2 = 1$, $b_3 = 2$ and $b_4 = 5$.
- Let $M_{ij}, i \leq j$ be the product $M_i * M_{i+1} * \dots * M_j$.

The product we wish to compute is M_{1n} by computing any one of the products $M_{1i} * M_{i+1,n}, 1 \leq i \leq n$.

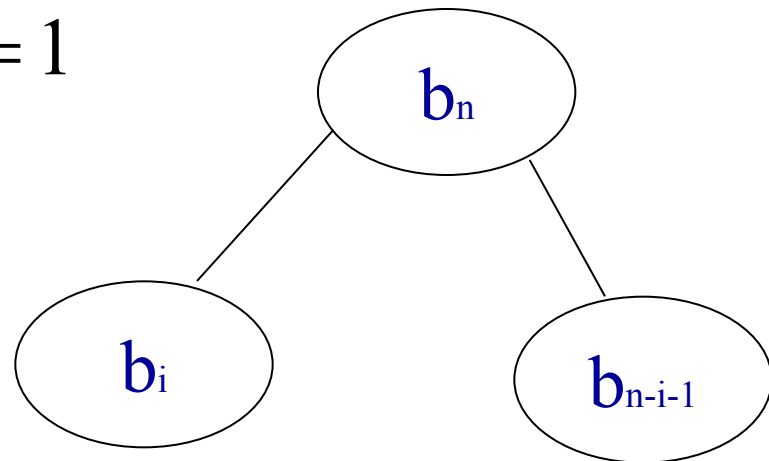
The number of distinct ways to obtain M_{1i} and $M_{i+1,n}$ are b_i and b_{n-i} , respectively. Therefore, we have:

$$b_n = \sum_{i=0}^{n-1} b_i b_{n-i}, n > 1$$

Counting Binary trees(7/10)

- Now instead let b_n be the number of distinct binary trees with n nodes. Again an expression for b_n in terms of n is what we want. Then we see that b_n is the sum of all the possible binary trees formed in the following way: a root and two subtrees with b_i and b_{n-i-1} nodes, for $0 \leq i < n$. This explanation says that

$$b_n = \sum_{i=0}^{n-1} b_i b_{n-i-1}, n \geq 1 \quad \text{and} \quad b_0 = 1$$



Counting Binary trees(8/10)

- Number of Distinct Binary Trees:
 - To obtain number of distinct binary trees with n nodes, we must solve the recurrence. To begin we let:
(5.6)
$$B(x) = \sum_{i \geq 0} b_i x^i$$
 - Which is the generating function for the number of binary trees. Next observe that by the recurrence relation we get the identity: $x B^2(x) = B(x) - 1$
 - Using the formula to solve quadratics and the fact (Eq. (5.5)) that $B(0) = b_0 = 1$, we get

$$B(x) = \frac{1 - \sqrt{1 - 4x}}{2x}$$

Counting Binary trees(9/10)

- Number of Distinct Binary Trees:
 - We can use the binomial theorem to expand $(1 - 4x)^{1/2}$ to obtain:

$$\begin{aligned} B(x) &= \frac{1}{2} \left[1 - \sum_{n \geq 0} \begin{bmatrix} 1/2 \\ n \end{bmatrix} (-4x)^n \right] \\ &= \sum_{m \geq 0} \begin{bmatrix} 1/2 \\ m+1 \end{bmatrix} (-1)^m 2^{2m+1} x^m \end{aligned} \tag{5.7}$$

Counting Binary trees(10/10)

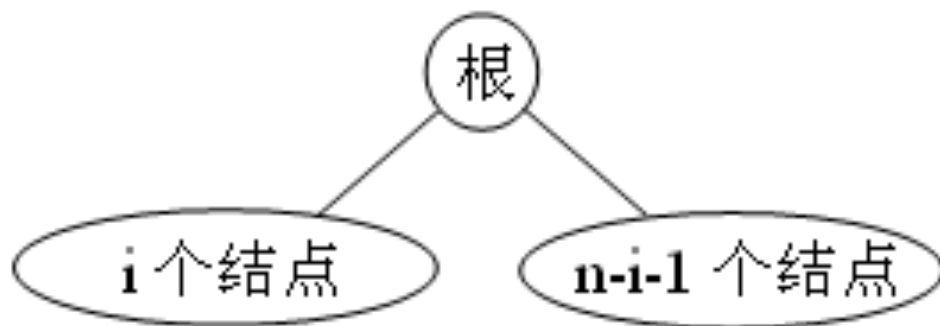
- Number of Distinct Binary Trees:
 - Comparing Eqs.(5.6) and (5.7) we see that b_n , which is the coefficient of x^n in $B(x)$, is : $\left[\begin{matrix} 1/2 \\ n+1 \end{matrix} \right] (-1)^n 2^{2n+1}$

Some simplification yields the more compact form

$$b_n = \frac{1}{n+1} \left[\begin{matrix} 2n \\ n \end{matrix} \right]$$

which is approximately $b_n = O\left(\frac{4^n}{n^{3/2}}\right)$

设 b_n 是具有 n 个结点的不同二叉树的个数。 b_n 实际是按以下方式形成的各种可能的二叉树个数之和：一个根和两个结点数分别为 i 和 $n-i-1$ 的子树($0 \leq i < n$)



对于每一个 i ，存在 b_i b_{n-i-1} 棵不同的树，因此有

$$b_n = \sum_{i=0}^{n-1} b_i b_{n-i-1}, \quad \text{当 } n \geq 1, \text{ 且 } b_0 = 1 \quad (4.3)$$

为了用 n 表示 b_n ，必须求解递归方程(4.3)。设

$$B(x) = \sum_{i \geq 0} b_i x^i \quad (4.4)$$

B(x)是二叉树个数的生成函数。由于

$$\begin{aligned} B(x) &= 1 + \sum_{i \geq 1} b_i x^i \\ &= 1 + \sum_{i \geq 1} \sum_{j=0}^{i-1} b_j b_{i-j-1} x^i \\ &= 1 + x \left(\sum_{i \geq 1} \sum_{j=0}^{i-1} b_j b_{i-j-1} x^{i-1} \right) \end{aligned}$$

$$= 1 + x \left(\sum_{i \geq 0} \sum_{j=0}^i b_j b_{i-j} x^i \right)$$

$$= 1 + x \left(\sum_{k, l \geq 0} b_k b_l x^{k+l} \right)$$

$$= 1 + x B^2(x)$$

即

$$x B^2(x) - B(x) + 1 = 0$$

解此一元二次方程，并注意 $B(0) = b_0 = 1$ （等式(4.3)），可得

$$B(x) = \frac{1 - \sqrt{1 - 4x}}{2x}$$

利用二项式公式展开 $(1 - 4x)^{1/2}$ 得到

$$\begin{aligned} B(x) &= \frac{1}{2x} \left[1 - \sum_{n \geq 0} \binom{1/2}{n} (-4x)^n \right] \\ &= \frac{1}{2x} \left[1 - \binom{1/2}{0} (-4x)^0 - \sum_{n \geq 1} \binom{1/2}{n} (-4x)^n \right] \end{aligned}$$

令 $n = m + 1$ ，可得

$$\begin{aligned} B(x) &= \frac{1}{2x} \left[- \sum_{m \geq 0} \binom{1/2}{m+1} (-1)^{m+1} (2^2)^{m+1} x^{m+1} \right] \\ &= \sum_{m \geq 0} \binom{1/2}{m+1} (-1)^m 2^{2m+1} x^m \end{aligned} \tag{4.5}$$

比较等式(4.4) 和(4.5), 并注意 b_n 是 $B(x)$ 中 x^n 的系数, 可得

$$\begin{aligned}
 b_n &= \binom{1/2}{n+1} (-1)^n 2^{2n+1} \\
 &= \frac{\frac{1}{2} \left(\frac{1}{2} - 1 \right) \cdots \left(\frac{1}{2} - n \right)}{(n+1)!} (-1)^n 2^{2n+1} \\
 &= \frac{\frac{1}{2} \cdot \frac{1}{2} \cdot \frac{3}{2} \cdots \frac{2n-1}{2}}{(n+1)!} 2^{n+1} 2^n
 \end{aligned}$$

$$= \frac{1.3.5....(2n-3).(2n-1)2^n}{(n+1)!} \cdot \frac{n!}{n!}$$

$$= \frac{1.3.5....(2n-3).(2n-1).2.4.6.....(2n-2).(2n)}{(n+1)! . n!}$$

$$= \frac{(2n)!}{(2n-n)! n!} \cdot \frac{1}{n+1}$$

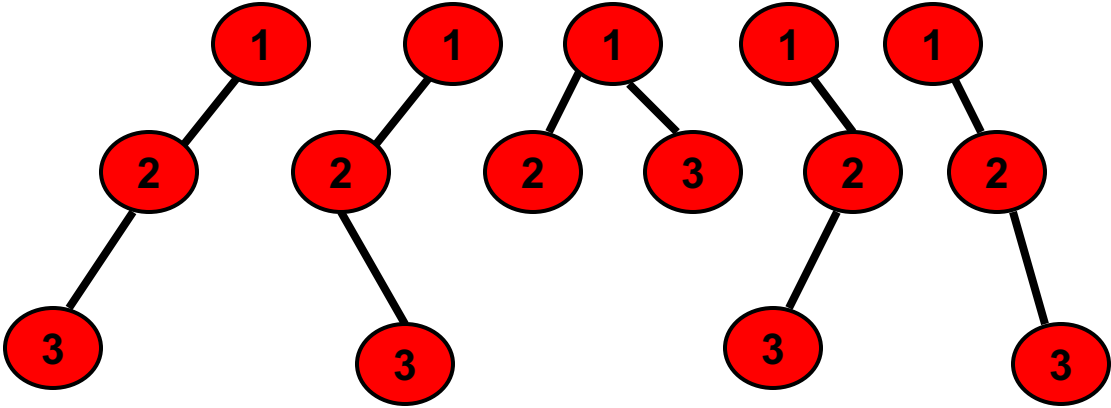
$$= \frac{1}{n+1} \binom{2n}{n} = O(4^n/n^{3/2})$$

树的计数

当 二叉树的结点个数 $n = 3$

前序序列为 1、2、3 时

- 不同的中序序列在中序遍历和相应的进出栈序列一一对应
- 不同的进出栈序列的总数是不同树形的二叉树的总和



1、进栈	1、进栈	1、进栈	1、进栈	1、进栈
2、进栈	2、进栈	2、进栈	1、出栈	1、出栈
3、进栈	2、出栈	2、出栈	2、进栈	2、进栈
3、出栈	3、进栈	1、出栈	3、进栈	2、出栈
2、出栈	3、出栈	3、进栈	3、出栈	3、进栈
1、出栈	1、出栈	3、出栈	2、出栈	3、出栈

树的计数

• 进出栈序列总数的计算为

$2n$ 个方格中放置 n 个1的组合数 — 不合理的序列总数

二叉树的结点个数 $n = 3$

1	1	1	0	0	0	合理
1	1	0	1	0	0	
1	1	0	0	1	0	
1	0	1	1	0	0	
1	0	1	0	1	0	
0	0	0	1	1	1	不合理
1	0	0	1	1	0	

树的计数

1	1	1	0	0	0
1	1	0	1	0	0
1	1	0	0	1	0
1	0	1	1	0	0
1	0	1	0	1	0
0	0	0	1	1	1
1	0	0	1	1	0

合
理

不
合
理

$n = 3$ 时，6 格放置 3 个 1 的序列情况：

$$\text{序列总数} = \binom{3}{2 \times 3}$$

$$\text{不合理的序列总数: } \binom{3+1}{2 \times 3}$$

因此， $n = 3$ 时进出栈序列的总数为：

$$\binom{3}{2 \times 3} - \binom{3+1}{2 \times 3}$$

推广到结点数为 n 时的进出栈序列总数：

$$\binom{n}{2 \times n} - \binom{n+1}{2n}$$

Trees

- Definition and terminologies
- Binary Trees
 - Full binary trees/complete binary trees
 - Representation methods
 - Traversal methods
 - Recursive/iterative
 - Applications
 - Iterators
 - Binary tree constructions

Trees

- Binary search trees
 - Definition
 - Search/Insertion/Removal
- Threaded Binary Trees
 - Head node
 - Traversal methods
 - Insertion/Removal

Trees

- Heaps
 - Definition
 - Insertion/Removal/Initialization
- Selection Trees
 - Winner trees
 - Loser Trees

Trees

- Forests
 - Transformation To/From Binary Trees
 - Traversal methods
- Union-Find
 - Height/Weight rule
 - Path Collapsing
- Huffman Trees